

---

# DICE Embeddings

*Release 0.1.3.2*

**Caglar Demir**

**Feb 13, 2026**

## Contents:

|           |                                                          |            |
|-----------|----------------------------------------------------------|------------|
| <b>1</b>  | <b>Dicee Manual</b>                                      | <b>2</b>   |
| <b>2</b>  | <b>Installation</b>                                      | <b>3</b>   |
| 2.1       | Installation from Source . . . . .                       | 3          |
| <b>3</b>  | <b>Download Knowledge Graphs</b>                         | <b>3</b>   |
| <b>4</b>  | <b>Knowledge Graph Embedding Models</b>                  | <b>3</b>   |
| <b>5</b>  | <b>How to Train</b>                                      | <b>3</b>   |
| <b>6</b>  | <b>Creating an Embedding Vector Database</b>             | <b>5</b>   |
| 6.1       | Learning Embeddings . . . . .                            | 5          |
| 6.2       | Loading Embeddings into Qdrant Vector Database . . . . . | 6          |
| 6.3       | Launching Webservice . . . . .                           | 6          |
| <b>7</b>  | <b>Answering Complex Queries</b>                         | <b>6</b>   |
| <b>8</b>  | <b>Predicting Missing Links</b>                          | <b>8</b>   |
| <b>9</b>  | <b>Downloading Pretrained Models</b>                     | <b>8</b>   |
| <b>10</b> | <b>How to Deploy</b>                                     | <b>8</b>   |
| <b>11</b> | <b>Docker</b>                                            | <b>8</b>   |
| <b>12</b> | <b>Coverage Report</b>                                   | <b>8</b>   |
| <b>13</b> | <b>How to cite</b>                                       | <b>10</b>  |
| <b>14</b> | <b>dicee</b>                                             | <b>12</b>  |
| 14.1      | Submodules . . . . .                                     | 12         |
| 14.2      | Attributes . . . . .                                     | 209        |
| 14.3      | Classes . . . . .                                        | 209        |
| 14.4      | Package Contents . . . . .                               | 210        |
|           | <b>Python Module Index</b>                               | <b>222</b> |

---

DICE Embeddings<sup>1</sup>: Hardware-agnostic Framework for Large-scale Knowledge Graph Embeddings:

## 1 Dicee Manual

**Version:** dicee 0.3.2

**GitHub repository:** <https://github.com/dice-group/dice-embeddings>

**Publisher and maintainer:** Caglar Demir<sup>2</sup>

**Contact:** [caglar.demir@upb.de](mailto:caglar.demir@upb.de)

**License:** OSI Approved :: MIT License

---

Dicee is a hardware-agnostic framework for large-scale knowledge graph embeddings.

Knowledge graph embedding research has mainly focused on learning continuous representations of knowledge graphs towards the link prediction problem. Recently developed frameworks can be effectively applied in a wide range of research-related applications. Yet, using these frameworks in real-world applications becomes more challenging as the size of the knowledge graph grows

We developed the DICE Embeddings framework (dicee) to compute embeddings for large-scale knowledge graphs in a hardware-agnostic manner. To achieve this goal, we rely on

1. **Pandas**<sup>3</sup> & Co. to use parallelism at preprocessing a large knowledge graph,
2. **PyTorch**<sup>4</sup> & Co. to learn knowledge graph embeddings via multi-CPU, GPUs, TPUs or computing cluster, and
3. **Huggingface**<sup>5</sup> to ease the deployment of pre-trained models.

**Why Pandas**<sup>6</sup> & Co. ? A large knowledge graph can be read and preprocessed (e.g. removing literals) by pandas, modin, or polars in parallel. Through polars, a knowledge graph having more than 1 billion triples can be read in parallel fashion. Importantly, using these frameworks allow us to perform all necessary computations on a single CPU as well as a cluster of computers.

**Why PyTorch**<sup>7</sup> & Co. ? PyTorch is one of the most popular machine learning frameworks available at the time of writing. PytorchLightning facilitates scaling the training procedure of PyTorch without boilerplate. In our framework, we combine **PyTorch**<sup>8</sup> & **PytorchLightning**<sup>9</sup>. Users can choose the trainer class (e.g., DDP by Pytorch) to train large knowledge graph embedding models with billions of parameters. PytorchLightning allows us to use state-of-the-art model parallelism techniques (e.g. Fully Sharded Training, FairScale, or DeepSpeed) without extra effort. With our framework, practitioners can directly use PytorchLightning for model parallelism to train gigantic embedding models.

**Why Huggingface**<sup>10</sup>? Seamlessly deploy and share pre-trained embedding models through the Huggingface ecosystem.

---

<sup>1</sup> <https://github.com/dice-group/dice-embeddings>

<sup>2</sup> <https://github.com/Demirrr>

<sup>3</sup> <https://pandas.pydata.org/>

<sup>4</sup> <https://pytorch.org/>

<sup>5</sup> <https://huggingface.co/>

<sup>6</sup> <https://pandas.pydata.org/>

<sup>7</sup> <https://pytorch.org/>

<sup>8</sup> <https://pytorch.org/>

<sup>9</sup> <https://www.pytorchlightning.ai/>

<sup>10</sup> <https://huggingface.co/>

## 2 Installation

### 2.1 Installation from Source

```
git clone https://github.com/dice-group/dice-embeddings.git
conda create -n dice python=3.10.13 --no-default-packages && conda activate dice &&
↪ cd dice-embeddings &&
pip3 install -e .
```

or

```
pip install dicee
```

## 3 Download Knowledge Graphs

```
wget https://files.dice-research.org/datasets/dice-embeddings/KGs.zip --no-check-
↪ certificate && unzip KGs.zip
```

To test the Installation

```
python -m pytest -p no:warnings -x # Runs >114 tests leading to > 15 mins
python -m pytest -p no:warnings --lf # run only the last failed test
python -m pytest -p no:warnings --ff # to run the failures first and then the rest of
↪ the tests.
```

## 4 Knowledge Graph Embedding Models

1. TransE, DistMult, ComplEx, ConEx, QMult, OMult, ConvO, ConvQ, Keci
2. All 44 models available in <https://github.com/pykeen/pykeen#models>

For more, please refer to examples.

## 5 How to Train

To Train a KGE model (KECI) and evaluate it on the train, validation, and test sets of the UMLS benchmark dataset.

```
from dicee.executer import Execute
from dicee.config import Namespace
args = Namespace()
args.model = 'Keci'
args.scoring_technique = "KvsAll" # 1vsAll, or AllvsAll, or NegSample
args.dataset_dir = "KGs/UMLS"
args.path_to_store_single_run = "Keci_UMLS"
args.num_epochs = 100
args.embedding_dim = 32
args.batch_size = 1024
reports = Execute(args).start()
print(reports["Train"]["MRR"]) # => 0.9912
print(reports["Test"]["MRR"]) # => 0.8155
# See the Keci_UMLS folder embeddings and all other files
```

where the data is in the following form

```
$ head -3 KGs/UMLS/train.txt
acquired_abnormality    location_of             experimental_model_of_disease
anatomical_abnormality  manifestation_of        physiologic_function
alga    isa      entity
```

A KGE model can also be trained from the command line

```
dicee --dataset_dir "KGs/UMLS" --model Keci --eval_model "train_val_test"
```

dicee automatically detects available GPUs and trains a model with distributed data parallels technique. Under the hood, dicee uses lightning as a default trainer.

```
# Train a model by only using the GPU-0
CUDA_VISIBLE_DEVICES=0 dicee --dataset_dir "KGs/UMLS" --model Keci --eval_model
↪ "train_val_test"
# Train a model by only using GPU-1
CUDA_VISIBLE_DEVICES=1 dicee --dataset_dir "KGs/UMLS" --model Keci --eval_model
↪ "train_val_test"
NCCL_P2P_DISABLE=1 CUDA_VISIBLE_DEVICES=0,1 python dicee/scripts/run.py --trainer PL -
↪ --dataset_dir "KGs/UMLS" --model Keci --eval_model "train_val_test"
```

Under the hood, dicee executes run.py script and uses lightning as a default trainer

```
# Two equivalent executions
# (1)
dicee --dataset_dir "KGs/UMLS" --model Keci --eval_model "train_val_test"
# Evaluate Keci on Train set: Evaluate Keci on Train set
# {'H@1': 0.9518788343558282, 'H@3': 0.9988496932515337, 'H@10': 1.0, 'MRR': 0.
↪ 9753123402351737}
# Evaluate Keci on Validation set: Evaluate Keci on Validation set
# {'H@1': 0.6932515337423313, 'H@3': 0.9041411042944786, 'H@10': 0.9754601226993865,
↪ 'MRR': 0.8072362996241839}
# Evaluate Keci on Test set: Evaluate Keci on Test set
# {'H@1': 0.6951588502269289, 'H@3': 0.9039334341906202, 'H@10': 0.9750378214826021,
↪ 'MRR': 0.8064032293278861}

# (2)
CUDA_VISIBLE_DEVICES=0,1 python dicee/scripts/run.py --trainer PL --dataset_dir "KGs/
↪ UMLS" --model Keci --eval_model "train_val_test"
# Evaluate Keci on Train set: Evaluate Keci on Train set
# {'H@1': 0.9518788343558282, 'H@3': 0.9988496932515337, 'H@10': 1.0, 'MRR': 0.
↪ 9753123402351737}
# Evaluate Keci on Train set: Evaluate Keci on Train set
# Evaluate Keci on Validation set: Evaluate Keci on Validation set
# {'H@1': 0.6932515337423313, 'H@3': 0.9041411042944786, 'H@10': 0.9754601226993865,
↪ 'MRR': 0.8072362996241839}
# Evaluate Keci on Test set: Evaluate Keci on Test set
# {'H@1': 0.6951588502269289, 'H@3': 0.9039334341906202, 'H@10': 0.9750378214826021,
↪ 'MRR': 0.8064032293278861}
```

Similarly, models can be easily trained with torchrun

```

torchrun --standalone --nnodes=1 --nproc_per_node=gpu dicee/scripts/run.py --trainer_
→torchDDP --dataset_dir "KGs/UMLS" --model Keci --eval_model "train_val_test"
# Evaluate Keci on Train set: Evaluate Keci on Train set: Evaluate Keci on Train set
# {'H@1': 0.9518788343558282, 'H@3': 0.9988496932515337, 'H@10': 1.0, 'MRR': 0.
→9753123402351737}
# Evaluate Keci on Validation set: Evaluate Keci on Validation set
# {'H@1': 0.6932515337423313, 'H@3': 0.9041411042944786, 'H@10': 0.9754601226993865,
→'MRR': 0.8072499937521418}
# Evaluate Keci on Test set: Evaluate Keci on Test set
{'H@1': 0.6951588502269289, 'H@3': 0.9039334341906202, 'H@10': 0.9750378214826021,
→'MRR': 0.8064032293278861}

```

You can also train a model in multi-node multi-gpu setting.

```

torchrun --nnodes 2 --nproc_per_node=gpu --node_rank 0 --rdzv_id 455 --rdzv_backend_
→c10d --rdzv_endpoint=nebula dicee/scripts/run.py --trainer torchDDP --dataset_dir_
→KGs/UMLS
torchrun --nnodes 2 --nproc_per_node=gpu --node_rank 1 --rdzv_id 455 --rdzv_backend_
→c10d --rdzv_endpoint=nebula dicee/scripts/run.py --trainer torchDDP --dataset_dir_
→KGs/UMLS

```

Train a KGE model by providing the path of a single file and store all parameters under newly created directory called KeciFamilyRun.

```

dicee --path_single_kg "KGs/Family/family-benchmark_rich_background.owl" --model Keci_
→--path_to_store_single_run KeciFamilyRun --backend rdflib

```

where the data is in the following form

```

$ head -3 KGs/Family/train.txt
_:1 <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://www.w3.org/2002/07/owl
→#Ontology> .
<http://www.benchmark.org/family#hasChild> <http://www.w3.org/1999/02/22-rdf-syntax-ns
→#type> <http://www.w3.org/2002/07/owl#ObjectProperty> .
<http://www.benchmark.org/family#hasParent> <http://www.w3.org/1999/02/22-rdf-syntax-
→ns#type> <http://www.w3.org/2002/07/owl#ObjectProperty> .

```

Apart from n-triples or standard link prediction dataset formats, we support ["owl", "nt", "turtle", "rdf/xml", "n3"]\*. Moreover, a KGE model can be also trained by providing an endpoint of a triple store.

```

dicee --sparql_endpoint "http://localhost:3030/mutagenesis/" --model Keci

```

For more, please refer to examples.

## 6 Creating an Embedding Vector Database

### 6.1 Learning Embeddings

```

# Train an embedding model
dicee --dataset_dir KGs/Countries-S1 --path_to_store_single_run CountryEmbeddings --
→model Keci --p 0 --q 1 --embedding_dim 32 --adaptive_swa

```

## 6.2 Loading Embeddings into Qdrant Vector Database

```
# Ensure that Qdrant available
# docker pull qdrant/qdrant && docker run -p 6333:6333 -p 6334:6334 -v $(pwd)/
↪qdrant_storage:/qdrant/storage:z qdrant/qdrant
diceeindex --path_model "CountryEmbeddings" --collection_name "dummy" --location
↪"localhost"
```

## 6.3 Launching Webservice

```
diceeserve --path_model "CountryEmbeddings" --collection_name "dummy" --collection_
↪location "localhost"
```

### Retrieve and Search

Get embedding of germany

```
curl -X 'GET' 'http://0.0.0.0:8000/api/get?q=germany' -H 'accept: application/json'
```

Get most similar things to europe

```
curl -X 'GET' 'http://0.0.0.0:8000/api/search?q=europe' -H 'accept: application/json'
{"result":[{"hit":"europe","score":1.0},
{"hit":"northern_europe","score":0.67126536},
{"hit":"western_europe","score":0.6010134},
{"hit":"puerto_rico","score":0.5051694},
{"hit":"southern_europe","score":0.4829831}]}
```

## 7 Answering Complex Queries

```
# pip install dicee
# wget https://files.dice-research.org/datasets/dice-embeddings/KGs.zip --no-check-
↪certificate & unzip KGs.zip
from dicee.executer import Execute
from dicee.config import Namespace
from dicee.knowledge_graph_embeddings import KGE
# (1) Train a KGE model
args = Namespace()
args.model = 'Keci'
args.p=0
args.q=1
args.optim = 'Adam'
args.scoring_technique = "AllvsAll"
args.path_single_kg = "KGs/Family/family-benchmark_rich_background.owl"
args.backend = "rdflib"
args.num_epochs = 200
args.batch_size = 1024
args.lr = 0.1
args.embedding_dim = 512
result = Execute(args).start()
# (2) Load the pre-trained model
```

(continues on next page)

(continued from previous page)

```
pre_trained_kge = KGE(path=result['path_experiment_folder'])
# (3) Single-hop query answering
# Query: ?E : \exist E.hasSibling(E, F9M167)
# Question: Who are the siblings of F9M167?
# Answer: [F9M157, F9F141], as (F9M167, hasSibling, F9M157) and (F9M167, hasSibling, ↵
↵F9F141)
predictions = pre_trained_kge.answer_multi_hop_query(query_type="1p",
                                                       query=('http://www.benchmark.org/
↵family#F9M167',
                                                       ('http://www.benchmark.
↵org/family#hasSibling',)),
                                                       tnorm="min", k=3)
top_entities = [topk_entity for topk_entity, query_score in predictions]
assert "http://www.benchmark.org/family#F9F141" in top_entities
assert "http://www.benchmark.org/family#F9M157" in top_entities
# (2) Two-hop query answering
# Query: ?D : \exist E.Married(D, E) \land hasSibling(E, F9M167)
# Question: To whom a sibling of F9M167 is married to?
# Answer: [F9F158, F9M142] as (F9M157 #married F9F158) and (F9F141 #married F9M142)
predictions = pre_trained_kge.answer_multi_hop_query(query_type="2p",
                                                       query=("http://www.benchmark.org/
↵family#F9M167",
                                                       ("http://www.benchmark.
↵org/family#hasSibling",
                                                       "http://www.benchmark.
↵org/family#married")),
                                                       tnorm="min", k=3)
top_entities = [topk_entity for topk_entity, query_score in predictions]
assert "http://www.benchmark.org/family#F9M142" in top_entities
assert "http://www.benchmark.org/family#F9F158" in top_entities
# (3) Three-hop query answering
# Query: ?T : \exist D.type(D,T) \land Married(D,E) \land hasSibling(E, F9M167)
# Question: What are the type of people who are married to a sibling of F9M167?
# (3) Answer: [Person, Male, Father] since F9M157 is [Brother Father Grandfather ↵
↵Male] and F9M142 is [Male Grandfather Father]
predictions = pre_trained_kge.answer_multi_hop_query(query_type="3p", query=("http://
↵www.benchmark.org/family#F9M167",
                                                       ("http://
↵www.benchmark.org/family#hasSibling",
                                                       "http://
↵www.benchmark.org/family#married",
                                                       "http://
↵www.w3.org/1999/02/22-rdf-syntax-ns#type")),
                                                       tnorm="min", k=5)
top_entities = [topk_entity for topk_entity, query_score in predictions]
print(top_entities)
assert "http://www.benchmark.org/family#Person" in top_entities
assert "http://www.benchmark.org/family#Father" in top_entities
assert "http://www.benchmark.org/family#Male" in top_entities
```

For more, please refer to examples/multi\_hop\_query\_answering.

## 8 Predicting Missing Links

```
from dicee import KGE
# (1) Train a knowledge graph embedding model..
# (2) Load a pretrained model
pre_trained_kge = KGE(path='../')
# (3) Predict missing links through head entity rankings
pre_trained_kge.predict_topk(h=[".."],r=[".."],topk=10)
# (4) Predict missing links through relation rankings
pre_trained_kge.predict_topk(h=[".."],t=[".."],topk=10)
# (5) Predict missing links through tail entity rankings
pre_trained_kge.predict_topk(r=[".."],t=[".."],topk=10)
```

## 9 Downloading Pretrained Models

```
from dicee import KGE
# (1) Load a pretrained ConEx on DBpedia
model = KGE(url="https://files.dice-research.org/projects/DiceEmbeddings/KINSHIP-Keci-
↳dim128-epoch256-KvsAll")
```

- For more please look at [dice-research.org/projects/DiceEmbeddings/](https://dice-research.org/projects/DiceEmbeddings/)<sup>11</sup>

## 10 How to Deploy

```
from dicee import KGE
KGE(path='../').deploy(share=True, top_k=10)
```

## 11 Docker

To build the Docker image:

```
docker build -t dice-embeddings .
```

To test the Docker image:

```
docker run --rm -v ~/.local/share/dicee/KGs:/dicee/KGs dice-embeddings ./main.py --
↳model AConEx --embedding_dim 16
```

## 12 Coverage Report

The coverage report is generated using `coverage.py`<sup>12</sup>:

| Name                 | Stmts | Miss  | Cover | Missing      |
|----------------------|-------|-------|-------|--------------|
| -----                | ----- | ----- | ----- | -----        |
| dicee/___init___ .py | 7     | 0     | 100%  |              |
| dicee/abstracts.py   | 338   | 115   | 66%   | 112-113, 115 |

(continues on next page)

<sup>11</sup> <https://files.dice-research.org/projects/DiceEmbeddings/>

<sup>12</sup> <https://coverage.readthedocs.io/en/7.6.0/>



(continued from previous page)

|                                                                                         |     |     |      |             |
|-----------------------------------------------------------------------------------------|-----|-----|------|-------------|
| ↪131, 154-155, 160, 173, 197, 240-254, 290, 303-306, 309-313, 353-364, 379-387, 402, ↪  |     |     |      |             |
| ↪413-417, 427-428, 434-436, 442-445, 448-453, 576-596, 602-606, 610-612, 631, 658-696   |     |     |      |             |
| dicee/callbacks.py                                                                      | 248 | 103 | 58%  | 50-55, ↪    |
| ↪67-73, 76, 88-93, 98-103, 106-109, 116-133, 138-142, 146-147, 247, 281-285, 291-292, ↪ |     |     |      |             |
| ↪310-316, 319, 324-325, 337-343, 349-358, 363-365, 410, 421-434, 438-473, 485-491       |     |     |      |             |
| dicee/config.py                                                                         | 97  | 2   | 98%  | 146-147     |
| dicee/dataset_classes.py                                                                | 430 | 146 | 66%  | 16, 44, ↪   |
| ↪57, 89-98, 104, 111-118, 121, 124, 127-151, 207-213, 216, 219-221, 324, 335-338, ↪     |     |     |      |             |
| ↪354, 420-421, 439, 562-581, 583, 587-599, 606-615, 618, 622-636, 780-787, 790-794, ↪   |     |     |      |             |
| ↪845, 866-878, 902-915, 937, 941-954, 964-967, 973, 985, 987, 989, 1012-1022            |     |     |      |             |
| dicee/eval_static_funcs.py                                                              | 256 | 100 | 61%  | 104, 109, ↪ |
| ↪114, 261-356, 363-414, 442, 465-468                                                    |     |     |      |             |
| dicee/evaluator.py                                                                      | 267 | 48  | 82%  | 48, 53, ↪   |
| ↪58, 77, 82-83, 86, 102, 119, 130, 134, 139, 173-184, 191-202, 310, 340-358, 452, ↪     |     |     |      |             |
| ↪462, 480-485                                                                           |     |     |      |             |
| dicee/executer.py                                                                       | 134 | 16  | 88%  | 53-57, ↪    |
| ↪166-176, 235-236, 283                                                                  |     |     |      |             |
| dicee/knowledge_graph.py                                                                | 82  | 10  | 88%  | 84, 94- ↪   |
| ↪95, 124, 128, 132-134, 137-138, 140                                                    |     |     |      |             |
| dicee/knowledge_graph_embeddings.py                                                     | 654 | 415 | 37%  | 25, 28- ↪   |
| ↪29, 37-50, 55-88, 91-125, 129-137, 171, 173-229, 261, 265, 276-277, 301-303, 311, ↪    |     |     |      |             |
| ↪339-362, 493, 497-519, 523-547, 580, 656, 665, 710-716, 748, 806-1171, 1202-1263, ↪    |     |     |      |             |
| ↪1267-1295, 1326, 1332                                                                  |     |     |      |             |
| dicee/models/___init___py                                                               | 9   | 0   | 100% |             |
| dicee/models/adopt.py                                                                   | 187 | 172 | 8%   | 50-86, ↪    |
| ↪99-110, 129-185, 195-242, 266-322, 346-448, 484-517                                    |     |     |      |             |
| dicee/models/base_model.py                                                              | 240 | 35  | 85%  | 30-35, ↪    |
| ↪64, 66, 92, 99-116, 171, 204, 244, 250, 259, 262, 266, 273, 277, 279, 294, 307-308, ↪  |     |     |      |             |
| ↪362, 365, 438, 450                                                                     |     |     |      |             |
| dicee/models/clifford.py                                                                | 470 | 278 | 41%  | 10, 12, ↪   |
| ↪16, 24-25, 52-56, 79-87, 101-103, 108-109, 140-160, 184, 191, 195-256, 273-277, 289, ↪ |     |     |      |             |
| ↪292, 297, 302, 346-361, 377-444, 464-470, 483, 486, 491, 496, 525-531, 544, 547, ↪     |     |     |      |             |
| ↪552, 557, 567-576, 592-593, 613-685, 696-699, 724-749, 773-806, 842-846, 859, 869, ↪   |     |     |      |             |
| ↪872, 877, 882, 887, 891, 895, 904-905, 935, 942, 947, 975-979, 1007-1016, 1026-1034, ↪ |     |     |      |             |
| ↪1052-1054, 1072-1074, 1090-1092                                                        |     |     |      |             |
| dicee/models/complex.py                                                                 | 162 | 25  | 85%  | 86-109, ↪   |
| ↪273-287                                                                                |     |     |      |             |
| dicee/models/dualE.py                                                                   | 59  | 10  | 83%  | 93-102, ↪   |
| ↪142-156                                                                                |     |     |      |             |
| dicee/models/ensemble.py                                                                | 89  | 67  | 25%  | 7-29, 31, ↪ |
| ↪34, 37, 40, 43, 46, 49, 52-54, 56-58, 64-68, 71-90, 93-94, 97-112, 131                 |     |     |      |             |
| dicee/models/function_space.py                                                          | 262 | 221 | 16%  | 10-23, ↪    |
| ↪27-36, 39-48, 52-69, 76-87, 90-99, 102-111, 115-127, 135-157, 160-166, 169-186, 189- ↪ |     |     |      |             |
| ↪195, 198-206, 209, 214-235, 244-247, 251-255, 259-268, 272-293, 302-308, 312-329, ↪    |     |     |      |             |
| ↪333-336, 345-353, 356, 367-373, 393-407, 425-439, 444-454, 462-466, 475-479            |     |     |      |             |
| dicee/models/literal.py                                                                 | 33  | 1   | 97%  | 82          |
| dicee/models/octonion.py                                                                | 227 | 83  | 63%  | 21-44, ↪    |
| ↪320-329, 334-345, 348-370, 374-416, 426-474                                            |     |     |      |             |
| dicee/models/pykeen_models.py                                                           | 55  | 5   | 91%  | 77-80, ↪    |
| ↪135                                                                                    |     |     |      |             |
| dicee/models/quaternion.py                                                              | 192 | 69  | 64%  | 7-21, 30- ↪ |
| ↪55, 68-72, 107, 185, 328-342, 345-364, 368-389, 399-426                                |     |     |      |             |

(continues on next page)

(continued from previous page)

|                                                                                       |      |      |      |            |
|---------------------------------------------------------------------------------------|------|------|------|------------|
| dicee/models/real.py                                                                  | 61   | 12   | 80%  | 37-42, ↵   |
| ↪70-73, 91, 107-110                                                                   |      |      |      |            |
| dicee/models/static_funcs.py                                                          | 10   | 0    | 100% |            |
| dicee/models/transformers.py                                                          | 234  | 189  | 19%  | 20-39, ↵   |
| ↪42, 56-71, 80-98, 101-112, 119-121, 124, 130-147, 151-176, 182-186, 189-193, 199-    |      |      |      |            |
| ↪203, 206-208, 225-252, 261-264, 267-272, 275-300, 306-311, 315-368, 372-394, 400-410 |      |      |      |            |
| dicee/query_generator.py                                                              | 374  | 346  | 7%   | 17-51, ↵   |
| ↪55, 61-64, 68-69, 77-91, 99-146, 154-187, 191-205, 211-268, 273-302, 306-442, 452-   |      |      |      |            |
| ↪471, 479-502, 509-513, 518, 523-529                                                  |      |      |      |            |
| dicee/read_preprocess_save_load_kg/___init___py                                       | 3    | 0    | 100% |            |
| dicee/read_preprocess_save_load_kg/preprocess.py                                      | 243  | 40   | 84%  | 33, 39, ↵  |
| ↪76, 100-125, 131, 136-149, 175, 205, 380-381                                         |      |      |      |            |
| dicee/read_preprocess_save_load_kg/read_from_disk.py                                  | 36   | 11   | 69%  | 34, 38-    |
| ↪40, 47, 55, 58-72                                                                    |      |      |      |            |
| dicee/read_preprocess_save_load_kg/save_load_disk.py                                  | 53   | 21   | 60%  | 29-30, ↵   |
| ↪38, 47-68                                                                            |      |      |      |            |
| dicee/read_preprocess_save_load_kg/util.py                                            | 236  | 125  | 47%  | 159, 173-  |
| ↪175, 179-180, 198-204, 207-209, 214-216, 230, 244-247, 252-260, 265-271, 276-281, ↵  |      |      |      |            |
| ↪286-291, 303-324, 330-386, 390-394, 398-399, 403, 407-408, 436, 441, 448-449         |      |      |      |            |
| dicee/sanity_checkers.py                                                              | 47   | 19   | 60%  | 8-12, 21-  |
| ↪31, 46, 51, 58, 69-79                                                                |      |      |      |            |
| dicee/static_funcs.py                                                                 | 483  | 194  | 60%  | 42, 52, ↵  |
| ↪58-63, 85, 92-96, 109-119, 129-131, 136, 143, 167, 172, 184, 190, 198, 202, 229-233, |      |      |      |            |
| ↪295, 303-309, 320-330, 341-361, 389, 413-414, 419-420, 437-438, 440-441, 443-444, ↵  |      |      |      |            |
| ↪452, 470-474, 491-494, 498-503, 507-511, 515-516, 522-524, 539-553, 558-561, 566-    |      |      |      |            |
| ↪569, 578-629, 634-646, 663-680, 683-691, 695-713, 724                                |      |      |      |            |
| dicee/static_funcs_training.py                                                        | 155  | 66   | 57%  | 7-10, ↵    |
| ↪222-319, 327-328                                                                     |      |      |      |            |
| dicee/static_preprocess_funcs.py                                                      | 98   | 43   | 56%  | 17-25, ↵   |
| ↪50, 57, 59, 70, 83-107, 112-115, 120-123, 128-131                                    |      |      |      |            |
| dicee/trainer/___init___py                                                            | 1    | 0    | 100% |            |
| dicee/trainer/dice_trainer.py                                                         | 151  | 18   | 88%  | 22, 30-    |
| ↪31, 33-35, 97, 104, 109-114, 152, 237, 280-283                                       |      |      |      |            |
| dicee/trainer/model_parallelism.py                                                    | 99   | 87   | 12%  | 10-25, ↵   |
| ↪30-116, 121-132, 136, 141-197                                                        |      |      |      |            |
| dicee/trainer/torch_trainer.py                                                        | 77   | 6    | 92%  | 31, 102, ↵ |
| ↪168, 179-181                                                                         |      |      |      |            |
| dicee/trainer/torch_trainer_ddp.py                                                    | 89   | 71   | 20%  | 11-14, ↵   |
| ↪43, 47-67, 78-94, 113-122, 126-136, 151-158, 168-191                                 |      |      |      |            |
| -----                                                                                 |      |      |      |            |
| TOTAL                                                                                 | 6948 | 3169 | 54%  |            |

## 13 How to cite

Currently, we are working on our manuscript describing our framework. If you really like our work and want to cite it now, feel free to chose one :)

```
# Keci
@inproceedings{demir2023clifford,
  title={Clifford Embeddings--A Generalized Approach for Embedding in Normed Algebras}
  ↪,
```

(continues on next page)

```

    author={Demir, Caglar and Ngonga Ngomo, Axel-Cyrille},
    booktitle={Joint European Conference on Machine Learning and Knowledge Discovery in
↪Databases},
    pages={567--582},
    year={2023},
    organization={Springer}
}
# LitCQD
@inproceedings{demir2023litcq,
    title={LitCQD: Multi-Hop Reasoning in Incomplete Knowledge Graphs with Numeric
↪Literals},
    author={Demir, Caglar and Wiebesiek, Michel and Lu, Renzhong and Ngonga Ngomo, Axel-
↪Cyrille and Heindorf, Stefan},
    booktitle={Joint European Conference on Machine Learning and Knowledge Discovery in
↪Databases},
    pages={617--633},
    year={2023},
    organization={Springer}
}
# DICE Embedding Framework
@article{demir2022hardware,
    title={Hardware-agnostic computation for large-scale knowledge graph embeddings},
    author={Demir, Caglar and Ngomo, Axel-Cyrille Ngonga},
    journal={Software Impacts},
    year={2022},
    publisher={Elsevier}
}
# KronE
@inproceedings{demir2022kronecker,
    title={Kronecker decomposition for knowledge graph embeddings},
    author={Demir, Caglar and Lienen, Julian and Ngonga Ngomo, Axel-Cyrille},
    booktitle={Proceedings of the 33rd ACM Conference on Hypertext and Social Media},
    pages={1--10},
    year={2022}
}
# QMult, OMult, ConvQ, ConvO
@InProceedings{pmlr-v157-demir21a,
    title = {Convolutional Hypercomplex Embeddings for Link Prediction},
    author = {Demir, Caglar and Moussallem, Diego and Heindorf, Stefan and Ngonga
↪Ngomo, Axel-Cyrille},
    booktitle = {Proceedings of The 13th Asian Conference on Machine Learning},
    pages = {656--671},
    year = {2021},
    editor = {Balasubramanian, Vineeth N. and Tsang, Ivor},
    volume = {157},
    series = {Proceedings of Machine Learning Research},
    month = {17--19 Nov},
    publisher = {PMLR},
    pdf = {https://proceedings.mlr.press/v157/demir21a/demir21a.pdf},
    url = {https://proceedings.mlr.press/v157/demir21a.html},
}
# ConEx

```

```
@inproceedings{demir2021convolutional,
  title={Convolutional Complex Knowledge Graph Embeddings},
  author={Caglar Demir and Axel-Cyrille Ngonga Ngomo},
  booktitle={Eighteenth Extended Semantic Web Conference - Research Track},
  year={2021},
  url={https://openreview.net/forum?id=6T45-4TFqaX}}
# Shallom
@inproceedings{demir2021shallow,
  title={A shallow neural model for relation prediction},
  author={Demir, Caglar and Moussallem, Diego and Ngomo, Axel-Cyrille Ngonga},
  booktitle={2021 IEEE 15th International Conference on Semantic Computing (ICSC)},
  pages={179--182},
  year={2021},
  organization={IEEE}}
```

For any questions or wishes, please contact: [caglar.demir@upb.de](mailto:caglar.demir@upb.de)

## 14 dicee

DICE Embeddings - Knowledge Graph Embedding Library.

A library for training and using knowledge graph embedding models with support for various scoring techniques and training strategies.

### Submodules:

evaluation: Model evaluation functions and Evaluator class models: KGE model implementations trainer: Training orchestration scripts: Utility scripts

### 14.1 Submodules

**`dicee.__main__`**

**`dicee.abstracts`**

### Classes

|                                      |                                                                                  |
|--------------------------------------|----------------------------------------------------------------------------------|
| <i>AbstractTrainer</i>               | Abstract class for Trainer class for knowledge graph embedding models            |
| <i>BaseInteractiveKGE</i>            | Abstract/base class for using knowledge graph embedding models interactively.    |
| <i>InteractiveQueryDecomposition</i> |                                                                                  |
| <i>AbstractCallback</i>              | Abstract class for Callback class for knowledge graph embedding models           |
| <i>AbstractPPECallback</i>           | Abstract class for Callback class for knowledge graph embedding models           |
| <i>BaseInteractiveTrainKGE</i>       | Abstract/base class for training knowledge graph embedding models interactively. |

## Module Contents

**class** dicee.abstracts.**AbstractTrainer** (*args, callbacks*)

Abstract class for Trainer class for knowledge graph embedding models

### Parameter

**args**

[str] ?

**callbacks: list**

?

**attributes**

**callbacks**

**is\_global\_zero** = True

**global\_rank** = 0

**local\_rank** = 0

**strategy** = None

**on\_fit\_start** (*\*args, \*\*kwargs*)

A function to call callbacks before the training starts.

### Parameter

**args**

**kwargs**

**rtype**

None

**on\_fit\_end** (*\*args, \*\*kwargs*)

A function to call callbacks at the end of the training.

### Parameter

**args**

**kwargs**

**rtype**

None

**on\_train\_epoch\_start** (*\*args, \*\*kwargs*)

A function to call callbacks at the start of an epoch.

### Parameter

args

kwargs

**rtype**

None

**on\_train\_epoch\_end** (\*args, \*\*kwargs)

A function to call callbacks at the end of an epoch.

### Parameter

args

kwargs

**rtype**

None

**on\_train\_batch\_end** (\*args, \*\*kwargs)

A function to call callbacks at the end of each mini-batch during training.

### Parameter

args

kwargs

**rtype**

None

**static save\_checkpoint** (full\_path: str, model) → None

A static function to save a model into disk

### Parameter

full\_path : str

model:

**rtype**

None

**class** dicee.abstracts.**BaseInteractiveKGE** (path: str = None, url: str = None,  
construct\_ensemble: bool = False, model\_name: str = None,  
apply\_semantic\_constraint: bool = False)

Abstract/base class for using knowledge graph embedding models interactively.

### Parameter

**path\_of\_pretrained\_model\_dir**

[str] ?

**construct\_ensemble: boolean**

?

model\_name: str apply\_semantic\_constraint : boolean

**construct\_ensemble = False**

**apply\_semantic\_constraint = False**

**configs**

**get\_eval\_report()** → dict

**get\_bpe\_token\_representation** (*str\_entity\_or\_relation: List[str] | str*) → List[List[int]] | List[int]

#### Parameters

**str\_entity\_or\_relation** (*corresponds to a str or a list of strings to be tokenized via BPE and shaped.*)

#### Return type

A list integer(s) or a list of lists containing integer(s)

**get\_padded\_bpe\_triple\_representation** (*triples: List[List[str]]*) → Tuple[List, List, List]

#### Parameters

**triples**

**set\_model\_train\_mode()** → None

Setting the model into training mode

#### Parameter

**set\_model\_eval\_mode()** → None

Setting the model into eval mode

#### Parameter

**property name**

**sample\_entity** (*n: int*) → List[str]

**sample\_relation** (*n: int*) → List[str]

**is\_seen** (*entity: str = None, relation: str = None*) → bool

**save()** → None

**get\_entity\_index** (*x: str*)

**get\_relation\_index** (*x: str*)

**index\_triple** (*head\_entity: List[str], relation: List[str], tail\_entity: List[str]*)  
→ Tuple[torch.LongTensor, torch.LongTensor, torch.LongTensor]

Index Triple

#### Parameter

**head\_entity:** List[str]

String representation of selected entities.

**relation:** List[str]

String representation of selected relations.

tail\_entity: List[str]

String representation of selected entities.

### Returns: Tuple

pytorch tensor of triple score

**add\_new\_entity\_embeddings** (entity\_name: str = None, embeddings: torch.FloatTensor = None)

**get\_entity\_embeddings** (items: List[str])

Return embedding of an entity given its string representation

### Parameter

**items:**

entities

**get\_relation\_embeddings** (items: List[str])

Return embedding of a relation given its string representation

### Parameter

**items:**

relations

**construct\_input\_and\_output** (head\_entity: List[str], relation: List[str], tail\_entity: List[str], labels)

Construct a data point :param head\_entity: :param relation: :param tail\_entity: :param labels: :return:

**parameters** ()

**class** dicee.abstracts.**InteractiveQueryDecomposition**

**t\_norm** (tens\_1: torch.Tensor, tens\_2: torch.Tensor, tnorm: str = 'min') → torch.Tensor

**tensor\_t\_norm** (subquery\_scores: torch.FloatTensor, tnorm: str = 'min') → torch.FloatTensor

Compute T-norm over  $[0,1]^{n \times d}$  where n denotes the number of hops and d denotes number of entities

**t\_conorm** (tens\_1: torch.Tensor, tens\_2: torch.Tensor, tconorm: str = 'min') → torch.Tensor

**negnorm** (tens\_1: torch.Tensor, lambda\_: float, neg\_norm: str = 'standard') → torch.Tensor

**class** dicee.abstracts.**AbstractCallback**

Bases: abc.ABC, lightning.pytorch.callbacks.Callback

Abstract class for Callback class for knowledge graph embedding models

### Parameter

**on\_init\_start** (\*args, \*\*kwargs)



### Parameter

trainer:

model:

**rtype**

None

**on\_init\_end** (*\*args, \*\*kwargs*)

Call at the beginning of the training.

### Parameter

trainer:

model:

**rtype**

None

**on\_fit\_start** (*trainer, model*)

Call at the beginning of the training.

### Parameter

trainer:

model:

**rtype**

None

**on\_train\_epoch\_end** (*trainer, model*)

Call at the end of each epoch during training.

### Parameter

trainer:

model:

**rtype**

None

**on\_train\_batch\_end** (*\*args, \*\*kwargs*)

Call at the end of each mini-batch during the training.

### Parameter

trainer:

model:

**rtype**

None

**on\_fit\_end** (*\*args, \*\*kwargs*)

Call at the end of the training.

### Parameter

trainer:

model:

**rtype**

None

```
class dicee.abstracts.AbstractPPECallback (num_epochs, path, epoch_to_start,  
      last_percent_to_consider)
```

Bases: [\*AbstractCallback\*](#)

Abstract class for Callback class for knowledge graph embedding models

### Parameter

**num\_epochs**

**path**

**sample\_counter** = 0

**epoch\_count** = 0

**alphas** = None

**on\_fit\_start** (trainer, model)

Call at the beginning of the training.

### Parameter

trainer:

model:

**rtype**

None

**on\_fit\_end** (trainer, model)

Call at the end of the training.

### Parameter

trainer:

model:

**rtype**

None

**store\_ensemble** (param\_ensemble) → None

```
class dicee.abstracts.BaseInteractiveTrainKGE
```

Abstract/base class for training knowledge graph embedding models interactively. This class provides methods for re-training KGE models and also Literal Embedding model.

**train\_triples** (h: List[str], r: List[str], t: List[str], labels: List[float], iteration=2, optimizer=None)

```
train_k_vs_all (h, r, iteration=1, lr=0.001)
```

Train k vs all :param head\_entity: :param relation: :param iteration: :param lr: :return:

```
train (kg, lr=0.1, epoch=10, batch_size=32, neg_sample_ratio=10, num_workers=1) → None
```

Retrained a pretrain model on an input KG via negative sampling.

```
train_literals (train_file_path: str = None, num_epochs: int = 100, lit_lr: float = 0.001,  
lit_normalization_type: str = 'z-norm', batch_size: int = 1024, sampling_ratio: float = None,  
random_seed=1, loader_backend: str = 'pandas', freeze_entity_embeddings: bool = True,  
gate_residual: bool = True, device: str = None, suffle_data: bool = True)
```

Trains the Literal Embeddings model using literal data.

### Parameters

- **train\_file\_path** (*str*) – Path to the training data file.
- **num\_epochs** (*int*) – Number of training epochs.
- **lit\_lr** (*float*) – Learning rate for the literal model.
- **norm\_type** (*str*) – Normalization type to use ('z-norm', 'min-max', or None).
- **batch\_size** (*int*) – Batch size for training.
- **sampling\_ratio** (*float*) – Ratio of training triples to use.
- **loader\_backend** (*str*) – Backend for loading the dataset ('pandas' or 'rdflib').
- **freeze\_entity\_embeddings** (*bool*) – If True, freeze the entity embeddings during training.
- **gate\_residual** (*bool*) – If True, use gate residual connections in the model.
- **device** (*str*) – Device to use for training ('cuda' or 'cpu'). If None, will use available GPU or CPU.
- **suffle\_data** (*bool*) – If True, shuffle the dataset before training.

## dicee.analyse\_experiments

This script should be moved to dicee/scripts Example: python dicee/analyse\_experiments.py --dir Experiments --features "model" "trainMRR" "testMRR"

## Classes

---

*Experiment*

---

## Functions

---

```
get_default_arguments()  
analyse(args)
```

---

## Module Contents

```
dicee.analyse_experiments.get_default_arguments()
```

```
class dicee.analyse_experiments.Experiment
```

```

model_name = []

callbacks = []

embedding_dim = []

num_params = []

num_epochs = []

batch_size = []

lr = []

byte_pair_encoding = []

aswa = []

path_dataset_folder = []

full_storage_path = []

pq = []

train_mrr = []

train_h1 = []

train_h3 = []

train_h10 = []

val_mrr = []

val_h1 = []

val_h3 = []

val_h10 = []

test_mrr = []

test_h1 = []

test_h3 = []

test_h10 = []

runtime = []

normalization = []

scoring_technique = []

save_experiment(x)

to_df()

dicee.analyse_experiments.analyse(args)

```

## dicee.callbacks

Callbacks for training lifecycle events.

Provides callback classes for various training events including epoch end, model saving, weight averaging, and evaluation.

### Classes

|                                    |                                                                                              |
|------------------------------------|----------------------------------------------------------------------------------------------|
| <i>AccumulateEpochLossCallback</i> | Callback to store epoch losses to a CSV file.                                                |
| <i>PrintCallback</i>               | Abstract class for Callback class for knowledge graph embedding models                       |
| <i>KGESaveCallback</i>             | Abstract class for Callback class for knowledge graph embedding models                       |
| <i>PseudoLabellingCallback</i>     | Abstract class for Callback class for knowledge graph embedding models                       |
| <i>Eval</i>                        | Abstract class for Callback class for knowledge graph embedding models                       |
| <i>KronE</i>                       | Abstract class for Callback class for knowledge graph embedding models                       |
| <i>Perturb</i>                     | A callback for a three-Level Perturbation                                                    |
| <i>PeriodicEvalCallback</i>        | Callback to periodically evaluate the model and optionally save checkpoints during training. |
| <i>LRScheduler</i>                 | Callback for managing learning rate scheduling and model snapshots.                          |

### Functions

|                                     |                                                  |
|-------------------------------------|--------------------------------------------------|
| <i>estimate_q</i> (eps)             | estimate rate of convergence q from sequence esp |
| <i>compute_convergence</i> (seq, i) |                                                  |

### Module Contents

```
class dicee.callbacks.AccumulateEpochLossCallback (path: str)
```

Bases: *dicee.abstracts.AbstractCallback*

Callback to store epoch losses to a CSV file.

#### Parameters

**path** – Directory path where the loss file will be saved.

**path**

**on\_fit\_end** (trainer, model) → None

Store epoch loss history to CSV file.

#### Parameters

- **trainer** – The trainer instance.
- **model** – The model being trained.

```
class dicee.callbacks.PrintCallback
```

Bases: *dicee.abstracts.AbstractCallback*

Abstract class for Callback class for knowledge graph embedding models

## Parameter

**start\_time**

**on\_fit\_start** (*trainer, pl\_module*)

Call at the beginning of the training.

## Parameter

trainer:

model:

**rtype**

None

**on\_fit\_end** (*trainer, pl\_module*)

Call at the end of the training.

## Parameter

trainer:

model:

**rtype**

None

**on\_train\_batch\_end** (*\*args, \*\*kwargs*)

Call at the end of each mini-batch during the training.

## Parameter

trainer:

model:

**rtype**

None

**on\_train\_epoch\_end** (*\*args, \*\*kwargs*)

Call at the end of each epoch during training.

## Parameter

trainer:

model:

**rtype**

None

**class** dicee.callbacks.KGESaveCallback (*every\_x\_epoch: int, max\_epochs: int, path: str*)

Bases: *dicee.abstracts.AbstractCallback*

Abstract class for Callback class for knowledge graph embedding models

## Parameter

**every\_x\_epoch**

**max\_epochs**

**epoch\_counter = 0**

**path**

**on\_train\_batch\_end** (*\*args, \*\*kwargs*)

Call at the end of each mini-batch during the training.

## Parameter

trainer:

model:

**rtype**

None

**on\_fit\_start** (*trainer, pl\_module*)

Call at the beginning of the training.

## Parameter

trainer:

model:

**rtype**

None

**on\_train\_epoch\_end** (*\*args, \*\*kwargs*)

Call at the end of each epoch during training.

## Parameter

trainer:

model:

**rtype**

None

**on\_fit\_end** (*\*args, \*\*kwargs*)

Call at the end of the training.

## Parameter

trainer:

model:

**rtype**

None

**on\_epoch\_end** (*model, trainer, \*\*kwargs*)

```
class dicee.callbacks.PseudoLabellingCallback (data_module, kg, batch_size)
```

Bases: *dicee.abstracts.AbstractCallback*

Abstract class for Callback class for knowledge graph embedding models

### Parameter

**data\_module**

**kg**

**num\_of\_epochs** = 0

**unlabelled\_size**

**batch\_size**

**create\_random\_data**()

**on\_epoch\_end** (*trainer, model*)

```
dicee.callbacks.estimate_q (eps)
```

estimate rate of convergence q from sequence esp

```
dicee.callbacks.compute_convergence (seq, i)
```

```
class dicee.callbacks.Eval (path, epoch_ratio: int = None)
```

Bases: *dicee.abstracts.AbstractCallback*

Abstract class for Callback class for knowledge graph embedding models

### Parameter

**path**

**reports** = []

**epoch\_ratio** = None

**epoch\_counter** = 0

**on\_fit\_start** (*trainer, model*)

Call at the beginning of the training.

### Parameter

trainer:

model:

**rtype**

None

**on\_fit\_end** (*trainer, model*)

Call at the end of the training.



### Parameter

trainer:

model:

**rtype**

None

**on\_train\_epoch\_end** (*trainer, model*)

Call at the end of each epoch during training.

### Parameter

trainer:

model:

**rtype**

None

**on\_train\_batch\_end** (*\*args, \*\*kwargs*)

Call at the end of each mini-batch during the training.

### Parameter

trainer:

model:

**rtype**

None

**class** dicee.callbacks.**KronE**

Bases: *dicee.abstracts.AbstractCallback*

Abstract class for Callback class for knowledge graph embedding models

### Parameter

**f** = None

**static batch\_kronecker\_product** (*a, b*)

Kronecker product of matrices a and b with leading batch dimensions. Batch dimensions are broadcast. The number of them must be the same. :type a: torch.Tensor :type b: torch.Tensor :rtype: torch.Tensor

**get\_kronecker\_triple\_representation** (*indexed\_triple: torch.LongTensor*)

Get kronecker embeddings

**on\_fit\_start** (*trainer, model*)

Call at the beginning of the training.

### Parameter

trainer:

model:

**rtype**

None

```
class dicee.callbacks.Perturb (level: str = 'input', ratio: float = 0.0, method: str = None,  
                             scaler: float = None, frequency=None)
```

Bases: `dicee.abstracts.AbstractCallback`

A callback for a three-Level Perturbation

Input Perturbation: During training an input  $x$  is perturbed by randomly replacing its element. In the context of knowledge graph embedding models,  $x$  can denote a triple, a tuple of an entity and a relation, or a tuple of two entities. A perturbation means that a component of  $x$  is randomly replaced by an entity or a relation.

Parameter Perturbation:

Output Perturbation:

```
level = 'input'
```

```
ratio = 0.0
```

```
method = None
```

```
scaler = None
```

```
frequency = None
```

```
on_train_batch_start (trainer, model, batch, batch_idx)
```

Called when the train batch begins.

```
class dicee.callbacks.PeriodicEvalCallback (experiment_path: str, max_epochs: int,  
                                           eval_every_n_epoch: int = 0, eval_at_epochs: list = None,  
                                           save_model_every_n_epoch: bool = True, n_epochs_eval_model: str = 'val_test')
```

Bases: `dicee.abstracts.AbstractCallback`

Callback to periodically evaluate the model and optionally save checkpoints during training.

Evaluates at regular intervals (every  $N$  epochs) or at explicitly specified epochs. Stores evaluation reports and model checkpoints.

```
experiment_dir
```

```
max_epochs
```

```
epoch_counter = 0
```

```
save_model_every_n_epoch = True
```

```
reports
```

```
n_epochs_eval_model = 'val_test'
```

```
default_eval_model = None
```

```
eval_epochs
```

```
on_fit_end (trainer, model)
```

Called at the end of training. Saves final evaluation report.

```
on_train_epoch_end (trainer, model)
```

Called at the end of each training epoch. Performs evaluation and checkpointing if scheduled.

```

class dicee.callbacks.LRScheduler(adaptive_lr_config: dict, total_epochs: int, experiment_dir: str,
                                eta_max: float = 0.1, snapshot_dir: str = 'snapshots')
    Bases: dicee.abstracts.AbstractCallback

    Callback for managing learning rate scheduling and model snapshots.

    Supports cosine annealing (“cca”), MMCCLR (“mmcclr”), and their deferred (warmup) variants: - “deferred_cca”
    - “deferred_mmcclr”

    At the end of each learning rate cycle, the model can optionally be saved as a snapshot.

    total_epochs

    experiment_dir

    snapshot_dir

    batches_per_epoch = None

    total_steps = None

    cycle_length = None

    warmup_steps = None

    lr_lambda = None

    scheduler = None

    step_count = 0

    snapshot_loss

    on_train_start(trainer, model)
        Initialize training parameters and LR scheduler at start of training.

    on_train_batch_end(trainer, model, outputs, batch, batch_idx)
        Step the LR scheduler and save model snapshot if needed after each batch.

    on_fit_end(trainer, model)
        Call at the end of the training.

    Parameter

    trainer:

    model:

    rtype
        None

```

## **`dicee.config`**

Configuration module for DICE embeddings.

Provides the Namespace class with default configuration values for training knowledge graph embedding models.

## Classes

|                  |                                                             |
|------------------|-------------------------------------------------------------|
| <i>Namespace</i> | Extended Namespace with default KGE training configuration. |
|------------------|-------------------------------------------------------------|

## Module Contents

```
class dicee.config.Namespace (**kwargs)
```

Bases: argparse.Namespace

Extended Namespace with default KGE training configuration.

Provides sensible defaults for all training parameters while allowing easy customization through command-line arguments or direct assignment.

**dataset\_dir: str = None**

The path of a folder containing train.txt, and/or valid.txt and/or test.txt

**save\_embeddings\_as\_csv: bool = False**

Embeddings of entities and relations are stored into CSV files to facilitate easy usage.

**storage\_path: str = 'Experiments'**

A directory named with time of execution under `storage_path` that contains related data about embeddings.

**path\_to\_store\_single\_run: str = None**

A single directory created that contains related data about embeddings.

**path\_single\_kg = None**

Path of a file corresponding to the input knowledge graph

**sparql\_endpoint = None**

An endpoint of a triple store.

**model: str = 'Keci'**

KGE model

**optim: str = 'Adam'**

Optimizer

**embedding\_dim: int = 64**

Size of continuous vector representation of an entity/relation

**num\_epochs: int = 150**

Number of pass over the training data

**batch\_size: int = 1024**

Mini-batch size if it is None, an automatic batch finder technique applied

**lr: float = 0.1**

Learning rate

**add\_noise\_rate: float = None**

The ratio of added random triples into training dataset

**gpus = None**

Number GPUs to be used during training

**callbacks**  
 10}}

**Type**  
 Callbacks, e.g., {"PPE"}

**Type**  
 {"last\_percent\_to\_consider"}

**backend: str = 'pandas'**  
 Backend to read, process, and index input knowledge graph. pandas, polars and rdflib available

**separator: str = '\\s+'**  
 separator for extracting head, relation and tail from a triple

**trainer: str = 'torchCPUTrainer'**  
 Trainer for knowledge graph embedding model

**scoring\_technique: str = 'KvsAll'**  
 Scoring technique for knowledge graph embedding models

**neg\_ratio: int = 0**  
 Negative ratio for a true triple in NegSample training\_technique

**weight\_decay: float = 0.0**  
 Weight decay for all trainable params

**normalization: str = 'None'**  
 LayerNorm, BatchNorm1d, or None

**init\_param: str = None**  
 xavier\_normal or None

**gradient\_accumulation\_steps: int = 0**  
 Not tested e

**num\_folds\_for\_cv: int = 0**  
 Number of folds for CV

**eval\_model: str = 'train\_val\_test'**  
 ["None", "train", "train\_val", "train\_val\_test", "test"]

**Type**  
 Evaluate trained model choices

**save\_model\_at\_every\_epoch: int = None**  
 Not tested

**label\_smoothing\_rate: float = 0.0**

**num\_core: int = 0**  
 Number of CPUs to be used in the mini-batch loading process

**random\_seed: int = 0**  
 Random Seed

**sample\_triples\_ratio: float = None**  
 Read some triples that are uniformly at random sampled. Ratio being between 0 and 1

**read\_only\_few: int = None**  
 Read only first few triples

**pykeen\_model\_kwargs**  
 Additional keyword arguments for pykeen models

**kernel\_size: int = 3**  
 Size of a square kernel in a convolution operation

**num\_of\_output\_channels: int = 32**  
 Number of slices in the generated feature map by convolution.

**p: int = 0**  
 P parameter of Clifford Embeddings

**q: int = 1**  
 Q parameter of Clifford Embeddings

**input\_dropout\_rate: float = 0.0**  
 Dropout rate on embeddings of input triples

**hidden\_dropout\_rate: float = 0.0**  
 Dropout rate on hidden representations of input triples

**feature\_map\_dropout\_rate: float = 0.0**  
 Dropout rate on a feature map generated by a convolution operation

**byte\_pair\_encoding: bool = False**  
 Byte pair encoding

**Type**  
 WIP

**adaptive\_swa: bool = False**  
 Adaptive stochastic weight averaging

**swa: bool = False**  
 Stochastic weight averaging

**swag: bool = False**  
 Stochastic weight averaging - Gaussian

**ema: bool = False**  
 Exponential Moving Average

**twa: bool = False**  
 Trainable weight averaging

**block\_size: int = None**  
 block size of LLM

**continual\_learning = None**  
 Path of a pretrained model size of LLM

**auto\_batch\_finding = False**  
 A flag for using auto batch finding

**eval\_every\_n\_epochs: int = 0**  
 Evaluate model every n epochs. If 0, no evaluation is applied.

**save\_every\_n\_epochs:** `bool = False`  
 Save model every n epochs. If True, save model at every epoch.

**eval\_at\_epochs:** `list = None`  
 List of epoch numbers at which to evaluate the model (e.g., 1 5 10).

**n\_epochs\_eval\_model:** `str = 'val_test'`  
 Evaluating link prediction performance on data splits while performing periodic evaluation.

**adaptive\_lr**  
 “cca”}

**Type**  
 Adaptive learning rate parameters, e.g., “{“scheduler\_name”

**swa\_start\_epoch:** `int = None`  
 Epoch at which to start applying stochastic weight averaging.

**swa\_c\_epochs:** `int = 1`  
 Number of epochs to average over for SWA, SWAG, EMA, TWA.

**\_\_iter\_\_()**

## dicee.dataset\_classes

Dataset classes for knowledge graph embedding training.

This package groups the various `torch.utils.data.Dataset` implementations used throughout DICE Embeddings into thematic sub-modules:

- `_bpe` – Byte-pair-encoding related datasets.
- `_negative_sampling` – Negative sampling based datasets.
- `_label_based` – Multi-label / multi-class scoring datasets.
- `_literal` – Literal (numeric) embedding dataset.
- `_factory` – `construct_dataset` / `reload_dataset` helpers.

All public names are re-exported here so that existing `from dicee.dataset_classes import ...` statements continue to work unchanged.

## Classes

|                                        |                                                                          |
|----------------------------------------|--------------------------------------------------------------------------|
| <i>BPE_NegativeSamplingDataset</i>     | Dataset for negative sampling with byte-pair encoded triples.            |
| <i>MultiClassClassificationDataset</i> | Dataset for autoregressive multi-class classification on sub-word units. |
| <i>MultiLabelDataset</i>               | Multi-label dataset for BPE-based KvsAll / AllvsAll training.            |
| <i>AllvsAll</i>                        | Dataset for AllvsAll training (multi-label, exhaustive).                 |
| <i>KvsAll</i>                          | Dataset for KvsAll training (multi-label).                               |
| <i>KvsSampleDataset</i>                | Dataset for KvsSample training (dynamic multi-label).                    |
| <i>OnevsAllDataset</i>                 | Dataset for the 1-vs-All training strategy (multi-class).                |
| <i>FixedNegSampleDataset</i>           | Pre-computed (fixed) negative sampling dataset.                          |

continues on next page

Table 7 – continued from previous page

|                                |                                                                               |
|--------------------------------|-------------------------------------------------------------------------------|
| <i>OnevsSample</i>             | Dataset for 1-vs-Sample training (dynamic multi-class with negatives).        |
| <i>TriplePredictionDataset</i> | Dataset for triple prediction with on-the-fly negative sampling.              |
| <i>LiteralDataset</i>          | Dataset for loading and processing literal data for Literal Embedding models. |

## Functions

|                                                                    |                                                                     |
|--------------------------------------------------------------------|---------------------------------------------------------------------|
| <i>construct_dataset</i> ( $\rightarrow$ torch.utils.data.Dataset) | Build the appropriate dataset for the given training configuration. |
| <i>reload_dataset</i> (path, form_of_labelling, ...)               | Reload training data from disk and construct a Pytorch dataset.     |

## Package Contents

**class** dicee.dataset\_classes.**BPE\_NegativeSamplingDataset** (*train\_set: torch.LongTensor*,  
*ordered\_shaped\_bpe\_entities: torch.LongTensor*, *neg\_ratio: int*)

Bases: torch.utils.data.Dataset

Dataset for negative sampling with byte-pair encoded triples.

Each sample is a BPE-encoded triple. The custom `collate_fn` constructs negatives by corrupting head or tail entities with random BPE entities.

### Parameters

- **train\_set** (*torch.LongTensor*) – Integer-encoded triples of shape  $(N, 3)$ .
- **ordered\_shaped\_bpe\_entities** (*torch.LongTensor*) – All BPE entity representations, ordered by entity index.
- **neg\_ratio** (*int*) – Number of negative samples per positive triple.

**train\_set**

**ordered\_bpe\_entities**

**num\_bpe\_entities**

**neg\_ratio**

**num\_datapoints**

**\_\_len\_\_()**

**\_\_getitem\_\_** (*idx*)

**collate\_fn** (*batch\_shaped\_bpe\_triples: List[Tuple[torch.Tensor, torch.Tensor]]*)

**class** dicee.dataset\_classes.**MultiClassClassificationDataset** (  
*subword\_units: numpy.ndarray*, *block\_size: int = 8*)

Bases: torch.utils.data.Dataset

Dataset for autoregressive multi-class classification on sub-word units.



Splits a flat sequence of sub-word token ids into overlapping windows of size `block_size` for next-token prediction.

#### Parameters

- **subword\_units** (*numpy.ndarray*) – 1-D array of sub-word token ids.
- **block\_size** (*int, optional*) – Context window length (default 8).

`train_data`

`block_size = 8`

`num_of_data_points`

`collate_fn = None`

`__len__()`

`__getitem__(idx)`

```
class dicee.dataset_classes.MultiLabelDataset (train_set: torch.LongTensor,  
        train_indices_target: torch.LongTensor, target_dim: int,  
        torch_ordered_shaped_bpe_entities: torch.LongTensor)
```

Bases: `torch.utils.data.Dataset`

Multi-label dataset for BPE-based KvsAll / AllvsAll training.

Each sample is a BPE-encoded (head, relation) pair together with a binary multi-label target vector over all entities.

#### Parameters

- **train\_set** (*torch.LongTensor*) – BPE-encoded input pairs of shape (N, 2, token\_length).
- **train\_indices\_target** (*torch.LongTensor*) – Per-sample lists of positive target entity indices.
- **target\_dim** (*int*) – Dimensionality of the target vector (number of entities).
- **torch\_ordered\_shaped\_bpe\_entities** (*torch.LongTensor*) – Ordered BPE entity representations.

`train_set`

`train_indices_target`

`target_dim`

`num_datapoints`

`torch_ordered_shaped_bpe_entities`

`collate_fn = None`

`__len__()`

`__getitem__(idx)`

```
class dicee.dataset_classes.AllvsAll (train_set_idx: numpy.ndarray, entity_idx, relation_idx,  
    label_smoothing_rate=0.0)
```

Bases: torch.utils.data.Dataset

Dataset for AllvsAll training (multi-label, exhaustive).

Extends the KvsAll idea: every *possible* (entity, relation) combination is included — not just those observed in the KG. Pairs without any known tail entities receive an all-zeros label vector.

#### Parameters

- **train\_set\_idx** (*numpy.ndarray*) – (N, 3) integer-indexed triples.
- **entity\_idx** (*dict*) – Entity-name → index mapping.
- **relation\_idx** (*dict*) – Relation-name → index mapping.
- **label\_smoothing\_rate** (*float, optional*) – Label smoothing coefficient (default 0.0).

**train\_data** = None

**train\_target** = None

**label\_smoothing\_rate**

**collate\_fn** = None

**target\_dim**

**\_\_len\_\_** ()

**\_\_getitem\_\_** (*idx*)

```
class dicee.dataset_classes.KvsAll (train_set_idx: numpy.ndarray, entity_idx, relation_idx, form,  
    store=None, label_smoothing_rate: float = 0.0)
```

Bases: torch.utils.data.Dataset

Dataset for KvsAll training (multi-label).

**D** := {(x, y)<sub>i</sub> }<sub>i=1</sub><sup>N</sup> where

- x = (h, r) is a unique (entity, relation) pair observed in the KG,
- y ∈ [0, 1]<sup>{**IEI**}</sup> is a multi-label vector with y<sub>j</sub> = 1 iff (h, r, e<sub>j</sub>) ∈ KG.

#### Parameters

- **train\_set\_idx** (*numpy.ndarray*) – (N, 3) integer-indexed triples.
- **entity\_idx** (*dict*) – Entity-name → index mapping.
- **relation\_idx** (*dict*) – Relation-name → index mapping.
- **form** (*str*) – 'EntityPrediction' or 'RelationPrediction'.
- **label\_smoothing\_rate** (*float, optional*) – Label smoothing coefficient (default 0.0).

**train\_data** = None

**train\_target** = None

**label\_smoothing\_rate**

**collate\_fn** = None

**\_\_len\_\_**()

**\_\_getitem\_\_**(idx)

**class** dicee.dataset\_classes.**KvsSampleDataset** (train\_set\_idx: numpy.ndarray, entity\_idxes, relation\_idxes, form, store=None, neg\_ratio=None, label\_smoothing\_rate: float = 0.0)

Bases: torch.utils.data.Dataset

Dataset for KvsSample training (dynamic multi-label).

Like KvsAll but sub-samples the target vector at each access to keep mini-batch sizes manageable when the entity set is large.

#### Parameters

- **train\_set\_idx** (numpy.ndarray) – (N, 3) integer-indexed triples.
- **entity\_idxes** (dict) – Entity-name → index mapping.
- **relation\_idxes** (dict) – Relation-name → index mapping.
- **form** (str) – 'EntityPrediction'.
- **neg\_ratio** (int) – Number of negative samples per positive target.
- **label\_smoothing\_rate** (float, optional) – Label smoothing coefficient (default 0.0).

**train\_data** = None

**train\_target** = None

**neg\_ratio** = None

**num\_entities**

**label\_smoothing\_rate**

**collate\_fn** = None

**max\_num\_of\_classes**

**\_\_len\_\_**()

**\_\_getitem\_\_**(idx)

**class** dicee.dataset\_classes.**OnevsAllDataset** (train\_set\_idx: numpy.ndarray, entity\_idxes)

Bases: torch.utils.data.Dataset

Dataset for the 1-vs-All training strategy (multi-class).

Each sample is a (head, relation) pair with a one-hot target vector whose single active position corresponds to the true tail entity.

#### Parameters

- **train\_set\_idx** (numpy.ndarray) – (N, 3) integer-indexed triples.
- **entity\_idxes** (dict) – Entity-name → index mapping (used to determine the target dimension).

```

train_data

target_dim

collate_fn = None

__len__()

__getitem__(idx)

```

```

class dicee.dataset_classes.FixedNegSampleDataset (train_set: numpy.ndarray, num_entities: int,
                                                    num_relations: int, neg_sample_ratio: int = 1, label_smoothing_rate: float = 0.0, seed: int = None)

```

Bases: torch.utils.data.Dataset

Pre-computed (fixed) negative sampling dataset.

At construction time every positive triple is paired with one random negative (head- or tail-corrupted) using vectorized operations for efficiency. The pairs are stored so that `__getitem__` is a simple lookup.

This is useful when you want deterministic negatives across epochs (e.g., for reproducibility or debugging).

#### Parameters

- **train\_set** (*numpy.ndarray*) – (N, 3) integer-indexed triples.
- **num\_entities** (*int*) – Total number of entities.
- **num\_relations** (*int*) – Total number of relations.
- **neg\_sample\_ratio** (*int, optional*) – Number of negative samples per positive triple (default 1).
- **label\_smoothing\_rate** (*float, optional*) – Label smoothing coefficient (default 0.0).

```

neg_sample_ratio = 1

num_entities

num_relations

label_smoothing_rate = 0.0

collate_fn = None

seed = None

train_triples

length

__len__() → int

__getitem__(idx: int) → Tuple[torch.Tensor, torch.Tensor]

```

```

class dicee.dataset_classes.OnevsSample (train_set: numpy.ndarray, num_entities: int,
                                          num_relations: int, neg_sample_ratio: int = None, label_smoothing_rate: float = 0.0)

```

Bases: torch.utils.data.Dataset

Dataset for 1-vs-Sample training (dynamic multi-class with negatives).

For every positive triple (h, r, t) the dataset draws `neg_sample_ratio` random entities as negatives and returns a label vector that marks the true tail and the negatives.

### Parameters

- **train\_set** (*numpy.ndarray*) – (N, 3) integer-indexed triples.
- **num\_entities** (*int*) – Total number of entities.
- **num\_relations** (*int*) – Total number of relations.
- **neg\_sample\_ratio** (*int*) – Number of negative samples per positive.
- **label\_smoothing\_rate** (*float, optional*) – Label smoothing coefficient (default 0.0).

**train\_data**

**num\_entities**

**num\_relations**

**neg\_sample\_ratio** = None

**label\_smoothing\_rate**

**collate\_fn** = None

**\_\_len\_\_**()

**\_\_getitem\_\_**(*idx*)

```
class dicee.dataset_classes.TriplePredictionDataset (train_set: numpy.ndarray,  
    num_entities: int, num_relations: int, neg_sample_ratio: int = 1, label_smoothing_rate: float = 0.0,  
    seed: int = None)
```

Bases: `torch.utils.data.Dataset`

Dataset for triple prediction with on-the-fly negative sampling.

Each item is a single positive triple; the custom `collate_fn` generates a batch of mixed positive and negative triples.

### Parameters

- **train\_set** (*numpy.ndarray*) – (N, 3) integer-indexed triples.
- **num\_entities** (*int*) – Total number of entities.
- **num\_relations** (*int*) – Total number of relations.
- **neg\_sample\_ratio** (*int, optional*) – Number of negative samples per positive triple (default 1).
- **label\_smoothing\_rate** (*float, optional*) – Label smoothing coefficient (default 0.0).

**label\_smoothing\_rate**

**neg\_sample\_ratio**

**seed** = None

**train\_set**

**length**

**num\_entities**

```

num_relations

__len__()

__getitem__(idx)

collate_fn(batch: List[torch.Tensor])

class dicee.dataset_classes.LiteralDataset (file_path: str, ent_idx: dict = None,
      normalization_type: str = 'z-norm', sampling_ratio: float = None, loader_backend: str = 'pandas')
Bases: torch.utils.data.Dataset

Dataset for loading and processing literal data for Literal Embedding models.

Handles loading, normalization, and preparation of (entity, attribute, value) triples. Supports z-score
and min-max normalization as well as optional sub-sampling for ablation studies.

```

#### Parameters

- **file\_path** (*str*) – Path to the training data file (CSV / TSV / RDF).
- **ent\_idx** (*dict*) – Entity-name → index mapping.
- **normalization\_type** (*str, optional*) – 'z-norm', 'min-max', or None (default 'z-norm').
- **sampling\_ratio** (*float or None, optional*) – Fraction of the training set to keep (default None = use all).
- **loader\_backend** (*str, optional*) – 'pandas' or 'rdflib' (default 'pandas').

```

train_file_path

loader_backend = 'pandas'

normalization_type = 'z-norm'

normalization_params

sampling_ratio = None

entity_to_idx = None

num_entities

__getitem__(index)

__len__()

static load_and_validate_literal_data (file_path: str = None, loader_backend: str = 'pandas')
      → pandas.DataFrame

Load and validate a literal data file.

```

#### Parameters

- **file\_path** (*str*) – Path to the data file.
- **loader\_backend** (*str*) – 'pandas' or 'rdflib'.

#### Returns

Three-column DataFrame with columns head, attribute, value.

#### Return type

pandas.DataFrame

**static denormalize** (*preds\_norm*, *attributes*, *normalization\_params*) → *numpy.ndarray*

Reverse the normalization applied during training.

#### Parameters

- **preds\_norm** (*numpy.ndarray*) – Normalized predictions.
- **attributes** (*list*) – Attribute names corresponding to each prediction.
- **normalization\_params** (*dict*) – Parameters stored during training.

#### Returns

Denormalized predictions.

#### Return type

*numpy.ndarray*

`dicee.dataset_classes.construct_dataset` (\* (Keyword-only parameters separator (PEP 3102)),  
*train\_set*: *numpy.ndarray* | *list*, *valid\_set*=None, *test\_set*=None, *ordered\_bpe\_entities*=None,  
*train\_target\_indices*=None, *target\_dim*: *int* = None, *entity\_to\_idx*: *dict*, *relation\_to\_idx*: *dict*,  
*form\_of\_labelling*: *str*, *scoring\_technique*: *str*, *neg\_ratio*: *int*, *label\_smoothing\_rate*: *float*,  
*byte\_pair\_encoding*=None, *block\_size*: *int* = None, *seed*: *int* = None) → *torch.utils.data.Dataset*

Build the appropriate dataset for the given training configuration.

#### Parameters

- **train\_set** (*numpy.ndarray* or *list*) – Raw integer-indexed triples.
- **entity\_to\_idx** (*dict*) – Name → index mappings.
- **relation\_to\_idx** (*dict*) – Name → index mappings.
- **form\_of\_labelling** (*str*) – 'EntityPrediction' or 'RelationPrediction'.
- **scoring\_technique** (*str*) – One of 'NegSample', 'FixedNegSample', '1vsAll', '1vsSample', 'KvsAll', 'AllvsAll', 'KvsSample'.
- **neg\_ratio** (*int*) – Negative sample ratio.
- **label\_smoothing\_rate** (*float*) – Label smoothing coefficient.

#### Return type

*torch.utils.data.Dataset*

`dicee.dataset_classes.reload_dataset` (*path*: *str*, *form\_of\_labelling*, *scoring\_technique*, *neg\_ratio*,  
*label\_smoothing\_rate*)

Reload training data from disk and construct a Pytorch dataset.

## **dicee.eval\_static\_funcs**

Static evaluation functions for KGE models.

This module provides backward compatibility by re-exporting from the new `dicee.evaluation` module.

Deprecated since version Use: `dicee.evaluation` submodules instead. This module will be removed in a future version.

## **Functions**

---

`evaluate_link_prediction_performance`(→  
*Dict*[*str*, *float*])

Evaluate link prediction performance with head and tail prediction.

continues on next page

Table 9 – continued from previous page

|                                                                        |                                                                    |
|------------------------------------------------------------------------|--------------------------------------------------------------------|
| <code>evaluate_link_prediction_performance_with_</code>                | Evaluate link prediction with reciprocal relations.                |
| <code>evaluate_link_prediction_performance_with_</code>                | Evaluate link prediction with BPE encoding (head and tail).        |
| <code>evaluate_link_prediction_performance_with_</code>                | Evaluate link prediction with BPE encoding and reciprocals.        |
| <code>evaluate_lp_bpe_k_vs_all(→ Dict[str, float])</code>              | Evaluate BPE link prediction with KvsAll scoring.                  |
| <code>evaluate_literal_prediction(→ Optional[pandas.DataFrame])</code> | Evaluate trained literal prediction model on a test file.          |
| <code>evaluate_ensemble_link_prediction_performance</code>             | Evaluate link prediction performance of an ensemble of KGE models. |

## Module Contents

`dicee.eval_static_funcs.evaluate_link_prediction_performance(model, triples, er_vocab: Dict[Tuple, List], re_vocab: Dict[Tuple, List]) → Dict[str, float]`

Evaluate link prediction performance with head and tail prediction.

Performs filtered evaluation where known correct answers are filtered out before computing ranks.

### Parameters

- **model** – KGE model wrapper with entity/relation mappings.
- **triples** – Test triples as list of (head, relation, tail) strings.
- **er\_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re\_vocab** – Mapping (relation, entity) -> list of valid head entities.

### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

`dicee.eval_static_funcs.evaluate_link_prediction_performance_with_reciprocals(model, triples, er_vocab: Dict[Tuple, List]) → Dict[str, float]`

Evaluate link prediction with reciprocal relations.

Optimized for models trained with reciprocal triples where only tail prediction is needed.

### Parameters

- **model** – KGE model wrapper.
- **triples** – Test triples as list of (head, relation, tail) strings.
- **er\_vocab** – Mapping (entity, relation) -> list of valid tail entities.

### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

`dicee.eval_static_funcs.evaluate_link_prediction_performance_with_bpe(model, within_entities: List[str], triples: List[Tuple[str]], er_vocab: Dict[Tuple, List], re_vocab: Dict[Tuple, List]) → Dict[str, float]`

Evaluate link prediction with BPE encoding (head and tail).

### Parameters

- **model** – KGE model wrapper with BPE support.
- **within\_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of (head, relation, tail) tuples.



- **er\_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re\_vocab** – Mapping (relation, entity) -> list of valid head entities.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.eval_static_funcs.evaluate_link_prediction_performance_with_bpe_reciprocals(
    model, within_entities: List[str], triples: List[List[str]], er_vocab: Dict[Tuple, List])
    → Dict[str, float]
```

Evaluate link prediction with BPE encoding and reciprocals.

#### Parameters

- **model** – KGE model wrapper with BPE support.
- **within\_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of [head, relation, tail] strings.
- **er\_vocab** – Mapping (entity, relation) -> list of valid tail entities.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.eval_static_funcs.evaluate_lp_bpe_k_vs_all(model, triples: List[List[str]],
    er_vocab: Dict | None = None, batch_size: int | None = None,
    func_triple_to_bpe_representation: Callable | None = None,
    str_to_bpe_entity_to_idx: Dict | None = None) → Dict[str, float]
```

Evaluate BPE link prediction with KvsAll scoring.

#### Parameters

- **model** – The KGE model wrapper.
- **triples** – List of string triples.
- **er\_vocab** – Entity-relation vocabulary for filtering.
- **batch\_size** – Batch size for processing.
- **func\_triple\_to\_bpe\_representation** – Function to convert triples to BPE.
- **str\_to\_bpe\_entity\_to\_idx** – Mapping from string entities to BPE indices.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

#### Raises

**ValueError** – If batch\_size is not provided.

```
dicee.eval_static_funcs.evaluate_literal_prediction(kge_model, eval_file_path: str = None,
    store_lit_preds: bool = True, eval_literals: bool = True, loader_backend: str = 'pandas',
    return_attr_error_metrics: bool = False) → pandas.DataFrame | None
```

Evaluate trained literal prediction model on a test file.

Evaluates the literal prediction capabilities of a KGE model by computing MAE and RMSE metrics for each attribute.

#### Parameters

- **kge\_model** – Trained KGE model with literal prediction capability.
- **eval\_file\_path** – Path to the evaluation file containing test literals.
- **store\_lit\_preds** – If True, stores predictions to CSV file.

- **eval\_literals** – If True, evaluates and prints error metrics.
- **loader\_backend** – Backend for loading dataset ('pandas' or 'rdflib').
- **return\_attr\_error\_metrics** – If True, returns the metrics DataFrame.

#### Returns

DataFrame with per-attribute MAE and RMSE if `return_attr_error_metrics` is True, otherwise None.

#### Raises

- **RuntimeError** – If the KGE model doesn't have a trained literal model.
- **AssertionError** – If model is invalid or test set has no valid data.

### Example

```
>>> from dicee import KGE
>>> from dicee.evaluation import evaluate_literal_prediction
>>> model = KGE(path="pretrained_model")
>>> metrics = evaluate_literal_prediction(
...     model,
...     eval_file_path="test_literals.csv",
...     return_attr_error_metrics=True
... )
>>> print(metrics)
```

`dicee.eval_static_funcs.evaluate_ensemble_link_prediction_performance` (*models: List, triples, er\_vocab: Dict[Tuple, List], weights: List[float] | None = None, batch\_size: int = 512, weighted\_averaging: bool = True, normalize\_scores: bool = True*) → Dict[str, float]

Evaluate link prediction performance of an ensemble of KGE models.

Combines predictions from multiple models using weighted or simple averaging, with optional score normalization.

#### Parameters

- **models** – List of KGE models (e.g., snapshots from training).
- **triples** – Test triples as numpy array or list, shape (N, 3), with integer indices (head, relation, tail).
- **er\_vocab** – Mapping (head\_idx, rel\_idx) -> list of tail indices for filtered evaluation.
- **weights** – Weights for model averaging. Required if `weighted_averaging` is True. Must sum to 1 for proper averaging.
- **batch\_size** – Batch size for processing triples.
- **weighted\_averaging** – If True, use weighted averaging of predictions. If False, use simple mean.
- **normalize\_scores** – If True, normalize scores to [0, 1] range per sample before averaging.

#### Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

#### Raises

**AssertionError** – If `weighted_averaging` is True but weights are not provided or have wrong length.

## Example

```
>>> from dicee.evaluation import evaluate_ensemble_link_prediction_performance
>>> models = [model1, model2, model3]
>>> weights = [0.5, 0.3, 0.2]
>>> results = evaluate_ensemble_link_prediction_performance(
...     models, test_triples, er_vocab,
...     weights=weights, weighted_averaging=True
... )
>>> print(f"MRR: {results['MRR']:.4f}")
```

## dicee.evaluation

Evaluation module for knowledge graph embedding models.

This module provides comprehensive evaluation capabilities for KGE models, including link prediction, literal prediction, and ensemble evaluation.

### Modules:

link\_prediction: Functions for evaluating link prediction performance  
literal\_prediction: Functions for evaluating literal/attribute prediction  
ensemble: Functions for ensemble model evaluation  
evaluator: Main Evaluator class for integrated evaluation  
utils: Shared utility functions for evaluation

## Example

```
>>> from dicee.evaluation import Evaluator
>>> from dicee.evaluation.link_prediction import evaluate_link_prediction_performance
>>> from dicee.evaluation.ensemble import evaluate_ensemble_link_prediction_
↪performance
```

## Submodules

### dicee.evaluation.ensemble

Ensemble evaluation functions.

This module provides functions for evaluating ensemble models, including weighted averaging and score normalization.

### Functions

---

|                                                            |                                                                    |
|------------------------------------------------------------|--------------------------------------------------------------------|
| <code>evaluate_ensemble_link_prediction_performance</code> | Evaluate link prediction performance of an ensemble of KGE models. |
|------------------------------------------------------------|--------------------------------------------------------------------|

---

## Module Contents

`dicee.evaluation.ensemble.evaluate_ensemble_link_prediction_performance` (*models*: List, *triples*, *er\_vocab*: Dict[Tuple, List], *weights*: List[float] | None = None, *batch\_size*: int = 512, *weighted\_averaging*: bool = True, *normalize\_scores*: bool = True) → Dict[str, float]

Evaluate link prediction performance of an ensemble of KGE models.

Combines predictions from multiple models using weighted or simple averaging, with optional score normalization.

### Parameters

- **models** – List of KGE models (e.g., snapshots from training).
- **triples** – Test triples as numpy array or list, shape (N, 3), with integer indices (head, relation, tail).
- **er\_vocab** – Mapping (head\_idx, rel\_idx) -> list of tail indices for filtered evaluation.
- **weights** – Weights for model averaging. Required if `weighted_averaging` is `True`. Must sum to 1 for proper averaging.
- **batch\_size** – Batch size for processing triples.
- **weighted\_averaging** – If `True`, use weighted averaging of predictions. If `False`, use simple mean.
- **normalize\_scores** – If `True`, normalize scores to [0, 1] range per sample before averaging.

#### Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

#### Raises

**AssertionError** – If `weighted_averaging` is `True` but weights are not provided or have wrong length.

#### Example

```
>>> from dicee.evaluation import evaluate_ensemble_link_prediction_performance
>>> models = [model1, model2, model3]
>>> weights = [0.5, 0.3, 0.2]
>>> results = evaluate_ensemble_link_prediction_performance(
...     models, test_triples, er_vocab,
...     weights=weights, weighted_averaging=True
... )
>>> print(f"MRR: {results['MRR']:.4f}")
```

### **`dicee.evaluation.evaluator`**

Main Evaluator class for KGE model evaluation.

This module provides the Evaluator class which orchestrates evaluation of knowledge graph embedding models across different datasets and scoring techniques.

#### Attributes

---

*VALID\_SCORING\_TECHNIQUES*

---

#### Classes

---

*Evaluator*

---

Evaluator class for KGE models in various downstream tasks.

---

## Module Contents

`dicee.evaluation.evaluator.VALID_SCORING_TECHNIQUES`

**class** `dicee.evaluation.evaluator.Evaluator` (*args*, *is\_continual\_training*: *bool* = *False*)

Evaluator class for KGE models in various downstream tasks.

Orchestrates link prediction evaluation with different scoring techniques including standard evaluation and byte-pair encoding based evaluation.

**er\_vocab**

Entity-relation to tail vocabulary for filtered ranking.

**re\_vocab**

Relation-entity (tail) to head vocabulary.

**ee\_vocab**

Entity-entity to relation vocabulary.

**num\_entities**

Total number of entities in the knowledge graph.

**num\_relations**

Total number of relations in the knowledge graph.

**args**

Configuration arguments.

**report**

Dictionary storing evaluation results.

**during\_training**

Whether evaluation is happening during training.

## Example

```
>>> from dicee.evaluation import Evaluator
>>> evaluator = Evaluator(args)
>>> results = evaluator.eval(dataset, model, 'EntityPrediction')
>>> print(f"Test MRR: {results['Test']['MRR']:.4f}")
```

**re\_vocab**: Dict | None = None

**er\_vocab**: Dict | None = None

**ee\_vocab**: Dict | None = None

**func\_triple\_to\_bpe\_representation** = None

**is\_continual\_training** = False

**num\_entities**: int | None = None

**num\_relations**: int | None = None

**domain\_constraints\_per\_rel** = None

**range\_constraints\_per\_rel** = None

**args**

**report**: Dict

**during\_training** = False

**vocab\_preparation** (*dataset*) → None

Prepare vocabularies from the dataset for evaluation.

Resolves any future objects and saves vocabularies to disk.

#### Parameters

**dataset** – Knowledge graph dataset with vocabulary attributes.

**eval** (*dataset*, *trained\_model*, *form\_of\_labelling*: str, *during\_training*: bool = False) → Dict | None

Evaluate the trained model on the dataset.

#### Parameters

- **dataset** – Knowledge graph dataset (KG instance).
- **trained\_model** – The trained KGE model.
- **form\_of\_labelling** – Type of labelling ('EntityPrediction' or 'RelationPrediction').
- **during\_training** – Whether evaluation is during training.

#### Returns

Dictionary of evaluation metrics, or None if evaluation is skipped.

**eval\_rank\_of\_head\_and\_tail\_entity** (\*, *train\_set*, *valid\_set*=None, *test\_set*=None, *trained\_model*)  
→ None

Evaluate with negative sampling scoring.

**eval\_rank\_of\_head\_and\_tail\_byte\_pair\_encoded\_entity** (\*, *train\_set*=None, *valid\_set*=None, *test\_set*=None, *ordered\_bpe\_entities*, *trained\_model*) → None

Evaluate with BPE-encoded entities and negative sampling.

**eval\_with\_byte** (\*, *raw\_train\_set*, *raw\_valid\_set*=None, *raw\_test\_set*=None, *trained\_model*, *form\_of\_labelling*) → None

Evaluate Byte model with generation.

**eval\_with\_bpe\_vs\_all** (\*, *raw\_train\_set*, *raw\_valid\_set*=None, *raw\_test\_set*=None, *trained\_model*, *form\_of\_labelling*) → None

Evaluate with BPE and KvsAll scoring.

**eval\_with\_vs\_all** (\*, *train\_set*, *valid\_set*=None, *test\_set*=None, *trained\_model*, *form\_of\_labelling*)  
→ None

Evaluate with KvsAll or 1vsAll scoring.

**evaluate\_lp\_k\_vs\_all** (*model*, *triple\_idx*, *info*: str | None = None, *form\_of\_labelling*: str | None = None) → Dict[str, float]

Filtered link prediction evaluation with KvsAll scoring.

#### Parameters

- **model** – The trained model to evaluate.
- **triple\_idx** – Integer-indexed test triples.
- **info** – Description to print.
- **form\_of\_labelling** – 'EntityPrediction' or 'RelationPrediction'.

### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

**evaluate\_lp\_with\_byte** (*model*, *triples*: List[List[str]], *info*: str | None = None) → Dict[str, float]

Evaluate Byte model with text generation.

### Parameters

- **model** – Byte model.
- **triples** – String triples.
- **info** – Description to print.

### Returns

Dictionary with placeholder metrics (-1 values).

**evaluate\_lp\_bpe\_k\_vs\_all** (*model*, *triples*: List[List[str]], *info*: str | None = None, *form\_of\_labelling*: str | None = None) → Dict[str, float]

Evaluate BPE model with KvsAll scoring.

### Parameters

- **model** – BPE-enabled model.
- **triples** – String triples.
- **info** – Description to print.
- **form\_of\_labelling** – Type of labelling.

### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

**evaluate\_lp** (*model*, *triple\_idx*, *info*: str) → Dict[str, float]

Evaluate link prediction with negative sampling.

### Parameters

- **model** – The model to evaluate.
- **triple\_idx** – Integer-indexed triples.
- **info** – Description to print.

### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

**dummy\_eval** (*trained\_model*, *form\_of\_labelling*: str) → None

Run evaluation from saved data (for continual training).

### Parameters

- **trained\_model** – The trained model.
- **form\_of\_labelling** – Type of labelling.

**eval\_with\_data** (*dataset*, *trained\_model*, *triple\_idx*: numpy.ndarray, *form\_of\_labelling*: str) → Dict[str, float]

Evaluate a trained model on a given dataset.

### Parameters

- **dataset** – Knowledge graph dataset.
- **trained\_model** – The trained model.

- **triple\_idx** – Integer-indexed triples to evaluate.
- **form\_of\_labelling** – Type of labelling.

#### Returns

Dictionary with evaluation metrics.

#### Raises

**ValueError** – If scoring technique is invalid.

## dicee.evaluation.link\_prediction

Link prediction evaluation functions.

This module provides various functions for evaluating link prediction performance of knowledge graph embedding models.

### Functions

|                                                                                                          |                                                                     |
|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <code>evaluate_link_prediction_performance(→ Dict[str, float])</code>                                    | Evaluate link prediction performance with head and tail prediction. |
| <code>evaluate_link_prediction_performance_with_reciprocals(→ Dict[str, float])</code>                   | Evaluate link prediction with reciprocal relations.                 |
| <code>evaluate_link_prediction_performance_with_reciprocals_and_bpe(→ Dict[str, float])</code>           | Evaluate link prediction with BPE encoding and reciprocals.         |
| <code>evaluate_link_prediction_performance_with_reciprocals_and_bpe_head_tail(→ Dict[str, float])</code> | Evaluate link prediction with BPE encoding (head and tail).         |
| <code>evaluate_lp(→ Dict[str, float])</code>                                                             | Evaluate link prediction with batched processing.                   |
| <code>evaluate_bpe_lp(→ Dict[str, float])</code>                                                         | Evaluate link prediction with BPE-encoded entities.                 |
| <code>evaluate_lp_bpe_k_vs_all(→ Dict[str, float])</code>                                                | Evaluate BPE link prediction with KvsAll scoring.                   |

### Module Contents

`dicee.evaluation.link_prediction.evaluate_link_prediction_performance(model, triples, er_vocab: Dict[Tuple, List], re_vocab: Dict[Tuple, List]) → Dict[str, float]`

Evaluate link prediction performance with head and tail prediction.

Performs filtered evaluation where known correct answers are filtered out before computing ranks.

#### Parameters

- **model** – KGE model wrapper with entity/relation mappings.
- **triples** – Test triples as list of (head, relation, tail) strings.
- **er\_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re\_vocab** – Mapping (relation, entity) -> list of valid head entities.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

`dicee.evaluation.link_prediction.evaluate_link_prediction_performance_with_reciprocals(model, triples, er_vocab: Dict[Tuple, List]) → Dict[str, float]`

Evaluate link prediction with reciprocal relations.

Optimized for models trained with reciprocal triples where only tail prediction is needed.

#### Parameters



- **model** – KGE model wrapper.
- **triples** – Test triples as list of (head, relation, tail) strings.
- **er\_vocab** – Mapping (entity, relation) -> list of valid tail entities.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

`dicee.evaluation.link_prediction.`

**evaluate\_link\_prediction\_performance\_with\_bpe\_reciprocals** (*model*,  
*within\_entities*: List[str], *triples*: List[List[str]], *er\_vocab*: Dict[Tuple, List]) → Dict[str, float]

Evaluate link prediction with BPE encoding and reciprocals.

#### Parameters

- **model** – KGE model wrapper with BPE support.
- **within\_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of [head, relation, tail] strings.
- **er\_vocab** – Mapping (entity, relation) -> list of valid tail entities.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

`dicee.evaluation.link_prediction.`**evaluate\_link\_prediction\_performance\_with\_bpe** (*model*, *within\_entities*: List[str], *triples*: List[Tuple[str]], *er\_vocab*: Dict[Tuple, List],  
*re\_vocab*: Dict[Tuple, List]) → Dict[str, float]

Evaluate link prediction with BPE encoding (head and tail).

#### Parameters

- **model** – KGE model wrapper with BPE support.
- **within\_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of (head, relation, tail) tuples.
- **er\_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re\_vocab** – Mapping (relation, entity) -> list of valid head entities.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

`dicee.evaluation.link_prediction.`**evaluate\_lp** (*model*, *triple\_idx*, *num\_entities*: int,  
*er\_vocab*: Dict[Tuple, List], *re\_vocab*: Dict[Tuple, List], *info*: str = 'Eval Starts',  
*batch\_size*: int = 128, *chunk\_size*: int = 1000) → Dict[str, float]

Evaluate link prediction with batched processing.

Memory-efficient evaluation using chunked entity scoring.

#### Parameters

- **model** – The KGE model to evaluate.
- **triple\_idx** – Integer-indexed triples as numpy array.
- **num\_entities** – Total number of entities.
- **er\_vocab** – Mapping (head\_idx, rel\_idx) -> list of tail indices.
- **re\_vocab** – Mapping (rel\_idx, tail\_idx) -> list of head indices.
- **info** – Description to print.

- **batch\_size** – Batch size for triple processing.
- **chunk\_size** – Chunk size for entity scoring.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.link_prediction.evaluate_bpe_lp(model, triple_idx: List[Tuple],
all_bpe_shaped_entities, er_vocab: Dict[Tuple, List], re_vocab: Dict[Tuple, List],
info: str = 'Eval Starts') → Dict[str, float]
```

Evaluate link prediction with BPE-encoded entities.

#### Parameters

- **model** – The KGE model to evaluate.
- **triple\_idx** – List of BPE-encoded triple tuples.
- **all\_bpe\_shaped\_entities** – All entities with BPE representations.
- **er\_vocab** – Mapping for tail filtering.
- **re\_vocab** – Mapping for head filtering.
- **info** – Description to print.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.link_prediction.evaluate_lp_bpe_k_vs_all(model, triples: List[List[str]],
er_vocab: Dict | None = None, batch_size: int | None = None,
func_triple_to_bpe_representation: Callable | None = None,
str_to_bpe_entity_to_idx: Dict | None = None) → Dict[str, float]
```

Evaluate BPE link prediction with KvsAll scoring.

#### Parameters

- **model** – The KGE model wrapper.
- **triples** – List of string triples.
- **er\_vocab** – Entity-relation vocabulary for filtering.
- **batch\_size** – Batch size for processing.
- **func\_triple\_to\_bpe\_representation** – Function to convert triples to BPE.
- **str\_to\_bpe\_entity\_to\_idx** – Mapping from string entities to BPE indices.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

#### Raises

**ValueError** – If batch\_size is not provided.

## **dicee.evaluation.literal\_prediction**

Literal prediction evaluation functions.

This module provides functions for evaluating literal/attribute prediction performance of knowledge graph embedding models.

## Functions

|                                                                         |                                                           |
|-------------------------------------------------------------------------|-----------------------------------------------------------|
| <code>evaluate_literal_prediction</code> (→ Optional[pandas.DataFrame]) | Evaluate trained literal prediction model on a test file. |
|-------------------------------------------------------------------------|-----------------------------------------------------------|

## Module Contents

```
dicee.evaluation.literal_prediction.evaluate_literal_prediction(kge_model,  
    eval_file_path: str = None, store_lit_preds: bool = True, eval_literals: bool = True,  
    loader_backend: str = 'pandas', return_attr_error_metrics: bool = False)  
    → pandas.DataFrame | None
```

Evaluate trained literal prediction model on a test file.

Evaluates the literal prediction capabilities of a KGE model by computing MAE and RMSE metrics for each attribute.

### Parameters

- **kge\_model** – Trained KGE model with literal prediction capability.
- **eval\_file\_path** – Path to the evaluation file containing test literals.
- **store\_lit\_preds** – If True, stores predictions to CSV file.
- **eval\_literals** – If True, evaluates and prints error metrics.
- **loader\_backend** – Backend for loading dataset ('pandas' or 'rdflib').
- **return\_attr\_error\_metrics** – If True, returns the metrics DataFrame.

### Returns

DataFrame with per-attribute MAE and RMSE if `return_attr_error_metrics` is True, otherwise None.

### Raises

- **RuntimeError** – If the KGE model doesn't have a trained literal model.
- **AssertionError** – If model is invalid or test set has no valid data.

## Example

```
>>> from dicee import KGE  
>>> from dicee.evaluation import evaluate_literal_prediction  
>>> model = KGE(path="pretrained_model")  
>>> metrics = evaluate_literal_prediction(  
...     model,  
...     eval_file_path="test_literals.csv",  
...     return_attr_error_metrics=True  
... )  
>>> print(metrics)
```

## `dicee.evaluation.utils`

Utility functions for evaluation module.

This module contains shared helper functions used across different evaluation components.

## Attributes

|                                 |
|---------------------------------|
| <code>DEFAULT_HITS_RANGE</code> |
| <code>ALL_HITS_RANGE</code>     |

## Functions

|                                                                    |                                                                       |
|--------------------------------------------------------------------|-----------------------------------------------------------------------|
| <code>make_iterable_verbose(→ Iterable)</code>                     | Wrap an iterable with tqdm progress bar if verbose is True.           |
| <code>compute_metrics_from_ranks(→ Dict[str, float])</code>        | Compute standard link prediction metrics from ranks.                  |
| <code>compute_metrics_from_ranks_simple(→ Dict[str, float])</code> | Compute link prediction metrics without scaling factor.               |
| <code>update_hits(→ None)</code>                                   | Update hits dictionary based on rank.                                 |
| <code>create_hits_dict(→ Dict[int, List[float]])</code>            | Create an initialized hits dictionary.                                |
| <code>efficient_zero_grad(→ None)</code>                           | Efficiently zero gradients using <code>parameter.grad = None</code> . |

## Module Contents

`dicee.evaluation.utils.DEFAULT_HITS_RANGE: List[int] = [1, 3, 10]`

`dicee.evaluation.utils.ALL_HITS_RANGE: List[int]`

`dicee.evaluation.utils.make_iterable_verbose(iterable_object: Iterable, verbose: bool, desc: str = 'Default', position: int | None = None, leave: bool = True) → Iterable`

Wrap an iterable with tqdm progress bar if verbose is True.

### Parameters

- **iterable\_object** – The iterable to potentially wrap.
- **verbose** – Whether to show progress bar.
- **desc** – Description for the progress bar.
- **position** – Position of the progress bar.
- **leave** – Whether to leave the progress bar after completion.

### Returns

The original iterable or a tqdm-wrapped version.

`dicee.evaluation.utils.compute_metrics_from_ranks(ranks: List[int], num_triples: int, hits_dict: Dict[int, List[float]], scale_factor: int = 1) → Dict[str, float]`

Compute standard link prediction metrics from ranks.

### Parameters

- **ranks** – List of ranks for each prediction.
- **num\_triples** – Total number of triples evaluated.
- **hits\_dict** – Dictionary mapping hit levels to lists of hits.
- **scale\_factor** – Factor to scale the denominator (e.g., 2 for head+tail).

### Returns

Dictionary containing H@1, H@3, H@10, and MRR metrics.

`dicee.evaluation.utils.compute_metrics_from_ranks_simple` (*ranks*: List[int], *num\_triples*: int, *hits\_dict*: Dict[int, List[float]]) → Dict[str, float]

Compute link prediction metrics without scaling factor.

#### Parameters

- **ranks** – List of ranks for each prediction.
- **num\_triples** – Total number of triples evaluated.
- **hits\_dict** – Dictionary mapping hit levels to lists of hits.

#### Returns

Dictionary containing H@1, H@3, H@10, and MRR metrics.

`dicee.evaluation.utils.update_hits` (*hits*: Dict[int, List[float]], *rank*: int, *hits\_range*: List[int] | None = None) → None

Update hits dictionary based on rank.

#### Parameters

- **hits** – Dictionary to update in-place.
- **rank** – The rank to check against hit levels.
- **hits\_range** – List of hit levels to check (default: ALL\_HITS\_RANGE).

`dicee.evaluation.utils.create_hits_dict` (*hits\_range*: List[int] | None = None) → Dict[int, List[float]]

Create an initialized hits dictionary.

#### Parameters

**hits\_range** – List of hit levels to initialize (default: ALL\_HITS\_RANGE).

#### Returns

Dictionary with empty lists for each hit level.

`dicee.evaluation.utils.efficient_zero_grad` (*model*) → None

Efficiently zero gradients using `parameter.grad = None`.

This is more efficient than `optimizer.zero_grad()` as it avoids memory operations.

See: [https://pytorch.org/tutorials/recipes/recipes/tuning\\_guide.html](https://pytorch.org/tutorials/recipes/recipes/tuning_guide.html)

#### Parameters

**model** – PyTorch model to zero gradients for.

## Attributes

---

`DEFAULT_HITS_RANGE`

`ALL_HITS_RANGE`

---

## Classes

---

`Evaluator`

Evaluator class for KGE models in various downstream tasks.

---

## Functions

|                                                                        |                                                                     |
|------------------------------------------------------------------------|---------------------------------------------------------------------|
| <code>evaluate_link_prediction_performance(→ Dict[str, float])</code>  | Evaluate link prediction performance with head and tail prediction. |
| <code>evaluate_link_prediction_performance_with_.</code>               | Evaluate link prediction with reciprocal relations.                 |
| <code>evaluate_link_prediction_performance_with_</code>                | Evaluate link prediction with BPE encoding (head and tail).         |
| <code>evaluate_link_prediction_performance_with_</code>                | Evaluate link prediction with BPE encoding and reciprocals.         |
| <code>evaluate_lp(→ Dict[str, float])</code>                           | Evaluate link prediction with batched processing.                   |
| <code>evaluate_lp_bpe_k_vs_all(→ Dict[str, float])</code>              | Evaluate BPE link prediction with KvsAll scoring.                   |
| <code>evaluate_bpe_lp(→ Dict[str, float])</code>                       | Evaluate link prediction with BPE-encoded entities.                 |
| <code>evaluate_literal_prediction(→ Optional[pandas.DataFrame])</code> | Evaluate trained literal prediction model on a test file.           |
| <code>evaluate_ensemble_link_prediction_performai</code>               | Evaluate link prediction performance of an ensemble of KGE models.  |
| <code>compute_metrics_from_ranks(→ Dict[str, float])</code>            | Compute standard link prediction metrics from ranks.                |
| <code>compute_metrics_from_ranks_simple(→ Dict[str, float])</code>     | Compute link prediction metrics without scaling factor.             |
| <code>make_iterable_verbose(→ Iterable)</code>                         | Wrap an iterable with tqdm progress bar if verbose is True.         |
| <code>update_hits(→ None)</code>                                       | Update hits dictionary based on rank.                               |
| <code>create_hits_dict(→ Dict[int, List[float]])</code>                | Create an initialized hits dictionary.                              |

## Package Contents

**class** `dicee.evaluation.Evaluator` (*args, is\_continual\_training: bool = False*)

Evaluator class for KGE models in various downstream tasks.

Orchestrates link prediction evaluation with different scoring techniques including standard evaluation and byte-pair encoding based evaluation.

**er\_vocab**

Entity-relation to tail vocabulary for filtered ranking.

**re\_vocab**

Relation-entity (tail) to head vocabulary.

**ee\_vocab**

Entity-entity to relation vocabulary.

**num\_entities**

Total number of entities in the knowledge graph.

**num\_relations**

Total number of relations in the knowledge graph.

**args**

Configuration arguments.

**report**

Dictionary storing evaluation results.

**during\_training**

Whether evaluation is happening during training.

## Example

```
>>> from dicee.evaluation import Evaluator
>>> evaluator = Evaluator(args)
>>> results = evaluator.eval(dataset, model, 'EntityPrediction')
>>> print(f"Test MRR: {results['Test']['MRR']:.4f}")
```

**re\_vocab:** Dict | None = None

**er\_vocab:** Dict | None = None

**ee\_vocab:** Dict | None = None

**func\_triple\_to\_bpe\_representation** = None

**is\_continual\_training** = False

**num\_entities:** int | None = None

**num\_relations:** int | None = None

**domain\_constraints\_per\_rel** = None

**range\_constraints\_per\_rel** = None

**args**

**report:** Dict

**during\_training** = False

**vocab\_preparation** (*dataset*) → None

Prepare vocabularies from the dataset for evaluation.

Resolves any future objects and saves vocabularies to disk.

### Parameters

**dataset** – Knowledge graph dataset with vocabulary attributes.

**eval** (*dataset*, *trained\_model*, *form\_of\_labelling*: str, *during\_training*: bool = False) → Dict | None

Evaluate the trained model on the dataset.

### Parameters

- **dataset** – Knowledge graph dataset (KG instance).
- **trained\_model** – The trained KGE model.
- **form\_of\_labelling** – Type of labelling ('EntityPrediction' or 'RelationPrediction').
- **during\_training** – Whether evaluation is during training.

### Returns

Dictionary of evaluation metrics, or None if evaluation is skipped.

**eval\_rank\_of\_head\_and\_tail\_entity** (\*, *train\_set*, *valid\_set*=None, *test\_set*=None, *trained\_model*) → None

Evaluate with negative sampling scoring.

**eval\_rank\_of\_head\_and\_tail\_byte\_pair\_encoded\_entity** (\*, *train\_set*=None, *valid\_set*=None, *test\_set*=None, *ordered\_bpe\_entities*, *trained\_model*) → None

Evaluate with BPE-encoded entities and negative sampling.

**eval\_with\_byte**(\* , raw\_train\_set, raw\_valid\_set=None, raw\_test\_set=None, trained\_model, form\_of\_labelling) → None

Evaluate ByteE model with generation.

**eval\_with\_bpe\_vs\_all**(\* , raw\_train\_set, raw\_valid\_set=None, raw\_test\_set=None, trained\_model, form\_of\_labelling) → None

Evaluate with BPE and KvsAll scoring.

**eval\_with\_vs\_all**(\* , train\_set, valid\_set=None, test\_set=None, trained\_model, form\_of\_labelling) → None

Evaluate with KvsAll or 1vsAll scoring.

**evaluate\_lp\_k\_vs\_all**(model, triple\_idx, info: str | None = None, form\_of\_labelling: str | None = None) → Dict[str, float]

Filtered link prediction evaluation with KvsAll scoring.

#### Parameters

- **model** – The trained model to evaluate.
- **triple\_idx** – Integer-indexed test triples.
- **info** – Description to print.
- **form\_of\_labelling** – ‘EntityPrediction’ or ‘RelationPrediction’.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

**evaluate\_lp\_with\_byte**(model, triples: List[List[str]], info: str | None = None) → Dict[str, float]

Evaluate ByteE model with text generation.

#### Parameters

- **model** – ByteE model.
- **triples** – String triples.
- **info** – Description to print.

#### Returns

Dictionary with placeholder metrics (-1 values).

**evaluate\_lp\_bpe\_k\_vs\_all**(model, triples: List[List[str]], info: str | None = None, form\_of\_labelling: str | None = None) → Dict[str, float]

Evaluate BPE model with KvsAll scoring.

#### Parameters

- **model** – BPE-enabled model.
- **triples** – String triples.
- **info** – Description to print.
- **form\_of\_labelling** – Type of labelling.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

**evaluate\_lp**(model, triple\_idx, info: str) → Dict[str, float]

Evaluate link prediction with negative sampling.

#### Parameters



- **model** – The model to evaluate.
- **triple\_idx** – Integer-indexed triples.
- **info** – Description to print.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

**dummy\_eval** (*trained\_model*, *form\_of\_labelling*: *str*) → None

Run evaluation from saved data (for continual training).

#### Parameters

- **trained\_model** – The trained model.
- **form\_of\_labelling** – Type of labelling.

**eval\_with\_data** (*dataset*, *trained\_model*, *triple\_idx*: *numpy.ndarray*, *form\_of\_labelling*: *str*)  
→ Dict[str, float]

Evaluate a trained model on a given dataset.

#### Parameters

- **dataset** – Knowledge graph dataset.
- **trained\_model** – The trained model.
- **triple\_idx** – Integer-indexed triples to evaluate.
- **form\_of\_labelling** – Type of labelling.

#### Returns

Dictionary with evaluation metrics.

#### Raises

**ValueError** – If scoring technique is invalid.

`dicee.evaluation.evaluate_link_prediction_performance` (*model*, *triples*,  
*er\_vocab*: Dict[Tuple, List], *re\_vocab*: Dict[Tuple, List]) → Dict[str, float]

Evaluate link prediction performance with head and tail prediction.

Performs filtered evaluation where known correct answers are filtered out before computing ranks.

#### Parameters

- **model** – KGE model wrapper with entity/relation mappings.
- **triples** – Test triples as list of (head, relation, tail) strings.
- **er\_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re\_vocab** – Mapping (relation, entity) -> list of valid head entities.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

`dicee.evaluation.evaluate_link_prediction_performance_with_reciprocals` (*model*, *triples*,  
*er\_vocab*: Dict[Tuple, List]) → Dict[str, float]

Evaluate link prediction with reciprocal relations.

Optimized for models trained with reciprocal triples where only tail prediction is needed.

#### Parameters

- **model** – KGE model wrapper.

- **triples** – Test triples as list of (head, relation, tail) strings.
- **er\_vocab** – Mapping (entity, relation) -> list of valid tail entities.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.evaluate_link_prediction_performance_with_bpe(model,
    within_entities: List[str], triples: List[Tuple[str]], er_vocab: Dict[Tuple, List],
    re_vocab: Dict[Tuple, List]) → Dict[str, float]
```

Evaluate link prediction with BPE encoding (head and tail).

#### Parameters

- **model** – KGE model wrapper with BPE support.
- **within\_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of (head, relation, tail) tuples.
- **er\_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re\_vocab** – Mapping (relation, entity) -> list of valid head entities.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.evaluate_link_prediction_performance_with_bpe_reciprocals(model,
    within_entities: List[str], triples: List[List[str]], er_vocab: Dict[Tuple, List]) → Dict[str, float]
```

Evaluate link prediction with BPE encoding and reciprocals.

#### Parameters

- **model** – KGE model wrapper with BPE support.
- **within\_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of [head, relation, tail] strings.
- **er\_vocab** – Mapping (entity, relation) -> list of valid tail entities.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.evaluate_lp(model, triple_idx, num_entities: int, er_vocab: Dict[Tuple, List],
    re_vocab: Dict[Tuple, List], info: str = 'Eval Starts', batch_size: int = 128, chunk_size: int = 1000)
    → Dict[str, float]
```

Evaluate link prediction with batched processing.

Memory-efficient evaluation using chunked entity scoring.

#### Parameters

- **model** – The KGE model to evaluate.
- **triple\_idx** – Integer-indexed triples as numpy array.
- **num\_entities** – Total number of entities.
- **er\_vocab** – Mapping (head\_idx, rel\_idx) -> list of tail indices.
- **re\_vocab** – Mapping (rel\_idx, tail\_idx) -> list of head indices.
- **info** – Description to print.
- **batch\_size** – Batch size for triple processing.

- **chunk\_size** – Chunk size for entity scoring.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.evaluate_lp_bpe_k_vs_all(model, triples: List[List[str]],
    er_vocab: Dict | None = None, batch_size: int | None = None,
    func_triple_to_bpe_representation: Callable | None = None,
    str_to_bpe_entity_to_idx: Dict | None = None) → Dict[str, float]
```

Evaluate BPE link prediction with KvsAll scoring.

#### Parameters

- **model** – The KGE model wrapper.
- **triples** – List of string triples.
- **er\_vocab** – Entity-relation vocabulary for filtering.
- **batch\_size** – Batch size for processing.
- **func\_triple\_to\_bpe\_representation** – Function to convert triples to BPE.
- **str\_to\_bpe\_entity\_to\_idx** – Mapping from string entities to BPE indices.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

#### Raises

**ValueError** – If batch\_size is not provided.

```
dicee.evaluation.evaluate_bpe_lp(model, triple_idx: List[Tuple], all_bpe_shaped_entities,
    er_vocab: Dict[Tuple, List], re_vocab: Dict[Tuple, List], info: str = 'Eval Starts') → Dict[str, float]
```

Evaluate link prediction with BPE-encoded entities.

#### Parameters

- **model** – The KGE model to evaluate.
- **triple\_idx** – List of BPE-encoded triple tuples.
- **all\_bpe\_shaped\_entities** – All entities with BPE representations.
- **er\_vocab** – Mapping for tail filtering.
- **re\_vocab** – Mapping for head filtering.
- **info** – Description to print.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.evaluate_literal_prediction(kge_model, eval_file_path: str = None,
    store_lit_preds: bool = True, eval_literals: bool = True, loader_backend: str = 'pandas',
    return_attr_error_metrics: bool = False) → pandas.DataFrame | None
```

Evaluate trained literal prediction model on a test file.

Evaluates the literal prediction capabilities of a KGE model by computing MAE and RMSE metrics for each attribute.

#### Parameters

- **kge\_model** – Trained KGE model with literal prediction capability.
- **eval\_file\_path** – Path to the evaluation file containing test literals.

- **store\_lit\_preds** – If True, stores predictions to CSV file.
- **eval\_literals** – If True, evaluates and prints error metrics.
- **loader\_backend** – Backend for loading dataset ('pandas' or 'rdflib').
- **return\_attr\_error\_metrics** – If True, returns the metrics DataFrame.

#### Returns

DataFrame with per-attribute MAE and RMSE if `return_attr_error_metrics` is True, otherwise None.

#### Raises

- **RuntimeError** – If the KGE model doesn't have a trained literal model.
- **AssertionError** – If model is invalid or test set has no valid data.

### Example

```
>>> from dicee import KGE
>>> from dicee.evaluation import evaluate_literal_prediction
>>> model = KGE(path="pretrained_model")
>>> metrics = evaluate_literal_prediction(
...     model,
...     eval_file_path="test_literals.csv",
...     return_attr_error_metrics=True
... )
>>> print(metrics)
```

`dicee.evaluation.evaluate_ensemble_link_prediction_performance(models: List, triples, er_vocab: Dict[Tuple, List], weights: List[float] | None = None, batch_size: int = 512, weighted_averaging: bool = True, normalize_scores: bool = True) → Dict[str, float]`

Evaluate link prediction performance of an ensemble of KGE models.

Combines predictions from multiple models using weighted or simple averaging, with optional score normalization.

#### Parameters

- **models** – List of KGE models (e.g., snapshots from training).
- **triples** – Test triples as numpy array or list, shape (N, 3), with integer indices (head, relation, tail).
- **er\_vocab** – Mapping (head\_idx, rel\_idx) -> list of tail indices for filtered evaluation.
- **weights** – Weights for model averaging. Required if `weighted_averaging` is True. Must sum to 1 for proper averaging.
- **batch\_size** – Batch size for processing triples.
- **weighted\_averaging** – If True, use weighted averaging of predictions. If False, use simple mean.
- **normalize\_scores** – If True, normalize scores to [0, 1] range per sample before averaging.

#### Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

#### Raises

**AssertionError** – If `weighted_averaging` is True but weights are not provided or have wrong length.

## Example

```
>>> from dicee.evaluation import evaluate_ensemble_link_prediction_performance
>>> models = [model1, model2, model3]
>>> weights = [0.5, 0.3, 0.2]
>>> results = evaluate_ensemble_link_prediction_performance(
...     models, test_triples, er_vocab,
...     weights=weights, weighted_averaging=True
... )
>>> print(f"MRR: {results['MRR']:.4f}")
```

`dicee.evaluation.compute_metrics_from_ranks` (*ranks: List[int], num\_triples: int, hits\_dict: Dict[int, List[float]], scale\_factor: int = 1*) → Dict[str, float]

Compute standard link prediction metrics from ranks.

### Parameters

- **ranks** – List of ranks for each prediction.
- **num\_triples** – Total number of triples evaluated.
- **hits\_dict** – Dictionary mapping hit levels to lists of hits.
- **scale\_factor** – Factor to scale the denominator (e.g., 2 for head+tail).

### Returns

Dictionary containing H@1, H@3, H@10, and MRR metrics.

`dicee.evaluation.compute_metrics_from_ranks_simple` (*ranks: List[int], num\_triples: int, hits\_dict: Dict[int, List[float]]*) → Dict[str, float]

Compute link prediction metrics without scaling factor.

### Parameters

- **ranks** – List of ranks for each prediction.
- **num\_triples** – Total number of triples evaluated.
- **hits\_dict** – Dictionary mapping hit levels to lists of hits.

### Returns

Dictionary containing H@1, H@3, H@10, and MRR metrics.

`dicee.evaluation.make_iterable_verbose` (*iterable\_object: Iterable, verbose: bool, desc: str = 'Default', position: int | None = None, leave: bool = True*) → Iterable

Wrap an iterable with tqdm progress bar if verbose is True.

### Parameters

- **iterable\_object** – The iterable to potentially wrap.
- **verbose** – Whether to show progress bar.
- **desc** – Description for the progress bar.
- **position** – Position of the progress bar.
- **leave** – Whether to leave the progress bar after completion.

### Returns

The original iterable or a tqdm-wrapped version.

`dicee.evaluation.update_hits (hits: Dict[int, List[float]], rank: int, hits_range: List[int] | None = None) → None`

Update hits dictionary based on rank.

#### Parameters

- **hits** – Dictionary to update in-place.
- **rank** – The rank to check against hit levels.
- **hits\_range** – List of hit levels to check (default: ALL\_HITS\_RANGE).

`dicee.evaluation.create_hits_dict (hits_range: List[int] | None = None) → Dict[int, List[float]]`

Create an initialized hits dictionary.

#### Parameters

- **hits\_range** – List of hit levels to initialize (default: ALL\_HITS\_RANGE).

#### Returns

Dictionary with empty lists for each hit level.

`dicee.evaluation.DEFAULT_HITS_RANGE: List[int] = [1, 3, 10]`

`dicee.evaluation.ALL_HITS_RANGE: List[int]`

## `dicee.evaluator`

Evaluator module for knowledge graph embedding models.

This module provides backward compatibility by re-exporting from the new `dicee.evaluation` module.

Deprecated since version Use: `dicee.evaluation.Evaluator` instead. This module will be removed in a future version.

## Classes

*Evaluator*

Evaluator class for KGE models in various downstream tasks.

## Module Contents

**class** `dicee.evaluator.Evaluator (args, is_continual_training: bool = False)`

Evaluator class for KGE models in various downstream tasks.

Orchestrates link prediction evaluation with different scoring techniques including standard evaluation and byte-pair encoding based evaluation.

**er\_vocab**

Entity-relation to tail vocabulary for filtered ranking.

**re\_vocab**

Relation-entity (tail) to head vocabulary.

**ee\_vocab**

Entity-entity to relation vocabulary.

**num\_entities**

Total number of entities in the knowledge graph.

**num\_relations**

Total number of relations in the knowledge graph.

**args**

Configuration arguments.

**report**

Dictionary storing evaluation results.

**during\_training**

Whether evaluation is happening during training.

## Example

```
>>> from dicee.evaluation import Evaluator
>>> evaluator = Evaluator(args)
>>> results = evaluator.eval(dataset, model, 'EntityPrediction')
>>> print(f"Test MRR: {results['Test']['MRR']:.4f}")
```

**re\_vocab:** Dict | None = None

**er\_vocab:** Dict | None = None

**ee\_vocab:** Dict | None = None

**func\_triple\_to\_bpe\_representation** = None

**is\_continual\_training** = False

**num\_entities:** int | None = None

**num\_relations:** int | None = None

**domain\_constraints\_per\_rel** = None

**range\_constraints\_per\_rel** = None

**args**

**report:** Dict

**during\_training** = False

**vocab\_preparation** (*dataset*) → None

Prepare vocabularies from the dataset for evaluation.

Resolves any future objects and saves vocabularies to disk.

### Parameters

**dataset** – Knowledge graph dataset with vocabulary attributes.

**eval** (*dataset*, *trained\_model*, *form\_of\_labelling*: str, *during\_training*: bool = False) → Dict | None

Evaluate the trained model on the dataset.

### Parameters

- **dataset** – Knowledge graph dataset (KG instance).
- **trained\_model** – The trained KGE model.
- **form\_of\_labelling** – Type of labelling ('EntityPrediction' or 'RelationPrediction').

- **during\_training** – Whether evaluation is during training.

#### Returns

Dictionary of evaluation metrics, or None if evaluation is skipped.

**eval\_rank\_of\_head\_and\_tail\_entity** (\*, *train\_set*, *valid\_set=None*, *test\_set=None*, *trained\_model*)  
→ None

Evaluate with negative sampling scoring.

**eval\_rank\_of\_head\_and\_tail\_byte\_pair\_encoded\_entity** (\*, *train\_set=None*, *valid\_set=None*,  
*test\_set=None*, *ordered\_bpe\_entities*, *trained\_model*) → None

Evaluate with BPE-encoded entities and negative sampling.

**eval\_with\_byte** (\*, *raw\_train\_set*, *raw\_valid\_set=None*, *raw\_test\_set=None*, *trained\_model*,  
*form\_of\_labelling*) → None

Evaluate Byte model with generation.

**eval\_with\_bpe\_vs\_all** (\*, *raw\_train\_set*, *raw\_valid\_set=None*, *raw\_test\_set=None*, *trained\_model*,  
*form\_of\_labelling*) → None

Evaluate with BPE and KvsAll scoring.

**eval\_with\_vs\_all** (\*, *train\_set*, *valid\_set=None*, *test\_set=None*, *trained\_model*, *form\_of\_labelling*)  
→ None

Evaluate with KvsAll or 1vsAll scoring.

**evaluate\_lp\_k\_vs\_all** (*model*, *triple\_idx*, *info: str | None = None*,  
*form\_of\_labelling: str | None = None*) → Dict[str, float]

Filtered link prediction evaluation with KvsAll scoring.

#### Parameters

- **model** – The trained model to evaluate.
- **triple\_idx** – Integer-indexed test triples.
- **info** – Description to print.
- **form\_of\_labelling** – ‘EntityPrediction’ or ‘RelationPrediction’.

#### Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

**evaluate\_lp\_with\_byte** (*model*, *triples: List[List[str]]*, *info: str | None = None*) → Dict[str, float]

Evaluate Byte model with text generation.

#### Parameters

- **model** – Byte model.
- **triples** – String triples.
- **info** – Description to print.

#### Returns

Dictionary with placeholder metrics (-1 values).

**evaluate\_lp\_bpe\_k\_vs\_all** (*model*, *triples: List[List[str]]*, *info: str | None = None*,  
*form\_of\_labelling: str | None = None*) → Dict[str, float]

Evaluate BPE model with KvsAll scoring.

#### Parameters

- **model** – BPE-enabled model.



- **triples** – String triples.
- **info** – Description to print.
- **form\_of\_labelling** – Type of labelling.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

**evaluate\_lp** (*model*, *triple\_idx*, *info*: *str*) → Dict[str, float]

Evaluate link prediction with negative sampling.

#### Parameters

- **model** – The model to evaluate.
- **triple\_idx** – Integer-indexed triples.
- **info** – Description to print.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

**dummy\_eval** (*trained\_model*, *form\_of\_labelling*: *str*) → None

Run evaluation from saved data (for continual training).

#### Parameters

- **trained\_model** – The trained model.
- **form\_of\_labelling** – Type of labelling.

**eval\_with\_data** (*dataset*, *trained\_model*, *triple\_idx*: *numpy.ndarray*, *form\_of\_labelling*: *str*)  
→ Dict[str, float]

Evaluate a trained model on a given dataset.

#### Parameters

- **dataset** – Knowledge graph dataset.
- **trained\_model** – The trained model.
- **triple\_idx** – Integer-indexed triples to evaluate.
- **form\_of\_labelling** – Type of labelling.

#### Returns

Dictionary with evaluation metrics.

#### Raises

**ValueError** – If scoring technique is invalid.

## dicee.executer

Executor module for training, retraining and evaluating KGE models.

This module provides the Execute and ContinuousExecute classes for managing the full lifecycle of knowledge graph embedding model training.

## Classes

|                          |                                                                    |
|--------------------------|--------------------------------------------------------------------|
| <i>Execute</i>           | Executor class for training, retraining and evaluating KGE models. |
| <i>ContinuousExecute</i> | A subclass of Execute Class for retraining                         |

## Module Contents

**class** `dicee.executer.Execute` (*args*, *continuous\_training: bool = False*)

Executor class for training, retraining and evaluating KGE models.

Handles the complete workflow: 1. Loading & Preprocessing & Serializing input data 2. Training & Validation & Testing 3. Storing all necessary information

**args**

Processed input arguments.

**distributed**

Whether distributed training is enabled.

**rank**

Process rank in distributed training.

**world\_size**

Total number of processes.

**local\_rank**

Local GPU rank.

**trainer**

Training handler instance.

**trained\_model**

The trained model after training completes.

**knowledge\_graph**

The loaded knowledge graph.

**report**

Dictionary storing training metrics and results.

**evaluator**

Model evaluation handler.

**distributed**

**args**

**is\_continual\_training** = False

**trainer:** `dicee.trainer.DICE_Trainer` | None = None

**trained\_model** = None

**knowledge\_graph:** `dicee.knowledge_graph.KG` | None = None

**report:** Dict

**evaluator:** `dicee.evaluator.Evaluator` | None = None

**start\_time:** float | None = None

**is\_rank\_zero()** → bool

**cleanup()**

**setup\_executor()** → None

Set up storage directories for the experiment.

Creates or reuses experiment directories based on configuration. Saves the configuration to a JSON file.

**create\_and\_store\_kg()** → None

Create knowledge graph and store as memory-mapped file.

Only executed on rank 0 in distributed training. Skips if memmap already exists.

**load\_from\_memmap()** → None

Load knowledge graph from memory-mapped file.

**save\_trained\_model()** → None

Save a knowledge graph embedding model

(1) Send model to eval mode and cpu.

(2) Store the memory footprint of the model.

(3) Save the model into disk.

(4) Update the stats of KG again ?

## Parameter

**rtype**

None

**end**(*form\_of\_labelling: str*) → dict

End training

(1) Store trained model.

(2) Report runtimes.

(3) Eval model if required.

## Parameter

**rtype**

A dict containing information about the training and/or evaluation

**write\_report()** → None

Report training related information in a report.json file

**start()** → dict

Start training

# (1) Loading the Data # (2) Create an evaluator object. # (3) Create a trainer object. # (4) Start the training

## Parameter

### rtype

A dict containing information about the training and/or evaluation

```
class dicee.executer.ContinuousExecute (args)
```

Bases: *Execute*

A subclass of Execute Class for retraining

- (1) Loading & Preprocessing & Serializing input data.
- (2) Training & Validation & Testing
- (3) Storing all necessary info

During the continual learning we can only modify \* **num\_epochs** \* parameter. Trained model stored in the same folder as the seed model for the training. Trained model is noted with the current time.

**continual\_start** () → dict

Start Continual Training

- (1) Initialize training.
- (2) Start continual training.
- (3) Save trained model.

## Parameter

### rtype

A dict containing information about the training and/or evaluation

## dicee.knowledge\_graph

Knowledge Graph module for data loading and preprocessing.

Provides the KG class for handling knowledge graph data including loading, preprocessing, and indexing operations.

## Classes

---

*KG*

Knowledge Graph container and processor.

---

## Module Contents

```
class dicee.knowledge_graph.KG (dataset_dir: str | None = None, byte_pair_encoding: bool = False,
padding: bool = False, add_noise_rate: float | None = None, sparql_endpoint: str | None = None,
path_single_kg: str | None = None, path_for_deserialization: str | None = None,
add_reciprocal: bool | None = None, eval_model: str | None = None,
read_only_few: int | None = None, sample_triples_ratio: float | None = None,
path_for_serialization: str | None = None, entity_to_idx: Dict | None = None,
relation_to_idx: Dict | None = None, backend: str | None = None,
training_technique: str | None = None, separator: str | None = None)
```

Knowledge Graph container and processor.

Handles loading, preprocessing, and indexing of knowledge graph data from various sources including files, SPARQL endpoints, and serialized formats.

**dataset\_dir**  
Path to directory containing train/valid/test files.

**num\_entities**  
Total number of unique entities.

**num\_relations**  
Total number of unique relations.

**train\_set**  
Indexed training triples as numpy array.

**valid\_set**  
Indexed validation triples (optional).

**test\_set**  
Indexed test triples (optional).

**entity\_to\_idx**  
Mapping from entity strings to indices.

**relation\_to\_idx**  
Mapping from relation strings to indices.

**dataset\_dir = None**

**sparql\_endpoint = None**

**path\_single\_kg = None**

**byte\_pair\_encoding = False**

**ordered\_shaped\_bpe\_tokens = None**

**add\_noise\_rate = None**

**num\_entities: int | None = None**

**num\_relations: int | None = None**

**path\_for\_deserialization = None**

**add\_reciprocal = None**

**eval\_model = None**

**read\_only\_few = None**

**sample\_triples\_ratio = None**

**path\_for\_serialization = None**

**entity\_to\_idx = None**

**relation\_to\_idx = None**

**backend = 'pandas'**

**training\_technique = None**

```

separator = None

raw_train_set = None

raw_valid_set = None

raw_test_set = None

train_set = None

valid_set = None

test_set = None

idx_entity_to_bpe_shaped: Dict

enc

num_tokens

num_bpe_entities: int | None = None

padding = False

dummy_id

max_length_subword_tokens: int | None = None

train_set_target = None

target_dim: int | None = None

train_target_indices = None

ordered_bpe_entities = None

description_of_input = None

describe() → None
    Generate a description string of the dataset statistics.

property_entities_str: List[str]
    Get list of all entity strings.

property_relations_str: List[str]
    Get list of all relation strings.

exists(h: str, r: str, t: str) → bool
    Check if a triple exists in the training set.

```

#### Parameters

- **h** – Head entity string.
- **r** – Relation string.
- **t** – Tail entity string.

#### Returns

True if the triple exists, False otherwise.

```

__iter__() → Iterator[Tuple[str, str, str]]
    Iterate over training triples as string tuples.

__len__() → int
    Return number of triples in the raw training set.

func_triple_to_bpe_representation(triple: List[str])

```

## dicee.knowledge\_graph\_embeddings

### Classes

|            |                                                                             |
|------------|-----------------------------------------------------------------------------|
| <i>KGE</i> | Knowledge Graph Embedding Class for interactive usage of pre-trained models |
|------------|-----------------------------------------------------------------------------|

### Module Contents

```

class dicee.knowledge_graph_embeddings.KGE (path=None, url=None, construct_ensemble=False,
      model_name=None)

    Bases: dicee.abstracts.BaseInteractiveKGE, dicee.abstracts.InteractiveQueryDecomposition, dicee.abstracts.BaseInteractiveTrainKGE

    Knowledge Graph Embedding Class for interactive usage of pre-trained models

    __str__()

    to(device: str) → None

    get_transductive_entity_embeddings(indices: torch.LongTensor | List[str], as_pytorch=False,
      as_numpy=False, as_list=True) → torch.FloatTensor | numpy.ndarray | List[float]

    create_vector_database(collection_name: str, distance: str, location: str = 'localhost',
      port: int = 6333)

    generate(h="", r="")

    eval_lp_performance(dataset=List[Tuple[str, str, str]], filtered=True)

    predict_missing_head_entity(relation: List[str] | str, tail_entity: List[str] | str, within=None,
      batch_size=2, topk=1, return_indices=False) → Tuple

    Given a relation and a tail entity, return top k ranked head entity.

     $\operatorname{argmax}_{\{e \in E\}} f(e, r, t)$ , where  $r \in R$ ,  $t \in E$ .

```

#### Parameter

relation: Union[List[str], str]  
 String representation of selected relations.

tail\_entity: Union[List[str], str]  
 String representation of selected entities.

k: int  
 Highest ranked k entities.

### Returns: Tuple

Highest K scores and entities

**predict\_missing\_relations** (*head\_entity: List[str] | str, tail\_entity: List[str] | str, within=None, batch\_size=2, topk=1, return\_indices=False*) → Tuple

Given a head entity and a tail entity, return top k ranked relations.

$\text{argmax}_{\{r \in R\}} f(h,r,t)$ , where  $h, t \in E$ .

### Parameter

head\_entity: List[str]

String representation of selected entities.

tail\_entity: List[str]

String representation of selected entities.

k: int

Highest ranked k entities.

### Returns: Tuple

Highest K scores and entities

**predict\_missing\_tail\_entity** (*head\_entity: List[str] | str, relation: List[str] | str, within: List[str] = None, batch\_size=2, topk=1, return\_indices=False*) → torch.FloatTensor

Given a head entity and a relation, return top k ranked entities

$\text{argmax}_{\{e \in E\}} f(h,r,e)$ , where  $h \in E$  and  $r \in R$ .

### Parameter

head\_entity: List[str]

String representation of selected entities.

tail\_entity: List[str]

String representation of selected entities.

### Returns: Tuple

scores

**predict** (\*, *h: List[str] | str = None, r: List[str] | str = None, t: List[str] | str = None, within=None, logits=True*) → torch.FloatTensor

#### Parameters

- **logits**
- **h**
- **r**
- **t**
- **within**



**predict\_topk** (\*, h: str | List[str] = None, r: str | List[str] = None, t: str | List[str] = None, topk: int = 10, within: List[str] = None, batch\_size: int = 1024)

Predict missing item in a given triple.

#### Returns

- If you query a single (h, r, ?) or (?, r, t) or (h, ?, t), returns List[(item, score)]
- If you query a batch of B, returns List of B such lists.

**triple\_score** (h: List[str] | str = None, r: List[str] | str = None, t: List[str] | str = None, logits=False)  
→ torch.FloatTensor

Predict triple score

#### Parameter

head\_entity: List[str]

String representation of selected entities.

relation: List[str]

String representation of selected relations.

tail\_entity: List[str]

String representation of selected entities.

logits: bool

If logits is True, unnormalized score returned

#### Returns: Tuple

pytorch tensor of triple score

**return\_multi\_hop\_query\_results** (aggregated\_query\_for\_all\_entities, k: int, only\_scores)

**single\_hop\_query\_answering** (query: tuple, only\_scores: bool = True, k: int = None)

**answer\_multi\_hop\_query** (query\_type: str = None, query: Tuple[str | Tuple[str, str], Ellipsis] = None, queries: List[Tuple[str | Tuple[str, str], Ellipsis]] = None, tnorm: str = 'prod', neg\_norm: str = 'standard', lambda\_: float = 0.0, k: int = 10, only\_scores=False)  
→ List[Tuple[str, torch.Tensor]]

# @TODO: Refactoring is needed # @TODO: Score computation for each query type should be done in a static function

Find an answer set for EPFO queries including negation and disjunction

#### Parameter

query\_type: str The type of the query, e.g., “2p”.

query: Union[str, Tuple[str, Tuple[str, str]]] The query itself, either a string or a nested tuple.

queries: List of Tuple[Union[str, Tuple[str, str]], ...]

tnorm: str The t-norm operator.

neg\_norm: str The negation norm.

**lambda\_**: float lambda parameter for sugeno and yager negation norms

k: int The top-k substitutions for intermediate variables.

### returns

- *List[Tuple[str, torch.Tensor]]*
- *Entities and corresponding scores sorted in the descening order of scores*

**find\_missing\_triples** (*confidence: float, entities: List[str] = None, relations: List[str] = None, topk: int = 10, at\_most: int = sys.maxsize*) → Set

Find missing triples

Iterative over a set of entities E and a set of relation R :

forall e in E and forall r in R f(e,r,x)

Return (e,r,x)

otin G and f(e,r,x) > confidence

confidence: float

A threshold for an output of a sigmoid function given a triple.

topk: int

Highest ranked k item to select triples with f(e,r,x) > confidence .

at\_most: int

Stop after finding at\_most missing triples

{(e,r,x) | f(e,r,x) > confidence and (e,r,x)

otin G

**predict\_literals** (*entity: List[str] | str = None, attribute: List[str] | str = None, denormalize\_preds: bool = True*) → numpy.ndarray

Predicts literal values for given entities and attributes.

### Parameters

- **entity** (*Union[List[str], str]*) – Entity or list of entities to predict literals for.
- **attribute** (*Union[List[str], str]*) – Attribute or list of attributes to predict literals for.
- **denormalize\_preds** (*bool*) – If True, denormalizes the predictions.

### Returns

Predictions for the given entities and attributes.

### Return type

numpy ndarray

## dicee.models

### Submodules

#### dicee.models.adopt

ADOPT Optimizer Implementation.

This module implements the ADOPT (Adaptive Optimization with Precise Tracking) algorithm, an advanced optimization method for training neural networks.

## ADOPT Overview:

ADOPT is an adaptive learning rate optimization algorithm that combines the benefits of momentum-based methods with per-parameter learning rate adaptation. Unlike Adam, which applies momentum to raw gradients, ADOPT normalizes gradients first and then applies momentum, leading to more stable training dynamics.

Key Features: - Gradient normalization before momentum application - Adaptive per-parameter learning rates - Optional gradient clipping that grows with training steps - Support for decoupled weight decay (AdamW-style) - Multiple execution modes: single-tensor, multi-tensor (foreach), and fused (planned)

## Algorithm Comparison:

Adam:  $m = \beta_1 * m + (1 - \beta_1) * g$ ,  $\theta = \theta - \alpha * m / \sqrt{v}$  ADOPT:  $m = \beta_1 * m + (1 - \beta_1) * g / \sqrt{v}$ ,  $\theta = \theta - \alpha * m$

The key difference is that ADOPT normalizes gradients before momentum, which provides better stability and can lead to improved convergence.

## Classes:

- ADOPT: Main optimizer class (extends torch.optim.Optimizer)

## Functions:

- adopt: Functional API for ADOPT algorithm computation
- \_single\_tensor\_adopt: Single-tensor implementation (TorchScript compatible)
- \_multi\_tensor\_adopt: Multi-tensor implementation using foreach operations

## Performance:

- Single-tensor: Default, compatible with torch.jit.script
- Multi-tensor (foreach): 2-3x faster on GPU through vectorization
- Fused (planned): Would provide maximum performance via specialized kernels

## Example:

```
>>> import torch
>>> from dicee.models.adopt import ADOPT
>>>
>>> model = torch.nn.Linear(10, 1)
>>> optimizer = ADOPT(model.parameters(), lr=0.001, weight_decay=0.01, decouple=True)
>>>
>>> # Training loop
>>> for epoch in range(num_epochs):
...     optimizer.zero_grad()
...     output = model(input)
...     loss = criterion(output, target)
...     loss.backward()
...     optimizer.step()
```

## References:

Original implementation: <https://github.com/iShohei220/adopt>

## Notes:

This implementation is based on the original ADOPT implementation and adapted to work with the PyTorch optimizer interface and the dicee framework.

## Classes

|                    |                  |
|--------------------|------------------|
| <code>ADOPT</code> | ADOPT Optimizer. |
|--------------------|------------------|

## Functions

|                                                                       |                                                           |
|-----------------------------------------------------------------------|-----------------------------------------------------------|
| <code>adopt(params, grads, exp_avgs, exp_avg_sqs, state_steps)</code> | Functional API that performs ADOPT algorithm computation. |
|-----------------------------------------------------------------------|-----------------------------------------------------------|

## Module Contents

```
class dicee.models.adopt.ADOPT (params: torch.optim.optimizer.ParamsT,  
    lr: float | torch.Tensor = 0.001, betas: Tuple[float, float] = (0.9, 0.9999), eps: float = 1e-06,  
    clip_lambda: Callable[[int], float] | None = lambda step: ..., weight_decay: float = 0.0,  
    decouple: bool = False, *, foreach: bool | None = None, maximize: bool = False,  
    capturable: bool = False, differentiable: bool = False, fused: bool | None = None)
```

Bases: torch.optim.optimizer.Optimizer

ADOPT Optimizer.

ADOPT is an adaptive learning rate optimization algorithm that combines momentum-based updates with adaptive per-parameter learning rates. It uses exponential moving averages of gradients and squared gradients, with gradient clipping for stability.

The algorithm performs the following key operations: 1. Normalizes gradients by the square root of the second moment estimate 2. Applies optional gradient clipping based on the training step 3. Updates parameters using momentum-smoothed normalized gradients 4. Supports decoupled weight decay (AdamW-style) or L2 regularization

### Mathematical formulation:

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \text{clip}(g_t / \sqrt{v_t}) \quad v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * g_t^2 \quad \theta_t = \theta_{t-1} - \alpha * m_t$$

where:

- $\theta_t$ : parameter at step  $t$
- $g_t$ : gradient at step  $t$
- $m_t$ : first moment estimate (momentum)
- $v_t$ : second moment estimate (variance)
- $\alpha$ : learning rate
- $\beta_1, \beta_2$ : exponential decay rates
- $\text{clip}()$ : optional gradient clipping function

## Reference:

Original implementation: <https://github.com/iShohei220/adopt>

## Parameters

- **params** (*ParamsT*) – Iterable of parameters to optimize or dicts defining parameter groups.
- **lr** (*float or Tensor, optional*) – Learning rate. Can be a float or 1-element Tensor. Default: 1e-3
- **betas** (*Tuple[float, float], optional*) – Coefficients ( $\beta_1, \beta_2$ ) for computing running averages of gradient and its square.  $\beta_1$  controls momentum,  $\beta_2$  controls variance. Default: (0.9, 0.9999)
- **eps** (*float, optional*) – Term added to denominator to improve numerical stability. Default: 1e-6
- **clip\_lambda** (*Callable[[int], float], optional*) – Function that takes the step number and returns the gradient clipping threshold. Common choices: - lambda step: step\*\*0.25 (default, gradually increases clipping threshold) - lambda step: 1.0 (constant clipping) - None (no clipping) Default: lambda step: step\*\*0.25
- **weight\_decay** (*float, optional*) – Weight decay coefficient (L2 penalty). Default: 0.0
- **decouple** (*bool, optional*) – If True, uses decoupled weight decay (AdamW-style), applying weight decay directly to parameters. If False, adds weight decay to gradients (L2 regularization). Default: False
- **foreach** (*bool, optional*) – If True, uses the faster foreach implementation for multi-tensor operations. Default: None (auto-select)
- **maximize** (*bool, optional*) – If True, maximizes parameters instead of minimizing. Useful for reinforcement learning. Default: False
- **capturable** (*bool, optional*) – If True, the optimizer is safe to capture in a CUDA graph. Requires learning rate as Tensor. Default: False
- **differentiable** (*bool, optional*) – If True, the optimization step can be differentiated. Useful for meta-learning. Default: False
- **fused** (*bool, optional*) – If True, uses fused kernel implementation (currently not supported). Default: None

## Raises

- **ValueError** – If learning rate, epsilon, betas, or weight\_decay are invalid.
- **RuntimeError** – If fused is enabled (not currently supported).
- **RuntimeError** – If lr is a Tensor with foreach=True and capturable=False.

## Example

```
>>> # Basic usage
>>> optimizer = ADOPT(model.parameters(), lr=0.001)
>>> optimizer.zero_grad()
>>> loss.backward()
>>> optimizer.step()
```

```
>>> # With decoupled weight decay
>>> optimizer = ADOPT(model.parameters(), lr=0.001, weight_decay=0.01,
↳decouple=True)
```

```
>>> # Custom gradient clipping
>>> optimizer = ADOPT(model.parameters(), clip_lambda=lambda step: max(1.0,
↳step**0.5))
```

#### Note

- For most use cases, the default hyperparameters work well
- Consider using decouple=True for better generalization (similar to AdamW)
- The clip\_lambda function helps stabilize training in early steps

### clip\_lambda

**\_\_setstate\_\_** (state)

Restore optimizer state from a checkpoint.

This method handles backward compatibility when loading optimizer state from older versions. It ensures all required fields are present with default values and properly converts step counters to tensors if needed.

Key responsibilities: 1. Set default values for newly added hyperparameters 2. Convert old-style scalar step counters to tensor format 3. Place step tensors on appropriate devices based on capturable/fused modes

#### Parameters

**state** (dict) – Optimizer state dictionary (typically from torch.load()).

#### Note

- This enables loading checkpoints saved with older ADOPT versions
- Step counters are converted to appropriate device/dtype for compatibility
- Capturable and fused modes require step tensors on parameter devices

**step** (closure=None)

Perform a single optimization step.

This method executes one iteration of the ADOPT optimization algorithm across all parameter groups. It orchestrates the following workflow:

1. Optionally evaluates a closure to recompute the loss (useful for algorithms like LBFGS or when loss needs multiple evaluations)
2. For each parameter group: - Collects parameters with gradients and their associated state - Extracts hyperparameters (betas, learning rate, etc.) - Calls the functional adopt() API to perform the actual update
3. Returns the loss value if a closure was provided

The functional API (adopt()) handles three execution modes: - Single-tensor: Updates one parameter at a time (default, JIT-compatible) - Multi-tensor (foreach): Batches operations for better performance - Fused: Uses fused CUDA kernels (not yet implemented)

Gradient scaling support: This method is compatible with automatic mixed precision (AMP) training. It can access `grad_scale` and `found_inf` attributes for gradient unscaling and inf/nan detection when used with `GradScaler`.

#### Parameters

**closure** (*Callable, optional*) – A callable that reevaluates the model and returns the loss. The closure should: - Enable gradients (`torch.enable_grad()`) - Compute forward pass - Compute loss - Compute backward pass - Return the loss value Example: `lambda: (loss := model(x), loss.backward(), loss)[-1]` Default: `None`

#### Returns

**The loss value returned by the closure, or `None` if no closure was provided.**

#### Return type

Optional[`Tensor`]

#### Example

```
>>> # Standard usage
>>> loss = criterion(model(input), target)
>>> loss.backward()
>>> optimizer.step()
```

```
>>> # With closure (e.g., for line search)
>>> def closure():
...     optimizer.zero_grad()
...     output = model(input)
...     loss = criterion(output, target)
...     loss.backward()
...     return loss
>>> loss = optimizer.step(closure)
```

#### Note

- Call `zero_grad()` before computing gradients for the next step
- CUDA graph capture is checked for safety when `capturable=True`
- The method is thread-safe for different parameter groups

`dicee.models.adopt.adopt` (*params: List[torch.Tensor], grads: List[torch.Tensor], exp\_avgs: List[torch.Tensor], exp\_avg\_sqs: List[torch.Tensor], state\_steps: List[torch.Tensor], foreach: bool | None = None, capturable: bool = False, differentiable: bool = False, fused: bool | None = None, grad\_scale: torch.Tensor | None = None, found\_inf: torch.Tensor | None = None, has\_complex: bool = False, \*, beta1: float, beta2: float, lr: float | torch.Tensor, clip\_lambda: Callable[[int], float] | None, weight\_decay: float, decouple: bool, eps: float, maximize: bool*)

Functional API that performs ADOPT algorithm computation.

This is the main functional interface for the ADOPT optimization algorithm. It dispatches to one of three implementations based on the execution mode:

1. **Single-tensor mode** (default): Updates parameters one at a time - Compatible with `torch.jit.script` - More flexible but slower - Used when `foreach=False` or automatically for small models

2. **Multi-tensor (foreach) mode:** Batches operations across tensors - 2-3x faster on GPU through vectorization  
- Groups tensors by device/dtype automatically - Used when foreach=True
3. **Fused mode:** Uses specialized fused kernels (not yet implemented) - Would provide maximum performance  
- Currently raises RuntimeError if enabled

### Algorithm overview (ADOPT):

ADOPT adapts learning rates per-parameter while using momentum on normalized gradients. The key innovation is normalizing gradients before momentum, which provides more stable training than standard Adam.

#### Mathematical formulation:

```
# Normalize gradient by its historical variance  $\text{normed\_g\_t} = \text{g\_t} / \sqrt{(\text{v\_t} + \epsilon)}$ 

# Optional gradient clipping for stability  $\text{normed\_g\_t} = \text{clip}(\text{normed\_g\_t}, \text{threshold}(t))$ 

# Momentum on normalized gradients (key difference from Adam)  $\text{m\_t} = \beta_1 * \text{m}_{\{t-1\}} + (1 - \beta_1) * \text{normed\_g\_t}$ 

# Parameter update  $\theta\_t = \theta_{\{t-1\}} - \alpha * \text{m\_t}$ 

# Update variance estimate  $\text{v\_t} = \beta_2 * \text{v}_{\{t-1\}} + (1 - \beta_2) * \text{g\_t}^2$ 
```

where:

- $\theta$ : parameters
- $g$ : gradients
- $m$ : first moment (momentum of normalized gradients)
- $v$ : second moment (variance of raw gradients)
- $\alpha$ : learning rate
- $\beta_1, \beta_2$ : exponential decay rates
- $\epsilon$ : numerical stability constant
- $\text{clip}()$ : gradient clipping function based on step

#### Automatic mode selection:

When foreach and fused are both None (default), the function automatically selects the best implementation based on:  
- Parameter types and devices - Whether differentiable mode is enabled - Learning rate type (float vs Tensor)  
- Capturable mode requirements

##### param params

Parameters to optimize.

##### type params

List[Tensor]

##### param grads

Gradients for each parameter.

##### type grads

List[Tensor]

##### param exp\_avgs

First moment estimates (momentum).

##### type exp\_avgs

List[Tensor]



**param exp\_avg\_sq**  
Second moment estimates (variance).

**type exp\_avg\_sq**  
List[Tensor]

**param state\_steps**  
Step counters (must be singleton tensors).

**type state\_steps**  
List[Tensor]

**param foreach**  
Whether to use multi-tensor implementation. None: auto-select based on configuration (default).

**type foreach**  
Optional[bool]

**param capturable**  
If True, ensure CUDA graph capture safety.

**type capturable**  
bool

**param differentiable**  
If True, allow gradients through optimization step.

**type differentiable**  
bool

**param fused**  
If True, use fused kernels (not implemented).

**type fused**  
Optional[bool]

**param grad\_scale**  
Gradient scaler for AMP training.

**type grad\_scale**  
Optional[Tensor]

**param found\_inf**  
Flag for inf/nan detection in AMP.

**type found\_inf**  
Optional[Tensor]

**param has\_complex**  
Whether any parameters are complex-valued.

**type has\_complex**  
bool

**param beta1**  
Exponential decay rate for first moment (momentum). Typical range: 0.9-0.95.

**type beta1**  
float

**param beta2**  
Exponential decay rate for second moment (variance). Typical range: 0.999-0.9999 (higher than Adam).

**type beta2**  
float

**param lr**  
Learning rate. Can be a scalar Tensor for dynamic learning rate with capturable=True.

**type lr**  
Union[float, Tensor]

**param clip\_lambda**  
Function that maps step number to gradient clipping threshold. None disables clipping.

**type clip\_lambda**  
Optional[Callable[[int], float]]

**param weight\_decay**  
Weight decay coefficient (L2 penalty).

**type weight\_decay**  
float

**param decouple**  
If True, use decoupled weight decay (AdamW-style). Recommended for better generalization.

**type decouple**  
bool

**param eps**  
Small constant for numerical stability in normalization.

**type eps**  
float

**param maximize**  
If True, maximize objective instead of minimize.

**type maximize**  
bool

**raises RuntimeError**  
If torch.jit.script is used with foreach or fused.

**raises RuntimeError**  
If state\_steps contains non-tensor elements.

**raises RuntimeError**  
If fused=True (not yet implemented).

**raises RuntimeError**  
If lr is Tensor with foreach=True and capturable=False.

## Example

```
>>> # Typically called by ADOPT optimizer, not directly
>>> adopt (
...     params=[p1, p2],
...     grads=[g1, g2],
...     exp_avgs=[m1, m2],
...     exp_avg_sqs=[v1, v2],
...     state_steps=[step1, step2],
...     beta1=0.9,
```

(continues on next page)

(continued from previous page)

```
...     beta2=0.9999,  
...     lr=0.001,  
...     clip_lambda=lambda s: s**0.25,  
...     weight_decay=0.01,  
...     decouple=True,  
...     eps=1e-6,  
...     maximize=False,  
... )
```

#### Note

- For distributed training, this API is compatible with torch/distributed/optim
- The foreach mode is generally preferred for GPU training
- Complex parameters are handled transparently by viewing as real
- First optimization step only initializes variance, doesn't update parameters

#### See also

- ADOPT class: High-level optimizer interface
- `_single_tensor_adopt`: Single-tensor implementation details
- `_multi_tensor_adopt`: Multi-tensor implementation details

## dicee.models.base\_model

### Classes

|                               |                                            |
|-------------------------------|--------------------------------------------|
| <code>BaseKGELightning</code> | Base class for all neural network modules. |
| <code>BaseKGE</code>          | Base class for all neural network modules. |
| <code>IdentityClass</code>    | Base class for all neural network modules. |

### Module Contents

```
class dicee.models.base_model.BaseKGELightning(*args, **kwargs)
```

Bases: `lightning.LightningModule`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn  
import torch.nn.functional as F
```

(continues on next page)

(continued from previous page)

```
class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**training\_step\_outputs** = []

**mem\_of\_model**() → Dict

Size of model in MB and number of params

**training\_step** (*batch, batch\_idx=None*)

Here you compute and return the training loss and some additional metrics for e.g. the progress bar or logger.

#### Parameters

- **batch** – The output of your data iterable, normally a `DataLoader`.
- **batch\_idx** – The index of this batch.
- **dataloader\_idx** – The index of the dataloader that produced this batch. (only if multiple dataloaders used)

#### Returns

- **Tensor** – The loss tensor
- **dict** – A dictionary which can include any keys, but must include the key 'loss' in the case of automatic optimization.
- **None** – In automatic optimization, this will skip to the next batch (but is not supported for multi-GPU, TPU, or DeepSpeed). For manual optimization, this has no special meaning, as returning the loss is not required.

In this step you'd normally do the forward pass and calculate the loss for a batch. You can also do fancier things like multiple forward passes or something model specific.

Example:

```
def training_step(self, batch, batch_idx):
    x, y, z = batch
    out = self.encoder(x)
    loss = self.loss(out, x)
    return loss
```

To use multiple optimizers, you can switch to ‘manual optimization’ and control their stepping:

```
def __init__(self):
    super().__init__()
    self.automatic_optimization = False

# Multiple optimizers (e.g.: GANs)
def training_step(self, batch, batch_idx):
    opt1, opt2 = self.optimizers()

    # do training_step with encoder
    ...
    opt1.step()
    # do training_step with decoder
    ...
    opt2.step()
```

#### Note

When `accumulate_grad_batches > 1`, the loss returned here will be automatically normalized by `accumulate_grad_batches` internally.

**loss\_function** (*yhat\_batch: torch.FloatTensor, y\_batch: torch.FloatTensor*)

#### Parameters

- **yhat\_batch**
- **y\_batch**

**on\_train\_epoch\_end** (*\*args, \*\*kwargs*)

Called in the training loop at the very end of the epoch.

To access all batch outputs at the end of the epoch, you can cache step outputs as an attribute of the `LightningModule` and access them in this hook:

```
class MyLightningModule(L.LightningModule):
    def __init__(self):
        super().__init__()
        self.training_step_outputs = []

    def training_step(self):
        loss = ...
        self.training_step_outputs.append(loss)
        return loss

    def on_train_epoch_end(self):
```

(continues on next page)

(continued from previous page)

```
# do something with all training_step outputs, for example:
epoch_mean = torch.stack(self.training_step_outputs).mean()
self.log("training_epoch_mean", epoch_mean)
# free up the memory
self.training_step_outputs.clear()
```

**test\_epoch\_end**(*outputs: List[Any]*)

**test\_dataloader**() → None

An iterable or collection of iterables specifying test samples.

For more information about multiple dataloaders, see this section.

For data processing use the following pattern:

- download in `prepare_data()`
- process and split in `setup()`

However, the above are only necessary for distributed processing.

#### **Warning**

do not assign state in `prepare_data`

- `test()`
- `prepare_data()`
- `setup()`

#### **Note**

Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.

#### **Note**

If you don't need a test dataset and a `test_step()`, you don't need to implement this method.

**val\_dataloader**() → None

An iterable or collection of iterables specifying validation samples.

For more information about multiple dataloaders, see this section.

The dataloader you return will not be reloaded unless you set **:param-ref:~lightning.pytorch.trainer.trainer.Trainer.reload\_dataloaders\_every\_n\_epochs** to a positive integer.

It's recommended that all data downloads and preparation happen in `prepare_data()`.

- `fit()`
- `validate()`

- `prepare_data()`
- `setup()`

#### Note

Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.

#### Note

If you don't need a validation dataset and a `validation_step()`, you don't need to implement this method.

**`predict_dataloader()`** → None

An iterable or collection of iterables specifying prediction samples.

For more information about multiple dataloaders, see this section.

It's recommended that all data downloads and preparation happen in `prepare_data()`.

- `predict()`
- `prepare_data()`
- `setup()`

#### Note

Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.

#### Returns

A `torch.utils.data.DataLoader` or a sequence of them specifying prediction samples.

**`train_dataloader()`** → None

An iterable or collection of iterables specifying training samples.

For more information about multiple dataloaders, see this section.

The dataloader you return will not be reloaded unless you set **`:param-ref:~lightning.pytorch.trainer.trainer.Trainer.reload_dataloaders_every_n_epochs``** to a positive integer.

For data processing use the following pattern:

- download in `prepare_data()`
- process and split in `setup()`

However, the above are only necessary for distributed processing.

### Warning

do not assign state in `prepare_data`

- `fit()`
- `prepare_data()`
- `setup()`

### Note

Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.

**`configure_optimizers`** (*parameters=None*)

Choose what optimizers and learning-rate schedulers to use in your optimization. Normally you'd need one. But in the case of GANs or similar you might have multiple. Optimization with multiple optimizers only works in the manual optimization mode.

#### Returns

Any of these 6 options.

- **Single optimizer.**
- **List or Tuple** of optimizers.
- **Two lists** - The first list has multiple optimizers, and the second has multiple LR schedulers (or multiple `lr_scheduler_config`).
- **Dictionary**, with an "optimizer" key, and (optionally) a "lr\_scheduler" key whose value is a single LR scheduler or `lr_scheduler_config`.
- **None** - Fit will run without any optimizer.

The `lr_scheduler_config` is a dictionary which contains the scheduler and its associated configuration. The default configuration is shown below.

```
lr_scheduler_config = {
    # REQUIRED: The scheduler instance
    "scheduler": lr_scheduler,
    # The unit of the scheduler's step size, could also be 'step'.
    # 'epoch' updates the scheduler on epoch end whereas 'step'
    # updates it after a optimizer update.
    "interval": "epoch",
    # How many epochs/steps should pass between calls to
    # `scheduler.step()`. 1 corresponds to updating the learning
    # rate after every epoch/step.
    "frequency": 1,
    # Metric to monitor for schedulers like `ReduceLROnPlateau`
    "monitor": "val_loss",
    # If set to `True`, will enforce that the value specified 'monitor'
    # is available when the scheduler is updated, thus stopping
    # training if not found. If set to `False`, it will only produce a warning
```

(continues on next page)



(continued from previous page)

```
"strict": True,
# If using the `LearningRateMonitor` callback to monitor the
# learning rate progress, this keyword can be used to specify
# a custom logged name
"name": None,
}
```

When there are schedulers in which the `.step()` method is conditioned on a value, such as the `torch.optim.lr_scheduler.ReduceLROnPlateau` scheduler, Lightning requires that the `lr_scheduler_config` contains the keyword "monitor" set to the metric name that the scheduler should be conditioned on.

Metrics can be made available to monitor by simply logging it using `self.log('metric_to_track', metric_val)` in your `LightningModule`.

#### Note

Some things to know:

- Lightning calls `.backward()` and `.step()` automatically in case of automatic optimization.
- If a learning rate scheduler is specified in `configure_optimizers()` with key "interval" (default "epoch") in the scheduler configuration, Lightning will call the scheduler's `.step()` method automatically in case of automatic optimization.
- If you use 16-bit precision (`precision=16`), Lightning will automatically handle the optimizer.
- If you use `torch.optim.LBFGS`, Lightning handles the closure function automatically for you.
- If you use multiple optimizers, you will have to switch to 'manual optimization' mode and step them yourself.
- If you need to control how often the optimizer steps, override the `optimizer_step()` hook.

```
class dicee.models.base_model.BaseKGE (args: dict)
```

Bases: `BaseKGELightning`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
```

(continues on next page)

```
x = F.relu(self.conv1(x))
return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

### Variables

`training (bool)` – Boolean represents whether this module is in training or evaluation mode.

#### args

`embedding_dim = None`

`num_entities = None`

`num_relations = None`

`num_tokens = None`

`learning_rate = None`

`apply_unit_norm = None`

`input_dropout_rate = None`

`hidden_dropout_rate = None`

`optimizer_name = None`

`feature_map_dropout_rate = None`

`kernel_size = None`

`num_of_output_channels = None`

`weight_decay = None`

#### loss

`selected_optimizer = None`

`normalizer_class = None`

`normalize_head_entity_embeddings`

`normalize_relation_embeddings`

`normalize_tail_entity_embeddings`

`hidden_normalizer`

```

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

forward_byte_pair_encoded_k_vs_all (x: torch.LongTensor)

    Parameters
         $\mathbf{x} (B \times 2 \times T)$ 

forward_byte_pair_encoded_triple (x: Tuple[torch.LongTensor, torch.LongTensor])
    byte pair encoded neural link predictors

    Parameters
    -----

init_params_with_sanity_checking ()

forward (x: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor],
        y_idx: torch.LongTensor = None)

    Parameters

        •  $\mathbf{x}$ 

        •  $\mathbf{y\_idx}$ 

        •  $\mathbf{ordered\_bpe\_entities}$ 

forward_triples (x: torch.LongTensor)  $\rightarrow$  torch.Tensor

    Parameters

         $\mathbf{x}$ 

forward_k_vs_all (*args, **kwargs)

forward_k_vs_sample (*args, **kwargs)

get_triple_representation (idx_hrt)

get_head_relation_representation (indexed_triple)

get_sentence_representation (x: torch.LongTensor)

    Parameters

        • ( $\mathbf{b} (x \text{ shape})$ )

        • 3

        •  $\mathbf{t}$ )

```

```
get_bpe_head_and_relation_representation(x: torch.LongTensor)
    → Tuple[torch.FloatTensor, torch.FloatTensor]
```

#### Parameters

$\mathbf{x} (B \times 2 \times T)$

```
get_embeddings() → Tuple[numpy.ndarray, numpy.ndarray]
```

```
class dicee.models.base_model.IdentityClass (args=None)
```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

`training (bool)` – Boolean represents whether this module is in training or evaluation mode.

`args = None`

`__call__(x)`

`static forward(x)`

`dicee.models.clifford`

#### Classes

*Keci*

Base class for all neural network modules.

continues on next page

Table 27 – continued from previous page

|                                 |                                                         |
|---------------------------------|---------------------------------------------------------|
| <i>KeciTransformer</i>          | Keci with Transformer architecture.                     |
| <i>TransformerBlock</i>         | A single transformer block with self-attention and MLP. |
| <i>TransformerSelfAttention</i> | Multi-head self-attention for the Keci Transformer.     |
| <i>TransformerMLP</i>           | MLP for the Keci Transformer.                           |
| <i>CKeci</i>                    | Without learning dimension scaling                      |
| <i>DeCaL</i>                    | Base class for all neural network modules.              |

## Module Contents

**class** dicee.models.clifford.**Keci**(args)

Bases: *dicee.models.base\_model.BaseKGE*

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**name** = 'Keci'

**p**

**q**

**r**

**requires\_grad\_for\_interactions** = True

**compute\_sigma\_pp** (*hp, rp*)

Compute  $\sigma_{pp} = \sum_{i=1}^{p-1} \sum_{k=i+1}^p (h_{i,r_k} - h_{k,r_i}) e_i e_k$

$\sigma_{pp}$  captures the interactions between along  $p$  bases For instance, let  $p = e_1, e_2, e_3$ , we compute interactions between  $e_1 e_2, e_1 e_3$ , and  $e_2 e_3$  This can be implemented with a nested two for loops

results = [] for i in range(p - 1):

**for k in range(i + 1, p):**

        results.append(hp[:, :, i] \* rp[:, :, k] - hp[:, :, k] \* rp[:, :, i])

sigma\_pp = torch.stack(results, dim=2) assert sigma\_pp.shape == (b, r, int((p \* (p - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all  $p$ , e.g.,  $e_1 e_1, e_1 e_2, e_1 e_3$ ,

$e_2 e_1, e_2 e_2, e_2 e_3, e_3 e_1, e_3 e_2, e_3 e_3$

Then select the triangular matrix without diagonals:  $e_1 e_2, e_1 e_3, e_2 e_3$ .

**compute\_sigma\_qq** (*hq, rq*)

Compute  $\sigma_{qq} = \sum_{j=1}^{p+q-1} \sum_{k=j+1}^{p+q} (h_{j,r_k} - h_{k,r_j}) e_j e_k$   $\sigma_{qq}$  captures the interactions between along  $q$  bases For instance, let  $q = e_1, e_2, e_3$ , we compute interactions between  $e_1 e_2, e_1 e_3$ , and  $e_2 e_3$  This can be implemented with a nested two for loops

results = [] for j in range(q - 1):

**for k in range(j + 1, q):**

        results.append(hq[:, :, j] \* rq[:, :, k] - hq[:, :, k] \* rq[:, :, j])

sigma\_qq = torch.stack(results, dim=2) assert sigma\_qq.shape == (b, r, int((q \* (q - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all  $p$ , e.g.,  $e_1 e_1, e_1 e_2, e_1 e_3$ ,

$e_2 e_1, e_2 e_2, e_2 e_3, e_3 e_1, e_3 e_2, e_3 e_3$

Then select the triangular matrix without diagonals:  $e_1 e_2, e_1 e_3, e_2 e_3$ .

**compute\_sigma\_pq** (\*, *hp, hq, rp, rq*)

$\sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_{i,r_j} - h_{j,r_i}) e_i e_j$

results = [] sigma\_pq = torch.zeros(b, r, p, q) for i in range(p):

**for j in range(q):**

        sigma\_pq[:, :, i, j] = hp[:, :, i] \* rq[:, :, j] - hq[:, :, j] \* rp[:, :, i]

print(sigma\_pq.shape)

**apply\_coefficients** (*hp, hq, rp, rq*)

Multiplying a base vector with its scalar coefficient

**clifford\_multiplication** (*h0, hp, hq, r0, rp, rq*)

Compute our CL multiplication

$h = h_0 + \sum_{i=1}^p h_i e_i + \sum_{j=p+1}^{p+q} h_j e_j$   $r = r_0 + \sum_{i=1}^p r_i e_i + \sum_{j=p+1}^{p+q} r_j e_j$

$e_i^2 = +1$  for  $i \leq p$   $e_j^2 = -1$  for  $p < j \leq p+q$   $e_i e_j = -e_j e_i$  for  $i$

$e_j$

$h r = \sigma_0 + \sigma_p + \sigma_q + \sigma_{pp} + \sigma_{qq} + \sigma_{pq}$  where

(1)  $\sigma_0 = h_0 r_0 + \sum_{i=1}^p (h_0 r_i - h_i r_0) e_i - \sum_{j=p+1}^{p+q} (h_j r_j) e_j$

- (2)  $\sigma_p = \sum_{i=1}^p (h_{0r_i} + h_{ir_0}) e_i$
- (3)  $\sigma_q = \sum_{j=p+1}^{p+q} (h_{0r_j} + h_{jr_0}) e_j$
- (4)  $\sigma_{pp} = \sum_{i=1}^{p-1} \sum_{k=i+1}^p (h_{ir_k} - h_{kr_i}) e_i e_k$
- (5)  $\sigma_{qq} = \sum_{j=1}^{p+q-1} \sum_{k=j+1}^{p+q} (h_{jr_k} - h_{kr_j}) e_j e_k$
- (6)  $\sigma_{pq} = \sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_{ir_j} - h_{jr_i}) e_i e_j$

**construct\_cl\_multivector** (*x: torch.FloatTensor, r: int, p: int, q: int*)  
 $\rightarrow$  tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]  
Construct a batch of multivectors  $Cl_{\{p,q\}}(\mathbb{R}^d)$

### Parameter

x: torch.FloatTensor with (n,d) shape

#### returns

- **a0** (*torch.FloatTensor with (n,r) shape*)
- **ap** (*torch.FloatTensor with (n,r,p) shape*)
- **aq** (*torch.FloatTensor with (n,r,q) shape*)

**forward\_k\_vs\_with\_explicit** (*x: torch.Tensor*)  
**k\_vs\_all\_score** (*bpe\_head\_ent\_emb, bpe\_rel\_ent\_emb, E*)  
**forward\_k\_vs\_all** (*x: torch.Tensor*)  $\rightarrow$  torch.FloatTensor

Kvsall training

- (1) Retrieve real-valued embedding vectors for heads and relations  $\mathbb{R}^d$ .
- (2) Construct head entity and relation embeddings according to  $Cl_{\{p,q\}}(\mathbb{R}^d)$ .
- (3) Perform Cl multiplication
- (4) Inner product of (3) and all entity embeddings

**forward\_k\_vs\_with\_explicit** and this functions are identical Parameter ——— x: torch.LongTensor with (n,2) shape :rtype: torch.FloatTensor with (n, **IEI**) shape

**construct\_batch\_selected\_cl\_multivector** (*x: torch.FloatTensor, r: int, p: int, q: int*)  
 $\rightarrow$  tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]  
Construct a batch of batchs multivectors  $Cl_{\{p,q\}}(\mathbb{R}^d)$

### Parameter

x: torch.FloatTensor with (n,k, d) shape

#### returns

- **a0** (*torch.FloatTensor with (n,k, m) shape*)
- **ap** (*torch.FloatTensor with (n,k, m, p) shape*)
- **aq** (*torch.FloatTensor with (n,k, m, q) shape*)

**forward\_k\_vs\_sample** (*x: torch.LongTensor, target\_entity\_idx: torch.LongTensor*)  $\rightarrow$  torch.FloatTensor

### Parameter

x: torch.LongTensor with (n,2) shape

target\_entity\_idx: torch.LongTensor with (n, k ) shape k denotes the selected number of examples.

**rtype**

torch.FloatTensor with (n, k) shape

**score** (*h, r, t*)

**forward\_triples** (*x: torch.Tensor*) → torch.FloatTensor

### Parameter

x: torch.LongTensor with (n,3) shape

**rtype**

torch.FloatTensor with (n) shape

**class** dicee.models.clifford.**KeciTransformer** (*args*)

Bases: *Keci*

Keci with Transformer architecture.

Concatenates h0, hp, hq, r0, rp, rq into a single embedding vector and processes through transformer.

**name** = 'KeciTransformer'

**use\_clifford\_mul**

**seq\_len** = 1

**transformer**

**lm\_head**

**forward\_k\_vs\_all** (*x: torch.Tensor*) → torch.FloatTensor

Kvsall training

### Parameter

x: torch.LongTensor with (n,2) shape

**rtype**

torch.FloatTensor with (n, **IEI**) shape

**class** dicee.models.clifford.**TransformerBlock** (*n\_embd, n\_head, dropout=0.0, bias=False*)

Bases: torch.nn.Module

A single transformer block with self-attention and MLP.

**ln\_1**

**attn**

**ln\_2**

**mlp**

**forward** (*x*)



```
class dicee.models.clifford.TransformerSelfAttention(n_embd, n_head, dropout=0.0,  

bias=False)
```

Bases: `torch.nn.Module`

Multi-head self-attention for the Keci Transformer.

**c\_attn**

**c\_proj**

**attn\_dropout**

**resid\_dropout**

**n\_head**

**n\_embd**

**dropout = 0.0**

**forward**(*x*)

```
class dicee.models.clifford.TransformerMLP(n_embd, dropout=0.0, bias=False)
```

Bases: `torch.nn.Module`

MLP for the Keci Transformer.

**c\_fc**

**gelu**

**c\_proj**

**dropout**

**forward**(*x*)

```
class dicee.models.clifford.CKeci(args)
```

Bases: *Keci*

Without learning dimension scaling

**name = 'CKeci'**

**requires\_grad\_for\_interactions = False**

```
class dicee.models.clifford.DeCaL(args)
```

Bases: `dicee.models.base_model.BaseKGE`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
```

(continues on next page)

(continued from previous page)

```
def __init__(self) -> None:
    super().__init__()
    self.conv1 = nn.Conv2d(1, 20, 5)
    self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**name** = 'DeCaL'

**entity\_embeddings**

**relation\_embeddings**

**p**

**q**

**r**

**re**

**forward\_triples** (*x: torch.Tensor*) → torch.FloatTensor

### Parameter

**x**: torch.LongTensor with (n, ) shape

**rtype**

torch.FloatTensor with (n) shape

**cl\_pqr** (*a: torch.tensor*) → torch.tensor

Input: tensor(batch\_size, emb\_dim) → output: tensor with 1+p+q+r components with size (batch\_size, emb\_dim/(1+p+q+r)) each.

1) takes a tensor of size (batch\_size, emb\_dim), split it into 1 + p + q + r components, hence 1+p+q+r must be a divisor of the emb\_dim. 2) Return a list of the 1+p+q+r components vectors, each are tensors of size (batch\_size, emb\_dim/(1+p+q+r))

**compute\_sigmas\_single** (*list\_h\_emb, list\_r\_emb, list\_t\_emb*)

here we compute all the sums with no others vectors interaction taken with the scalar product with t, that is,

$$s0 = h_0 r_0 t_0 s1 = \sum_{i=1}^p h_i r_i t_0 s2 = \sum_{j=p+1}^{p+q} h_j r_j t_0 s3 = \sum_{i=1}^q (h_0 r_i t_i + h_i r_0 t_i) s4 = \sum_{i=p+1}^{p+q} (h_0 r_i t_i + h_i r_0 t_i) s5 = \sum_{i=p+q+1}^{p+q+r} (h_0 r_i t_i + h_i r_0 t_i)$$

and return:

$$\sigma_0 t = \sigma_0 \cdot t_0 = s0 + s1 - s2 s3, s4 \text{ and } s5$$

**compute\_sigmas\_multivect** (*list\_h\_emb, list\_r\_emb*)

Here we compute and return all the sums with vectors interaction for the same and different bases.

For same bases vectors interaction we have

$$\sigma_p p = \sum_{i=1}^{p-1} \sum_{i'=i+1}^p (h_i r_{i'} - h_{i'} r_i) (\text{model the interactions between } e_i \text{ and } e_{i'} \text{ for } 1 \leq i, i' \leq p) \sigma_q q = \sum_{j=p+1}^{p+q-1} \sum_{j'=j+1}^{p+q} (h_j r_{j'} - h_{j'} r_j)$$

For different base vector interactions, we have

$$\sigma_p q = \sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) (\text{interactions between } e_i \text{ and } e_j \text{ for } 1 \leq i \leq p \text{ and } p+1 \leq j \leq p+q) \sigma_p r = \sum_{i=1}^p \sum_{j=p+q+1}^{p+q+r} (h_i r_j - h_j r_i)$$

**forward\_k\_vs\_all** (*x: torch.Tensor*) → torch.FloatTensor

Kvsall training

- (1) Retrieve real-valued embedding vectors for heads and relations
- (2) Construct head entity and relation embeddings according to  $Cl_{\{p,q,r\}}(\mathbb{R}^d)$ .
- (3) Perform Cl multiplication
- (4) Inner product of (3) and all entity embeddings

**forward\_k\_vs\_with\_explicit** and this functions are identical Parameter ——— x: torch.LongTensor with (n, ) shape :rtype: torch.FloatTensor with (n, **IEI**) shape

**apply\_coefficients** (*h0, hp, hq, hk, r0, rp, rq, rk*)

Multiplying a base vector with its scalar coefficient

**construct\_cl\_multivector** (*x: torch.FloatTensor, re: int, p: int, q: int, r: int*)  
→ tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Construct a batch of multivectors  $Cl_{\{p,q,r\}}(\mathbb{R}^d)$

## Parameter

x: torch.FloatTensor with (n,d) shape

**returns**

- **a0** (*torch.FloatTensor*)
- **ap** (*torch.FloatTensor*)
- **aq** (*torch.FloatTensor*)
- **ar** (*torch.FloatTensor*)

**compute\_sigma\_pp** (*hp, rp*)

Compute .. math:

$$\sigma_{p,p} = \sum_{i=1}^{p-1} \sum_{i'=i+1}^p (x_i y_{i'} - x_{i'} y_i)$$

$\sigma_{pp}$  captures the interactions between along  $p$  bases For instance, let  $p$  e\_1, e\_2, e\_3, we compute interactions between e\_1 e\_2, e\_1 e\_3, and e\_2 e\_3 This can be implemented with a nested two for loops

results = [] for i in range(p - 1):

**for k in range(i + 1, p):**

        results.append(hp[:, :, i] \* rp[:, :, k] - hp[:, :, k] \* rp[:, :, i])

sigma\_pp = torch.stack(results, dim=2) assert sigma\_pp.shape == (b, r, int((p \* (p - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all  $p$ , e.g., e1e1, e1e2, e1e3,

e2e1, e2e2, e2e3, e3e1, e3e2, e3e3

Then select the triangular matrix without diagonals: e1e2, e1e3, e2e3.

**compute\_sigma\_qq** (*hq, rq*)

Compute

$$\sigma_{q,q}^* = \sum_{j=p+1}^{p+q-1} \sum_{j'=j+1}^{p+q} (x_j y_{j'} - x_{j'} y_j) Eq.16$$

$\sigma_{qq}$  captures the interactions between along  $q$  bases For instance, let  $q$  e\_1, e\_2, e\_3, we compute interactions between e\_1 e\_2, e\_1 e\_3, and e\_2 e\_3 This can be implemented with a nested two for loops

results = [] for j in range(q - 1):

**for k in range(j + 1, q):**

        results.append(hq[:, :, j] \* rq[:, :, k] - hq[:, :, k] \* rq[:, :, j])

sigma\_qq = torch.stack(results, dim=2) assert sigma\_qq.shape == (b, r, int((q \* (q - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all  $p$ , e.g., e1e1, e1e2, e1e3,

e2e1, e2e2, e2e3, e3e1, e3e2, e3e3

Then select the triangular matrix without diagonals: e1e2, e1e3, e2e3.

**compute\_sigma\_rr** (*hk, rk*)

$$\sigma_{r,r}^* = \sum_{k=p+q+1}^{p+q+r-1} \sum_{k'=k+1}^p (x_k y_{k'} - x_{k'} y_k)$$

**compute\_sigma\_pq** (\*, *hp, hq, rp, rq*)

Compute

$$\sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

results = [] sigma\_pq = torch.zeros(b, r, p, q) for i in range(p):

**for j in range(q):**

        sigma\_pq[:, :, i, j] = hp[:, :, i] \* rq[:, :, j] - hq[:, :, j] \* rp[:, :, i]

```

print(sigma_pq.shape)
compute_sigma_pr(* , hp, hk, rp, rk)
    Compute

```

$$\sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

```

results = []
sigma_pq = torch.zeros(b, r, p, q)
for i in range(p):
    for j in range(q):
        sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]
print(sigma_pq.shape)
compute_sigma_qr(* , hq, hk, rq, rk)

```

$$\sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

```

results = []
sigma_pq = torch.zeros(b, r, p, q)
for i in range(p):
    for j in range(q):
        sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]
print(sigma_pq.shape)

```

## dicее.models.complex

### Classes

|                |                                                           |
|----------------|-----------------------------------------------------------|
| <i>ConEx</i>   | Convolutional ComplEx Knowledge Graph Embeddings          |
| <i>AConEx</i>  | Additive Convolutional ComplEx Knowledge Graph Embeddings |
| <i>ComplEx</i> | Base class for all neural network modules.                |

### Module Contents

```

class dicее.models.complex.ConEx(args)
    Bases: dicее.models.base_model.BaseKGE
    Convolutional ComplEx Knowledge Graph Embeddings
    name = 'ConEx'
    conv2d
    fc_num_input
    fc1
    norm_fc1
    bn_conv2d
    feature_map_dropout

```

```
residual_convolution (C_1: Tuple[torch.Tensor, torch.Tensor],
                       C_2: Tuple[torch.Tensor, torch.Tensor]) → torch.FloatTensor
```

Compute residual score of two complex-valued embeddings. :param C\_1: a tuple of two pytorch tensors that corresponds complex-valued embeddings :param C\_2: a tuple of two pytorch tensors that corresponds complex-valued embeddings :return:

```
forward_k_vs_all (x: torch.Tensor) → torch.FloatTensor
```

```
forward_triples (x: torch.Tensor) → torch.FloatTensor
```

#### Parameters

**x**

```
forward_k_vs_sample (x: torch.Tensor, target_entity_idx: torch.Tensor)
```

```
class dicee.models.complex.AConEx (args)
```

Bases: [dicee.models.base\\_model.BaseKGE](#)

Additive Convolutional ComplEx Knowledge Graph Embeddings

```
name = 'AConEx'
```

```
conv2d
```

```
fc_num_input
```

```
fc1
```

```
norm_fc1
```

```
bn_conv2d
```

```
feature_map_dropout
```

```
residual_convolution (C_1: Tuple[torch.Tensor, torch.Tensor],
                       C_2: Tuple[torch.Tensor, torch.Tensor]) → torch.FloatTensor
```

Compute residual score of two complex-valued embeddings. :param C\_1: a tuple of two pytorch tensors that corresponds complex-valued embeddings :param C\_2: a tuple of two pytorch tensors that corresponds complex-valued embeddings :return:

```
forward_k_vs_all (x: torch.Tensor) → torch.FloatTensor
```

```
forward_triples (x: torch.Tensor) → torch.FloatTensor
```

#### Parameters

**x**

```
forward_k_vs_sample (x: torch.Tensor, target_entity_idx: torch.Tensor)
```

```
class dicee.models.complex.Complex (args)
```

Bases: [dicee.models.base\\_model.BaseKGE](#)

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**name** = 'Complex'

**static score** (*head\_ent\_emb: torch.FloatTensor, rel\_ent\_emb: torch.FloatTensor, tail\_ent\_emb: torch.FloatTensor*)

**static k\_vs\_all\_score** (*emb\_h: torch.FloatTensor, emb\_r: torch.FloatTensor, emb\_E: torch.FloatTensor*)

### Parameters

- **emb\_h**
- **emb\_r**
- **emb\_E**

**forward\_k\_vs\_all** (*x: torch.LongTensor*)  $\rightarrow$  *torch.FloatTensor*

**forward\_k\_vs\_sample** (*x: torch.LongTensor, target\_entity\_idx: torch.LongTensor*)

**dicee.models.dualE**

### Classes

*DualE*

Dual Quaternion Knowledge Graph Embeddings  
(<https://ojs.aaai.org/index.php/AAAI/article/download/16850/16657>)

## Module Contents

**class** `dicee.models.dualE.DualE` (*args*)

Bases: `dicee.models.base_model.BaseKGE`

Dual Quaternion Knowledge Graph Embeddings (<https://ojs.aaai.org/index.php/AAAI/article/download/16850/16657>)

**name** = 'DualE'

**entity\_embeddings**

**relation\_embeddings**

**num\_ent** = None

**kvsall\_score** (*e\_1\_h, e\_2\_h, e\_3\_h, e\_4\_h, e\_5\_h, e\_6\_h, e\_7\_h, e\_8\_h, e\_1\_t, e\_2\_t, e\_3\_t, e\_4\_t, e\_5\_t, e\_6\_t, e\_7\_t, e\_8\_t, r\_1, r\_2, r\_3, r\_4, r\_5, r\_6, r\_7, r\_8*) → torch.tensor

KvsAll scoring function

### Input

x: torch.LongTensor with (n, ) shape

### Output

torch.FloatTensor with (n) shape

**forward\_triples** (*idx\_triple: torch.tensor*) → torch.tensor

Negative Sampling forward pass:

### Input

x: torch.LongTensor with (n, ) shape

### Output

torch.FloatTensor with (n) shape

**forward\_k\_vs\_all** (*x*)

KvsAll forward pass

### Input

x: torch.LongTensor with (n, ) shape

### Output

torch.FloatTensor with (n) shape

**T** (*x: torch.tensor*) → torch.tensor

Transpose function

Input: Tensor with shape (nxm) Output: Tensor with shape (mxn)



## dicee.models.ensemble

### Classes

---

|                    |
|--------------------|
| <i>EnsembleKGE</i> |
|--------------------|

---

### Module Contents

**class** `dicee.models.ensemble.EnsembleKGE` (*models: list = None, seed\_model=None, pretrained\_models: List = None*)

`name`

`train_mode = True`

`args`

`named_children()`

`property example_input_array`

`parameters()`

`modules()`

`__iter__()`

`__len__()`

`eval()`

`to(device)`

`state_dict()`  
Return the state dict of the ensemble.

`load_state_dict(state_dict, strict=True)`  
Load the state dict into the ensemble.

`mem_of_model()`

`__call__(x_batch)`

`step()`

`get_embeddings()`

`__str__()`

## dicee.models.function\_space

### Classes

---

|               |                                  |
|---------------|----------------------------------|
| <i>FMult</i>  | Learning Knowledge Neural Graphs |
| <i>GFMult</i> | Learning Knowledge Neural Graphs |

---

continues on next page

Table 31 – continued from previous page

|                |                                                                                                                 |
|----------------|-----------------------------------------------------------------------------------------------------------------|
| <i>FMult2</i>  | Learning Knowledge Neural Graphs                                                                                |
| <i>LFMult1</i> | Embedding with trigonometric functions. We represent all entities and relations in the complex number space as: |
| <i>LFMult</i>  | Embedding with polynomial functions. We represent all entities and relations in the polynomial space as:        |

## Module Contents

```

class dicee.models.function_space.FMult(args)
    Bases: dicee.models.base_model.BaseKGE
    Learning Knowledge Neural Graphs
    name = 'FMult'
    entity_embeddings
    relation_embeddings
    k
    num_sample = 50
    gamma
    roots
    weights
    compute_func(weights: torch.FloatTensor, x) → torch.FloatTensor
    chain_func(weights, x: torch.FloatTensor)
    forward_triples(idx_triple: torch.Tensor) → torch.Tensor

    Parameters
    x

class dicee.models.function_space.GFMult(args)
    Bases: dicee.models.base_model.BaseKGE
    Learning Knowledge Neural Graphs
    name = 'GFMult'
    entity_embeddings
    relation_embeddings
    k
    num_sample = 250
    roots
    weights
    compute_func(weights: torch.FloatTensor, x) → torch.FloatTensor

```

**chain\_func** (*weights, x: torch.FloatTensor*)

**forward\_triples** (*idx\_triple: torch.Tensor*) → torch.Tensor

#### Parameters

**x**

**class** dicee.models.function\_space.**FMult2** (*args*)

Bases: *dicee.models.base\_model.BaseKGE*

Learning Knowledge Neural Graphs

**name** = 'FMult2'

**n\_layers** = 3

**k**

**n** = 50

**score\_func** = 'compositional'

**discrete\_points**

**entity\_embeddings**

**relation\_embeddings**

**build\_func** (*Vec*)

**build\_chain\_funcs** (*list\_Vec*)

**compute\_func** (*W, b, x*) → torch.FloatTensor

**function** (*list\_W, list\_b*)

**trapezoid** (*list\_W, list\_b*)

**forward\_triples** (*idx\_triple: torch.Tensor*) → torch.Tensor

#### Parameters

**x**

**class** dicee.models.function\_space.**LFMult1** (*args*)

Bases: *dicee.models.base\_model.BaseKGE*

Embedding with trigonometric functions. We represent all entities and relations in the complex number space as:  $f(x) = \sum_{k=0}^{d-1} w_k e^{kix}$ . and use the three differents scoring function as in the paper to evaluate the score

**name** = 'LFMult1'

**entity\_embeddings**

**relation\_embeddings**

**forward\_triples** (*idx\_triple*)

#### Parameters

**x**

```

tri_score (h, r, t)

vtp_score (h, r, t)

class dicee.models.function_space.LFMult (args)
    Bases: dicee.models.base_model.BaseKGE

    Embedding with polynomial functions. We represent all entities and relations in the polynomial space as:  $f(x) = \sum_{i=0}^{d-1} a_i x^i$  and use the three different scoring function as in the paper to evaluate the score. We also consider combining with Neural Networks.

    name = 'LFMult'

    entity_embeddings

    relation_embeddings

    degree

    m

    x_values

    forward_triples (idx_triple)

        Parameters
        x

    construct_multi_coeff (x)

    poly_NN (x, coefh, coefr, coeft)
        Constructing a 2 layers NN to represent the embeddings.  $h = \sigma(w_h^T x + b_h)$ ,  $r = \sigma(w_r^T x + b_r)$ ,  $t = \sigma(w_t^T x + b_t)$ 

    linear (x, w, b)

    scalar_batch_NN (a, b, c)
        element wise multiplication between a,b and c: Inputs : a, b, c ==> torch.tensor of size batch_size x m x d
        Output : a tensor of size batch_size x d

    tri_score (coeff_h, coeff_r, coeff_t)
        this part implement the trilinear scoring techniques:

        
$$\text{score}(h,r,t) = \int_0^1 h(x)r(x)t(x) dx = \sum_{i,j,k=0}^{d-1} \text{dfrac}\{a_i*b_j*c_k\}\{1+(i+j+k)\%d\}$$

        1. generate the range for i,j and k from [0 d-1]
        2. perform  $\text{dfrac}\{a_i*b_j*c_k\}\{1+(i+j+k)\%d\}$  in parallel for every batch
        3. take the sum over each batch

    vtp_score (h, r, t)
        this part implement the vector triple product scoring techniques:

        
$$\text{score}(h,r,t) = \int_0^1 h(x)r(x)t(x) dx = \sum_{i,j,k=0}^{d-1} \text{dfrac}\{a_i*c_j*b_k - b_i*c_j*a_k\}\{(1+(i+j)\%d)(1+k)\}$$

        1. generate the range for i,j and k from [0 d-1]
        2. Compute the first and second terms of the sum
        3. Multiply with then denominator and take the sum

```

4. take the sum over each batch

**comp\_func** (*h, r, t*)

this part implement the function composition scoring techniques: i.e. score = <hor, t>

**polynomial** (*coeff, x, degree*)

This function takes a matrix tensor of coefficients (*coeff*), a tensor vector of points *x* and range of integer  $[0, 1, \dots, d]$  and return a vector tensor ( $\text{coeff}[0][0] + \text{coeff}[0][1]x + \dots + \text{coeff}[0][d]x^d$ ,

$\text{coeff}[1][0] + \text{coeff}[1][1]x + \dots + \text{coeff}[1][d]x^d$ )

**pop** (*coeff, x, degree*)

This function allow us to evaluate the composition of two polynomes without for loops :) it takes a matrix tensor of coefficients (*coeff*), a matrix tensor of points *x* and range of integer  $[0, 1, \dots, d]$

and return a tensor ( $\text{coeff}[0][0] + \text{coeff}[0][1]x + \dots + \text{coeff}[0][d]x^d$ ,

$\text{coeff}[1][0] + \text{coeff}[1][1]x + \dots + \text{coeff}[1][d]x^d$ )

## dicee.models.literal

### Classes

*LiteralEmbeddings*

A model for learning and predicting numerical literals using pre-trained KGE.

### Module Contents

**class** `dicee.models.literal.LiteralEmbeddings` (*num\_of\_data\_properties: int, embedding\_dims: int, entity\_embeddings: torch.tensor, dropout: float = 0.3, gate\_residual=True, freeze\_entity\_embeddings=True*)

Bases: `torch.nn.Module`

A model for learning and predicting numerical literals using pre-trained KGE.

**num\_of\_data\_properties**

Number of data properties (attributes).

**Type**

int

**embedding\_dims**

Dimension of the embeddings.

**Type**

int

**entity\_embeddings**

Pre-trained entity embeddings.

**Type**

torch.tensor

**dropout**

Dropout rate for regularization.

**Type**

float

**gate\_residual**

Whether to use gated residual connections.

**Type**

bool

**freeze\_entity\_embeddings**

Whether to freeze the entity embeddings during training.

**Type**

bool

**embedding\_dim**

**num\_of\_data\_properties**

**hidden\_dim**

**gate\_residual = True**

**freeze\_entity\_embeddings = True**

**entity\_embeddings**

**data\_property\_embeddings**

**fc**

**fc\_out**

**dropout**

**gated\_residual\_proj**

**layer\_norm**

**forward**(*entity\_idx*, *attr\_idx*)

**Parameters**

- **entity\_idx** (*Tensor*) – Entity indices (batch).
- **attr\_idx** (*Tensor*) – Attribute (Data property) indices (batch).

**Returns**

scalar predictions.

**Return type**

Tensor

**property device**

## **dicee.models.octonion**

### **Classes**

|              |                                            |
|--------------|--------------------------------------------|
| <i>OMult</i> | Base class for all neural network modules. |
| <i>ConvO</i> | Base class for all neural network modules. |

continues on next page

Table 33 – continued from previous page

|               |                                                            |
|---------------|------------------------------------------------------------|
| <i>AConvO</i> | Additive Convolutional Octonion Knowledge Graph Embeddings |
|---------------|------------------------------------------------------------|

## Functions

|                                             |
|---------------------------------------------|
| <code>octonion_mul(*, O_1, O_2)</code>      |
| <code>octonion_mul_norm(*, O_1, O_2)</code> |

## Module Contents

`dicee.models.octonion.octonion_mul(*, O_1, O_2)`

`dicee.models.octonion.octonion_mul_norm(*, O_1, O_2)`

**class** `dicee.models.octonion.OMult(args)`

Bases: `dicee.models.base_model.BaseKGE`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

## Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**name** = 'OMult'

```
static octonion_normalizer(emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4,
                           emb_rel_e5, emb_rel_e6, emb_rel_e7)
```

```
score(head_ent_emb: torch.FloatTensor, rel_ent_emb: torch.FloatTensor,
      tail_ent_emb: torch.FloatTensor)
```

```
k_vs_all_score(bpe_head_ent_emb, bpe_rel_ent_emb, E)
```

```
forward_k_vs_all(x)
```

Completed. Given a head entity and a relation (h,r), we compute scores for all possible triples,i.e.,  
[score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and  
relations => shape (size of batch,| Entities|)

```
class dicee.models.octonion.ConvO(args: dict)
```

Bases: `dicee.models.base_model.BaseKGE`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**name** = 'ConvO'

**conv2d**

**fc\_num\_input**

**fc1**



```

bn_conv2d

norm_fc1

feature_map_dropout

static octonion_normalizer(emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4,
                           emb_rel_e5, emb_rel_e6, emb_rel_e7)

residual_convolution(O_1, O_2)

forward_triples(x: torch.Tensor) → torch.Tensor

```

#### Parameters

**x**

```
forward_k_vs_all(x: torch.Tensor)
```

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch, **Entities**)

```
class dicee.models.octonion.AConvO(args: dict)
```

Bases: `dicee.models.base_model.BaseKGE`

Additive Convolutional Octonion Knowledge Graph Embeddings

**name** = 'AConvO'

**conv2d**

**fc\_num\_input**

**fc1**

**bn\_conv2d**

**norm\_fc1**

**feature\_map\_dropout**

```
static octonion_normalizer(emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4,
                           emb_rel_e5, emb_rel_e6, emb_rel_e7)
```

```
residual_convolution(O_1, O_2)
```

```
forward_triples(x: torch.Tensor) → torch.Tensor
```

#### Parameters

**x**

```
forward_k_vs_all(x: torch.Tensor)
```

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch, **Entities**)

**dicee.models.pykeen\_models**

## Classes

## Module Contents

**class** `dicee.models.pykeen_models.PykeenKGE` (*args: dict*)

Bases: `dicee.models.base_model.BaseKGE`

A class for using knowledge graph embedding models implemented in Pykeen

Notes: Pykeen\_DistMult: C Pykeen\_ComplEx: Pykeen\_QuatE: Pykeen\_MuRE: Pykeen\_CP: Pykeen\_HolE: Pykeen\_HolE: Pykeen\_HolE: Pykeen\_TransD: Pykeen\_TransE: Pykeen\_TransF: Pykeen\_TransH: Pykeen\_TransR:

**model\_kwargs**

**name**

**model**

**loss\_history** = []

**args**

**entity\_embeddings** = None

**relation\_embeddings** = None

**forward\_k\_vs\_all** (*x: torch.LongTensor*)

# => Explicit version by this we can apply bn and dropout

# (1) Retrieve embeddings of heads and relations + apply Dropout & Normalization if given. h, r = self.get\_head\_relation\_representation(x) # (2) Reshape (1). if self.last\_dim > 0:

h = h.reshape(len(x), self.embedding\_dim, self.last\_dim) r = r.reshape(len(x), self.embedding\_dim, self.last\_dim)

# (3) Reshape all entities. if self.last\_dim > 0:

t = self.entity\_embeddings.weight.reshape(self.num\_entities, self.embedding\_dim, self.last\_dim)

**else:**

t = self.entity\_embeddings.weight

# (4) Call the score\_t from interactions to generate triple scores. return self.interaction.score\_t(h=h, r=r, all\_entities=t, slice\_size=1)

**forward\_triples** (*x: torch.LongTensor*) → torch.FloatTensor

# => Explicit version by this we can apply bn and dropout

# (1) Retrieve embeddings of heads, relations and tails and apply Dropout & Normalization if given. h, r, t = self.get\_triple\_representation(x) # (2) Reshape (1). if self.last\_dim > 0:

h = h.reshape(len(x), self.embedding\_dim, self.last\_dim) r = r.reshape(len(x), self.embedding\_dim, self.last\_dim) t = t.reshape(len(x), self.embedding\_dim, self.last\_dim)

# (3) Compute the triple score return self.interaction.score(h=h, r=r, t=t, slice\_size=None, slice\_dim=0)

**abstractmethod forward\_k\_vs\_sample** (*x: torch.LongTensor, target\_entity\_idx*)

## dicee.models.quaternion

### Classes

|               |                                                              |
|---------------|--------------------------------------------------------------|
| <i>QMult</i>  | Base class for all neural network modules.                   |
| <i>ConvQ</i>  | Convolutional Quaternion Knowledge Graph Embeddings          |
| <i>AConvQ</i> | Additive Convolutional Quaternion Knowledge Graph Embeddings |

### Functions

|                                                    |
|----------------------------------------------------|
| <i>quaternion_mul_with_unit_norm</i> (*, Q_1, Q_2) |
|----------------------------------------------------|

### Module Contents

`dicee.models.quaternion.quaternion_mul_with_unit_norm(*, Q_1, Q_2)`

**class** `dicee.models.quaternion.QMult` (*args*)

Bases: `dicee.models.base_model.BaseKGE`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

`name = 'QMult'`

`explicit = True`

`quaternion_multiplication_followed_by_inner_product (h, r, t)`

#### Parameters

- **h** – shape: (\*batch\_dims, dim) The head representations.
- **r** – shape: (\*batch\_dims, dim) The head representations.
- **t** – shape: (\*batch\_dims, dim) The tail representations.

#### Returns

Triple scores.

**static** `quaternion_normalizer (x: torch.FloatTensor) → torch.FloatTensor`

Normalize the length of relation vectors, if the forward constraint has not been applied yet.

Absolute value of a quaternion

$$|a + bi + cj + dk| = \sqrt{a^2 + b^2 + c^2 + d^2}$$

L2 norm of quaternion vector:

$$\|x\|^2 = \sum_{i=1}^d |x_i|^2 = \sum_{i=1}^d (x_i.re^2 + x_i.im_1^2 + x_i.im_2^2 + x_i.im_3^2)$$

#### Parameters

**x** – The vector.

#### Returns

The normalized vector.

**score** (`head_ent_emb: torch.FloatTensor, rel_ent_emb: torch.FloatTensor, tail_ent_emb: torch.FloatTensor`)

**k\_vs\_all\_score** (`bpe_head_ent_emb, bpe_rel_ent_emb, E`)

#### Parameters

- **bpe\_head\_ent\_emb**
- **bpe\_rel\_ent\_emb**
- **E**

**forward\_k\_vs\_all** (`x`)

#### Parameters

**x**

**forward\_k\_vs\_sample** (`x, target_entity_idx`)

Completed. Given a head entity and a relation (h,r), we compute scores for all possible triples,i.e., [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch,| Entities|)

**class** `dicee.models.quaternion.ConvQ (args)`

Bases: `dicee.models.base_model.BaseKGE`

Convolutional Quaternion Knowledge Graph Embeddings

```

name = 'ConvQ'

entity_embeddings

relation_embeddings

conv2d

fc_num_input

fc1

bn_conv1

bn_conv2

feature_map_dropout

residual_convolution(Q_1, Q_2)

forward_triples(indexed_triple: torch.Tensor) → torch.Tensor

```

#### Parameters

**x**

```
forward_k_vs_all(x: torch.Tensor)
```

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch,|Entities)

```
class dicee.models.quaternion.AConvQ(args)
```

Bases: *[dicee.models.base\\_model.BaseKGE](#)*

Additive Convolutional Quaternion Knowledge Graph Embeddings

```
name = 'AConvQ'
```

```
entity_embeddings
```

```
relation_embeddings
```

```
conv2d
```

```
fc_num_input
```

```
fc1
```

```
bn_conv1
```

```
bn_conv2
```

```
feature_map_dropout
```

```
residual_convolution(Q_1, Q_2)
```

```
forward_triples(indexed_triple: torch.Tensor) → torch.Tensor
```

#### Parameters

**x**

**forward\_k\_vs\_all** (*x: torch.Tensor*)

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch,|Entities)

## dicee.models.real

### Classes

|                   |                                                                                                                                    |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <i>DistMult</i>   | Embedding Entities and Relations for Learning and Inference in Knowledge Bases                                                     |
| <i>TransE</i>     | Translating Embeddings for Modeling                                                                                                |
| <i>Shallom</i>    | A shallow neural model for relation prediction ( <a href="https://arxiv.org/abs/2101.09090">https://arxiv.org/abs/2101.09090</a> ) |
| <i>Pyke</i>       | A Physical Embedding Model for Knowledge Graphs                                                                                    |
| <i>CoKEConfig</i> | Configuration for the CoKE (Contextualized Knowledge Graph Embedding) model.                                                       |
| <i>CoKE</i>       | Contextualized Knowledge Graph Embedding (CoKE) model.                                                                             |

### Module Contents

**class** dicee.models.real.**DistMult** (*args*)

Bases: *dicee.models.base\_model.BaseKGE*

Embedding Entities and Relations for Learning and Inference in Knowledge Bases <https://arxiv.org/abs/1412.6575>

**name** = 'DistMult'

**k\_vs\_all\_score** (*emb\_h: torch.FloatTensor, emb\_r: torch.FloatTensor, emb\_E: torch.FloatTensor*)

#### Parameters

- **emb\_h**
- **emb\_r**
- **emb\_E**

**forward\_k\_vs\_all** (*x: torch.LongTensor*)

**forward\_k\_vs\_sample** (*x: torch.LongTensor, target\_entity\_idx: torch.LongTensor*)

**score** (*h, r, t*)

**class** dicee.models.real.**TransE** (*args*)

Bases: *dicee.models.base\_model.BaseKGE*

Translating Embeddings for Modeling Multi-relational Data <https://proceedings.neurips.cc/paper/2013/file/1cecc7a77928ca8133fa24680a88d2f9-Paper.pdf>

**name** = 'TransE'

**margin** = 4

**score** (*head\_ent\_emb, rel\_ent\_emb, tail\_ent\_emb*)

```

forward_k_vs_all (x: torch.Tensor) → torch.FloatTensor

class dicee.models.real.Shallom(args)
    Bases: dicee.models.base_model.BaseKGE
    A shallow neural model for relation prediction (https://arxiv.org/abs/2101.09090)
    name = 'Shallom'

    shallom

    get_embeddings () → Tuple[numpy.ndarray, None]

    forward_k_vs_all (x) → torch.FloatTensor

    forward_triples (x) → torch.FloatTensor

        Parameters
        x

        Returns

class dicee.models.real.Pyke(args)
    Bases: dicee.models.base_model.BaseKGE
    A Physical Embedding Model for Knowledge Graphs
    name = 'Pyke'

    dist_func

    margin = 1.0

    forward_triples (x: torch.LongTensor)

        Parameters
        x

class dicee.models.real.CoKEConfig
    Configuration for the CoKE (Contextualized Knowledge Graph Embedding) model.

    block_size
        Sequence length for transformer (3 for triples: head, relation, tail)

    vocab_size
        Total vocabulary size (num_entities + num_relations)

    n_layer
        Number of transformer layers

    n_head
        Number of attention heads per layer

    n_embd
        Embedding dimension (set to match model embedding_dim)

    dropout
        Dropout rate applied throughout the model

    bias
        Whether to use bias in linear layers

```

```

causal
    Whether to use causal masking (False for bidirectional attention)

block_size: int = 3

vocab_size: int = None

n_layer: int = 6

n_head: int = 8

n_embd: int = None

dropout: float = 0.3

bias: bool = True

causal: bool = False

```

```

class dicee.models.real.CoKE (args, config: CoKEConfig = CoKEConfig())

```

Bases: `dicee.models.base_model.BaseKGE`

Contextualized Knowledge Graph Embedding (CoKE) model. Based on: <https://arxiv.org/pdf/1911.02168>.

CoKE uses a transformer encoder to learn contextualized representations of entities and relations. For link prediction, it predicts masked elements in (head, relation, tail) triples using bidirectional attention, similar to BERT's masked language modeling approach.

The model creates a sequence [head\_emb, relation\_emb, mask\_emb], adds positional embeddings, and processes it through transformer layers to predict the tail entity.

```

name = 'CoKE'

```

```

config

```

```

pos_emb

```

```

mask_emb

```

```

blocks

```

```

ln_f

```

```

coke_dropout

```

```

forward_k_vs_all (x: torch.Tensor)

```

```

score (emb_h, emb_r, emb_t)

```

```

forward_k_vs_sample (x: torch.LongTensor, target_entity_idx: torch.LongTensor)

```

## `dicee.models.static_funcs`

### Functions

---

```

quaternion_mul(→ Tuple[torch.Tensor, torch.Tensor,    Perform quaternion multiplication
...)
```

---



## Module Contents

```
dicee.models.static_funcs.quaternion_mul(* , Q_1, Q_2)
→ Tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]
```

Perform quaternion multiplication :param Q\_1: :param Q\_2: :return:

## `dicee.models.transformers`

Full definition of a GPT Language Model, all of it in this single file. References: 1) the official GPT-2 TensorFlow implementation released by OpenAI: <https://github.com/openai/gpt-2/blob/master/src/model.py> 2) huggingface/transformers PyTorch implementation: [https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling\\_gpt2.py](https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling_gpt2.py)

## Classes

|                      |                                                                                |
|----------------------|--------------------------------------------------------------------------------|
| <i>ByteE</i>         | Base class for all neural network modules.                                     |
| <i>LayerNorm</i>     | LayerNorm but with an optional bias. PyTorch doesn't support simply bias=False |
| <i>SelfAttention</i> | Base class for all neural network modules.                                     |
| <i>MLP</i>           | Base class for all neural network modules.                                     |
| <i>Block</i>         | Base class for all neural network modules.                                     |
| <i>GPTConfig</i>     |                                                                                |
| <i>GPT</i>           | Base class for all neural network modules.                                     |

## Module Contents

```
class dicee.models.transformers.ByteE(*args, **kwargs)
```

Bases: `dicee.models.base_model.BaseKGE`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**name** = 'ByteE'

**config**

**temperature** = 0.5

**topk** = 2

**transformer**

**lm\_head**

**loss\_function** (*yhat\_batch, y\_batch*)

#### Parameters

- **yhat\_batch**
- **y\_batch**

**forward** (*x: torch.LongTensor*)

#### Parameters

**x** (*B by T tensor*)

**generate** (*idx, max\_new\_tokens, temperature=1.0, top\_k=None*)

Take a conditioning sequence of indices *idx* (LongTensor of shape (b,t)) and complete the sequence *max\_new\_tokens* times, feeding the predictions back into the model each time. Most likely you'll want to make sure to be in `model.eval()` mode of operation for this.

**training\_step** (*batch, batch\_idx=None*)

Here you compute and return the training loss and some additional metrics for e.g. the progress bar or logger.

#### Parameters

- **batch** – The output of your data iterable, normally a `DataLoader`.
- **batch\_idx** – The index of this batch.
- **dataloader\_idx** – The index of the dataloader that produced this batch. (only if multiple dataloaders used)

#### Returns

- **Tensor** - The loss tensor
- **dict** - A dictionary which can include any keys, but must include the key 'loss' in the case of automatic optimization.
- **None** - In automatic optimization, this will skip to the next batch (but is not supported for multi-GPU, TPU, or DeepSpeed). For manual optimization, this has no special meaning, as returning the loss is not required.

In this step you'd normally do the forward pass and calculate the loss for a batch. You can also do fancier things like multiple forward passes or something model specific.

Example:

```
def training_step(self, batch, batch_idx):
    x, y, z = batch
    out = self.encoder(x)
    loss = self.loss(out, x)
    return loss
```

To use multiple optimizers, you can switch to 'manual optimization' and control their stepping:

```
def __init__(self):
    super().__init__()
    self.automatic_optimization = False

# Multiple optimizers (e.g.: GANs)
def training_step(self, batch, batch_idx):
    opt1, opt2 = self.optimizers()

    # do training_step with encoder
    ...
    opt1.step()
    # do training_step with decoder
    ...
    opt2.step()
```

#### Note

When `accumulate_grad_batches > 1`, the loss returned here will be automatically normalized by `accumulate_grad_batches` internally.

```
class dicee.models.transformers.LayerNorm(ndim, bias)
```

Bases: `torch.nn.Module`

LayerNorm but with an optional bias. PyTorch doesn't support simply `bias=False`

**weight**

**bias**

**forward**(*input*)

```
class dicee.models.transformers.SelfAttention(config)
```

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

`training (bool)` – Boolean represents whether this module is in training or evaluation mode.

`c_attn`

`c_proj`

`attn_dropout`

`resid_dropout`

`n_head`

`n_embd`

`dropout`

`causal`

`flash = True`

`forward(x)`

```
class dicee.models.transformers.MLP (config)
```

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**c\_fc**

**gelu**

**c\_proj**

**dropout**

**forward** (*x*)

```
class dicee.models.transformers.Block(config)
```

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

```

(continues on next page)

```
def forward(self, x):
    x = F.relu(self.conv1(x))
    return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

### **Note**

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

### **Variables**

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**ln\_1**

**attn**

**ln\_2**

**mlp**

**forward** (*x*)

```
class dicee.models.transformers.GPTConfig
```

**block\_size**: int = 1024

**vocab\_size**: int = 50304

**n\_layer**: int = 12

**n\_head**: int = 12

**n\_embd**: int = 768

**dropout**: float = 0.0

**bias**: bool = False

**causal**: bool = True

```
class dicee.models.transformers.GPT(config)
```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

## Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**config**

**transformer**

**lm\_head**

**get\_num\_params** (*non\_embedding=True*)

Return the number of parameters in the model. For non-embedding count (default), the position embeddings get subtracted. The token embeddings would too, except due to the parameter sharing these params are actually used as weights in the final layer, so we include them.

**forward** (*idx, targets=None*)

**crop\_block\_size** (*block\_size*)

**classmethod from\_pretrained** (*model\_type, override\_args=None*)

**configure\_optimizers** (*weight\_decay, learning\_rate, betas, device\_type*)

**estimate\_mfu** (*fwdbwd\_per\_iter, dt*)

estimate model flops utilization (MFU) in units of A100 bfloat16 peak FLOPS

## Classes

|                         |                                            |
|-------------------------|--------------------------------------------|
| <i>ADOPT</i>            | ADOPT Optimizer.                           |
| <i>BaseKGELightning</i> | Base class for all neural network modules. |
| <i>BaseKGE</i>          | Base class for all neural network modules. |

continues on next page

Table 41 – continued from previous page

|                        |                                                                                                                                                                                                |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>IdentityClass</i>   | Base class for all neural network modules.                                                                                                                                                     |
| <i>BaseKGE</i>         | Base class for all neural network modules.                                                                                                                                                     |
| <i>Block</i>           | Base class for all neural network modules.                                                                                                                                                     |
| <i>DistMult</i>        | Embedding Entities and Relations for Learning and Inference in Knowledge Bases                                                                                                                 |
| <i>TransE</i>          | Translating Embeddings for Modeling                                                                                                                                                            |
| <i>Shallom</i>         | A shallow neural model for relation prediction ( <a href="https://arxiv.org/abs/2101.09090">https://arxiv.org/abs/2101.09090</a> )                                                             |
| <i>Pyke</i>            | A Physical Embedding Model for Knowledge Graphs                                                                                                                                                |
| <i>CoKEConfig</i>      | Configuration for the CoKE (Contextualized Knowledge Graph Embedding) model.                                                                                                                   |
| <i>CoKE</i>            | Contextualized Knowledge Graph Embedding (CoKE) model.                                                                                                                                         |
| <i>BaseKGE</i>         | Base class for all neural network modules.                                                                                                                                                     |
| <i>ConEx</i>           | Convolutional ComplEx Knowledge Graph Embeddings                                                                                                                                               |
| <i>AConEx</i>          | Additive Convolutional ComplEx Knowledge Graph Embeddings                                                                                                                                      |
| <i>ComplEx</i>         | Base class for all neural network modules.                                                                                                                                                     |
| <i>BaseKGE</i>         | Base class for all neural network modules.                                                                                                                                                     |
| <i>IdentityClass</i>   | Base class for all neural network modules.                                                                                                                                                     |
| <i>QMult</i>           | Base class for all neural network modules.                                                                                                                                                     |
| <i>ConvQ</i>           | Convolutional Quaternion Knowledge Graph Embeddings                                                                                                                                            |
| <i>AConvQ</i>          | Additive Convolutional Quaternion Knowledge Graph Embeddings                                                                                                                                   |
| <i>BaseKGE</i>         | Base class for all neural network modules.                                                                                                                                                     |
| <i>IdentityClass</i>   | Base class for all neural network modules.                                                                                                                                                     |
| <i>OMult</i>           | Base class for all neural network modules.                                                                                                                                                     |
| <i>ConvO</i>           | Base class for all neural network modules.                                                                                                                                                     |
| <i>AConvO</i>          | Additive Convolutional Octonion Knowledge Graph Embeddings                                                                                                                                     |
| <i>Keci</i>            | Base class for all neural network modules.                                                                                                                                                     |
| <i>CKeci</i>           | Without learning dimension scaling                                                                                                                                                             |
| <i>DeCaL</i>           | Base class for all neural network modules.                                                                                                                                                     |
| <i>KeciTransformer</i> | Keci with Transformer architecture.                                                                                                                                                            |
| <i>BaseKGE</i>         | Base class for all neural network modules.                                                                                                                                                     |
| <i>PykeenKGE</i>       | A class for using knowledge graph embedding models implemented in Pykeen                                                                                                                       |
| <i>BaseKGE</i>         | Base class for all neural network modules.                                                                                                                                                     |
| <i>FMult</i>           | Learning Knowledge Neural Graphs                                                                                                                                                               |
| <i>GFMult</i>          | Learning Knowledge Neural Graphs                                                                                                                                                               |
| <i>FMult2</i>          | Learning Knowledge Neural Graphs                                                                                                                                                               |
| <i>LFMult1</i>         | Embedding with trigonometric functions. We represent all entities and relations in the complex number space as:                                                                                |
| <i>LFMult</i>          | Embedding with polynomial functions. We represent all entities and relations in the polynomial space as:                                                                                       |
| <i>DualE</i>           | Dual Quaternion Knowledge Graph Embeddings ( <a href="https://ojs.aaai.org/index.php/AAAI/article/download/16850/16657">https://ojs.aaai.org/index.php/AAAI/article/download/16850/16657</a> ) |



## Functions

```
quaternion_mul(→ Tuple[torch.Tensor, torch.Tensor, Perform quaternion multiplication
...))
quaternion_mul_with_unit_norm(*, Q_1, Q_2)
octonion_mul(*, O_1, O_2)
octonion_mul_norm(*, O_1, O_2)
```

## Package Contents

```
class dicee.models.ADOPT (params: torch.optim.optimizer.ParamsT, lr: float | torch.Tensor = 0.001,
    betas: Tuple[float, float] = (0.9, 0.9999), eps: float = 1e-06,
    clip_lambda: Callable[[int], float] | None = lambda step: ..., weight_decay: float = 0.0,
    decouple: bool = False, *, foreach: bool | None = None, maximize: bool = False,
    capturable: bool = False, differentiable: bool = False, fused: bool | None = None)
```

Bases: `torch.optim.optimizer.Optimizer`

ADOPT Optimizer.

ADOPT is an adaptive learning rate optimization algorithm that combines momentum-based updates with adaptive per-parameter learning rates. It uses exponential moving averages of gradients and squared gradients, with gradient clipping for stability.

The algorithm performs the following key operations: 1. Normalizes gradients by the square root of the second moment estimate 2. Applies optional gradient clipping based on the training step 3. Updates parameters using momentum-smoothed normalized gradients 4. Supports decoupled weight decay (AdamW-style) or L2 regularization

### Mathematical formulation:

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \text{clip}(g_t / \sqrt{v_t}) \quad v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * g_t^2 \quad \theta_t = \theta_{t-1} - \alpha * m_t$$

where:

- $\theta_t$ : parameter at step  $t$
- $g_t$ : gradient at step  $t$
- $m_t$ : first moment estimate (momentum)
- $v_t$ : second moment estimate (variance)
- $\alpha$ : learning rate
- $\beta_1, \beta_2$ : exponential decay rates
- `clip()`: optional gradient clipping function

### Reference:

Original implementation: <https://github.com/iShohei220/adopt>

### Parameters

- **params** (*ParamsT*) – Iterable of parameters to optimize or dicts defining parameter groups.
- **lr** (*float or Tensor, optional*) – Learning rate. Can be a float or 1-element Tensor. Default: 1e-3
- **betas** (*Tuple[float, float], optional*) – Coefficients ( $\beta_1, \beta_2$ ) for computing running averages of gradient and its square.  $\beta_1$  controls momentum,  $\beta_2$  controls variance. Default: (0.9, 0.9999)

- **eps** (*float, optional*) – Term added to denominator to improve numerical stability. Default: 1e-6
- **clip\_lambda** (*Callable[[int], float], optional*) – Function that takes the step number and returns the gradient clipping threshold. Common choices: - lambda step: step\*\*0.25 (default, gradually increases clipping threshold) - lambda step: 1.0 (constant clipping) - None (no clipping) Default: lambda step: step\*\*0.25
- **weight\_decay** (*float, optional*) – Weight decay coefficient (L2 penalty). Default: 0.0
- **decouple** (*bool, optional*) – If True, uses decoupled weight decay (AdamW-style), applying weight decay directly to parameters. If False, adds weight decay to gradients (L2 regularization). Default: False
- **foreach** (*bool, optional*) – If True, uses the faster foreach implementation for multi-tensor operations. Default: None (auto-select)
- **maximize** (*bool, optional*) – If True, maximizes parameters instead of minimizing. Useful for reinforcement learning. Default: False
- **capturable** (*bool, optional*) – If True, the optimizer is safe to capture in a CUDA graph. Requires learning rate as Tensor. Default: False
- **differentiable** (*bool, optional*) – If True, the optimization step can be differentiated. Useful for meta-learning. Default: False
- **fused** (*bool, optional*) – If True, uses fused kernel implementation (currently not supported). Default: None

#### Raises

- **ValueError** – If learning rate, epsilon, betas, or weight\_decay are invalid.
- **RuntimeError** – If fused is enabled (not currently supported).
- **RuntimeError** – If lr is a Tensor with foreach=True and capturable=False.

#### Example

```
>>> # Basic usage
>>> optimizer = ADOPT(model.parameters(), lr=0.001)
>>> optimizer.zero_grad()
>>> loss.backward()
>>> optimizer.step()
```

```
>>> # With decoupled weight decay
>>> optimizer = ADOPT(model.parameters(), lr=0.001, weight_decay=0.01,
↳ decouple=True)
```

```
>>> # Custom gradient clipping
>>> optimizer = ADOPT(model.parameters(), clip_lambda=lambda step: max(1.0,
↳ step**0.5))
```

#### Note

- For most use cases, the default hyperparameters work well
- Consider using decouple=True for better generalization (similar to AdamW)

- The `clip_lambda` function helps stabilize training in early steps

**`clip_lambda`**

**`__setstate__`** (*state*)

Restore optimizer state from a checkpoint.

This method handles backward compatibility when loading optimizer state from older versions. It ensures all required fields are present with default values and properly converts step counters to tensors if needed.

Key responsibilities: 1. Set default values for newly added hyperparameters 2. Convert old-style scalar step counters to tensor format 3. Place step tensors on appropriate devices based on capturable/fused modes

#### Parameters

**`state`** (*dict*) – Optimizer state dictionary (typically from `torch.load()`).

#### Note

- This enables loading checkpoints saved with older ADOPT versions
- Step counters are converted to appropriate device/dtype for compatibility
- Capturable and fused modes require step tensors on parameter devices

**`step`** (*closure=None*)

Perform a single optimization step.

This method executes one iteration of the ADOPT optimization algorithm across all parameter groups. It orchestrates the following workflow:

1. Optionally evaluates a closure to recompute the loss (useful for algorithms like LBFGS or when loss needs multiple evaluations)
2. For each parameter group: - Collects parameters with gradients and their associated state - Extracts hyperparameters (betas, learning rate, etc.) - Calls the functional `adopt()` API to perform the actual update
3. Returns the loss value if a closure was provided

The functional API (`adopt()`) handles three execution modes: - Single-tensor: Updates one parameter at a time (default, JIT-compatible) - Multi-tensor (foreach): Batches operations for better performance - Fused: Uses fused CUDA kernels (not yet implemented)

Gradient scaling support: This method is compatible with automatic mixed precision (AMP) training. It can access `grad_scale` and `found_inf` attributes for gradient unscaling and inf/nan detection when used with `GradScaler`.

#### Parameters

**`closure`** (*Callable, optional*) – A callable that reevaluates the model and returns the loss. The closure should: - Enable gradients (`torch.enable_grad()`) - Compute forward pass - Compute loss - Compute backward pass - Return the loss value Example: `lambda: (loss := model(x), loss.backward(), loss)[-1]` Default: `None`

#### Returns

**The loss value returned by the closure, or `None` if no closure was provided.**

**Return type**  
Optional[Tensor]

### Example

```
>>> # Standard usage
>>> loss = criterion(model(input), target)
>>> loss.backward()
>>> optimizer.step()
```

```
>>> # With closure (e.g., for line search)
>>> def closure():
...     optimizer.zero_grad()
...     output = model(input)
...     loss = criterion(output, target)
...     loss.backward()
...     return loss
>>> loss = optimizer.step(closure)
```

#### Note

- Call `zero_grad()` before computing gradients for the next step
- CUDA graph capture is checked for safety when `capturable=True`
- The method is thread-safe for different parameter groups

```
class dicee.models.BaseKGE Lightning(*args, **kwargs)
```

Bases: `lightning.LightningModule`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**training\_step\_outputs** = []

**mem\_of\_model**() → Dict

Size of model in MB and number of params

**training\_step** (*batch*, *batch\_idx=None*)

Here you compute and return the training loss and some additional metrics for e.g. the progress bar or logger.

### Parameters

- **batch** – The output of your data iterable, normally a `DataLoader`.
- **batch\_idx** – The index of this batch.
- **dataloader\_idx** – The index of the dataloader that produced this batch. (only if multiple dataloaders used)

### Returns

- **Tensor** – The loss tensor
- **dict** – A dictionary which can include any keys, but must include the key 'loss' in the case of automatic optimization.
- **None** – In automatic optimization, this will skip to the next batch (but is not supported for multi-GPU, TPU, or DeepSpeed). For manual optimization, this has no special meaning, as returning the loss is not required.

In this step you'd normally do the forward pass and calculate the loss for a batch. You can also do fancier things like multiple forward passes or something model specific.

Example:

```
def training_step(self, batch, batch_idx):
    x, y, z = batch
    out = self.encoder(x)
    loss = self.loss(out, x)
    return loss
```

To use multiple optimizers, you can switch to 'manual optimization' and control their stepping:

```
def __init__(self):
    super().__init__()
    self.automatic_optimization = False

# Multiple optimizers (e.g.: GANs)
def training_step(self, batch, batch_idx):
    opt1, opt2 = self.optimizers()
```

(continues on next page)

(continued from previous page)

```
# do training_step with encoder
...
opt1.step()
# do training_step with decoder
...
opt2.step()
```

### Note

When `accumulate_grad_batches > 1`, the loss returned here will be automatically normalized by `accumulate_grad_batches` internally.

**loss\_function** (*yhat\_batch*: *torch.FloatTensor*, *y\_batch*: *torch.FloatTensor*)

#### Parameters

- **yhat\_batch**
- **y\_batch**

**on\_train\_epoch\_end** (\*args, \*\*kwargs)

Called in the training loop at the very end of the epoch.

To access all batch outputs at the end of the epoch, you can cache step outputs as an attribute of the `LightningModule` and access them in this hook:

```
class MyLightningModule(L.LightningModule):
    def __init__(self):
        super().__init__()
        self.training_step_outputs = []

    def training_step(self):
        loss = ...
        self.training_step_outputs.append(loss)
        return loss

    def on_train_epoch_end(self):
        # do something with all training_step outputs, for example:
        epoch_mean = torch.stack(self.training_step_outputs).mean()
        self.log("training_epoch_mean", epoch_mean)
        # free up the memory
        self.training_step_outputs.clear()
```

**test\_epoch\_end** (outputs: *List[Any]*)

**test\_dataloader** () → None

An iterable or collection of iterables specifying test samples.

For more information about multiple dataloaders, see this section.

For data processing use the following pattern:

- download in `prepare_data()`
- process and split in `setup()`

However, the above are only necessary for distributed processing.

 **Warning**

do not assign state in `prepare_data`

- `test()`
- `prepare_data()`
- `setup()`

 **Note**

Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.

 **Note**

If you don't need a test dataset and a `test_step()`, you don't need to implement this method.

`val_dataloader()` → None

An iterable or collection of iterables specifying validation samples.

For more information about multiple dataloaders, see this section.

The dataloader you return will not be reloaded unless you set **:param-ref:~lightning.pytorch.trainer.trainer.Trainer.reload\_dataloaders\_every\_n\_epochs** to a positive integer.

It's recommended that all data downloads and preparation happen in `prepare_data()`.

- `fit()`
- `validate()`
- `prepare_data()`
- `setup()`

 **Note**

Lightning tries to add the correct sampler for distributed and arbitrary hardware There is no need to set it yourself.

 **Note**

If you don't need a validation dataset and a `validation_step()`, you don't need to implement this method.

`predict_dataloader()` → None

An iterable or collection of iterables specifying prediction samples.

For more information about multiple dataloaders, see this section.

It's recommended that all data downloads and preparation happen in `prepare_data()`.

- `predict()`
- `prepare_data()`
- `setup()`

#### Note

Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.

#### Returns

A `torch.utils.data.DataLoader` or a sequence of them specifying prediction samples.

`train_dataloader()` → None

An iterable or collection of iterables specifying training samples.

For more information about multiple dataloaders, see this section.

The dataloader you return will not be reloaded unless you set **:param-ref:~lightning.pytorch.trainer.trainer.Trainer.reload\_dataloaders\_every\_n\_epochs** to a positive integer.

For data processing use the following pattern:

- download in `prepare_data()`
- process and split in `setup()`

However, the above are only necessary for distributed processing.

#### Warning

do not assign state in `prepare_data`

- `fit()`
- `prepare_data()`
- `setup()`

#### Note

Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.



**configure\_optimizers** (*parameters=None*)

Choose what optimizers and learning-rate schedulers to use in your optimization. Normally you'd need one. But in the case of GANs or similar you might have multiple. Optimization with multiple optimizers only works in the manual optimization mode.

### Returns

Any of these 6 options.

- **Single optimizer.**
- **List or Tuple** of optimizers.
- **Two lists** - The first list has multiple optimizers, and the second has multiple LR schedulers (or multiple `lr_scheduler_config`).
- **Dictionary**, with an "optimizer" key, and (optionally) a "lr\_scheduler" key whose value is a single LR scheduler or `lr_scheduler_config`.
- **None** - Fit will run without any optimizer.

The `lr_scheduler_config` is a dictionary which contains the scheduler and its associated configuration. The default configuration is shown below.

```
lr_scheduler_config = {
    # REQUIRED: The scheduler instance
    "scheduler": lr_scheduler,
    # The unit of the scheduler's step size, could also be 'step'.
    # 'epoch' updates the scheduler on epoch end whereas 'step'
    # updates it after a optimizer update.
    "interval": "epoch",
    # How many epochs/steps should pass between calls to
    # `scheduler.step()`. 1 corresponds to updating the learning
    # rate after every epoch/step.
    "frequency": 1,
    # Metric to monitor for schedulers like `ReduceLROnPlateau`
    "monitor": "val_loss",
    # If set to `True`, will enforce that the value specified 'monitor'
    # is available when the scheduler is updated, thus stopping
    # training if not found. If set to `False`, it will only produce a warning
    "strict": True,
    # If using the `LearningRateMonitor` callback to monitor the
    # learning rate progress, this keyword can be used to specify
    # a custom logged name
    "name": None,
}
```

When there are schedulers in which the `.step()` method is conditioned on a value, such as the `torch.optim.lr_scheduler.ReduceLROnPlateau` scheduler, Lightning requires that the `lr_scheduler_config` contains the keyword "monitor" set to the metric name that the scheduler should be conditioned on.

Metrics can be made available to monitor by simply logging it using `self.log('metric_to_track', metric_val)` in your `LightningModule`.

### Note

Some things to know:

- Lightning calls `.backward()` and `.step()` automatically in case of automatic optimization.
- If a learning rate scheduler is specified in `configure_optimizers()` with key "interval" (default "epoch") in the scheduler configuration, Lightning will call the scheduler's `.step()` method automatically in case of automatic optimization.
- If you use 16-bit precision (`precision=16`), Lightning will automatically handle the optimizer.
- If you use `torch.optim.LBFGS`, Lightning handles the closure function automatically for you.
- If you use multiple optimizers, you will have to switch to 'manual optimization' mode and step them yourself.
- If you need to control how often the optimizer steps, override the `optimizer_step()` hook.

```
class dicee.models.BaseKGE(args: dict)
```

Bases: *BaseKGELightning*

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**args**

```

embedding_dim = None

num_entities = None

num_relations = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

loss

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

forward_byte_pair_encoded_k_vs_all (x: torch.LongTensor)

```

#### Parameters

$\mathbf{x} (B \times 2 \times T)$

**forward\_byte\_pair\_encoded\_triple** (*x*: *Tuple[torch.LongTensor, torch.LongTensor]*)  
 byte pair encoded neural link predictors

**Parameters**

-----

**init\_params\_with\_sanity\_checking** ()

**forward** (*x*: *torch.LongTensor* | *Tuple[torch.LongTensor, torch.LongTensor]*,  
*y\_idx*: *torch.LongTensor* = *None*)

**Parameters**

- **x**
- **y\_idx**
- **ordered\_bpe\_entities**

**forward\_triples** (*x*: *torch.LongTensor*) → *torch.Tensor*

**Parameters**

**x**

**forward\_k\_vs\_all** (*\*args*, *\*\*kwargs*)

**forward\_k\_vs\_sample** (*\*args*, *\*\*kwargs*)

**get\_triple\_representation** (*idx\_hrt*)

**get\_head\_relation\_representation** (*indexed\_triple*)

**get\_sentence\_representation** (*x*: *torch.LongTensor*)

**Parameters**

- (**b** (*x* *shape*))
- **3**
- **t**)

**get\_bpe\_head\_and\_relation\_representation** (*x*: *torch.LongTensor*)  
 → *Tuple[torch.FloatTensor, torch.FloatTensor]*

**Parameters**

**x** (*B* × 2 × *T*)

**get\_embeddings** () → *Tuple[numpy.ndarray, numpy.ndarray]*

**class** *dicee.models.IdentityClass* (*args=None*)

Bases: *torch.nn.Module*

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**args** = None

**\_\_call\_\_** (*x*)

**static forward** (*x*)

**class** `dicdee.models.BaseKGE` (*args: dict*)

Bases: `BaseKGELightning`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

`training` (*bool*) – Boolean represents whether this module is in training or evaluation mode.

#### `args`

`embedding_dim = None`

`num_entities = None`

`num_relations = None`

`num_tokens = None`

`learning_rate = None`

`apply_unit_norm = None`

`input_dropout_rate = None`

`hidden_dropout_rate = None`

`optimizer_name = None`

`feature_map_dropout_rate = None`

`kernel_size = None`

`num_of_output_channels = None`

`weight_decay = None`

#### `loss`

`selected_optimizer = None`

`normalizer_class = None`

`normalize_head_entity_embeddings`

`normalize_relation_embeddings`

`normalize_tail_entity_embeddings`

`hidden_normalizer`

`param_init`

`input_dp_ent_real`

`input_dp_rel_real`

```

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

forward_byte_pair_encoded_k_vs_all (x: torch.LongTensor)

    Parameters
         $\mathbf{x}$  ( $B \times 2 \times T$ )

forward_byte_pair_encoded_triple (x: Tuple[torch.LongTensor, torch.LongTensor])
    byte pair encoded neural link predictors

    Parameters
    -----

init_params_with_sanity_checking()

forward (x: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor],
        y_idx: torch.LongTensor = None)

    Parameters

    •  $\mathbf{x}$ 

    •  $\mathbf{y\_idx}$ 

    • ordered_bpe_entities

forward_triples (x: torch.LongTensor) → torch.Tensor

    Parameters
         $\mathbf{x}$ 

forward_k_vs_all (*args, **kwargs)

forward_k_vs_sample (*args, **kwargs)

get_triple_representation (idx_hrt)

get_head_relation_representation (indexed_triple)

get_sentence_representation (x: torch.LongTensor)

    Parameters

    • ( $\mathbf{b}$  ( $x$  shape))

    • 3

    •  $\mathbf{t}$ )

get_bpe_head_and_relation_representation (x: torch.LongTensor)
    → Tuple[torch.FloatTensor, torch.FloatTensor]

    Parameters
         $\mathbf{x}$  ( $B \times 2 \times T$ )

```

`get_embeddings()` → Tuple[numpy.ndarray, numpy.ndarray]

`class dicee.models.Block(config)`

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

`training (bool)` – Boolean represents whether this module is in training or evaluation mode.

`ln_1`

`attn`

`ln_2`

`mlp`

`forward(x)`

`class dicee.models.DistMult(args)`

Bases: `dicee.models.base_model.BaseKGE`

Embedding Entities and Relations for Learning and Inference in Knowledge Bases <https://arxiv.org/abs/1412.6575>

`name = 'DistMult'`



**k\_vs\_all\_score** (*emb\_h: torch.FloatTensor, emb\_r: torch.FloatTensor, emb\_E: torch.FloatTensor*)

**Parameters**

- **emb\_h**
- **emb\_r**
- **emb\_E**

**forward\_k\_vs\_all** (*x: torch.LongTensor*)

**forward\_k\_vs\_sample** (*x: torch.LongTensor, target\_entity\_idx: torch.LongTensor*)

**score** (*h, r, t*)

**class** `dicee.models.TransE` (*args*)

Bases: `dicee.models.base_model.BaseKGE`

Translating Embeddings for Modeling Multi-relational Data <https://proceedings.neurips.cc/paper/2013/file/1cecc7a77928ca8133fa24680a88d2f9-Paper.pdf>

**name** = 'TransE'

**margin** = 4

**score** (*head\_ent\_emb, rel\_ent\_emb, tail\_ent\_emb*)

**forward\_k\_vs\_all** (*x: torch.Tensor*) → torch.FloatTensor

**class** `dicee.models.Shallom` (*args*)

Bases: `dicee.models.base_model.BaseKGE`

A shallow neural model for relation prediction (<https://arxiv.org/abs/2101.09090>)

**name** = 'Shallom'

**shallom**

**get\_embeddings** () → Tuple[numpy.ndarray, None]

**forward\_k\_vs\_all** (*x*) → torch.FloatTensor

**forward\_triples** (*x*) → torch.FloatTensor

**Parameters**

**x**

**Returns**

**class** `dicee.models.Pyke` (*args*)

Bases: `dicee.models.base_model.BaseKGE`

A Physical Embedding Model for Knowledge Graphs

**name** = 'Pyke'

**dist\_func**

**margin** = 1.0

**forward\_triples** (*x*: *torch.LongTensor*)

### Parameters

**x**

**class** `dicee.models.CoKEConfig`

Configuration for the CoKE (Contextualized Knowledge Graph Embedding) model.

**block\_size**

Sequence length for transformer (3 for triples: head, relation, tail)

**vocab\_size**

Total vocabulary size (num\_entities + num\_relations)

**n\_layer**

Number of transformer layers

**n\_head**

Number of attention heads per layer

**n\_embd**

Embedding dimension (set to match model embedding\_dim)

**dropout**

Dropout rate applied throughout the model

**bias**

Whether to use bias in linear layers

**causal**

Whether to use causal masking (False for bidirectional attention)

**block\_size**: `int = 3`

**vocab\_size**: `int = None`

**n\_layer**: `int = 6`

**n\_head**: `int = 8`

**n\_embd**: `int = None`

**dropout**: `float = 0.3`

**bias**: `bool = True`

**causal**: `bool = False`

**class** `dicee.models.CoKE` (*args*, *config*: *CoKEConfig* = *CoKEConfig*())

Bases: `dicee.models.base_model.BaseKGE`

Contextualized Knowledge Graph Embedding (CoKE) model. Based on: <https://arxiv.org/pdf/1911.02168>.

CoKE uses a transformer encoder to learn contextualized representations of entities and relations. For link prediction, it predicts masked elements in (head, relation, tail) triples using bidirectional attention, similar to BERT's masked language modeling approach.

The model creates a sequence [head\_emb, relation\_emb, mask\_emb], adds positional embeddings, and processes it through transformer layers to predict the tail entity.

```

name = 'CoKE'

config

pos_emb

mask_emb

blocks

ln_f

coke_dropout

forward_k_vs_all(x: torch.Tensor)

score(emb_h, emb_r, emb_t)

forward_k_vs_sample(x: torch.LongTensor, target_entity_idx: torch.LongTensor)

```

```
class dicee.models.BaseKGE(args: dict)
```

Bases: *BaseKGELightning*

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**args**

```

embedding_dim = None

num_entities = None

num_relations = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

loss

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

forward_byte_pair_encoded_k_vs_all (x: torch.LongTensor)

```

#### Parameters

$\mathbf{x} (B \times 2 \times T)$

```

forward_byte_pair_encoded_triple (x: Tuple[torch.LongTensor, torch.LongTensor])
    byte pair encoded neural link predictors

    Parameters
    -----

init_params_with_sanity_checking()

forward (x: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor],
         y_idx: torch.LongTensor = None)

    Parameters
    • x
    • y_idx
    • ordered_bpe_entities

forward_triples (x: torch.LongTensor) → torch.Tensor

    Parameters
    • x

forward_k_vs_all (*args, **kwargs)

forward_k_vs_sample (*args, **kwargs)

get_triple_representation (idx_hrt)

get_head_relation_representation (indexed_triple)

get_sentence_representation (x: torch.LongTensor)

    Parameters
    • (b (x shape)
    • 3
    • t)

get_bpe_head_and_relation_representation (x: torch.LongTensor)
    → Tuple[torch.FloatTensor, torch.FloatTensor]

    Parameters
    • x (B x 2 x T)

get_embeddings () → Tuple[numpy.ndarray, numpy.ndarray]

class dicee.models.ConEx (args)
    Bases: dicee.models.base_model.BaseKGE
    Convolutional ComplEx Knowledge Graph Embeddings
    name = 'ConEx'

    conv2d

    fc_num_input

    fc1

```

```

norm_fc1

bn_conv2d

feature_map_dropout

residual_convolution (C_1: Tuple[torch.Tensor, torch.Tensor],
                      C_2: Tuple[torch.Tensor, torch.Tensor]) → torch.FloatTensor
    Compute residual score of two complex-valued embeddings. :param C_1: a tuple of two pytorch tensors
    that corresponds complex-valued embeddings :param C_2: a tuple of two pytorch tensors that corresponds
    complex-valued embeddings :return:

forward_k_vs_all (x: torch.Tensor) → torch.FloatTensor

forward_triples (x: torch.Tensor) → torch.FloatTensor

    Parameters
    x

forward_k_vs_sample (x: torch.Tensor, target_entity_idx: torch.Tensor)

class dicee.models.AConEx (args)
    Bases: dicee.models.base_model.BaseKGE
    Additive Convolutional ComplEx Knowledge Graph Embeddings
    name = 'AConEx'

    conv2d

    fc_num_input

    fc1

    norm_fc1

    bn_conv2d

    feature_map_dropout

    residual_convolution (C_1: Tuple[torch.Tensor, torch.Tensor],
                          C_2: Tuple[torch.Tensor, torch.Tensor]) → torch.FloatTensor
        Compute residual score of two complex-valued embeddings. :param C_1: a tuple of two pytorch tensors
        that corresponds complex-valued embeddings :param C_2: a tuple of two pytorch tensors that corresponds
        complex-valued embeddings :return:

    forward_k_vs_all (x: torch.Tensor) → torch.FloatTensor

    forward_triples (x: torch.Tensor) → torch.FloatTensor

    Parameters
    x

    forward_k_vs_sample (x: torch.Tensor, target_entity_idx: torch.Tensor)

class dicee.models.ComplEx (args)
    Bases: dicee.models.base_model.BaseKGE
    Base class for all neural network modules.
    Your models should also subclass this class.

```

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**name** = 'Complex'

**static score** (*head\_ent\_emb: torch.FloatTensor, rel\_ent\_emb: torch.FloatTensor, tail\_ent\_emb: torch.FloatTensor*)

**static k\_vs\_all\_score** (*emb\_h: torch.FloatTensor, emb\_r: torch.FloatTensor, emb\_E: torch.FloatTensor*)

#### Parameters

- **emb\_h**
- **emb\_r**
- **emb\_E**

**forward\_k\_vs\_all** (*x: torch.LongTensor*) → torch.FloatTensor

**forward\_k\_vs\_sample** (*x: torch.LongTensor, target\_entity\_idx: torch.LongTensor*)

**dicee.models.quaternion\_mul** (\*, *Q\_1, Q\_2*)  
→ Tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]

Perform quaternion multiplication :param Q\_1: :param Q\_2: :return:

**class** dicee.models.**BaseKGE** (*args: dict*)

Bases: *BaseKGELightning*

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**args**

**embedding\_dim** = None

**num\_entities** = None

**num\_relations** = None

**num\_tokens** = None

**learning\_rate** = None

**apply\_unit\_norm** = None

**input\_dropout\_rate** = None

**hidden\_dropout\_rate** = None

**optimizer\_name** = None

**feature\_map\_dropout\_rate** = None

**kernel\_size** = None

**num\_of\_output\_channels** = None

**weight\_decay** = None



```

loss

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

forward_byte_pair_encoded_k_vs_all (x: torch.LongTensor)

    Parameters
         $\mathbf{x}$  ( $B \times 2 \times T$ )

forward_byte_pair_encoded_triple (x: Tuple[torch.LongTensor, torch.LongTensor])
    byte pair encoded neural link predictors

    Parameters
    -----

init_params_with_sanity_checking()

forward (x: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor],
        y_idx: torch.LongTensor = None)

    Parameters

        •  $\mathbf{x}$ 

        •  $\mathbf{y\_idx}$ 

        • ordered_bpe_entities

forward_triples (x: torch.LongTensor)  $\rightarrow$  torch.Tensor

    Parameters

         $\mathbf{x}$ 

forward_k_vs_all (*args, **kwargs)

forward_k_vs_sample (*args, **kwargs)

```

```

get_triple_representation(idx_hrt)

get_head_relation_representation(indexed_triple)

get_sentence_representation(x: torch.LongTensor)

```

#### Parameters

- **b** ( $x$  shape)
- 3
- **t**)

```

get_bpe_head_and_relation_representation(x: torch.LongTensor)
→ Tuple[torch.FloatTensor, torch.FloatTensor]

```

#### Parameters

**x** ( $B \times 2 \times T$ )

```

get_embeddings() → Tuple[numpy.ndarray, numpy.ndarray]

```

```

class dicee.models.IdentityClass(args=None)

```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

```
args = None
```

```
__call__(x)
```

```
static forward(x)
```

```
dicee.models.quaternion_mul_with_unit_norm(*, Q_1, Q_2)
```

```
class dicee.models.QMult(args)
```

Bases: `dicee.models.base_model.BaseKGE`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

`training (bool)` – Boolean represents whether this module is in training or evaluation mode.

```
name = 'QMult'
```

```
explicit = True
```

```
quaternion_multiplication_followed_by_inner_product (h, r, t)
```

#### Parameters

- **h** – shape: `(*batch_dims, dim)` The head representations.
- **r** – shape: `(*batch_dims, dim)` The head representations.
- **t** – shape: `(*batch_dims, dim)` The tail representations.

### Returns

Triple scores.

**static quaternion\_normalizer** ( $x$ : *torch.FloatTensor*)  $\rightarrow$  *torch.FloatTensor*

Normalize the length of relation vectors, if the forward constraint has not been applied yet.

Absolute value of a quaternion

$$|a + bi + cj + dk| = \sqrt{a^2 + b^2 + c^2 + d^2}$$

L2 norm of quaternion vector:

$$\|x\|^2 = \sum_{i=1}^d |x_i|^2 = \sum_{i=1}^d (x_i.re^2 + x_i.im_1^2 + x_i.im_2^2 + x_i.im_3^2)$$

### Parameters

**x** – The vector.

### Returns

The normalized vector.

**score** (*head\_ent\_emb*: *torch.FloatTensor*, *rel\_ent\_emb*: *torch.FloatTensor*,  
*tail\_ent\_emb*: *torch.FloatTensor*)

**k\_vs\_all\_score** (*bpe\_head\_ent\_emb*, *bpe\_rel\_ent\_emb*, *E*)

### Parameters

- **bpe\_head\_ent\_emb**
- **bpe\_rel\_ent\_emb**
- **E**

**forward\_k\_vs\_all** (*x*)

### Parameters

**x**

**forward\_k\_vs\_sample** (*x*, *target\_entity\_idx*)

Completed. Given a head entity and a relation (h,r), we compute scores for all possible triples,i.e.,  
[score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and  
relations => shape (size of batch,| Entities|)

**class** *dicee.models.ConvQ* (*args*)

Bases: *dicee.models.base\_model.BaseKGE*

Convolutional Quaternion Knowledge Graph Embeddings

**name** = 'ConvQ'

**entity\_embeddings**

**relation\_embeddings**

**conv2d**

**fc\_num\_input**

**fc1**

```

bn_conv1

bn_conv2

feature_map_dropout

residual_convolution(Q_1, Q_2)

forward_triples(indexed_triple: torch.Tensor) → torch.Tensor

```

#### Parameters

**x**

```
forward_k_vs_all(x: torch.Tensor)
```

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch,|Entities)

```
class dicee.models.AConvQ(args)
```

Bases: *dicee.models.base\_model.BaseKGE*

Additive Convolutional Quaternion Knowledge Graph Embeddings

**name** = 'AConvQ'

**entity\_embeddings**

**relation\_embeddings**

**conv2d**

**fc\_num\_input**

**fc1**

**bn\_conv1**

**bn\_conv2**

**feature\_map\_dropout**

**residual\_convolution**(Q\_1, Q\_2)

**forward\_triples**(indexed\_triple: torch.Tensor) → torch.Tensor

#### Parameters

**x**

```
forward_k_vs_all(x: torch.Tensor)
```

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch,|Entities)

```
class dicee.models.BaseKGE(args: dict)
```

Bases: *BaseKGELightning*

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

`training (bool)` – Boolean represents whether this module is in training or evaluation mode.

`args`

`embedding_dim = None`

`num_entities = None`

`num_relations = None`

`num_tokens = None`

`learning_rate = None`

`apply_unit_norm = None`

`input_dropout_rate = None`

`hidden_dropout_rate = None`

`optimizer_name = None`

`feature_map_dropout_rate = None`

`kernel_size = None`

`num_of_output_channels = None`

`weight_decay = None`

`loss`

```

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

forward_byte_pair_encoded_k_vs_all (x: torch.LongTensor)

    Parameters
         $\mathbf{x} (B \times 2 \times T)$ 

forward_byte_pair_encoded_triple (x: Tuple[torch.LongTensor, torch.LongTensor])
    byte pair encoded neural link predictors

    Parameters
    -----

init_params_with_sanity_checking ()

forward (x: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor],
        y_idx: torch.LongTensor = None)

    Parameters

        •  $\mathbf{x}$ 

        •  $\mathbf{y\_idx}$ 

        • ordered_bpe_entities

forward_triples (x: torch.LongTensor)  $\rightarrow$  torch.Tensor

    Parameters
         $\mathbf{x}$ 

forward_k_vs_all (*args, **kwargs)

forward_k_vs_sample (*args, **kwargs)

get_triple_representation (idx_hrt)

```

`get_head_relation_representation(indexed_triple)`

`get_sentence_representation(x: torch.LongTensor)`

#### Parameters

- **b** ( $x$  shape)
- 3
- **t**)

`get_bpe_head_and_relation_representation(x: torch.LongTensor)`  
→ Tuple[torch.FloatTensor, torch.FloatTensor]

#### Parameters

**x** ( $B \times 2 \times T$ )

`get_embeddings()` → Tuple[numpy.ndarray, numpy.ndarray]

`class dicee.models.IdentityClass(args=None)`

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**args** = None



```
__call__(x)
```

```
static forward(x)
```

```
dicee.models.octonion_mul(*, O_1, O_2)
```

```
dicee.models.octonion_mul_norm(*, O_1, O_2)
```

```
class dicee.models.OMult(args)
```

Bases: `dicee.models.base_model.BaseKGE`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

```
name = 'OMult'
```

```
static octonion_normalizer(emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4,
                           emb_rel_e5, emb_rel_e6, emb_rel_e7)
```

```
score(head_ent_emb: torch.FloatTensor, rel_ent_emb: torch.FloatTensor,
      tail_ent_emb: torch.FloatTensor)
```

```
k_vs_all_score(bpe_head_ent_emb, bpe_rel_ent_emb, E)
```

```
forward_k_vs_all(x)
```

Completed. Given a head entity and a relation (h,r), we compute scores for all possible triples,i.e.,  $[\text{score}(h,r,x) | x \text{ in Entities}] \Rightarrow [0.0,0.1,\dots,0.8]$ , shape= $\Rightarrow (1, \text{Entities!})$  Given a batch of head entities and relations  $\Rightarrow$  shape (size of batch,| Entities!)

```
class dicee.models.ConvO(args: dict)
```

Bases: `dicee.models.base_model.BaseKGE`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

`training (bool)` – Boolean represents whether this module is in training or evaluation mode.

`name = 'ConvO'`

`conv2d`

`fc_num_input`

`fc1`

`bn_conv2d`

`norm_fc1`

`feature_map_dropout`

`static octonion_normalizer(emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4, emb_rel_e5, emb_rel_e6, emb_rel_e7)`

`residual_convolution(O_1, O_2)`

**forward\_triples** (*x: torch.Tensor*) → torch.Tensor

#### Parameters

**x**

**forward\_k\_vs\_all** (*x: torch.Tensor*)

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch,|Entities)

```
class dicee.models.AConvO(args: dict)
```

Bases: *dicee.models.base\_model.BaseKGE*

Additive Convolutional Octonion Knowledge Graph Embeddings

**name** = 'AConvO'

**conv2d**

**fc\_num\_input**

**fc1**

**bn\_conv2d**

**norm\_fc1**

**feature\_map\_dropout**

**static octonion\_normalizer** (*emb\_rel\_e0, emb\_rel\_e1, emb\_rel\_e2, emb\_rel\_e3, emb\_rel\_e4, emb\_rel\_e5, emb\_rel\_e6, emb\_rel\_e7*)

**residual\_convolution** (*O\_1, O\_2*)

**forward\_triples** (*x: torch.Tensor*) → torch.Tensor

#### Parameters

**x**

**forward\_k\_vs\_all** (*x: torch.Tensor*)

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch,|Entities)

```
class dicee.models.Keci(args)
```

Bases: *dicee.models.base\_model.BaseKGE*

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
```

(continues on next page)

```

super().__init__()
self.conv1 = nn.Conv2d(1, 20, 5)
self.conv2 = nn.Conv2d(20, 20, 5)

def forward(self, x):
    x = F.relu(self.conv1(x))
    return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**name** = 'Keci'

**p**

**q**

**r**

**requires\_grad\_for\_interactions** = True

**compute\_sigma\_pp** (*hp, rp*)

Compute  $\text{sigma\_pp} = \sum_{i=1}^{p-1} \sum_{k=i+1}^p (h_i r_k - h_k r_i) e_i e_k$

$\text{sigma\_pp}$  captures the interactions between along  $p$  bases For instance, let  $p$   $e_1, e_2, e_3$ , we compute interactions between  $e_1 e_2, e_1 e_3$ , and  $e_2 e_3$  This can be implemented with a nested two for loops

results = [] for i in range(p - 1):

    for k in range(i + 1, p):

        results.append(hp[:, :, i] \* rp[:, :, k] - hp[:, :, k] \* rp[:, :, i])

sigma\_pp = torch.stack(results, dim=2) assert sigma\_pp.shape == (b, r, int((p \* (p - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all  $p$ , e.g.,  $e_1 e_1, e_1 e_2, e_1 e_3$ ,

$e_2 e_1, e_2 e_2, e_2 e_3, e_3 e_1, e_3 e_2, e_3 e_3$

Then select the triangular matrix without diagonals:  $e_1 e_2, e_1 e_3, e_2 e_3$ .

**compute\_sigma\_qq** (*hq, rq*)

Compute  $\text{sigma\_qq} = \sum_{j=1}^{p+q-1} \sum_{k=j+1}^{p+q} (h_j r_k - h_k r_j) e_j e_k \text{sigma\_q}$  captures the interactions between along  $q$  bases For instance, let  $q$   $e_1, e_2, e_3$ , we compute interactions between  $e_1 e_2, e_1 e_3$ , and  $e_2 e_3$  This can be implemented with a nested two for loops

results = [] for j in range(q - 1):

    for k in range(j + 1, q):

        results.append(hq[:, :, j] \* rq[:, :, k] - hq[:, :, k] \* rq[:, :, j])

```
sigma_qq = torch.stack(results, dim=2) assert sigma_qq.shape == (b, r, int((q * (q - 1)) / 2))
```

Yet, this computation would be quite inefficient. Instead, we compute interactions along all  $p$ , e.g.,  $e_1e_1$ ,  $e_1e_2$ ,  $e_1e_3$ ,

$e_2e_1$ ,  $e_2e_2$ ,  $e_2e_3$ ,  $e_3e_1$ ,  $e_3e_2$ ,  $e_3e_3$

Then select the triangular matrix without diagonals:  $e_1e_2$ ,  $e_1e_3$ ,  $e_2e_3$ .

```
compute_sigma_pq(*, hp, hq, rp, rq)
```

```
sum_{i=1}^p sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j
```

```
results = [] sigma_pq = torch.zeros(b, r, p, q) for i in range(p):
```

```
    for j in range(q):
```

```
        sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]
```

```
    print(sigma_pq.shape)
```

```
apply_coefficients(hp, hq, rp, rq)
```

Multiplying a base vector with its scalar coefficient

```
clifford_multiplication(h0, hp, hq, r0, rp, rq)
```

Compute our CL multiplication

```
h = h_0 + sum_{i=1}^p h_i e_i + sum_{j=p+1}^{p+q} h_j e_j
r = r_0 + sum_{i=1}^p r_i e_i + sum_{j=p+1}^{p+q} r_j e_j
```

$e_i^2 = +1$  for  $i \leq p$   $e_j^2 = -1$  for  $p < j \leq p+q$   $e_i e_j = -e_j e_i$  for  $i$

$e_j$

$h r = \text{sigma}_0 + \text{sigma}_p + \text{sigma}_q + \text{sigma}_{\{pp\}} + \text{sigma}_{\{q\}} + \text{sigma}_{\{pq\}}$  where

(1)  $\text{sigma}_0 = h_0 r_0 + \sum_{i=1}^p (h_0 r_i - h_i r_0) e_i - \sum_{j=p+1}^{p+q} (h_j r_j) e_j$

(2)  $\text{sigma}_p = \sum_{i=1}^p (h_0 r_i + h_i r_0) e_i$

(3)  $\text{sigma}_q = \sum_{j=p+1}^{p+q} (h_0 r_j + h_j r_0) e_j$

(4)  $\text{sigma}_{\{pp\}} = \sum_{i=1}^p \sum_{k=i+1}^p (h_i r_k - h_k r_i) e_i e_k$

(5)  $\text{sigma}_{\{qq\}} = \sum_{j=1}^{p+q-1} \sum_{k=j+1}^{p+q} (h_j r_k - h_k r_j) e_j e_k$

(6)  $\text{sigma}_{\{pq\}} = \sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$

```
construct_cl_multivector(x: torch.FloatTensor, r: int, p: int, q: int)
```

```
→ tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]
```

Construct a batch of multivectors  $\text{Cl}_{\{p,q\}}(\mathbb{R}^d)$

## Parameter

$x$ : torch.FloatTensor with  $(n,d)$  shape

**returns**

- **a0** (torch.FloatTensor with  $(n,r)$  shape)
- **ap** (torch.FloatTensor with  $(n,r,p)$  shape)
- **aq** (torch.FloatTensor with  $(n,r,q)$  shape)

```
forward_k_vs_with_explicit(x: torch.Tensor)
```

```
k_vs_all_score(bpe_head_ent_emb, bpe_rel_ent_emb, E)
```

**forward\_k\_vs\_all** (*x*: torch.Tensor) → torch.FloatTensor

Kvsall training

- (1) Retrieve real-valued embedding vectors for heads and relations  $\mathbb{R}^d$ .
- (2) Construct head entity and relation embeddings according to  $Cl_{\{p,q\}}(\mathbb{R}^d)$ .
- (3) Perform Cl multiplication
- (4) Inner product of (3) and all entity embeddings

forward\_k\_vs\_with\_explicit and this function are identical Parameter ——— *x*: torch.LongTensor with (n,2) shape :rtype: torch.FloatTensor with (n, **IE**) shape

**construct\_batch\_selected\_cl\_multivector** (*x*: torch.FloatTensor, *r*: int, *p*: int, *q*: int)  
→ tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Construct a batch of batchs multivectors  $Cl_{\{p,q\}}(\mathbb{R}^d)$

### Parameter

*x*: torch.FloatTensor with (n,k, d) shape

#### returns

- **a0** (torch.FloatTensor with (n,k, m) shape)
- **ap** (torch.FloatTensor with (n,k, m, p) shape)
- **aq** (torch.FloatTensor with (n,k, m, q) shape)

**forward\_k\_vs\_sample** (*x*: torch.LongTensor, *target\_entity\_idx*: torch.LongTensor) → torch.FloatTensor

### Parameter

*x*: torch.LongTensor with (n,2) shape

*target\_entity\_idx*: torch.LongTensor with (n, k ) shape k denotes the selected number of examples.

#### rtype

torch.FloatTensor with (n, k) shape

**score** (*h*, *r*, *t*)

**forward\_triples** (*x*: torch.Tensor) → torch.FloatTensor

### Parameter

*x*: torch.LongTensor with (n,3) shape

#### rtype

torch.FloatTensor with (n) shape

**class** dicee.models.CKeci (*args*)

Bases: *Keci*

Without learning dimension scaling

**name** = 'CKeci'

**requires\_grad\_for\_interactions** = False

```
class dicee.models.DeCaL(args)
```

Bases: `dicee.models.base_model.BaseKGE`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

### Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**name** = 'DeCaL'

**entity\_embeddings**

**relation\_embeddings**

**p**

**q**

**r**

**re**

**forward\_triples** (*x: torch.Tensor*)  $\rightarrow$  torch.FloatTensor

### Parameter

**x**: torch.LongTensor with (n, ) shape

**rtype**

torch.FloatTensor with (n) shape

**cl\_pqr** (*a: torch.tensor*) → torch.tensor

Input: tensor(batch\_size, emb\_dim) → output: tensor with 1+p+q+r components with size (batch\_size, emb\_dim/(1+p+q+r)) each.

1) takes a tensor of size (batch\_size, emb\_dim), split it into 1 + p + q + r components, hence 1+p+q+r must be a divisor of the emb\_dim. 2) Return a list of the 1+p+q+r components vectors, each are tensors of size (batch\_size, emb\_dim/(1+p+q+r))

**compute\_sigmas\_single** (*list\_h\_emb, list\_r\_emb, list\_t\_emb*)

here we compute all the sums with no others vectors interaction taken with the scalar product with t, that is,

$$s0 = h_0 r_0 t_0 s1 = \sum_{i=1}^p h_i r_i t_0 s2 = \sum_{j=p+1}^{p+q} h_j r_j t_0 s3 = \sum_{i=1}^q (h_0 r_i t_i + h_i r_0 t_i) s4 = \sum_{i=p+1}^{p+q} (h_0 r_i t_i + h_i r_0 t_i) s5 = \sum_{i=p+q+1}^{p+q+r} (h_0 r_i t_i + h_i r_0 t_i)$$

and return:

$$sigma_0 t = \sigma_0 \cdot t_0 = s0 + s1 - s2 s3, s4 \text{ and } s5$$

**compute\_sigmas\_multivect** (*list\_h\_emb, list\_r\_emb*)

Here we compute and return all the sums with vectors interaction for the same and different bases.

For same bases vectors interaction we have

$$\sigma_p p = \sum_{i=1}^{p-1} \sum_{i'=i+1}^p (h_i r_{i'} - h_{i'} r_i) (model \text{ the interactions between } e_i \text{ and } e_{i'} \text{ for } 1 \leq i, i' \leq p) \sigma_q q = \sum_{j=p+1}^{p+q-1} \sum_{j'=j+1}^{p+q} (h_j r_{j'} - h_{j'} r_j)$$

For different base vector interactions, we have

$$\sigma_p q = \sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) (interactions \text{ between } e_i \text{ and } e_j \text{ for } 1 \leq i \leq p \text{ and } p+1 \leq j \leq p+q) \sigma_p r = \sum_{i=1}^p \sum_{j=p+q+1}^{p+q+r} (h_i r_j - h_j r_i)$$

**forward\_k\_vs\_all** (*x: torch.Tensor*) → torch.FloatTensor

Kvsall training

- (1) Retrieve real-valued embedding vectors for heads and relations
- (2) Construct head entity and relation embeddings according to  $Cl_{\{p,q,r\}}(\mathbb{R}^d)$ .
- (3) Perform Cl multiplication
- (4) Inner product of (3) and all entity embeddings

forward\_k\_vs\_with\_explicit and this functions are identical Parameter ——— x: torch.LongTensor with (n, ) shape :rtype: torch.FloatTensor with (n, **IE**) shape

**apply\_coefficients** (*h0, hp, hq, hk, r0, rp, rq, rk*)

Multiplying a base vector with its scalar coefficient

**construct\_cl\_multivector** (*x: torch.FloatTensor, re: int, p: int, q: int, r: int*)  
→ tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Construct a batch of multivectors  $Cl_{\{p,q,r\}}(\mathbb{R}^d)$



## Parameter

x: torch.FloatTensor with (n,d) shape

### returns

- **a0** (torch.FloatTensor)
- **ap** (torch.FloatTensor)
- **aq** (torch.FloatTensor)
- **ar** (torch.FloatTensor)

**compute\_sigma\_pp** (hp, rp)

Compute .. math:

$$\sigma_{pp} = \sum_{i=1}^{p-1} \sum_{i'=i+1}^p (x_i y_{i'} - x_{i'} y_i)$$

sigma\_{pp} captures the interactions between along p bases For instance, let p e\_1, e\_2, e\_3, we compute interactions between e\_1 e\_2, e\_1 e\_3, and e\_2 e\_3 This can be implemented with a nested two for loops

results = [] for i in range(p - 1):

**for k in range(i + 1, p):**

        results.append(hp[:, :, i] \* rp[:, :, k] - hp[:, :, k] \* rp[:, :, i])

sigma\_pp = torch.stack(results, dim=2) assert sigma\_pp.shape == (b, r, int((p \* (p - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p, e.g., e1e1, e1e2, e1e3,

e2e1, e2e2, e2e3, e3e1, e3e2, e3e3

Then select the triangular matrix without diagonals: e1e2, e1e3, e2e3.

**compute\_sigma\_qq** (hq, rq)

Compute

$$\sigma_{qq}^* = \sum_{j=p+1}^{p+q-1} \sum_{j'=j+1}^{p+q} (x_j y_{j'} - x_{j'} y_j) \quad (16)$$

sigma\_{qq} captures the interactions between along q bases For instance, let q e\_1, e\_2, e\_3, we compute interactions between e\_1 e\_2, e\_1 e\_3, and e\_2 e\_3 This can be implemented with a nested two for loops

results = [] for j in range(q - 1):

**for k in range(j + 1, q):**

        results.append(hq[:, :, j] \* rq[:, :, k] - hq[:, :, k] \* rq[:, :, j])

sigma\_qq = torch.stack(results, dim=2) assert sigma\_qq.shape == (b, r, int((q \* (q - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p, e.g., e1e1, e1e2, e1e3,

e2e1, e2e2, e2e3, e3e1, e3e2, e3e3

Then select the triangular matrix without diagonals: e1e2, e1e3, e2e3.

**compute\_sigma\_rr** (hk, rk)

$$\sigma_{rr}^* = \sum_{k=p+q+1}^{p+q+r-1} \sum_{k'=k+1}^p (x_k y_{k'} - x_{k'} y_k)$$

**compute\_sigma\_pq**(\* , hp, hq, rp, rq)

Compute

$$\sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

results = [] sigma\_pq = torch.zeros(b, r, p, q) for i in range(p):

**for j in range(q):**

        sigma\_pq[:, :, i, j] = hp[:, :, i] \* rq[:, :, j] - hq[:, :, j] \* rp[:, :, i]

print(sigma\_pq.shape)

**compute\_sigma\_pr**(\* , hp, hk, rp, rk)

Compute

$$\sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

results = [] sigma\_pq = torch.zeros(b, r, p, q) for i in range(p):

**for j in range(q):**

        sigma\_pq[:, :, i, j] = hp[:, :, i] \* rq[:, :, j] - hq[:, :, j] \* rp[:, :, i]

print(sigma\_pq.shape)

**compute\_sigma\_qr**(\* , hq, hk, rq, rk)

$$\sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

results = [] sigma\_pq = torch.zeros(b, r, p, q) for i in range(p):

**for j in range(q):**

        sigma\_pq[:, :, i, j] = hp[:, :, i] \* rq[:, :, j] - hq[:, :, j] \* rp[:, :, i]

print(sigma\_pq.shape)

**class** dicee.models.**KeciTransformer**(args)

Bases: [Keci](#)

Keci with Transformer architecture.

Concatenates h0, hp, hq, r0, rp, rq into a single embedding vector and processes through transformer.

**name** = 'KeciTransformer'

**use\_clifford\_mul**

**seq\_len** = 1

**transformer**

**lm\_head**

**forward\_k\_vs\_all**(x: torch.Tensor) → torch.FloatTensor

Kvsall training

## Parameter

x: torch.LongTensor with (n,2) shape

### rtype

torch.FloatTensor with (n, **IE**) shape

**class** dicee.models.**BaseKGE** (args: dict)

Bases: *BaseKGELightning*

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

## Variables

**training** (*bool*) – Boolean represents whether this module is in training or evaluation mode.

**args**

**embedding\_dim** = None

**num\_entities** = None

**num\_relations** = None

**num\_tokens** = None

**learning\_rate** = None

**apply\_unit\_norm** = None

**input\_dropout\_rate** = None

```

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

loss

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

forward_byte_pair_encoded_k_vs_all (x: torch.LongTensor)

    Parameters
         $\mathbf{x} (B \times 2 \times T)$ 

forward_byte_pair_encoded_triple (x: Tuple[torch.LongTensor, torch.LongTensor])
    byte pair encoded neural link predictors

    Parameters
    -----

init_params_with_sanity_checking()

forward (x: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor],
        y_idx: torch.LongTensor = None)

    Parameters

    •  $\mathbf{x}$ 

    •  $\mathbf{y\_idx}$ 

```

```

        • ordered_bpe_entities
forward_triples (x: torch.LongTensor) → torch.Tensor

    Parameters
    x

forward_k_vs_all (*args, **kwargs)

forward_k_vs_sample (*args, **kwargs)

get_triple_representation (idx_hrt)

get_head_relation_representation (indexed_triple)

get_sentence_representation (x: torch.LongTensor)

    Parameters
        • (b (x shape)
        • 3
        • t)

get_bpe_head_and_relation_representation (x: torch.LongTensor)
    → Tuple[torch.FloatTensor, torch.FloatTensor]

    Parameters
        x (B × 2 × T)

get_embeddings () → Tuple[numpy.ndarray, numpy.ndarray]

class dicee.models.PykeenKGE (args: dict)
    Bases: dicee.models.base_model.BaseKGE

    A class for using knowledge graph embedding models implemented in Pykeen

    Notes: Pykeen_DistMult: C Pykeen_ComplEx: Pykeen_QuatE: Pykeen_MuRE: Pykeen_CP: Pykeen_HolE: Py-
    keen_HolE: Pykeen_HolE: Pykeen_TransD: Pykeen_TransE: Pykeen_TransF: Pykeen_TransH: Pykeen_TransR:

    model_kwargs

    name

    model

    loss_history = []

    args

    entity_embeddings = None

    relation_embeddings = None

forward_k_vs_all (x: torch.LongTensor)
    # => Explicit version by this we can apply bn and dropout
    # (1) Retrieve embeddings of heads and relations + apply Dropout & Normalization if given. h, r =
    self.get_head_relation_representation(x) # (2) Reshape (1). if self.last_dim > 0:
        h = h.reshape(len(x), self.embedding_dim, self.last_dim) r = r.reshape(len(x), self.embedding_dim,
        self.last_dim)

```

```

# (3) Reshape all entities. if self.last_dim > 0:
    t = self.entity_embeddings.weight.reshape(self.num_entities, self.embedding_dim, self.last_dim)

else:
    t = self.entity_embeddings.weight

# (4) Call the score_t from interactions to generate triple scores. return self.interaction.score_t(h=h, r=r,
all_entities=t, slice_size=1)

forward_triples (x: torch.LongTensor) → torch.FloatTensor
# => Explicit version by this we can apply bn and dropout

# (1) Retrieve embeddings of heads, relations and tails and apply Dropout & Normalization if given. h, r, t =
self.get_triple_representation(x) # (2) Reshape (1). if self.last_dim > 0:
    h = h.reshape(len(x), self.embedding_dim, self.last_dim) r = r.reshape(len(x), self.embedding_dim,
    self.last_dim) t = t.reshape(len(x), self.embedding_dim, self.last_dim)

# (3) Compute the triple score return self.interaction.score(h=h, r=r, t=t, slice_size=None, slice_dim=0)

abstractmethod forward_k_vs_sample (x: torch.LongTensor, target_entity_idx)

```

**class** dicee.models.**BaseKGE** (args: dict)

Bases: *BaseKGELightning*

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

#### Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

#### Variables

**training** (bool) – Boolean represents whether this module is in training or evaluation mode.

```

args

embedding_dim = None

num_entities = None

num_relations = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

loss

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

forward_byte_pair_encoded_k_vs_all (x: torch.LongTensor)

```

#### Parameters

$\mathbf{x} (B \times 2 \times T)$

```

forward_byte_pair_encoded_triple (x: Tuple[torch.LongTensor, torch.LongTensor])
    byte pair encoded neural link predictors

    Parameters
    -----

init_params_with_sanity_checking()

forward (x: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor],
         y_idx: torch.LongTensor = None)

    Parameters
    • x
    • y_idx
    • ordered_bpe_entities

forward_triples (x: torch.LongTensor) → torch.Tensor

    Parameters
    • x

forward_k_vs_all (*args, **kwargs)

forward_k_vs_sample (*args, **kwargs)

get_triple_representation (idx_hrt)

get_head_relation_representation (indexed_triple)

get_sentence_representation (x: torch.LongTensor)

    Parameters
    • (b (x shape)
    • 3
    • t)

get_bpe_head_and_relation_representation (x: torch.LongTensor)
    → Tuple[torch.FloatTensor, torch.FloatTensor]

    Parameters
    • x (B × 2 × T)

get_embeddings () → Tuple[numpy.ndarray, numpy.ndarray]

class dicee.models.FMult (args)
    Bases: dicee.models.base_model.BaseKGE
    Learning Knowledge Neural Graphs
    name = 'FMult'
    entity_embeddings
    relation_embeddings
    k

```



```

num_sample = 50

gamma

roots

weights

compute_func (weights: torch.FloatTensor, x) → torch.FloatTensor

chain_func (weights, x: torch.FloatTensor)

forward_triples (idx_triple: torch.Tensor) → torch.Tensor

```

#### Parameters

**x**

```

class dicee.models.GFMult (args)
    Bases: dicee.models.base_model.BaseKGE
    Learning Knowledge Neural Graphs
    name = 'GFMult'
    entity_embeddings
    relation_embeddings
    k
    num_sample = 250
    roots
    weights
    compute_func (weights: torch.FloatTensor, x) → torch.FloatTensor
    chain_func (weights, x: torch.FloatTensor)
    forward_triples (idx_triple: torch.Tensor) → torch.Tensor

```

#### Parameters

**x**

```

class dicee.models.FMult2 (args)
    Bases: dicee.models.base_model.BaseKGE
    Learning Knowledge Neural Graphs
    name = 'FMult2'
    n_layers = 3
    k
    n = 50
    score_func = 'compositional'
    discrete_points

```

```

entity_embeddings

relation_embeddings

build_func (Vec)

build_chain_funcs (list_Vec)

compute_func (W, b, x) → torch.FloatTensor

function (list_W, list_b)

trapezoid (list_W, list_b)

forward_triples (idx_triple: torch.Tensor) → torch.Tensor

```

#### Parameters

**x**

```
class dicee.models.LFMult1 (args)
```

Bases: *[dicee.models.base\\_model.BaseKGE](#)*

Embedding with trigonometric functions. We represent all entities and relations in the complex number space as:  $f(x) = \sum_{k=0}^{d-1} w_k e^{kix}$ . and use the three different scoring function as in the paper to evaluate the score

```
name = 'LFMult1'
```

```
entity_embeddings
```

```
relation_embeddings
```

```
forward_triples (idx_triple)
```

#### Parameters

**x**

```
tri_score (h, r, t)
```

```
vtp_score (h, r, t)
```

```
class dicee.models.LFMult (args)
```

Bases: *[dicee.models.base\\_model.BaseKGE](#)*

Embedding with polynomial functions. We represent all entities and relations in the polynomial space as:  $f(x) = \sum_{i=0}^{d-1} a_i x^i$  and use the three different scoring function as in the paper to evaluate the score. We also consider combining with Neural Networks.

```
name = 'LFMult'
```

```
entity_embeddings
```

```
relation_embeddings
```

```
degree
```

```
m
```

```
x_values
```

**forward\_triples** (*idx\_triple*)

#### Parameters

**x**

**construct\_multi\_coeff** (*x*)

**poly\_NN** (*x, coefh, coefr, coeft*)

Constructing a 2 layers NN to represent the embeddings.  $h = \text{sigma}(w_h^T x + b_h)$ ,  $r = \text{sigma}(w_r^T x + b_r)$ ,  $t = \text{sigma}(w_t^T x + b_t)$

**linear** (*x, w, b*)

**scalar\_batch\_NN** (*a, b, c*)

element wise multiplication between a,b and c: Inputs : a, b, c ==> torch.tensor of size batch\_size x m x d  
Output : a tensor of size batch\_size x d

**tri\_score** (*coeff\_h, coeff\_r, coeff\_t*)

this part implement the trilinear scoring techniques:

$$\text{score}(h,r,t) = \int_0^1 h(x)r(x)t(x) dx = \sum_{i,j,k=0}^{d-1} \text{dfrac}\{a_i*b_j*c_k\} \{1+(i+j+k)*d\}$$

1. generate the range for i,j and k from [0 d-1]
2. perform  $\text{dfrac}\{a_i*b_j*c_k\} \{1+(i+j+k)*d\}$  in parallel for every batch
3. take the sum over each batch

**vtp\_score** (*h, r, t*)

this part implement the vector triple product scoring techniques:

$$\text{score}(h,r,t) = \int_0^1 h(x)r(x)t(x) dx = \sum_{i,j,k=0}^{d-1} \text{dfrac}\{a_i*c_j*b_k - b_i*c_j*a_k\} \{(1+(i+j)*d)(1+k)\}$$

1. generate the range for i,j and k from [0 d-1]
2. Compute the first and second terms of the sum
3. Multiply with then denominator and take the sum
4. take the sum over each batch

**comp\_func** (*h, r, t*)

this part implement the function composition scoring techniques: i.e. score = <hor, t>

**polynomial** (*coeff, x, degree*)

This function takes a matrix tensor of coefficients (coeff), a tensor vector of points x and range of integer [0,1,...d] and return a vector tensor (coeff[0][0] + coeff[0][1]x +...+ coeff[0][d]x^d,

$$\text{coeff}[1][0] + \text{coeff}[1][1]x + \dots + \text{coeff}[1][d]x^d)$$

**pop** (*coeff, x, degree*)

This function allow us to evaluate the composition of two polynomes without for loops :) it takes a matrix tensor of coefficients (coeff), a matrix tensor of points x and range of integer [0,1,...d]

**and return a tensor (coeff[0][0] + coeff[0][1]x +...+ coeff[0][d]x^d,**

$$\text{coeff}[1][0] + \text{coeff}[1][1]x + \dots + \text{coeff}[1][d]x^d)$$

**class** dicee.models.DualE (*args*)

Bases: *dicee.models.base\_model.BaseKGE*

Dual Quaternion Knowledge Graph Embeddings (<https://ojs.aaai.org/index.php/AAAI/article/download/16850/16657>)

```

name = 'DualE'

entity_embeddings

relation_embeddings

num_ent = None

kvsall_score(e_1_h, e_2_h, e_3_h, e_4_h, e_5_h, e_6_h, e_7_h, e_8_h, e_1_t, e_2_t, e_3_t, e_4_t,
            e_5_t, e_6_t, e_7_t, e_8_t, r_1, r_2, r_3, r_4, r_5, r_6, r_7, r_8) → torch.tensor
KvsAll scoring function

```

### Input

x: torch.LongTensor with (n, ) shape

### Output

torch.FloatTensor with (n) shape

```

forward_triples(idx_triple: torch.tensor) → torch.tensor
Negative Sampling forward pass:

```

### Input

x: torch.LongTensor with (n, ) shape

### Output

torch.FloatTensor with (n) shape

```

forward_k_vs_all(x)
KvsAll forward pass

```

### Input

x: torch.LongTensor with (n, ) shape

### Output

torch.FloatTensor with (n) shape

```

T(x: torch.tensor) → torch.tensor
Transpose function

```

Input: Tensor with shape (nxm) Output: Tensor with shape (mxn)

## dicee.query\_generator

### Classes

---

*QueryGenerator*

---

## Module Contents

```
class dicee.query_generator.QueryGenerator(train_path: str, val_path: str, test_path: str,
    ent2id: Dict = None, rel2id: Dict = None, seed: int = 1, gen_valid: bool = False,
    gen_test: bool = True)

    train_path

    val_path

    test_path

    gen_valid = False

    gen_test = True

    seed = 1

    max_ans_num = 1000000.0

    mode

    ent2id = None

    rel2id: Dict = None

    ent_in: Dict

    ent_out: Dict

    query_name_to_struct

    list2tuple(list_data)

    tuple2list(x: List | Tuple) → List | Tuple
        Convert a nested tuple to a nested list.

    set_global_seed(seed: int)
        Set seed

    construct_graph(paths: List[str]) → Tuple[Dict, Dict]
        Construct graph from triples Returns dicts with incoming and outgoing edges

    fill_query(query_structure: List[str | List], ent_in: Dict, ent_out: Dict, answer: int) → bool
        Private method for fill_query logic.

    achieve_answer(query: List[str | List], ent_in: Dict, ent_out: Dict) → set
        Private method for achieve_answer logic. @TODO: Document the code

    write_links(ent_out, small_ent_out)

    ground_queries(query_structure: List[str | List], ent_in: Dict, ent_out: Dict, small_ent_in: Dict,
        small_ent_out: Dict, gen_num: int, query_name: str)
        Generating queries and achieving answers

    unmap(query_type, queries, tp_answers, fp_answers, fn_answers)

    unmap_query(query_structure, query, id2ent, id2rel)
```

**generate\_queries** (*query\_struct: List, gen\_num: int, query\_type: str*)

Passing incoming and outgoing edges to ground queries depending on mode [train valid or text] and getting queries and answers in return @ TODO: create a class for each single query struct

**save\_queries** (*query\_type: str, gen\_num: int, save\_path: str*)

**abstractmethod load\_queries** (*path*)

**get\_queries** (*query\_type: str, gen\_num: int*)

**static save\_queries\_and\_answers** (*path: str, data: List[Tuple[str, Tuple[collections.defaultdict]]]*)  
→ None

Save Queries into Disk

**static load\_queries\_and\_answers** (*path: str*) → List[Tuple[str, Tuple[collections.defaultdict]]]

Load Queries from Disk to Memory

## **dicee.read\_preprocess\_save\_load\_kg**

### **Submodules**

#### **dicee.read\_preprocess\_save\_load\_kg.preprocess**

### **Classes**

*PreprocessKG*

Preprocess the data in memory

### **Module Contents**

**class** dicee.read\_preprocess\_save\_load\_kg.preprocess.**PreprocessKG** (*kg*)

Preprocess the data in memory

**kg**

**start** () → None

Preprocess train, valid and test datasets stored in knowledge graph instance

#### **Parameter**

**rtype**

None

**preprocess\_with\_byte\_pair\_encoding** ()

**preprocess\_with\_byte\_pair\_encoding\_with\_padding** () → None

Preprocess with byte pair encoding and add padding

**preprocess\_with\_pandas** () → None

Preprocess with pandas: add reciprocal triples, construct vocabulary, and index datasets

**preprocess\_with\_polars** () → None

Preprocess with polars: add reciprocal triples and create indexed datasets

`sequential_vocabulary_construction()` → None

- (1) Read input data into memory
- (2) Remove triples with a condition
- (3) **Serialize vocabularies in a pandas dataframe where**  
=> the index is integer and => a single column is string (e.g. URI)

`dicee.read_preprocess_save_load_kg.read_from_disk`

## Classes

---

*ReadFromDisk*

Read the data from disk into memory

---

## Module Contents

`class dicee.read_preprocess_save_load_kg.read_from_disk.ReadFromDisk(kg)`

Read the data from disk into memory

**kg**

`start()` → None

Read a knowledge graph from disk into memory

Data will be available at the `train_set`, `test_set`, `valid_set` attributes.

### Parameter

None

**rtype**

None

`add_noisy_triples_into_training()`

`dicee.read_preprocess_save_load_kg.save_load_disk`

## Classes

---

*LoadSaveToDisk*

## Module Contents

`class dicee.read_preprocess_save_load_kg.save_load_disk.LoadSaveToDisk(kg)`

**kg**

`save()`

`load()`

## dicee.read\_preprocess\_save\_load\_kg.util

### Functions

|                                                                                        |                                                                                                                          |
|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>polars_dataframe_indexer</code> ( $\rightarrow$ polars.DataFrame)                | Replaces 'subject', 'relation', and 'object' columns in the input Polars DataFrame with their corresponding index values |
| <code>pandas_dataframe_indexer</code> ( $\rightarrow$ pandas.DataFrame)                | Replaces 'subject', 'relation', and 'object' columns in the input Pandas DataFrame with their corresponding index values |
| <code>apply_reciprocal_or_noise</code> (add_reciprocal, eval_model)                    | Add reciprocal triples if conditions are met                                                                             |
| <code>timeit</code> (func)                                                             |                                                                                                                          |
| <code>read_with_polars</code> ( $\rightarrow$ polars.DataFrame)                        | Load and Preprocess via Polars                                                                                           |
| <code>read_with_pandas</code> (data_path[, read_only_few, ...])                        | Load and Preprocess via Pandas                                                                                           |
| <code>read_from_disk</code> ( $\rightarrow$ Tuple[polars.DataFrame, pandas.DataFrame]) |                                                                                                                          |
| <code>count_triples</code> ( $\rightarrow$ int)                                        | Returns the total number of triples in the triple store.                                                                 |
| <code>fetch_worker</code> (endpoint, offsets, chunk_size, ...)                         | Worker process: fetch assigned chunks and save to disk with per-worker tqdm.                                             |
| <code>read_from_triple_store_with_polars</code> (endpoint[, ...])                      | Main function to read all triples in parallel, save as Parquet, and load into Polars dataframe.                          |
| <code>read_from_triple_store_with_pandas</code> ([endpoint])                           | Read triples from triple store into pandas dataframe                                                                     |
| <code>get_er_vocab</code> (data[, file_path])                                          |                                                                                                                          |
| <code>get_re_vocab</code> (data[, file_path])                                          |                                                                                                                          |
| <code>get_ee_vocab</code> (data[, file_path])                                          |                                                                                                                          |
| <code>create_constraints</code> (triples[, file_path])                                 |                                                                                                                          |
| <code>load_with_pandas</code> ( $\rightarrow$ None)                                    | Deserialize data                                                                                                         |
| <code>save_numpy_ndarray</code> (*, data, file_path)                                   |                                                                                                                          |
| <code>load_numpy_ndarray</code> (*, file_path)                                         |                                                                                                                          |
| <code>save_pickle</code> (*, data[, file_path])                                        |                                                                                                                          |
| <code>load_pickle</code> (*[, file_path])                                              |                                                                                                                          |
| <code>create_recipriocal_triples</code> (x)                                            | Add inverse triples into dask dataframe                                                                                  |
| <code>dataset_sanity_checking</code> ( $\rightarrow$ None)                             |                                                                                                                          |

### Module Contents

`dicee.read_preprocess_save_load_kg.util.polars_dataframe_indexer` (  
    *df\_polars*: *polars.DataFrame*, *idx\_entity*: *polars.DataFrame*, *idx\_relation*: *polars.DataFrame*)  
     $\rightarrow$  *polars.DataFrame*

Replaces 'subject', 'relation', and 'object' columns in the input Polars DataFrame with their corresponding index values from the entity and relation index DataFrames.

This function processes the DataFrame in three main steps: 1. Replace the 'relation' values with the corresponding index from *idx\_relation*. 2. Replace the 'subject' values with the corresponding index from *idx\_entity*. 3. Replace the 'object' values with the corresponding index from *idx\_entity*.

#### Parameters:

##### **df\_polars**

[polars.DataFrame] The input Polars DataFrame containing columns: 'subject', 'relation', and 'object'.

##### **idx\_entity**



[polars.DataFrame] A Polars DataFrame that contains the mapping between entity names and their corresponding indices. Must have columns: 'entity' and 'index'.

#### **idx\_relation**

[polars.DataFrame] A Polars DataFrame that contains the mapping between relation names and their corresponding indices. Must have columns: 'relation' and 'index'.

#### **Returns:**

#### **polars.DataFrame**

A DataFrame with the 'subject', 'relation', and 'object' columns replaced by their corresponding indices.

#### **Example Usage:**

```
>>> df_polars = pl.DataFrame({
    "subject": ["Alice", "Bob", "Charlie"],
    "relation": ["knows", "works_with", "lives_in"],
    "object": ["Dave", "Eve", "Frank"]
})
>>> idx_entity = pl.DataFrame({
    "entity": ["Alice", "Bob", "Charlie", "Dave", "Eve", "Frank"],
    "index": [0, 1, 2, 3, 4, 5]
})
>>> idx_relation = pl.DataFrame({
    "relation": ["knows", "works_with", "lives_in"],
    "index": [0, 1, 2]
})
>>> polars_dataframe_indexer(df_polars, idx_entity, idx_relation)
```

#### **Steps:**

1. Join the input DataFrame *df\_polars* on the 'relation' column with *idx\_relation* to replace the relations with their indices.
2. Join on 'subject' to replace it with the corresponding entity index using a left join on *idx\_entity*.
3. Join on 'object' to replace it with the corresponding entity index using a left join on *idx\_entity*.
4. Select only the 'subject', 'relation', and 'object' columns to return the final result.

```
dicee.read_preprocess_save_load_kg.util.pandas_dataframe_indexer(
    df_pandas: pandas.DataFrame, idx_entity: pandas.DataFrame, idx_relation: pandas.DataFrame
) → pandas.DataFrame
```

Replaces 'subject', 'relation', and 'object' columns in the input Pandas DataFrame with their corresponding index values from the entity and relation index DataFrames.

#### **Parameters:**

#### **df\_pandas**

[pd.DataFrame] The input Pandas DataFrame containing columns: 'subject', 'relation', and 'object'.

#### **idx\_entity**

[pd.DataFrame] A Pandas DataFrame that contains the mapping between entity names and their corresponding indices. Must have columns: 'entity' and 'index'.

### **idx\_relation**

[pd.DataFrame] A Pandas DataFrame that contains the mapping between relation names and their corresponding indices. Must have columns: 'relation' and 'index'.

### **Returns:**

#### **pd.DataFrame**

A DataFrame with the 'subject', 'relation', and 'object' columns replaced by their corresponding indices.

```
dicee.read_preprocess_save_load_kg.util.apply_reciprocal_or_noise (add_reciprocal: bool,  
    eval_model: str, df: object = None, info: str = None)
```

Add reciprocal triples if conditions are met

```
dicee.read_preprocess_save_load_kg.util.timeit (func)
```

```
dicee.read_preprocess_save_load_kg.util.read_with_polars (data_path,  
    read_only_few: int = None, sample_triples_ratio: float = None, separator: str = None)  
    → polars.DataFrame
```

Load and Preprocess via Polars

```
dicee.read_preprocess_save_load_kg.util.read_with_pandas (data_path,  
    read_only_few: int = None, sample_triples_ratio: float = None, separator: str = None)
```

Load and Preprocess via Pandas

```
dicee.read_preprocess_save_load_kg.util.read_from_disk (data_path: str,  
    read_only_few: int = None, sample_triples_ratio: float = None, backend: str = None,  
    separator: str = None) → Tuple[polars.DataFrame, pandas.DataFrame]
```

```
dicee.read_preprocess_save_load_kg.util.count_triples (endpoint: str) → int
```

Returns the total number of triples in the triple store.

```
dicee.read_preprocess_save_load_kg.util.fetch_worker (endpoint: str, offsets: list[int],  
    chunk_size: int, output_dir: str, worker_id: int)
```

Worker process: fetch assigned chunks and save to disk with per-worker tqdm.

```
dicee.read_preprocess_save_load_kg.util.read_from_triple_store_with_polars (  
    endpoint: str, chunk_size: int = 500000, output_dir: str = 'triples_parquet')
```

Main function to read all triples in parallel, save as Parquet, and load into Polars dataframe.

```
dicee.read_preprocess_save_load_kg.util.read_from_triple_store_with_pandas (  
    endpoint: str = None)
```

Read triples from triple store into pandas dataframe

```
dicee.read_preprocess_save_load_kg.util.get_er_vocab (data, file_path: str = None)
```

```
dicee.read_preprocess_save_load_kg.util.get_re_vocab (data, file_path: str = None)
```

```
dicee.read_preprocess_save_load_kg.util.get_ee_vocab (data, file_path: str = None)
```

```
dicee.read_preprocess_save_load_kg.util.create_constraints (triples, file_path: str = None)
```

(1) Extract domains and ranges of relations

(2) Store a mapping from relations to entities that are outside of the domain and range. Create constrained entities based on the range of relations :param triples: :return: Tuple[dict, dict]

```
dicee.read_preprocess_save_load_kg.util.load_with_pandas (self) → None
```

Deserialize data

```
dicee.read_preprocess_save_load_kg.util.save_numpy_ndarray(* , data: numpy.ndarray,  
    file_path: str)
```

```
dicee.read_preprocess_save_load_kg.util.load_numpy_ndarray(* , file_path: str)
```

```
dicee.read_preprocess_save_load_kg.util.save_pickle(* , data: object, file_path=str)
```

```
dicee.read_preprocess_save_load_kg.util.load_pickle(* , file_path=str)
```

```
dicee.read_preprocess_save_load_kg.util.create_recipriocal_triples(x)
```

Add inverse triples into dask dataframe :param x: :return:

```
dicee.read_preprocess_save_load_kg.util.dataset_sanity_checking(  
    train_set: numpy.ndarray, num_entities: int, num_relations: int) → None
```

#### Parameters

- **train\_set**
- **num\_entities**
- **num\_relations**

#### Returns

### Classes

|                       |                                     |
|-----------------------|-------------------------------------|
| <i>PreprocessKG</i>   | Preprocess the data in memory       |
| <i>LoadSaveToDisk</i> |                                     |
| <i>ReadFromDisk</i>   | Read the data from disk into memory |

### Package Contents

```
class dicee.read_preprocess_save_load_kg.PreprocessKG(kg)
```

Preprocess the data in memory

**kg**

**start**() → None

Preprocess train, valid and test datasets stored in knowledge graph instance

#### Parameter

**rtype**

None

**preprocess\_with\_byte\_pair\_encoding**()

**preprocess\_with\_byte\_pair\_encoding\_with\_padding**() → None

Preprocess with byte pair encoding and add padding

**preprocess\_with\_pandas**() → None

Preprocess with pandas: add reciprocal triples, construct vocabulary, and index datasets

**preprocess\_with\_polars**() → None

Preprocess with polars: add reciprocal triples and create indexed datasets

**sequential\_vocabulary\_construction()** → None

- (1) Read input data into memory
- (2) Remove triples with a condition
- (3) **Serialize vocabularies in a pandas dataframe where**  
=> the index is integer and => a single column is string (e.g. URI)

**class** dicee.read\_preprocess\_save\_load\_kg.**LoadSaveToDisk**(kg)

**kg**

**save()**

**load()**

**class** dicee.read\_preprocess\_save\_load\_kg.**ReadFromDisk**(kg)

Read the data from disk into memory

**kg**

**start()** → None

Read a knowledge graph from disk into memory

Data will be available at the train\_set, test\_set, valid\_set attributes.

### Parameter

None

**rtype**

None

**add\_noisy\_triples\_into\_training()**

## **dicee.sanity\_checkers**

### **Functions**

|                                                          |                                                      |
|----------------------------------------------------------|------------------------------------------------------|
| <code>is_sparql_endpoint_alive([sparql_endpoint])</code> |                                                      |
| <code>validate_knowledge_graph(args)</code>              | Validating the source of knowledge graph             |
| <code>sanity_checking_with_arguments(args)</code>        |                                                      |
| <code>sanity_check_callback_args(args)</code>            | Perform sanity checks on callback-related arguments. |

### **Module Contents**

`dicee.sanity_checkers.is_sparql_endpoint_alive(sparql_endpoint: str = None)`

`dicee.sanity_checkers.validate_knowledge_graph(args)`

Validating the source of knowledge graph

`dicee.sanity_checkers.sanity_checking_with_arguments(args)`

`dicee.sanity_checkers.sanity_check_callback_args(args)`

Perform sanity checks on callback-related arguments.

## dicee.scripts

### Submodules

#### dicee.scripts.index\_serve

```
$ docker pull qdrant/qdrant && docker run -p 6333:6333 -p 6334:6334 -v $(pwd)/qdrant_storage:/qdrant/storage:z  
qdrant/qdrant $ dicee_vector_db -index -serve -path CountryEmbeddings -collection "countries_vdb"
```

### Attributes

|                              |
|------------------------------|
| <code>app</code>             |
| <code>neural_searcher</code> |

### Classes

|                                |
|--------------------------------|
| <code>NeuralSearcher</code>    |
| <code>StringListRequest</code> |

### Functions

|                                               |
|-----------------------------------------------|
| <code>get_default_arguments()</code>          |
| <code>index(args)</code>                      |
| <code>root()</code>                           |
| <code>search_embeddings(q)</code>             |
| <code>retrieve_embeddings(q)</code>           |
| <code>search_embeddings_batch(request)</code> |
| <code>serve(args)</code>                      |
| <code>main()</code>                           |

### Module Contents

```
dicee.scripts.index_serve.get_default_arguments()  
  
dicee.scripts.index_serve.index(args)  
  
dicee.scripts.index_serve.app  
  
dicee.scripts.index_serve.neural_searcher = None  
  
class dicee.scripts.index_serve.NeuralSearcher(args)  
  
    collection_name  
  
    entity_to_idx = None  
  
    qdrant_client  
  
    topk = 5  
  
    retrieve_embedding(entity: str = None, entities: List[str] = None) → List
```

```

    search(entity: str)

async dicee.scripts.index_serve.root()

async dicee.scripts.index_serve.search_embeddings(q: str)

async dicee.scripts.index_serve.retrieve_embeddings(q: str)

class dicee.scripts.index_serve.StringListRequest
    Bases: pydantic.BaseModel

    queries: List[str]

    reducer: str | None = None

async dicee.scripts.index_serve.search_embeddings_batch(request: StringListRequest)

dicee.scripts.index_serve.serve(args)

dicee.scripts.index_serve.main()

```

## dicee.scripts.run

### Functions

|                                                                          |                                                         |
|--------------------------------------------------------------------------|---------------------------------------------------------|
| <code>get_default_arguments([description])</code><br><code>main()</code> | Extends pytorch_lightning Trainer's arguments with ours |
|--------------------------------------------------------------------------|---------------------------------------------------------|

## Module Contents

```

dicee.scripts.run.get_default_arguments(description=None)
    Extends pytorch_lightning Trainer's arguments with ours

dicee.scripts.run.main()

```

## dicee.static\_funcs

Static utility functions for DICE embeddings.

This module provides utility functions for model initialization, data loading, serialization, and various helper operations.

### Attributes

|                             |  |
|-----------------------------|--|
| <code>MODEL_REGISTRY</code> |  |
|-----------------------------|--|

### Functions

|                                                               |                                                  |
|---------------------------------------------------------------|--------------------------------------------------|
| <code>create_recipriocal_triples(→ pandas.DataFrame)</code>   | Add inverse triples to a DataFrame.              |
| <code>get_er_vocab(→ Dict[Tuple[int, int], List[int]])</code> | Build entity-relation to tail vocabulary.        |
| <code>get_re_vocab(→ Dict[Tuple[int, int], List[int]])</code> | Build relation-entity (tail) to head vocabulary. |

continues on next page

Table 55 – continued from previous page

|                                                                          |                                                                                      |
|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <code>get_ee_vocab(→ Dict[Tuple[int, int], List[int]])</code>            | Build entity-entity to relation vocabulary.                                          |
| <code>timeit(→ Callable)</code>                                          | Decorator to measure and print execution time and memory usage.                      |
| <code>save_pickle(→ None)</code>                                         | Save data to a pickle file.                                                          |
| <code>load_pickle(→ object)</code>                                       | Load data from a pickle file.                                                        |
| <code>load_term_mapping(→ Union[dict, polars.DataFrame])</code>          | Load term-to-index mapping from pickle or CSV file.                                  |
| <code>select_model(args[, is_continual_training, storage_path])</code>   |                                                                                      |
| <code>load_model(→ Tuple[object, Tuple[dict, dict]])</code>              | Load weights and initialize pytorch module from namespace arguments                  |
| <code>load_model_ensemble(...)</code>                                    | Construct Ensemble Of weights and initialize pytorch module from namespace arguments |
| <code>save_numpy_ndarray(*, data, file_path)</code>                      |                                                                                      |
| <code>numpy_data_type_changer(→ numpy.ndarray)</code>                    | Detect most efficient data type for a given triples                                  |
| <code>save_checkpoint_model(→ None)</code>                               | Store Pytorch model into disk                                                        |
| <code>store(→ None)</code>                                               |                                                                                      |
| <code>add_noisy_triples(→ pandas.DataFrame)</code>                       | Add randomly constructed triples                                                     |
| <code>read_or_load_kg(args, cls)</code>                                  |                                                                                      |
| <code>intialize_model(...)</code>                                        | Initialize a knowledge graph embedding model.                                        |
| <code>load_json(→ Dict)</code>                                           | Load JSON file into a dictionary.                                                    |
| <code>save_embeddings(→ None)</code>                                     | Save embeddings to a CSV file.                                                       |
| <code>vocab_to_parquet(vocab_to_idx, name, ...)</code>                   |                                                                                      |
| <code>create_experiment_folder(→ str)</code>                             | Create a timestamped experiment folder.                                              |
| <code>continual_training_setup_executor(→ None)</code>                   |                                                                                      |
| <code>exponential_function(→ torch.FloatTensor)</code>                   |                                                                                      |
| <code>load_numpy(→ numpy.ndarray)</code>                                 |                                                                                      |
| <code>evaluate(entity_to_idx, scores, easy_answers, hard_answers)</code> | # @TODO: CD: Renamed this function                                                   |
| <code>download_file(url[, destination_folder])</code>                    |                                                                                      |
| <code>download_files_from_url(→ None)</code>                             |                                                                                      |
| <code>download_pretrained_model(→ str)</code>                            |                                                                                      |
| <code>write_csv_from_model_parallel(path)</code>                         | Create                                                                               |
| <code>from_pretrained_model_write_embeddings_into_csv(path, None)</code> |                                                                                      |

## Module Contents

`dicee.static_funcs.MODEL_REGISTRY: Dict[str, Tuple[Type, str]]`

`dicee.static_funcs.create_recipriocal_triples(df: pandas.DataFrame) → pandas.DataFrame`

Add inverse triples to a DataFrame.

For each triple (s, p, o), creates an inverse triple (o, p\_inverse, s).

### Parameters

**df** – DataFrame with ‘subject’, ‘relation’, and ‘object’ columns.

### Returns

DataFrame with original and inverse triples concatenated.

`dicee.static_funcs.get_er_vocab(data: numpy.ndarray, file_path: str | None = None)`

→ Dict[Tuple[int, int], List[int]]

Build entity-relation to tail vocabulary.

### Parameters

- **data** – Array of triples with shape (n, 3) where columns are (head, relation, tail).
- **file\_path** – Optional path to save the vocabulary as pickle.

### Returns

Dictionary mapping (head, relation) pairs to list of tail entities.

```
dicee.static_funcs.get_re_vocab(data: numpy.ndarray, file_path: str | None = None)  
→ Dict[Tuple[int, int], List[int]]
```

Build relation-entity (tail) to head vocabulary.

### Parameters

- **data** – Array of triples with shape (n, 3) where columns are (head, relation, tail).
- **file\_path** – Optional path to save the vocabulary as pickle.

### Returns

Dictionary mapping (relation, tail) pairs to list of head entities.

```
dicee.static_funcs.get_ee_vocab(data: numpy.ndarray, file_path: str | None = None)  
→ Dict[Tuple[int, int], List[int]]
```

Build entity-entity to relation vocabulary.

### Parameters

- **data** – Array of triples with shape (n, 3) where columns are (head, relation, tail).
- **file\_path** – Optional path to save the vocabulary as pickle.

### Returns

Dictionary mapping (head, tail) pairs to list of relations.

```
dicee.static_funcs.timeit(func: Callable) → Callable
```

Decorator to measure and print execution time and memory usage.

### Parameters

**func** – Function to be timed.

### Returns

Wrapped function that prints timing information.

```
dicee.static_funcs.save_pickle(*, data: object | None = None, file_path: str) → None
```

Save data to a pickle file.

Note: Consider using more portable formats (JSON, Parquet) for new code.

### Parameters

- **data** – Object to serialize. If None, nothing is saved.
- **file\_path** – Path where the pickle file will be saved.

```
dicee.static_funcs.load_pickle(file_path: str) → object
```

Load data from a pickle file.

Note: Consider using more portable formats (JSON, Parquet) for new code.

### Parameters

**file\_path** – Path to the pickle file.

### Returns

Deserialized object from the pickle file.



`dicee.static_funcs.load_term_mapping (file_path: str) → dict | polars.DataFrame`

Load term-to-index mapping from pickle or CSV file.

Attempts to load from pickle first, falls back to CSV if not found.

#### Parameters

**file\_path** – Base path without extension.

#### Returns

Dictionary or Polars DataFrame containing the mapping.

`dicee.static_funcs.select_model (args: dict, is_continual_training: bool = None, storage_path: str = None)`

`dicee.static_funcs.load_model (path_of_experiment_folder: str, model_name='model.pt', verbose=0) → Tuple[object, Tuple[dict, dict]]`

Load weights and initialize pytorch module from namespace arguments

`dicee.static_funcs.load_model_ensemble (path_of_experiment_folder: str) → Tuple[dicee.models.base_model.BaseKGE, Tuple[pandas.DataFrame, pandas.DataFrame]]`

Construct Ensemble Of weights and initialize pytorch module from namespace arguments

- (1) Detect models under given path
- (2) Accumulate parameters of detected models
- (3) Normalize parameters
- (4) Insert (3) into model.

`dicee.static_funcs.save_numpy_ndarray (*, data: numpy.ndarray, file_path: str)`

`dicee.static_funcs.numpy_data_type_changer (train_set: numpy.ndarray, num: int) → numpy.ndarray`

Detect most efficient data type for a given triples :param train\_set: :param num: :return:

`dicee.static_funcs.save_checkpoint_model (model, path: str) → None`

Store Pytorch model into disk

`dicee.static_funcs.store (trained_model, model_name: str = 'model', full_storage_path: str = None, save_embeddings_as_csv=False) → None`

`dicee.static_funcs.add_noisy_triples (train_set: pandas.DataFrame, add_noise_rate: float) → pandas.DataFrame`

Add randomly constructed triples :param train\_set: :param add\_noise\_rate: :return:

`dicee.static_funcs.read_or_load_kg (args, cls)`

`dicee.static_funcs.intialize_model (args: Dict, verbose: int = 0) → Tuple[dicee.models.base_model.BaseKGE, str]`

Initialize a knowledge graph embedding model.

#### Parameters

- **args** – Dictionary containing model configuration including ‘model’ key.
- **verbose** – Verbosity level. If > 0, prints initialization message.

#### Returns

Tuple of (initialized model, form of labelling string).

#### Raises

**ValueError** – If the model name is not recognized.

`dicee.static_funcs.load_json(path: str) → Dict`

Load JSON file into a dictionary.

**Parameters**

**path** – Path to the JSON file.

**Returns**

Dictionary containing the JSON data.

**Raises**

- **FileNotFoundError** – If the file does not exist.
- **json.JSONDecodeError** – If the file contains invalid JSON.

`dicee.static_funcs.save_embeddings(embeddings: numpy.ndarray, indexes: List, path: str) → None`

Save embeddings to a CSV file.

**Parameters**

- **embeddings** – NumPy array of embeddings with shape (n\_items, embedding\_dim).
- **indexes** – List of index labels (entity/relation names).
- **path** – Output file path.

`dicee.static_funcs.vocab_to_parquet(vocab_to_idx, name, path_for_serialization, print_into)`

`dicee.static_funcs.create_experiment_folder(folder_name: str = 'Experiments') → str`

Create a timestamped experiment folder.

**Parameters**

**folder\_name** – Base directory name for experiments.

**Returns**

Full path to the created experiment folder.

`dicee.static_funcs.continual_training_setup_executor(executor) → None`

`dicee.static_funcs.exponential_function(x: numpy.ndarray, lam: float, ascending_order=True)`  
→ torch.FloatTensor

`dicee.static_funcs.load_numpy(path) → numpy.ndarray`

`dicee.static_funcs.evaluate(entity_to_idx, scores, easy_answers, hard_answers)`

# @TODO: CD: Renamed this function Evaluate multi hop query answering on different query types

`dicee.static_funcs.download_file(url, destination_folder='.')`

`dicee.static_funcs.download_files_from_url(base_url: str, destination_folder='.') → None`

**Parameters**

- **base\_url** (e.g. <https://files.dice-research.org/projects/DiceEmbeddings/KINSHIP-Keci-dim128-epoch256-KvsAll>)
- **destination\_folder** (e.g. `"KINSHIP-Keci-dim128-epoch256-KvsAll"`)

`dicee.static_funcs.download_pretrained_model(url: str) → str`

`dicee.static_funcs.write_csv_from_model_parallel(path: str)`

Create

`dicee.static_funcs.from_pretrained_model_write_embeddings_into_csv(path: str) → None`

## dicee.static\_funcs\_training

Training-related static functions.

This module provides backward compatibility by re-exporting evaluation functions from the new `dicee.evaluation` module, along with training utilities.

Deprecated since version Evaluation: functions have moved to `dicee.evaluation`. Use that module for new code. This module will continue to export training utilities.

## Functions

|                                                                |                                                                       |
|----------------------------------------------------------------|-----------------------------------------------------------------------|
| <code>evaluate_lp</code> ( $\rightarrow$ Dict[str, float])     | Evaluate link prediction with batched processing.                     |
| <code>evaluate_bpe_lp</code> ( $\rightarrow$ Dict[str, float]) | Evaluate link prediction with BPE-encoded entities.                   |
| <code>make_iterable_verbose</code> ( $\rightarrow$ Iterable)   | Wrap an iterable with tqdm progress bar if verbose is True.           |
| <code>efficient_zero_grad</code> ( $\rightarrow$ None)         | Efficiently zero gradients using <code>parameter.grad = None</code> . |

## Module Contents

`dicee.static_funcs_training.evaluate_lp`(*model*, *triple\_idx*, *num\_entities*: int, *er\_vocab*: Dict[Tuple, List], *re\_vocab*: Dict[Tuple, List], *info*: str = 'Eval Starts', *batch\_size*: int = 128, *chunk\_size*: int = 1000)  $\rightarrow$  Dict[str, float]

Evaluate link prediction with batched processing.

Memory-efficient evaluation using chunked entity scoring.

### Parameters

- **model** – The KGE model to evaluate.
- **triple\_idx** – Integer-indexed triples as numpy array.
- **num\_entities** – Total number of entities.
- **er\_vocab** – Mapping (head\_idx, rel\_idx) -> list of tail indices.
- **re\_vocab** – Mapping (rel\_idx, tail\_idx) -> list of head indices.
- **info** – Description to print.
- **batch\_size** – Batch size for triple processing.
- **chunk\_size** – Chunk size for entity scoring.

### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

`dicee.static_funcs_training.evaluate_bpe_lp`(*model*, *triple\_idx*: List[Tuple], *all\_bpe\_shaped\_entities*, *er\_vocab*: Dict[Tuple, List], *re\_vocab*: Dict[Tuple, List], *info*: str = 'Eval Starts')  $\rightarrow$  Dict[str, float]

Evaluate link prediction with BPE-encoded entities.

### Parameters

- **model** – The KGE model to evaluate.
- **triple\_idx** – List of BPE-encoded triple tuples.
- **all\_bpe\_shaped\_entities** – All entities with BPE representations.
- **er\_vocab** – Mapping for tail filtering.

- **re\_vocab** – Mapping for head filtering.
- **info** – Description to print.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

`dicee.static_funcs_training.make_iterable_verbose(iterable_object: Iterable, verbose: bool, desc: str = 'Default', position: int | None = None, leave: bool = True) → Iterable`

Wrap an iterable with tqdm progress bar if verbose is True.

#### Parameters

- **iterable\_object** – The iterable to potentially wrap.
- **verbose** – Whether to show progress bar.
- **desc** – Description for the progress bar.
- **position** – Position of the progress bar.
- **leave** – Whether to leave the progress bar after completion.

#### Returns

The original iterable or a tqdm-wrapped version.

`dicee.static_funcs_training.efficient_zero_grad(model) → None`

Efficiently zero gradients using `parameter.grad = None`.

This is more efficient than `optimizer.zero_grad()` as it avoids memory operations.

See: [https://pytorch.org/tutorials/recipes/recipes/tuning\\_guide.html](https://pytorch.org/tutorials/recipes/recipes/tuning_guide.html)

#### Parameters

**model** – PyTorch model to zero gradients for.

## dicee.static\_preprocess\_funcs

### Attributes

|                         |
|-------------------------|
| <code>enable_log</code> |
|-------------------------|

### Functions

|                                                                  |                                    |
|------------------------------------------------------------------|------------------------------------|
| <code>timeit(func)</code>                                        |                                    |
| <code>preprocesses_input_args(args)</code>                       | Sanity Checking in input arguments |
| <code>create_constraints(→ Tuple[dict, dict, dict, dict])</code> |                                    |
| <code>get_er_vocab(data)</code>                                  |                                    |
| <code>get_re_vocab(data)</code>                                  |                                    |
| <code>get_ee_vocab(data)</code>                                  |                                    |
| <code>mapping_from_first_two_cols_to_third(train_se</code>       |                                    |

## Module Contents

`dicee.static_preprocess_funcs.enable_log = False`

`dicee.static_preprocess_funcs.timeit(func)`

```
dicee.static_preprocess_funcs.preprocesses_input_args (args)
```

Sanity Checking in input arguments

```
dicee.static_preprocess_funcs.create_constraints (triples: numpy.ndarray)
```

→ Tuple[dict, dict, dict, dict]

(1) Extract domains and ranges of relations

(2) Store a mapping from relations to entities that are outside of the domain and range. Create constraints entities based on the range of relations :param triples: :return:

```
dicee.static_preprocess_funcs.get_er_vocab (data)
```

```
dicee.static_preprocess_funcs.get_re_vocab (data)
```

```
dicee.static_preprocess_funcs.get_ee_vocab (data)
```

```
dicee.static_preprocess_funcs.mapping_from_first_two_cols_to_third (train_set_idx)
```

## **dicee.trainer**

### **Submodules**

#### **dicee.trainer.dice\_trainer**

DICE Trainer module for knowledge graph embedding training.

Provides the DICE\_Trainer class which supports multiple training backends including PyTorch Lightning, DDP, and custom CPU/GPU trainers.

### **Classes**

|                           |                        |
|---------------------------|------------------------|
| <code>DICE_Trainer</code> | DICE_Trainer implement |
|---------------------------|------------------------|

### **Functions**

|                                                     |                                                            |
|-----------------------------------------------------|------------------------------------------------------------|
| <code>load_term_mapping</code> (→ polars.DataFrame) | Load term-to-index mapping from CSV file.                  |
| <code>initialize_trainer</code> (...)               | Initialize the appropriate trainer based on configuration. |
| <code>get_callbacks</code> (→ List)                 | Create list of callbacks based on configuration.           |

### **Module Contents**

```
dicee.trainer.dice_trainer.load_term_mapping (file_path: str) → polars.DataFrame
```

Load term-to-index mapping from CSV file.

#### **Parameters**

**file\_path** – Base path without extension.

#### **Returns**

Polars DataFrame containing the mapping.

```
dicee.trainer.dice_trainer.initialize_trainer (args, callbacks: List)
```

→ `dicee.trainer.torch_trainer.TorchTrainer` | `dicee.trainer.model_parallelism.TensorParallel` | `dicee.trainer.torch_trainer_ddp`

Initialize the appropriate trainer based on configuration.

### Parameters

- **args** – Configuration arguments containing trainer type.
- **callbacks** – List of training callbacks.

### Returns

Initialized trainer instance.

### Raises

**AssertionError** – If trainer is None after initialization.

`dicee.trainer.dice_trainer.get_callbacks(args) → List`

Create list of callbacks based on configuration.

### Parameters

**args** – Configuration arguments.

### Returns

List of callback instances.

```
class dicee.trainer.dice_trainer.DICE_Trainer (args, is_continual_training: bool, storage_path,
        evaluator=None)
```

### DICE\_Trainer implement

- 1- Pytorch Lightning trainer (<https://pytorch-lightning.readthedocs.io/en/stable/common/trainer.html>)
- 2- Multi-GPU Trainer(<https://pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html>)
- 3- CPU Trainer

args

is\_continual\_training:bool

storage\_path:str

evaluator:

report:dict

**report**

**args**

**trainer = None**

**is\_continual\_training**

**storage\_path**

**evaluator = None**

**form\_of\_labelling = None**

**continual\_start** (*knowledge\_graph*)

- (1) Initialize training.
- (2) Load model
- (3) Load trainer (3) Fit model

## Parameter

### returns

- *model*
- **form\_of\_labelling** (*str*)

**initialize\_trainer** (*callbacks: List*)

→ `lightning.Trainer` | `diccee.trainer.model_parallelism.TensorParallel` | `diccee.trainer.torch_trainer.TorchTrainer` | `diccee.t`

Initialize Trainer from input arguments

**initialize\_or\_load\_model** ()

**init\_dataloader** (*dataset: torch.utils.data.Dataset*) → `torch.utils.data.DataLoader`

**init\_dataset** () → `torch.utils.data.Dataset`

**start** (*knowledge\_graph: diccee.knowledge\_graph.KG* | *numpy.memmap*)

→ `Tuple[diccee.models.base_model.BaseKGE, str]`

Start the training

- (1) Initialize Trainer
- (2) Initialize or load a pretrained KGE model

in DDP setup, we need to load the memory map of already read/index KG.

**k\_fold\_cross\_validation** (*dataset*) → `Tuple[diccee.models.base_model.BaseKGE, str]`

Perform K-fold Cross-Validation

1. Obtain K train and test splits.
2. **For each split,**
  - 2.1 initialize trainer and model
  - 2.2. Train model with configuration provided in args.
  - 2.3. Compute the mean reciprocal rank (MRR) score of the model on the test respective split.
3. Report the mean and average MRR .

### Parameters

- **self**
- **dataset**

### Returns

*model*

## diccee.trainer.model\_parallelism

### Classes

*TensorParallel*

Abstract class for Trainer class for knowledge graph embedding models

## Functions

|                                                                                    |
|------------------------------------------------------------------------------------|
| <code>extract_input_outputs(z[, device])</code>                                    |
| <code>find_good_batch_size(train_loader,</code><br><code>tp_ensemble_model)</code> |
| <code>forward_backward_update_loss(→ float)</code>                                 |

## Module Contents

```
dicee.trainer.model_parallelism.extract_input_outputs (z: list, device=None)
dicee.trainer.model_parallelism.find_good_batch_size (train_loader, tp_ensemble_model)
dicee.trainer.model_parallelism.forward_backward_update_loss (z: Tuple, ensemble_model)
→ float
```

```
class dicee.trainer.model_parallelism.TensorParallel (args, callbacks)
```

Bases: `dicee.abstracts.AbstractTrainer`

Abstract class for Trainer class for knowledge graph embedding models

### Parameter

**args**

[str] ?

**callbacks: list**

?

**fit** (\*args, \*\*kwargs)

Train model

## `dicee.trainer.torch_trainer`

## Classes

|                           |                                                                  |
|---------------------------|------------------------------------------------------------------|
| <code>TorchTrainer</code> | TorchTrainer for using single GPU or multi CPUs on a single node |
|---------------------------|------------------------------------------------------------------|

## Module Contents

```
class dicee.trainer.torch_trainer.TorchTrainer (args, callbacks)
```

Bases: `dicee.abstracts.AbstractTrainer`

TorchTrainer for using single GPU or multi CPUs on a single node

Arguments

callbacks: list of Abstract callback instances

**loss\_function = None**

**optimizer = None**



**model** = None

**train\_data loaders** = None

**training\_step** = None

**process**

**fit** (\*args, train\_data loaders, \*\*kwargs) → None

Training starts

Arguments

**kwargs: Tuple**

empty dictionary

**Return type**

batch loss (float)

**forward\_backward\_update** (x\_batch: torch.Tensor, y\_batch: torch.Tensor) → torch.Tensor

Compute forward, loss, backward, and parameter update

Arguments

**Return type**

batch loss (float)

**extract\_input\_outputs\_set\_device** (batch: list) → Tuple

Construct inputs and outputs from a batch of inputs with outputs From a batch of inputs and put

Arguments

**Return type**

(tuple) mini-batch on select device

## **dicee.trainer.torch\_trainer\_ddp**

### **Classes**

---

*TorchDDPTrainer*

A Trainer based on torch.nn.parallel.DistributedDataParallel

*NodeTrainer*

---

### **Functions**

---

*make\_iterable\_verbose*(→ Iterable)

---

### **Module Contents**

dicee.trainer.torch\_trainer\_ddp.**make\_iterable\_verbose** (iterable\_object, verbose,  
desc='Default', position=None, leave=True) → Iterable

```

class dicee.trainer.torch_trainer_ddp.TorchDDPTrainer (args, callbacks)
    Bases: dicee.abstracts.AbstractTrainer

        A Trainer based on torch.nn.parallel.DistributedDataParallel

        Arguments

    entity_idx
        mapping.

    relation_idx
        mapping.

    form
        ?

    store
        ?

    label_smoothing_rate
        Using hard targets (0,1) drives weights to infinity. An outlier produces enormous gradients.

    Return type
        torch.utils.data.Dataset

    fit (*args, **kwargs)
        Train model

class dicee.trainer.torch_trainer_ddp.NodeTrainer (trainer, model: torch.nn.Module,
    train_dataset_loader: torch.utils.data.DataLoader, callbacks, num_epochs: int)

    trainer

    local_rank

    global_rank

    optimizer

    train_dataset_loader

    loss_func

    callbacks

    model

    num_epochs

    loss_history = []

    ctx

    scaler

    extract_input_outputs (z: list)

    train ()
        Training loop for DDP

```

## Classes

*DICE\_Trainer*

DICE\_Trainer implement

## Package Contents

```
class dicee.trainer.DICE_Trainer (args, is_continual_training: bool, storage_path, evaluator=None)
```

### DICE\_Trainer implement

- 1- Pytorch Lightning trainer (<https://pytorch-lightning.readthedocs.io/en/stable/common/trainer.html>)
- 2- Multi-GPU Trainer(<https://pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html>)
- 3- CPU Trainer

args

is\_continual\_training:bool

storage\_path:str

evaluator:

report:dict

**report**

**args**

**trainer = None**

**is\_continual\_training**

**storage\_path**

**evaluator = None**

**form\_of\_labelling = None**

**continual\_start** (*knowledge\_graph*)

- (1) Initialize training.
- (2) Load model
- (3) Load trainer (3) Fit model

### Parameter

**returns**

- *model*
- **form\_of\_labelling** (*str*)

**initialize\_trainer** (*callbacks: List*)

→ *lightning.Trainer* | *dicee.trainer.model\_parallelism.TensorParallel* | *dicee.trainer.torch\_trainer.TorchTrainer* | *dicee.t*

Initialize Trainer from input arguments

**initialize\_or\_load\_model** ()

**init\_dataloader** (*dataset: torch.utils.data.Dataset*) → torch.utils.data.DataLoader

**init\_dataset** () → torch.utils.data.Dataset

**start** (*knowledge\_graph: dicee.knowledge\_graph.KG | numpy.memmap*)  
→ Tuple[*dicee.models.base\_model.BaseKGE*, str]

Start the training

(1) Initialize Trainer

(2) Initialize or load a pretrained KGE model

in DDP setup, we need to load the memory map of already read/index KG.

**k\_fold\_cross\_validation** (*dataset*) → Tuple[*dicee.models.base\_model.BaseKGE*, str]

Perform K-fold Cross-Validation

1. Obtain K train and test splits.

2. **For each split,**

2.1 initialize trainer and model 2.2. Train model with configuration provided in args. 2.3. Compute the mean reciprocal rank (MRR) score of the model on the test respective split.

3. Report the mean and average MRR .

#### Parameters

- **self**
- **dataset**

#### Returns

model

## **dicee.weight\_averaging**

### **Classes**

|             |                                                                          |
|-------------|--------------------------------------------------------------------------|
| <i>ASWA</i> | Adaptive stochastic weight averaging                                     |
| <i>SWA</i>  | Stochastic Weight Averaging callback.                                    |
| <i>SWAG</i> | Stochastic Weight Averaging - Gaussian (SWAG).                           |
| <i>EMA</i>  | Exponential Moving Average (EMA) callback.                               |
| <i>TWA</i>  | Train with Weight Averaging (TWA) using subspace projection + averaging. |

### **Module Contents**

**class** dicee.weight\_averaging.**ASWA** (*num\_epochs, path*)

Bases: *dicee.abstracts.AbstractCallback*

Adaptive stochastic weight averaging ASWE keeps track of the validation performance and update s the ensemble model accordingly.

**path**

**num\_epochs**

**initial\_eval\_setting = None**

```
epoch_count = 0

alphas = []

val_aswa = -1

on_fit_end(trainer, model)
    Call at the end of the training.
```

### Parameter

trainer:

model:

**rtype**

None

```
static compute_mrr(trainer, model) → float
```

```
get_aswa_state_dict(model)
```

```
decide(running_model_state_dict, ensemble_state_dict, val_running_model,
        mrr_updated_ensemble_model)
```

Perform Hard Update, software or rejection

### Parameters

- **running\_model\_state\_dict**
- **ensemble\_state\_dict**
- **val\_running\_model**
- **mrr\_updated\_ensemble\_model**

```
on_train_epoch_end(trainer, model)
```

Call at the end of each epoch during training.

### Parameter

trainer:

model:

**rtype**

None

```
class dicee.weight_averaging.SWA(swa_start_epoch, swa_c_epochs: int = 1, lr_init: float = 0.1,
    swa_lr: float = 0.05, max_epochs: int = None)
```

Bases: *[dicee.abstracts.AbstractCallback](#)*

Stochastic Weight Averaging callback.

**Initialize SWA callback.**

**swa\_start\_epoch: int**

The epoch at which to start SWA.

**swa\_c\_epochs: int**

The number of epochs to use for SWA.

```

lr_init: float
    The initial learning rate.

swa_lr: float
    The learning rate to use during SWA.

max_epochs: int
    The maximum number of epochs. args.num_epochs

swa_start_epoch

swa_c_epochs = 1

swa_lr = 0.05

lr_init = 0.1

max_epochs = None

swa_model = None

swa_n = 0

current_epoch = -1

static moving_average (swa_model, running_model, alpha)
    Update SWA model with moving average of current model. Math: # SWA update:  $\theta_{\text{swa}} \leftarrow (1 - \alpha) * \theta_{\text{swa}} + \alpha * \theta$  #  $\alpha = 1 / (n + 1)$ , where n = number of models already averaged # alpha is tracked via self.swa_n in code

on_train_epoch_start (trainer, model)
    Update learning rate according to SWA schedule.

on_train_epoch_end (trainer, model)
    Apply SWA averaging if conditions are met.

on_fit_end (trainer, model)
    Replace main model with SWA model at the end of training.

class dicee.weight_averaging.SWAG (swa_start_epoch: int = 1, lr_init: float = 0.1,
    swa_lr: float = 0.05, max_epochs: int = None, max_num_models: int = 20,
    var_clamp: float = 1e-30)

Bases: dicee.abstracts.AbstractCallback

Stochastic Weight Averaging - Gaussian (SWAG). Parameters

swa_start_epoch
    [int] Epoch at which to start collecting weights.

swa_c_epochs
    [int] Interval of epochs between updates.

lr_init
    [float] Initial LR.

swa_lr
    [float] LR in SWA / GSWA phase.

max_epochs
    [int] Total number of epochs.

```

```

    max_num_models
        [int] Number of models to keep for low-rank covariance approx.

    var_clamp
        [float] Clamp low variance for stability.

swa_start_epoch

swa_c_epochs = 1

swa_lr = 0.05

lr_init = 0.1

max_epochs = None

max_num_models = 20

var_clamp = 1e-30

mean = None

sq_mean = None

deviations = []

gswa_n = 0

current_epoch = -1

get_mean_and_var()
    Return mean + variance (diagonal part).

sample(base_model, scale=0.5)
    Sample new model from SWAG posterior distribution.

    Math: # From SWAG, posterior is approximated as:  $\theta \sim N(\text{mean}, \Sigma)$  # where  $\Sigma \approx \text{diag}(\text{var}) + (1/(K-1)) * D D^T$  # - mean = running average of weights # - var = elementwise variance (sq_mean - mean^2) # - D = [dev_1, dev_2, ..., dev_K], deviations from mean (low-rank approx) # - K = number of collected models

    # Sampling step: # 1.  $\theta_{\text{diag}} = \text{mean} + \text{scale} * \text{std} \odot \varepsilon$ , where  $\varepsilon \sim N(0, I)$  # 2.  $\theta_{\text{lowrank}} = \theta_{\text{diag}} + (D z) / \text{sqrt}(K-1)$ , where  $z \sim N(0, I_K)$  # Final sample =  $\theta_{\text{lowrank}}$ 

on_train_epoch_start(trainer, model)
    Update LR schedule (same as SWA).

on_train_epoch_end(trainer, model)
    Collect Gaussian stats at the end of epochs after swa_start.

on_fit_end(trainer, model)
    Set model weights to the collected SWAG mean at the end of training.

class dicee.weight_averaging.EMA(ema_start_epoch: int, decay: float = 0.999,
    max_epochs: int = None, ema_c_epochs: int = 1)

    Bases: dicee.abstracts.AbstractCallback

    Exponential Moving Average (EMA) callback.

    Parameters
        • ema_start_epoch (int) – Epoch to start EMA.

```

- **decay** (*float*) – EMA decay rate (typical: 0.99 - 0.9999) Math:  $\theta_{\text{ema}} \leftarrow \text{decay} * \theta_{\text{ema}} + (1 - \text{decay}) * \theta$
- **max\_epochs** (*int*) – Maximum number of epochs.

**ema\_start\_epoch**

**decay** = 0.999

**max\_epochs** = None

**ema\_c\_epochs** = 1

**ema\_model** = None

**current\_epoch** = -1

**static ema\_update** (*ema\_model, running\_model, decay: float*)

Update EMA model with exponential moving average of current model. Math: # EMA update:  $\theta_{\text{ema}} \leftarrow (1 - \alpha) * \theta_{\text{ema}} + \alpha * \theta$  #  $\alpha = 1 - \text{decay}$ , where decay is the EMA smoothing factor (typical 0.99 - 0.999) #  $\alpha$  controls how much of the current model  $\theta$  contributes to the EMA # decay is fixed in code -> can be extended to scheduled

**on\_train\_epoch\_start** (*trainer, model*)

Track current epoch.

**on\_train\_epoch\_end** (*trainer, model*)

Update EMA if past start epoch.

**on\_fit\_end** (*trainer, model*)

Replace main model with EMA model at the end of training.

**class** dicee.weight\_averaging.**TWA** (*twa\_start\_epoch: int, lr\_init: float, num\_samples: int = 5, reg\_lambda: float = 0.0, max\_epochs: int = None, twa\_c\_epochs: int = 1*)

Bases: *dicee.abstracts.AbstractCallback*

Train with Weight Averaging (TWA) using subspace projection + averaging.

### Parameters

**twa\_start\_epoch**

[int] Epoch to start TWA.

**lr\_init**

[float] Learning rate used for  $\beta$  updates.

**num\_samples**

[int] Number of sampled weight snapshots to build projection subspace.

**reg\_lambda**

[float] Regularization coefficient for  $\beta$  updates.

**max\_epochs**

[int] Total number of training epochs.

**twa\_c\_epochs**

[int] Interval of epochs between TWA updates.

**twa\_start\_epoch**

**num\_samples** = 5



```

reg_lambda = 0.0

max_epochs = None

lr_init

twa_c_epochs = 1

current_epoch = -1

weight_samples = []

twa_model = None

base_weights = None

P = None

beta = None

sample_weights(model)
    Collect sampled weights from the current model and maintain rolling buffer.

build_projection(weight_samples, k=None)
    Build projection subspace from collected weight samples. :param weight_samples: list of flat weight tensors
    [(D,), ...] :param k: number of basis vectors to keep. Defaults to min(N, D).

    Returns
        (D,) base weight vector (average) P: (D, k) projection matrix with top-k basis directions

    Return type
        mean_w

on_train_epoch_start(trainer, model)
    Track epoch.

on_train_epoch_end(trainer, model)
    Main TWA logic: build subspace and update in  $\beta$  space.

    # Math: # TWA weight update: #  $w_{twa} = \text{mean\_w} + P * \beta$  #  $\text{mean\_w} = (1/n) * \sum_i w_i$  (SWA baseline)
    #  $\beta \leftarrow (1 - \eta * \lambda) * \beta - \eta * P^T * g$  #  $g$  = gradient of training loss w.r.t. full model weights
    #  $\eta$  = learning rate,  $\lambda$  = ridge regularization #  $P$  = orthonormal basis spanning sampled checkpoints
    { $w_i$ }

on_fit_end(trainer, model)
    Replace with TWA model at the end.

```

## 14.2 Attributes

---

`__version__`

---

## 14.3 Classes

---

`Execute`

---

Executor class for training, retraining and evaluating KGE models.

continues on next page

Table 69 – continued from previous page

|                       |                                                                             |
|-----------------------|-----------------------------------------------------------------------------|
| <i>KGE</i>            | Knowledge Graph Embedding Class for interactive usage of pre-trained models |
| <i>QueryGenerator</i> |                                                                             |
| <i>DICE_Trainer</i>   | DICE_Trainer implement                                                      |
| <i>Evaluator</i>      | Evaluator class for KGE models in various downstream tasks.                 |

## 14.4 Package Contents

**class** `dicee.Execute` (*args*, *continuous\_training*: *bool = False*)

Executor class for training, retraining and evaluating KGE models.

Handles the complete workflow: 1. Loading & Preprocessing & Serializing input data 2. Training & Validation & Testing 3. Storing all necessary information

**args**

Processed input arguments.

**distributed**

Whether distributed training is enabled.

**rank**

Process rank in distributed training.

**world\_size**

Total number of processes.

**local\_rank**

Local GPU rank.

**trainer**

Training handler instance.

**trained\_model**

The trained model after training completes.

**knowledge\_graph**

The loaded knowledge graph.

**report**

Dictionary storing training metrics and results.

**evaluator**

Model evaluation handler.

**distributed**

**args**

**is\_continual\_training** = `False`

**trainer**: `dicee.trainer.DICE_Trainer` | `None` = `None`

**trained\_model** = `None`

**knowledge\_graph**: `dicee.knowledge_graph.KG` | `None` = `None`

**report:** Dict

**evaluator:** `dicee.evaluator.Evaluator` | None = None

**start\_time:** float | None = None

**is\_rank\_zero()** → bool

**cleanup()**

**setup\_executor()** → None  
 Set up storage directories for the experiment.  
 Creates or reuses experiment directories based on configuration. Saves the configuration to a JSON file.

**create\_and\_store\_kg()** → None  
 Create knowledge graph and store as memory-mapped file.  
 Only executed on rank 0 in distributed training. Skips if memmap already exists.

**load\_from\_memmap()** → None  
 Load knowledge graph from memory-mapped file.

**save\_trained\_model()** → None  
 Save a knowledge graph embedding model  
 (1) Send model to eval mode and cpu.  
 (2) Store the memory footprint of the model.  
 (3) Save the model into disk.  
 (4) Update the stats of KG again ?

### Parameter

**rtype**  
 None

**end**(*form\_of\_labelling: str*) → dict  
 End training  
 (1) Store trained model.  
 (2) Report runtimes.  
 (3) Eval model if required.

### Parameter

**rtype**  
 A dict containing information about the training and/or evaluation

**write\_report()** → None  
 Report training related information in a report.json file

**start()** → dict  
 Start training  
 # (1) Loading the Data # (2) Create an evaluator object. # (3) Create a trainer object. # (4) Start the training

## Parameter

### rtype

A dict containing information about the training and/or evaluation

```
class dicee.KGE (path=None, url=None, construct_ensemble=False, model_name=None)
```

Bases: `dicee.abstracts.BaseInteractiveKGE`, `dicee.abstracts.InteractiveQueryDecomposition`, `dicee.abstracts.BaseInteractiveTrainKGE`

Knowledge Graph Embedding Class for interactive usage of pre-trained models

`__str__()`

`to(device: str) → None`

`get_transductive_entity_embeddings(indices: torch.LongTensor | List[str], as_pytorch=False, as_numpy=False, as_list=True) → torch.FloatTensor | numpy.ndarray | List[float]`

`create_vector_database(collection_name: str, distance: str, location: str = 'localhost', port: int = 6333)`

`generate(h="", r="")`

`eval_lp_performance(dataset=List[Tuple[str, str, str]], filtered=True)`

`predict_missing_head_entity(relation: List[str] | str, tail_entity: List[str] | str, within=None, batch_size=2, topk=1, return_indices=False) → Tuple`

Given a relation and a tail entity, return top k ranked head entity.

$\operatorname{argmax}_{\{e \in E\}} f(e, r, t)$ , where  $r \in R$ ,  $t \in E$ .

## Parameter

relation: Union[List[str], str]

String representation of selected relations.

tail\_entity: Union[List[str], str]

String representation of selected entities.

k: int

Highest ranked k entities.

## Returns: Tuple

Highest K scores and entities

`predict_missing_relations(head_entity: List[str] | str, tail_entity: List[str] | str, within=None, batch_size=2, topk=1, return_indices=False) → Tuple`

Given a head entity and a tail entity, return top k ranked relations.

$\operatorname{argmax}_{\{r \in R\}} f(h, r, t)$ , where  $h, t \in E$ .

## Parameter

head\_entity: List[str]

String representation of selected entities.

tail\_entity: List[str]

String representation of selected entities.

k: int

Highest ranked k entities.

### Returns: Tuple

Highest K scores and entities

**predict\_missing\_tail\_entity** (*head\_entity: List[str] | str, relation: List[str] | str,*  
*within: List[str] = None, batch\_size=2, topk=1, return\_indices=False*) → torch.FloatTensor

Given a head entity and a relation, return top k ranked entities

$\text{argmax}_{\{e \in E\}} f(h, r, e)$ , where  $h \in E$  and  $r \in R$ .

### Parameter

head\_entity: List[str]

String representation of selected entities.

tail\_entity: List[str]

String representation of selected entities.

### Returns: Tuple

scores

**predict** (\*, *h: List[str] | str = None, r: List[str] | str = None, t: List[str] | str = None, within=None,*  
*logits=True*) → torch.FloatTensor

#### Parameters

- **logits**
- **h**
- **r**
- **t**
- **within**

**predict\_topk** (\*, *h: str | List[str] = None, r: str | List[str] = None, t: str | List[str] = None, topk: int = 10,*  
*within: List[str] = None, batch\_size: int = 1024*)

Predict missing item in a given triple.

#### Returns

- If you query a single (h, r, ?) or (?, r, t) or (h, ?, t), returns List[(item, score)]
- If you query a batch of B, returns List of B such lists.

**triple\_score** (*h: List[str] | str = None, r: List[str] | str = None, t: List[str] | str = None, logits=False*)  
→ torch.FloatTensor

Predict triple score

## Parameter

head\_entity: List[str]

String representation of selected entities.

relation: List[str]

String representation of selected relations.

tail\_entity: List[str]

String representation of selected entities.

logits: bool

If logits is True, unnormalized score returned

## Returns: Tuple

pytorch tensor of triple score

**return\_multi\_hop\_query\_results** (*aggregated\_query\_for\_all\_entities*, *k*: int, *only\_scores*)

**single\_hop\_query\_answering** (*query*: tuple, *only\_scores*: bool = True, *k*: int = None)

**answer\_multi\_hop\_query** (*query\_type*: str = None, *query*: Tuple[str | Tuple[str, str], Ellipsis] = None, *queries*: List[Tuple[str | Tuple[str, str], Ellipsis]] = None, *tnorm*: str = 'prod', *neg\_norm*: str = 'standard', *lambda\_*: float = 0.0, *k*: int = 10, *only\_scores*=False) → List[Tuple[str, torch.Tensor]]

# @TODO: Refactoring is needed # @TODO: Score computation for each query type should be done in a static function

Find an answer set for EPFO queries including negation and disjunction

## Parameter

query\_type: str The type of the query, e.g., “2p”.

query: Union[str, Tuple[str, Tuple[str, str]]] The query itself, either a string or a nested tuple.

queries: List of Tuple[Union[str, Tuple[str, str]], ...]

tnorm: str The t-norm operator.

neg\_norm: str The negation norm.

**lambda\_**: float lambda parameter for sugeno and yager negation norms

k: int The top-k substitutions for intermediate variables.

### returns

- List[Tuple[str, torch.Tensor]]
- Entities and corresponding scores sorted in the descening order of scores

**find\_missing\_triples** (*confidence*: float, *entities*: List[str] = None, *relations*: List[str] = None, *topk*: int = 10, *at\_most*: int = sys.maxsize) → Set

Find missing triples

Iterative over a set of entities E and a set of relation R :

orall e in E and orall r in R f(e,r,x)

```

    Return (e,r,x)
otin G and f(e,r,x) > confidence
    confidence: float
    A threshold for an output of a sigmoid function given a triple.
    topk: int
    Highest ranked k item to select triples with f(e,r,x) > confidence .
    at_most: int
    Stop after finding at_most missing triples
    {(e,r,x) | f(e,r,x) > confidence land (e,r,x)
otin G
predict_literals (entity: List[str] | str = None, attribute: List[str] | str = None,
    denormalize_preds: bool = True) → numpy.ndarray
    Predicts literal values for given entities and attributes.

Parameters
    • entity (Union[List[str], str]) – Entity or list of entities to predict literals for.
    • attribute (Union[List[str], str]) – Attribute or list of attributes to predict literals
        for.
    • denormalize_preds (bool) – If True, denormalizes the predictions.

Returns
    Predictions for the given entities and attributes.

Return type
    numpy ndarray

class dicee.QueryGenerator (train_path: str, val_path: str, test_path: str, ent2id: Dict = None,
    rel2id: Dict = None, seed: int = 1, gen_valid: bool = False, gen_test: bool = True)

    train_path
    val_path
    test_path
    gen_valid = False
    gen_test = True
    seed = 1
    max_ans_num = 1000000.0
    mode
    ent2id = None
    rel2id: Dict = None
    ent_in: Dict

```

```

ent_out: Dict

query_name_to_struct

list2tuple(list_data)

tuple2list(x: List | Tuple) → List | Tuple
    Convert a nested tuple to a nested list.

set_global_seed(seed: int)
    Set seed

construct_graph(paths: List[str]) → Tuple[Dict, Dict]
    Construct graph from triples Returns dicts with incoming and outgoing edges

fill_query(query_structure: List[str | List], ent_in: Dict, ent_out: Dict, answer: int) → bool
    Private method for fill_query logic.

achieve_answer(query: List[str | List], ent_in: Dict, ent_out: Dict) → set
    Private method for achieve_answer logic. @TODO: Document the code

write_links(ent_out, small_ent_out)

ground_queries(query_structure: List[str | List], ent_in: Dict, ent_out: Dict, small_ent_in: Dict,
               small_ent_out: Dict, gen_num: int, query_name: str)
    Generating queries and achieving answers

unmap(query_type, queries, tp_answers, fp_answers, fn_answers)

unmap_query(query_structure, query, id2ent, id2rel)

generate_queries(query_struct: List, gen_num: int, query_type: str)
    Passing incoming and outgoing edges to ground queries depending on mode [train valid or text] and getting
    queries and answers in return @ TODO: create a class for each single query struct

save_queries(query_type: str, gen_num: int, save_path: str)

abstractmethod load_queries(path)

get_queries(query_type: str, gen_num: int)

static save_queries_and_answers(path: str, data: List[Tuple[str, Tuple[collections.defaultdict]]])
    → None
    Save Queries into Disk

static load_queries_and_answers(path: str) → List[Tuple[str, Tuple[collections.defaultdict]]]
    Load Queries from Disk to Memory

class dicee.DICE_Trainer(args, is_continual_training: bool, storage_path, evaluator=None)

DICE_Trainer implement
1- Pytorch Lightning trainer (https://pytorch-lightning.readthedocs.io/en/stable/common/trainer.html)
2- Multi-GPU Trainer(https://pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html)
3- CPU Trainer

args
is_continual_training:bool
storage_path:str

```



```

    evaluator:
    report:dict
report
args
trainer = None
is_continual_training
storage_path
evaluator = None
form_of_labelling = None
continual_start (knowledge_graph)

```

- (1) Initialize training.
- (2) Load model
- (3) Load trainer (3) Fit model

## Parameter

### returns

- *model*
- **form\_of\_labelling** (*str*)

```

initialize_trainer (callbacks: List)
    → lightning.Trainer | dicke.trainer.model_parallelism.TensorParallel | dicke.trainer.torch_trainer.TorchTrainer | dicke.

```

Initialize Trainer from input arguments

```

initialize_or_load_model ()

```

```

init_dataloader (dataset: torch.utils.data.Dataset) → torch.utils.data.DataLoader

```

```

init_dataset () → torch.utils.data.Dataset

```

```

start (knowledge_graph: dicke.knowledge_graph.KG | numpy.memmap)
    → Tuple[dicke.models.base_model.BaseKGE, str]

```

Start the training

- (1) Initialize Trainer
- (2) Initialize or load a pretrained KGE model

in DDP setup, we need to load the memory map of already read/index KG.

```

k_fold_cross_validation (dataset) → Tuple[dicke.models.base_model.BaseKGE, str]

```

Perform K-fold Cross-Validation

1. Obtain K train and test splits.
2. **For each split,**
  - 2.1 initialize trainer and model
  - 2.2. Train model with configuration provided in args.
  - 2.3. Compute the mean reciprocal rank (MRR) score of the model on the test respective split.

3. Report the mean and average MRR .

#### Parameters

- **self**
- **dataset**

#### Returns

model

```
class dicee.Evaluator(args, is_continual_training: bool = False)
```

Evaluator class for KGE models in various downstream tasks.

Orchestrates link prediction evaluation with different scoring techniques including standard evaluation and byte-pair encoding based evaluation.

**er\_vocab**

Entity-relation to tail vocabulary for filtered ranking.

**re\_vocab**

Relation-entity (tail) to head vocabulary.

**ee\_vocab**

Entity-entity to relation vocabulary.

**num\_entities**

Total number of entities in the knowledge graph.

**num\_relations**

Total number of relations in the knowledge graph.

**args**

Configuration arguments.

**report**

Dictionary storing evaluation results.

**during\_training**

Whether evaluation is happening during training.

#### Example

```
>>> from dicee.evaluation import Evaluator
>>> evaluator = Evaluator(args)
>>> results = evaluator.eval(dataset, model, 'EntityPrediction')
>>> print(f"Test MRR: {results['Test']['MRR']:.4f}")
```

**re\_vocab:** Dict | None = None

**er\_vocab:** Dict | None = None

**ee\_vocab:** Dict | None = None

**func\_triple\_to\_bpe\_representation** = None

**is\_continual\_training** = False

```

num_entities: int | None = None

num_relations: int | None = None

domain_constraints_per_rel = None

range_constraints_per_rel = None

args

report: Dict

during_training = False

```

**vocab\_preparation** (*dataset*) → None

Prepare vocabularies from the dataset for evaluation.

Resolves any future objects and saves vocabularies to disk.

#### Parameters

**dataset** – Knowledge graph dataset with vocabulary attributes.

**eval** (*dataset, trained\_model, form\_of\_labelling: str, during\_training: bool = False*) → Dict | None

Evaluate the trained model on the dataset.

#### Parameters

- **dataset** – Knowledge graph dataset (KG instance).
- **trained\_model** – The trained KGE model.
- **form\_of\_labelling** – Type of labelling ('EntityPrediction' or 'RelationPrediction').
- **during\_training** – Whether evaluation is during training.

#### Returns

Dictionary of evaluation metrics, or None if evaluation is skipped.

**eval\_rank\_of\_head\_and\_tail\_entity** (\*, *train\_set, valid\_set=None, test\_set=None, trained\_model*) → None

Evaluate with negative sampling scoring.

**eval\_rank\_of\_head\_and\_tail\_byte\_pair\_encoded\_entity** (\*, *train\_set=None, valid\_set=None, test\_set=None, ordered\_bpe\_entities, trained\_model*) → None

Evaluate with BPE-encoded entities and negative sampling.

**eval\_with\_byte** (\*, *raw\_train\_set, raw\_valid\_set=None, raw\_test\_set=None, trained\_model, form\_of\_labelling*) → None

Evaluate Byte model with generation.

**eval\_with\_bpe\_vs\_all** (\*, *raw\_train\_set, raw\_valid\_set=None, raw\_test\_set=None, trained\_model, form\_of\_labelling*) → None

Evaluate with BPE and KvsAll scoring.

**eval\_with\_vs\_all** (\*, *train\_set, valid\_set=None, test\_set=None, trained\_model, form\_of\_labelling*) → None

Evaluate with KvsAll or 1vsAll scoring.

**evaluate\_lp\_k\_vs\_all** (*model, triple\_idx, info: str | None = None, form\_of\_labelling: str | None = None*) → Dict[str, float]

Filtered link prediction evaluation with KvsAll scoring.

#### Parameters

- **model** – The trained model to evaluate.
- **triple\_idx** – Integer-indexed test triples.
- **info** – Description to print.
- **form\_of\_labelling** – ‘EntityPrediction’ or ‘RelationPrediction’.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

**evaluate\_lp\_with\_byte** (*model, triples: List[List[str]], info: str | None = None*) → Dict[str, float]

Evaluate Byte model with text generation.

#### Parameters

- **model** – Byte model.
- **triples** – String triples.
- **info** – Description to print.

#### Returns

Dictionary with placeholder metrics (-1 values).

**evaluate\_lp\_bpe\_k\_vs\_all** (*model, triples: List[List[str]], info: str | None = None, form\_of\_labelling: str | None = None*) → Dict[str, float]

Evaluate BPE model with KvsAll scoring.

#### Parameters

- **model** – BPE-enabled model.
- **triples** – String triples.
- **info** – Description to print.
- **form\_of\_labelling** – Type of labelling.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

**evaluate\_lp** (*model, triple\_idx, info: str*) → Dict[str, float]

Evaluate link prediction with negative sampling.

#### Parameters

- **model** – The model to evaluate.
- **triple\_idx** – Integer-indexed triples.
- **info** – Description to print.

#### Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

**dummy\_eval** (*trained\_model, form\_of\_labelling: str*) → None

Run evaluation from saved data (for continual training).

#### Parameters

- **trained\_model** – The trained model.
- **form\_of\_labelling** – Type of labelling.

**eval\_with\_data** (*dataset, trained\_model, triple\_idx: numpy.ndarray, form\_of\_labelling: str*)  
→ Dict[str, float]

Evaluate a trained model on a given dataset.

#### Parameters

- **dataset** – Knowledge graph dataset.
- **trained\_model** – The trained model.
- **triple\_idx** – Integer-indexed triples to evaluate.
- **form\_of\_labelling** – Type of labelling.

#### Returns

Dictionary with evaluation metrics.

#### Raises

**ValueError** – If scoring technique is invalid.

`dicee.__version__ = '0.3.2'`

## Python Module Index

### d

- `dicee`, 12
- `dicee.__main__`, 12
- `dicee.abstracts`, 12
- `dicee.analyse_experiments`, 19
- `dicee.callbacks`, 21
- `dicee.config`, 27
- `dicee.dataset_classes`, 31
- `dicee.eval_static_funcs`, 39
- `dicee.evaluation`, 43
  - `dicee.evaluation.ensemble`, 43
  - `dicee.evaluation.evaluator`, 44
  - `dicee.evaluation.link_prediction`, 48
  - `dicee.evaluation.literal_prediction`, 50
  - `dicee.evaluation.utils`, 51
- `dicee.evaluator`, 62
- `dicee.executer`, 65
- `dicee.knowledge_graph`, 68
- `dicee.knowledge_graph_embeddings`, 71
- `dicee.models`, 74
  - `dicee.models.adopt`, 74
  - `dicee.models.base_model`, 83
  - `dicee.models.clifford`, 92
  - `dicee.models.complex`, 101
  - `dicee.models.dualE`, 103
  - `dicee.models.ensemble`, 105
  - `dicee.models.function_space`, 105
  - `dicee.models.literal`, 109
  - `dicee.models.octonion`, 110
  - `dicee.models.pykeen_models`, 113
  - `dicee.models.quaternion`, 115
  - `dicee.models.real`, 118
  - `dicee.models.static_funcs`, 120
  - `dicee.models.transformers`, 121
- `dicee.query_generator`, 180
- `dicee.read_preprocess_save_load_kg`, 182
- `dicee.read_preprocess_save_load_kg.preprocess`, 182
- `dicee.read_preprocess_save_load_kg.read_from_disk`, 183
- `dicee.read_preprocess_save_load_kg.save_load_disk`, 183
- `dicee.read_preprocess_save_load_kg.util`, 184
- `dicee.sanity_checkers`, 188
- `dicee.scripts`, 189
  - `dicee.scripts.index_serve`, 189
  - `dicee.scripts.run`, 190
- `dicee.static_funcs`, 190
- `dicee.static_funcs_training`, 195
- `dicee.static_preprocess_funcs`, 196
- `dicee.trainer`, 197
  - `dicee.trainer.dice_trainer`, 197
  - `dicee.trainer.model_parallelism`, 199
  - `dicee.trainer.torch_trainer`, 200
  - `dicee.trainer.torch_trainer_ddp`, 201
- `dicee.weight_averaging`, 204

# Index

## Non-alphabetical

`__call__()` (*dicee.models.base\_model.IdentityClass method*), 92  
`__call__()` (*dicee.models.ensemble.EnsembleKGE method*), 105  
`__call__()` (*dicee.models.IdentityClass method*), 141, 155, 160  
`__getitem__()` (*dicee.dataset\_classes.AllvsAll method*), 34  
`__getitem__()` (*dicee.dataset\_classes.BPE\_NegativeSamplingDataset method*), 32  
`__getitem__()` (*dicee.dataset\_classes.FixedNegSampleDataset method*), 36  
`__getitem__()` (*dicee.dataset\_classes.KvsAll method*), 35  
`__getitem__()` (*dicee.dataset\_classes.KvsSampleDataset method*), 35  
`__getitem__()` (*dicee.dataset\_classes.LiteralDataset method*), 38  
`__getitem__()` (*dicee.dataset\_classes.MultiClassClassificationDataset method*), 33  
`__getitem__()` (*dicee.dataset\_classes.MultiLabelDataset method*), 33  
`__getitem__()` (*dicee.dataset\_classes.OnevsAllDataset method*), 36  
`__getitem__()` (*dicee.dataset\_classes.OnevsSample method*), 37  
`__getitem__()` (*dicee.dataset\_classes.TriplePredictionDataset method*), 38  
`__iter__()` (*dicee.config.Namespace method*), 31  
`__iter__()` (*dicee.knowledge\_graph.KG method*), 70  
`__iter__()` (*dicee.models.ensemble.EnsembleKGE method*), 105  
`__len__()` (*dicee.dataset\_classes.AllvsAll method*), 34  
`__len__()` (*dicee.dataset\_classes.BPE\_NegativeSamplingDataset method*), 32  
`__len__()` (*dicee.dataset\_classes.FixedNegSampleDataset method*), 36  
`__len__()` (*dicee.dataset\_classes.KvsAll method*), 35  
`__len__()` (*dicee.dataset\_classes.KvsSampleDataset method*), 35  
`__len__()` (*dicee.dataset\_classes.LiteralDataset method*), 38  
`__len__()` (*dicee.dataset\_classes.MultiClassClassificationDataset method*), 33  
`__len__()` (*dicee.dataset\_classes.MultiLabelDataset method*), 33  
`__len__()` (*dicee.dataset\_classes.OnevsAllDataset method*), 36  
`__len__()` (*dicee.dataset\_classes.OnevsSample method*), 37  
`__len__()` (*dicee.dataset\_classes.TriplePredictionDataset method*), 38  
`__len__()` (*dicee.knowledge\_graph.KG method*), 71  
`__len__()` (*dicee.models.ensemble.EnsembleKGE method*), 105  
`__setstate__()` (*dicee.models.ADOPT method*), 131  
`__setstate__()` (*dicee.models.adopt.ADOPT method*), 78  
`__str__()` (*dicee.KGE method*), 212  
`__str__()` (*dicee.knowledge\_graph\_embeddings.KGE method*), 71  
`__str__()` (*dicee.models.ensemble.EnsembleKGE method*), 105  
`__version__` (*in module dicee*), 221

## A

`AbstractCallback` (*class in dicee.abstracts*), 16  
`AbstractPPECallback` (*class in dicee.abstracts*), 18  
`AbstractTrainer` (*class in dicee.abstracts*), 13  
`AccumulateEpochLossCallback` (*class in dicee.callbacks*), 21  
`achieve_answer()` (*dicee.query\_generator.QueryGenerator method*), 181  
`achieve_answer()` (*dicee.QueryGenerator method*), 216  
`AConEx` (*class in dicee.models*), 150  
`AConEx` (*class in dicee.models.complex*), 102  
`AConvO` (*class in dicee.models*), 163  
`AConvO` (*class in dicee.models.octonion*), 113  
`AConvQ` (*class in dicee.models*), 157  
`AConvQ` (*class in dicee.models.quaternion*), 117  
`adaptive_lr` (*dicee.config.Namespace attribute*), 31  
`adaptive_swa` (*dicee.config.Namespace attribute*), 30  
`add_new_entity_embeddings()` (*dicee.abstracts.BaseInteractiveKGE method*), 16  
`add_noise_rate` (*dicee.config.Namespace attribute*), 28  
`add_noise_rate` (*dicee.knowledge\_graph.KG attribute*), 69  
`add_noisy_triples()` (*in module dicee.static\_funcs*), 193  
`add_noisy_triples_into_training()` (*dicee.read\_preprocess\_save\_load\_kg.read\_from\_disk.ReadFromDisk method*), 183  
`add_noisy_triples_into_training()` (*dicee.read\_preprocess\_save\_load\_kg.ReadFromDisk method*), 188  
`add_reciprocal` (*dicee.knowledge\_graph.KG attribute*), 69  
`ADOPT` (*class in dicee.models*), 129  
`ADOPT` (*class in dicee.models.adopt*), 76  
`adopt()` (*in module dicee.models.adopt*), 79  
`ALL_HITS_RANGE` (*in module dicee.evaluation*), 62  
`ALL_HITS_RANGE` (*in module dicee.evaluation.utils*), 52

AllvsAll (class in *dicce.dataset\_classes*), 33  
 alphas (*dicce.abstracts.AbstractPPECallback* attribute), 18  
 alphas (*dicce.weight\_averaging.ASWA* attribute), 205  
 analyse () (in module *dicce.analyse\_experiments*), 20  
 answer\_multi\_hop\_query () (*dicce.KGE* method), 214  
 answer\_multi\_hop\_query () (*dicce.knowledge\_graph\_embeddings.KGE* method), 73  
 app (in module *dicce.scripts.index\_serve*), 189  
 apply\_coefficients () (*dicce.models.clifford.DeCaL* method), 99  
 apply\_coefficients () (*dicce.models.clifford.Keci* method), 94  
 apply\_coefficients () (*dicce.models.DeCaL* method), 168  
 apply\_coefficients () (*dicce.models.Keci* method), 165  
 apply\_reciprocal\_or\_noise () (in module *dicce.read\_preprocess\_save\_load\_kg.util*), 186  
 apply\_semantic\_constraint (*dicce.abstracts.BaseInteractiveKGE* attribute), 15  
 apply\_unit\_norm (*dicce.models.base\_model.BaseKGE* attribute), 90  
 apply\_unit\_norm (*dicce.models.BaseKGE* attribute), 139, 142, 148, 152, 158, 171, 175  
 args (*dicce.DICE\_Trainer* attribute), 217  
 args (*dicce.evaluation.Evaluator* attribute), 54, 55  
 args (*dicce.evaluation.evaluator.Evaluator* attribute), 45  
 args (*dicce.Evaluator* attribute), 218, 219  
 args (*dicce.evaluator.Evaluator* attribute), 63  
 args (*dicce.Execute* attribute), 210  
 args (*dicce.executer.Execute* attribute), 66  
 args (*dicce.models.base\_model.BaseKGE* attribute), 90  
 args (*dicce.models.base\_model.IdentityClass* attribute), 92  
 args (*dicce.models.BaseKGE* attribute), 138, 142, 147, 152, 158, 171, 174  
 args (*dicce.models.ensemble.EnsembleKGE* attribute), 105  
 args (*dicce.models.IdentityClass* attribute), 141, 154, 160  
 args (*dicce.models.pykeen\_models.PykeenKGE* attribute), 114  
 args (*dicce.models.PykeenKGE* attribute), 173  
 args (*dicce.trainer.DICE\_Trainer* attribute), 203  
 args (*dicce.trainer.dice\_trainer.DICE\_Trainer* attribute), 198  
 ASWA (class in *dicce.weight\_averaging*), 204  
 aswa (*dicce.analyse\_experiments.Experiment* attribute), 20  
 attn (*dicce.models.Block* attribute), 144  
 attn (*dicce.models.clifford.TransformerBlock* attribute), 96  
 attn (*dicce.models.transformers.Block* attribute), 126  
 attn\_dropout (*dicce.models.clifford.TransformerSelfAttention* attribute), 97  
 attn\_dropout (*dicce.models.transformers.SelfAttention* attribute), 124  
 attributes (*dicce.abstracts.AbstractTrainer* attribute), 13  
 auto\_batch\_finding (*dicce.config.Namespace* attribute), 30

## B

backend (*dicce.config.Namespace* attribute), 29  
 backend (*dicce.knowledge\_graph.KG* attribute), 69  
 base\_weights (*dicce.weight\_averaging.TWA* attribute), 209  
 BaseInteractiveKGE (class in *dicce.abstracts*), 14  
 BaseInteractiveTrainKGE (class in *dicce.abstracts*), 18  
 BaseKGE (class in *dicce.models*), 138, 141, 147, 151, 157, 171, 174  
 BaseKGE (class in *dicce.models.base\_model*), 89  
 BaseKGELightning (class in *dicce.models*), 132  
 BaseKGELightning (class in *dicce.models.base\_model*), 83  
 batch\_kronecker\_product () (*dicce.callbacks.KronE* static method), 25  
 batch\_size (*dicce.analyse\_experiments.Experiment* attribute), 20  
 batch\_size (*dicce.callbacks.PseudoLabellingCallback* attribute), 24  
 batch\_size (*dicce.config.Namespace* attribute), 28  
 batches\_per\_epoch (*dicce.callbacks.LRScheduler* attribute), 27  
 beta (*dicce.weight\_averaging.TWA* attribute), 209  
 bias (*dicce.models.CoKEConfig* attribute), 146  
 bias (*dicce.models.real.CoKEConfig* attribute), 119, 120  
 bias (*dicce.models.transformers.GPTConfig* attribute), 126  
 bias (*dicce.models.transformers.LayerNorm* attribute), 123  
 Block (class in *dicce.models*), 144  
 Block (class in *dicce.models.transformers*), 125  
 block\_size (*dicce.config.Namespace* attribute), 30  
 block\_size (*dicce.dataset\_classes.MultiClassClassificationDataset* attribute), 33  
 block\_size (*dicce.models.base\_model.BaseKGE* attribute), 91  
 block\_size (*dicce.models.BaseKGE* attribute), 139, 143, 148, 153, 159, 172, 175



block\_size (*dicee.models.CoKEConfig* attribute), 146  
 block\_size (*dicee.models.real.CoKEConfig* attribute), 119, 120  
 block\_size (*dicee.models.transformers.GPTConfig* attribute), 126  
 blocks (*dicee.models.CoKE* attribute), 147  
 blocks (*dicee.models.real.CoKE* attribute), 120  
 bn\_conv1 (*dicee.models.AConvQ* attribute), 157  
 bn\_conv1 (*dicee.models.ConvQ* attribute), 156  
 bn\_conv1 (*dicee.models.quaternion.AConvQ* attribute), 117  
 bn\_conv1 (*dicee.models.quaternion.ConvQ* attribute), 117  
 bn\_conv2 (*dicee.models.AConvQ* attribute), 157  
 bn\_conv2 (*dicee.models.ConvQ* attribute), 157  
 bn\_conv2 (*dicee.models.quaternion.AConvQ* attribute), 117  
 bn\_conv2 (*dicee.models.quaternion.ConvQ* attribute), 117  
 bn\_conv2d (*dicee.models.AConEx* attribute), 150  
 bn\_conv2d (*dicee.models.AConvO* attribute), 163  
 bn\_conv2d (*dicee.models.complex.AConEx* attribute), 102  
 bn\_conv2d (*dicee.models.complex.ConEx* attribute), 101  
 bn\_conv2d (*dicee.models.ConEx* attribute), 150  
 bn\_conv2d (*dicee.models.ConvO* attribute), 162  
 bn\_conv2d (*dicee.models.octonion.AConvO* attribute), 113  
 bn\_conv2d (*dicee.models.octonion.ConvO* attribute), 112  
 BPE\_NegativeSamplingDataset (*class in dicee.dataset\_classes*), 32  
 build\_chain\_funcs () (*dicee.models.FMult2* method), 178  
 build\_chain\_funcs () (*dicee.models.function\_space.FMult2* method), 107  
 build\_func () (*dicee.models.FMult2* method), 178  
 build\_func () (*dicee.models.function\_space.FMult2* method), 107  
 build\_projection () (*dicee.weight\_averaging.TWA* method), 209  
 Byte (*class in dicee.models.transformers*), 121  
 byte\_pair\_encoding (*dicee.analyse\_experiments.Experiment* attribute), 20  
 byte\_pair\_encoding (*dicee.config.Namespace* attribute), 30  
 byte\_pair\_encoding (*dicee.knowledge\_graph.KG* attribute), 69  
 byte\_pair\_encoding (*dicee.models.base\_model.BaseKGE* attribute), 91  
 byte\_pair\_encoding (*dicee.models.BaseKGE* attribute), 139, 143, 148, 153, 159, 172, 175

## C

c\_attn (*dicee.models.clifford.TransformerSelfAttention* attribute), 97  
 c\_attn (*dicee.models.transformers.SelfAttention* attribute), 124  
 c\_fc (*dicee.models.clifford.TransformerMLP* attribute), 97  
 c\_fc (*dicee.models.transformers.MLP* attribute), 125  
 c\_proj (*dicee.models.clifford.TransformerMLP* attribute), 97  
 c\_proj (*dicee.models.clifford.TransformerSelfAttention* attribute), 97  
 c\_proj (*dicee.models.transformers.MLP* attribute), 125  
 c\_proj (*dicee.models.transformers.SelfAttention* attribute), 124  
 callbacks (*dicee.abstracts.AbstractTrainer* attribute), 13  
 callbacks (*dicee.analyse\_experiments.Experiment* attribute), 20  
 callbacks (*dicee.config.Namespace* attribute), 28  
 callbacks (*dicee.trainer.torch\_trainer\_ddp.NodeTrainer* attribute), 202  
 causal (*dicee.models.CoKEConfig* attribute), 146  
 causal (*dicee.models.real.CoKEConfig* attribute), 119, 120  
 causal (*dicee.models.transformers.GPTConfig* attribute), 126  
 causal (*dicee.models.transformers.SelfAttention* attribute), 124  
 chain\_func () (*dicee.models.FMult* method), 177  
 chain\_func () (*dicee.models.function\_space.FMult* method), 106  
 chain\_func () (*dicee.models.function\_space.GFMult* method), 106  
 chain\_func () (*dicee.models.GFMult* method), 177  
 CKeci (*class in dicee.models*), 166  
 CKeci (*class in dicee.models.clifford*), 97  
 cl\_pqr () (*dicee.models.clifford.DeCaL* method), 98  
 cl\_pqr () (*dicee.models.DeCaL* method), 168  
 cleanup () (*dicee.Execute* method), 211  
 cleanup () (*dicee.executer.Execute* method), 67  
 clifford\_multiplication () (*dicee.models.clifford.Keci* method), 94  
 clifford\_multiplication () (*dicee.models.Keci* method), 165  
 clip\_lambda (*dicee.models.ADOPT* attribute), 131  
 clip\_lambda (*dicee.models.adopt.ADOPT* attribute), 78  
 CoKE (*class in dicee.models*), 146  
 CoKE (*class in dicee.models.real*), 120

coke\_dropout (*dicee.models.CoKE attribute*), 147  
 coke\_dropout (*dicee.models.real.CoKE attribute*), 120  
 CoKEConfig (*class in dicee.models*), 146  
 CoKEConfig (*class in dicee.models.real*), 119  
 collate\_fn (*dicee.dataset\_classes.AllvsAll attribute*), 34  
 collate\_fn (*dicee.dataset\_classes.FixedNegSampleDataset attribute*), 36  
 collate\_fn (*dicee.dataset\_classes.KvsAll attribute*), 35  
 collate\_fn (*dicee.dataset\_classes.KvsSampleDataset attribute*), 35  
 collate\_fn (*dicee.dataset\_classes.MultiClassClassificationDataset attribute*), 33  
 collate\_fn (*dicee.dataset\_classes.MultiLabelDataset attribute*), 33  
 collate\_fn (*dicee.dataset\_classes.OnevsAllDataset attribute*), 36  
 collate\_fn (*dicee.dataset\_classes.OnevsSample attribute*), 37  
 collate\_fn () (*dicee.dataset\_classes.BPE\_NegativeSamplingDataset method*), 32  
 collate\_fn () (*dicee.dataset\_classes.TriplePredictionDataset method*), 38  
 collection\_name (*dicee.scripts.index\_serve.NeuralSearcher attribute*), 189  
 comp\_func () (*dicee.models.function\_space.LFMMult method*), 109  
 comp\_func () (*dicee.models.LFMMult method*), 179  
 ComplEx (*class in dicee.models*), 150  
 ComplEx (*class in dicee.models.complex*), 102  
 compute\_convergence () (*in module dicee.callbacks*), 24  
 compute\_func () (*dicee.models.FMMult method*), 177  
 compute\_func () (*dicee.models.FMMult2 method*), 178  
 compute\_func () (*dicee.models.function\_space.FMMult method*), 106  
 compute\_func () (*dicee.models.function\_space.FMMult2 method*), 107  
 compute\_func () (*dicee.models.function\_space.GFMMult method*), 106  
 compute\_func () (*dicee.models.GFMMult method*), 177  
 compute\_metrics\_from\_ranks () (*in module dicee.evaluation*), 61  
 compute\_metrics\_from\_ranks () (*in module dicee.evaluation.utils*), 52  
 compute\_metrics\_from\_ranks\_simple () (*in module dicee.evaluation*), 61  
 compute\_metrics\_from\_ranks\_simple () (*in module dicee.evaluation.utils*), 52  
 compute\_mrr () (*dicee.weight\_averaging.ASWA static method*), 205  
 compute\_sigma\_pp () (*dicee.models.clifford.DeCaL method*), 99  
 compute\_sigma\_pp () (*dicee.models.clifford.Keci method*), 93  
 compute\_sigma\_pp () (*dicee.models.DeCaL method*), 169  
 compute\_sigma\_pp () (*dicee.models.Keci method*), 164  
 compute\_sigma\_pq () (*dicee.models.clifford.DeCaL method*), 100  
 compute\_sigma\_pq () (*dicee.models.clifford.Keci method*), 94  
 compute\_sigma\_pq () (*dicee.models.DeCaL method*), 170  
 compute\_sigma\_pq () (*dicee.models.Keci method*), 165  
 compute\_sigma\_pr () (*dicee.models.clifford.DeCaL method*), 101  
 compute\_sigma\_pr () (*dicee.models.DeCaL method*), 170  
 compute\_sigma\_qq () (*dicee.models.clifford.DeCaL method*), 100  
 compute\_sigma\_qq () (*dicee.models.clifford.Keci method*), 94  
 compute\_sigma\_qq () (*dicee.models.DeCaL method*), 169  
 compute\_sigma\_qq () (*dicee.models.Keci method*), 164  
 compute\_sigma\_qr () (*dicee.models.clifford.DeCaL method*), 101  
 compute\_sigma\_qr () (*dicee.models.DeCaL method*), 170  
 compute\_sigma\_rr () (*dicee.models.clifford.DeCaL method*), 100  
 compute\_sigma\_rr () (*dicee.models.DeCaL method*), 169  
 compute\_sigmas\_multivect () (*dicee.models.clifford.DeCaL method*), 99  
 compute\_sigmas\_multivect () (*dicee.models.DeCaL method*), 168  
 compute\_sigmas\_single () (*dicee.models.clifford.DeCaL method*), 98  
 compute\_sigmas\_single () (*dicee.models.DeCaL method*), 168  
 ConEx (*class in dicee.models*), 149  
 ConEx (*class in dicee.models.complex*), 101  
 config (*dicee.models.CoKE attribute*), 147  
 config (*dicee.models.real.CoKE attribute*), 120  
 config (*dicee.models.transformers.BytE attribute*), 122  
 config (*dicee.models.transformers.GPT attribute*), 127  
 configs (*dicee.abstracts.BaseInteractiveKGE attribute*), 15  
 configure\_optimizers () (*dicee.models.base\_model.BaseKGELightning method*), 88  
 configure\_optimizers () (*dicee.models.BaseKGELightning method*), 136  
 configure\_optimizers () (*dicee.models.transformers.GPT method*), 127  
 construct\_batch\_selected\_cl\_multivector () (*dicee.models.clifford.Keci method*), 95  
 construct\_batch\_selected\_cl\_multivector () (*dicee.models.Keci method*), 166  
 construct\_cl\_multivector () (*dicee.models.clifford.DeCaL method*), 99  
 construct\_cl\_multivector () (*dicee.models.clifford.Keci method*), 95  
 construct\_cl\_multivector () (*dicee.models.DeCaL method*), 168

`construct_cl_multivector()` (*dicee.models.Keci method*), 165  
`construct_dataset()` (*in module dicee.dataset\_classes*), 39  
`construct_ensemble` (*dicee.abstracts.BaseInteractiveKGE attribute*), 14  
`construct_graph()` (*dicee.query\_generator.QueryGenerator method*), 181  
`construct_graph()` (*dicee.QueryGenerator method*), 216  
`construct_input_and_output()` (*dicee.abstracts.BaseInteractiveKGE method*), 16  
`construct_multi_coeff()` (*dicee.models.function\_space.LFMulti method*), 108  
`construct_multi_coeff()` (*dicee.models.LFMulti method*), 179  
`continual_learning` (*dicee.config.Namespace attribute*), 30  
`continual_start()` (*dicee.DICE\_Trainer method*), 217  
`continual_start()` (*dicee.executer.ContinuousExecute method*), 68  
`continual_start()` (*dicee.trainer.DICE\_Trainer method*), 203  
`continual_start()` (*dicee.trainer.dice\_trainer.DICE\_Trainer method*), 198  
`continual_training_setup_executor()` (*in module dicee.static\_funcs*), 194  
`ContinuousExecute` (*class in dicee.executer*), 68  
`conv2d` (*dicee.models.AConEx attribute*), 150  
`conv2d` (*dicee.models.AConvO attribute*), 163  
`conv2d` (*dicee.models.AConvQ attribute*), 157  
`conv2d` (*dicee.models.complex.AConEx attribute*), 102  
`conv2d` (*dicee.models.complex.ConEx attribute*), 101  
`conv2d` (*dicee.models.ConEx attribute*), 149  
`conv2d` (*dicee.models.ConvO attribute*), 162  
`conv2d` (*dicee.models.ConvQ attribute*), 156  
`conv2d` (*dicee.models.octonion.AConvO attribute*), 113  
`conv2d` (*dicee.models.octonion.ConvO attribute*), 112  
`conv2d` (*dicee.models.quaternion.AConvQ attribute*), 117  
`conv2d` (*dicee.models.quaternion.ConvQ attribute*), 117  
`ConvO` (*class in dicee.models*), 162  
`ConvO` (*class in dicee.models.octonion*), 112  
`ConvQ` (*class in dicee.models*), 156  
`ConvQ` (*class in dicee.models.quaternion*), 116  
`count_triples()` (*in module dicee.read\_preprocess\_save\_load\_kg.util*), 186  
`create_and_store_kg()` (*dicee.Execute method*), 211  
`create_and_store_kg()` (*dicee.executer.Execute method*), 67  
`create_constraints()` (*in module dicee.read\_preprocess\_save\_load\_kg.util*), 186  
`create_constraints()` (*in module dicee.static\_preprocess\_funcs*), 197  
`create_experiment_folder()` (*in module dicee.static\_funcs*), 194  
`create_hits_dict()` (*in module dicee.evaluation*), 62  
`create_hits_dict()` (*in module dicee.evaluation.utils*), 53  
`create_random_data()` (*dicee.callbacks.PseudoLabellingCallback method*), 24  
`create_recipriocal_triples()` (*in module dicee.read\_preprocess\_save\_load\_kg.util*), 187  
`create_recipriocal_triples()` (*in module dicee.static\_funcs*), 191  
`create_vector_database()` (*dicee.KGE method*), 212  
`create_vector_database()` (*dicee.knowledge\_graph\_embeddings.KGE method*), 71  
`crop_block_size()` (*dicee.models.transformers.GPT method*), 127  
`ctx` (*dicee.trainer.torch\_trainer\_ddp.NodeTrainer attribute*), 202  
`current_epoch` (*dicee.weight\_averaging.EMA attribute*), 208  
`current_epoch` (*dicee.weight\_averaging.SWA attribute*), 206  
`current_epoch` (*dicee.weight\_averaging.SWAG attribute*), 207  
`current_epoch` (*dicee.weight\_averaging.TWA attribute*), 209  
`cycle_length` (*dicee.callbacks.LRScheduler attribute*), 27

## D

`data_module` (*dicee.callbacks.PseudoLabellingCallback attribute*), 24  
`data_property_embeddings` (*dicee.models.literal.LiteralEmbeddings attribute*), 110  
`dataset_dir` (*dicee.config.Namespace attribute*), 28  
`dataset_dir` (*dicee.knowledge\_graph.KG attribute*), 68, 69  
`dataset_sanity_checking()` (*in module dicee.read\_preprocess\_save\_load\_kg.util*), 187  
`DeCaL` (*class in dicee.models*), 166  
`DeCaL` (*class in dicee.models.clifford*), 97  
`decay` (*dicee.weight\_averaging.EMA attribute*), 208  
`decide()` (*dicee.weight\_averaging.ASWA method*), 205  
`default_eval_model` (*dicee.callbacks.PeriodicEvalCallback attribute*), 26  
`DEFAULT_HITS_RANGE` (*in module dicee.evaluation*), 62  
`DEFAULT_HITS_RANGE` (*in module dicee.evaluation.utils*), 52  
`degree` (*dicee.models.function\_space.LFMulti attribute*), 108  
`degree` (*dicee.models.LFMulti attribute*), 178

- `denormalize()` (*dicee.dataset\_classes.LiteralDataset static method*), 38
- `describe()` (*dicee.knowledge\_graph.KG method*), 70
- `description_of_input` (*dicee.knowledge\_graph.KG attribute*), 70
- `deviations` (*dicee.weight\_averaging.SWAG attribute*), 207
- `device` (*dicee.models.literal.LiteralEmbeddings property*), 110
- `DICE_Trainer` (*class in dicee*), 216
- `DICE_Trainer` (*class in dicee.trainer*), 203
- `DICE_Trainer` (*class in dicee.trainer.dice\_trainer*), 198
- `dicee`
  - `module`, 12
- `dicee.__main__`
  - `module`, 12
- `dicee.abstracts`
  - `module`, 12
- `dicee.analyse_experiments`
  - `module`, 19
- `dicee.callbacks`
  - `module`, 21
- `dicee.config`
  - `module`, 27
- `dicee.dataset_classes`
  - `module`, 31
- `dicee.eval_static_funcs`
  - `module`, 39
- `dicee.evaluation`
  - `module`, 43
- `dicee.evaluation.ensemble`
  - `module`, 43
- `dicee.evaluation.evaluator`
  - `module`, 44
- `dicee.evaluation.link_prediction`
  - `module`, 48
- `dicee.evaluation.literal_prediction`
  - `module`, 50
- `dicee.evaluation.utils`
  - `module`, 51
- `dicee.evaluator`
  - `module`, 62
- `dicee.executer`
  - `module`, 65
- `dicee.knowledge_graph`
  - `module`, 68
- `dicee.knowledge_graph_embeddings`
  - `module`, 71
- `dicee.models`
  - `module`, 74
- `dicee.models.adopt`
  - `module`, 74
- `dicee.models.base_model`
  - `module`, 83
- `dicee.models.clifford`
  - `module`, 92
- `dicee.models.complex`
  - `module`, 101
- `dicee.models.dualE`
  - `module`, 103
- `dicee.models.ensemble`
  - `module`, 105
- `dicee.models.function_space`
  - `module`, 105
- `dicee.models.literal`
  - `module`, 109
- `dicee.models.octonion`
  - `module`, 110
- `dicee.models.pykeen_models`
  - `module`, 113
- `dicee.models.quaternion`
  - `module`, 115

- `dicee.models.real`
  - module, [118](#)
- `dicee.models.static_funcs`
  - module, [120](#)
- `dicee.models.transformers`
  - module, [121](#)
- `dicee.query_generator`
  - module, [180](#)
- `dicee.read_preprocess_save_load_kg`
  - module, [182](#)
- `dicee.read_preprocess_save_load_kg.preprocess`
  - module, [182](#)
- `dicee.read_preprocess_save_load_kg.read_from_disk`
  - module, [183](#)
- `dicee.read_preprocess_save_load_kg.save_load_disk`
  - module, [183](#)
- `dicee.read_preprocess_save_load_kg.util`
  - module, [184](#)
- `dicee.sanity_checkers`
  - module, [188](#)
- `dicee.scripts`
  - module, [189](#)
- `dicee.scripts.index_serve`
  - module, [189](#)
- `dicee.scripts.run`
  - module, [190](#)
- `dicee.static_funcs`
  - module, [190](#)
- `dicee.static_funcs_training`
  - module, [195](#)
- `dicee.static_preprocess_funcs`
  - module, [196](#)
- `dicee.trainer`
  - module, [197](#)
- `dicee.trainer.dice_trainer`
  - module, [197](#)
- `dicee.trainer.model_parallelism`
  - module, [199](#)
- `dicee.trainer.torch_trainer`
  - module, [200](#)
- `dicee.trainer.torch_trainer_ddp`
  - module, [201](#)
- `dicee.weight_averaging`
  - module, [204](#)
- `discrete_points (dicee.models.FMult2 attribute)`, [177](#)
- `discrete_points (dicee.models.function_space.FMult2 attribute)`, [107](#)
- `dist_func (dicee.models.Pyke attribute)`, [145](#)
- `dist_func (dicee.models.real.Pyke attribute)`, [119](#)
- `DistMult (class in dicee.models)`, [144](#)
- `DistMult (class in dicee.models.real)`, [118](#)
- `distributed (dicee.Execute attribute)`, [210](#)
- `distributed (dicee.executer.Execute attribute)`, [66](#)
- `domain_constraints_per_rel (dicee.evaluation.Evaluator attribute)`, [55](#)
- `domain_constraints_per_rel (dicee.evaluation.evaluator.Evaluator attribute)`, [45](#)
- `domain_constraints_per_rel (dicee.Evaluator attribute)`, [219](#)
- `domain_constraints_per_rel (dicee.evaluator.Evaluator attribute)`, [63](#)
- `download_file()` (in module `dicee.static_funcs`), [194](#)
- `download_files_from_url()` (in module `dicee.static_funcs`), [194](#)
- `download_pretrained_model()` (in module `dicee.static_funcs`), [194](#)
- `dropout (dicee.models.clifford.TransformerMLP attribute)`, [97](#)
- `dropout (dicee.models.clifford.TransformerSelfAttention attribute)`, [97](#)
- `dropout (dicee.models.CoKEConfig attribute)`, [146](#)
- `dropout (dicee.models.literal.LiteralEmbeddings attribute)`, [109](#), [110](#)
- `dropout (dicee.models.real.CoKEConfig attribute)`, [119](#), [120](#)
- `dropout (dicee.models.transformers.GPTConfig attribute)`, [126](#)
- `dropout (dicee.models.transformers.MLP attribute)`, [125](#)
- `dropout (dicee.models.transformers.SelfAttention attribute)`, [124](#)
- `DualE (class in dicee.models)`, [179](#)

DualE (class in *dicее.models.dualE*), 104  
 dummy\_eval () (*dicее.evaluation.Evaluator* method), 57  
 dummy\_eval () (*dicее.evaluation.evaluator.Evaluator* method), 47  
 dummy\_eval () (*dicее.Evaluator* method), 220  
 dummy\_eval () (*dicее.evaluator.Evaluator* method), 65  
 dummy\_id (*dicее.knowledge\_graph.KG* attribute), 70  
 during\_training (*dicее.evaluation.Evaluator* attribute), 54, 55  
 during\_training (*dicее.evaluation.evaluator.Evaluator* attribute), 45, 46  
 during\_training (*dicее.Evaluator* attribute), 218, 219  
 during\_training (*dicее.evaluator.Evaluator* attribute), 63

## E

ee\_vocab (*dicее.evaluation.Evaluator* attribute), 54, 55  
 ee\_vocab (*dicее.evaluation.evaluator.Evaluator* attribute), 45  
 ee\_vocab (*dicее.Evaluator* attribute), 218  
 ee\_vocab (*dicее.evaluator.Evaluator* attribute), 62, 63  
 efficient\_zero\_grad () (in module *dicее.evaluation.utils*), 53  
 efficient\_zero\_grad () (in module *dicее.static\_funcs\_training*), 196  
 EMA (class in *dicее.weight\_averaging*), 207  
 ema (*dicее.config.Namespace* attribute), 30  
 ema\_c\_epochs (*dicее.weight\_averaging.EMA* attribute), 208  
 ema\_model (*dicее.weight\_averaging.EMA* attribute), 208  
 ema\_start\_epoch (*dicее.weight\_averaging.EMA* attribute), 208  
 ema\_update () (*dicее.weight\_averaging.EMA* static method), 208  
 embedding\_dim (*dicее.analyse\_experiments.Experiment* attribute), 20  
 embedding\_dim (*dicее.config.Namespace* attribute), 28  
 embedding\_dim (*dicее.models.base\_model.BaseKGE* attribute), 90  
 embedding\_dim (*dicее.models.BaseKGE* attribute), 138, 142, 147, 152, 158, 171, 175  
 embedding\_dim (*dicее.models.literal.LiteralEmbeddings* attribute), 110  
 embedding\_dims (*dicее.models.literal.LiteralEmbeddings* attribute), 109  
 enable\_log (in module *dicее.static\_preprocess\_funcs*), 196  
 enc (*dicее.knowledge\_graph.KG* attribute), 70  
 end () (*dicее.Execute* method), 211  
 end () (*dicее.executer.Execute* method), 67  
 EnsembleKGE (class in *dicее.models.ensemble*), 105  
 ent2id (*dicее.query\_generator.QueryGenerator* attribute), 181  
 ent2id (*dicее.QueryGenerator* attribute), 215  
 ent\_in (*dicее.query\_generator.QueryGenerator* attribute), 181  
 ent\_in (*dicее.QueryGenerator* attribute), 215  
 ent\_out (*dicее.query\_generator.QueryGenerator* attribute), 181  
 ent\_out (*dicее.QueryGenerator* attribute), 215  
 entities\_str (*dicее.knowledge\_graph.KG* property), 70  
 entity\_embeddings (*dicее.models.AConvQ* attribute), 157  
 entity\_embeddings (*dicее.models.clifford.DeCaL* attribute), 98  
 entity\_embeddings (*dicее.models.ConvQ* attribute), 156  
 entity\_embeddings (*dicее.models.DeCaL* attribute), 167  
 entity\_embeddings (*dicее.models.DualE* attribute), 180  
 entity\_embeddings (*dicее.models.dualE.DualE* attribute), 104  
 entity\_embeddings (*dicее.models.FMult* attribute), 176  
 entity\_embeddings (*dicее.models.FMult2* attribute), 177  
 entity\_embeddings (*dicее.models.function\_space.FMult* attribute), 106  
 entity\_embeddings (*dicее.models.function\_space.FMult2* attribute), 107  
 entity\_embeddings (*dicее.models.function\_space.GFMult* attribute), 106  
 entity\_embeddings (*dicее.models.function\_space.LFMult* attribute), 108  
 entity\_embeddings (*dicее.models.function\_space.LFMult1* attribute), 107  
 entity\_embeddings (*dicее.models.GFMult* attribute), 177  
 entity\_embeddings (*dicее.models.LFMult* attribute), 178  
 entity\_embeddings (*dicее.models.LFMult1* attribute), 178  
 entity\_embeddings (*dicее.models.literal.LiteralEmbeddings* attribute), 109, 110  
 entity\_embeddings (*dicее.models.pykeen\_models.PykeenKGE* attribute), 114  
 entity\_embeddings (*dicее.models.PykeenKGE* attribute), 173  
 entity\_embeddings (*dicее.models.quaternion.AConvQ* attribute), 117  
 entity\_embeddings (*dicее.models.quaternion.ConvQ* attribute), 117  
 entity\_to\_idx (*dicее.dataset\_classes.LiteralDataset* attribute), 38  
 entity\_to\_idx (*dicее.knowledge\_graph.KG* attribute), 69  
 entity\_to\_idx (*dicее.scripts.index\_serve.NeuralSearcher* attribute), 189  
 epoch\_count (*dicее.abstracts.AbstractPPECallback* attribute), 18



epoch\_count (*dicee.weight\_averaging.ASWA attribute*), 204  
 epoch\_counter (*dicee.callbacks.Eval attribute*), 24  
 epoch\_counter (*dicee.callbacks.KGESaveCallback attribute*), 23  
 epoch\_counter (*dicee.callbacks.PeriodicEvalCallback attribute*), 26  
 epoch\_ratio (*dicee.callbacks.Eval attribute*), 24  
 er\_vocab (*dicee.evaluation.Evaluator attribute*), 54, 55  
 er\_vocab (*dicee.evaluation.evaluator.Evaluator attribute*), 45  
 er\_vocab (*dicee.Evaluator attribute*), 218  
 er\_vocab (*dicee.evaluator.Evaluator attribute*), 62, 63  
 estimate\_mfu() (*dicee.models.transformers.GPT method*), 127  
 estimate\_q() (*in module dicee.callbacks*), 24  
 Eval (*class in dicee.callbacks*), 24  
 eval() (*dicee.evaluation.Evaluator method*), 55  
 eval() (*dicee.evaluation.evaluator.Evaluator method*), 46  
 eval() (*dicee.Evaluator method*), 219  
 eval() (*dicee.evaluator.Evaluator method*), 63  
 eval() (*dicee.models.ensemble.EnsembleKGE method*), 105  
 eval\_at\_epochs (*dicee.config.Namespace attribute*), 31  
 eval\_epochs (*dicee.callbacks.PeriodicEvalCallback attribute*), 26  
 eval\_every\_n\_epochs (*dicee.config.Namespace attribute*), 30  
 eval\_lp\_performance() (*dicee.KGE method*), 212  
 eval\_lp\_performance() (*dicee.knowledge\_graph\_embeddings.KGE method*), 71  
 eval\_model (*dicee.config.Namespace attribute*), 29  
 eval\_model (*dicee.knowledge\_graph.KG attribute*), 69  
 eval\_rank\_of\_head\_and\_tail\_byte\_pair\_encoded\_entity() (*dicee.evaluation.Evaluator method*), 55  
 eval\_rank\_of\_head\_and\_tail\_byte\_pair\_encoded\_entity() (*dicee.evaluation.evaluator.Evaluator method*), 46  
 eval\_rank\_of\_head\_and\_tail\_byte\_pair\_encoded\_entity() (*dicee.Evaluator method*), 219  
 eval\_rank\_of\_head\_and\_tail\_byte\_pair\_encoded\_entity() (*dicee.evaluator.Evaluator method*), 64  
 eval\_rank\_of\_head\_and\_tail\_entity() (*dicee.evaluation.Evaluator method*), 55  
 eval\_rank\_of\_head\_and\_tail\_entity() (*dicee.evaluation.evaluator.Evaluator method*), 46  
 eval\_rank\_of\_head\_and\_tail\_entity() (*dicee.Evaluator method*), 219  
 eval\_rank\_of\_head\_and\_tail\_entity() (*dicee.evaluator.Evaluator method*), 64  
 eval\_with\_bpe\_vs\_all() (*dicee.evaluation.Evaluator method*), 56  
 eval\_with\_bpe\_vs\_all() (*dicee.evaluation.evaluator.Evaluator method*), 46  
 eval\_with\_bpe\_vs\_all() (*dicee.Evaluator method*), 219  
 eval\_with\_bpe\_vs\_all() (*dicee.evaluator.Evaluator method*), 64  
 eval\_with\_byte() (*dicee.evaluation.Evaluator method*), 55  
 eval\_with\_byte() (*dicee.evaluation.evaluator.Evaluator method*), 46  
 eval\_with\_byte() (*dicee.Evaluator method*), 219  
 eval\_with\_byte() (*dicee.evaluator.Evaluator method*), 64  
 eval\_with\_data() (*dicee.evaluation.Evaluator method*), 57  
 eval\_with\_data() (*dicee.evaluation.evaluator.Evaluator method*), 47  
 eval\_with\_data() (*dicee.Evaluator method*), 220  
 eval\_with\_data() (*dicee.evaluator.Evaluator method*), 65  
 eval\_with\_vs\_all() (*dicee.evaluation.Evaluator method*), 56  
 eval\_with\_vs\_all() (*dicee.evaluation.evaluator.Evaluator method*), 46  
 eval\_with\_vs\_all() (*dicee.Evaluator method*), 219  
 eval\_with\_vs\_all() (*dicee.evaluator.Evaluator method*), 64  
 evaluate() (*in module dicee.static\_funcs*), 194  
 evaluate\_bpe\_lp() (*in module dicee.evaluation*), 59  
 evaluate\_bpe\_lp() (*in module dicee.evaluation.link\_prediction*), 50  
 evaluate\_bpe\_lp() (*in module dicee.static\_funcs\_training*), 195  
 evaluate\_ensemble\_link\_prediction\_performance() (*in module dicee.eval\_static\_funcs*), 42  
 evaluate\_ensemble\_link\_prediction\_performance() (*in module dicee.evaluation*), 60  
 evaluate\_ensemble\_link\_prediction\_performance() (*in module dicee.evaluation.ensemble*), 43  
 evaluate\_link\_prediction\_performance() (*in module dicee.eval\_static\_funcs*), 40  
 evaluate\_link\_prediction\_performance() (*in module dicee.evaluation*), 57  
 evaluate\_link\_prediction\_performance() (*in module dicee.evaluation.link\_prediction*), 48  
 evaluate\_link\_prediction\_performance\_with\_bpe() (*in module dicee.eval\_static\_funcs*), 40  
 evaluate\_link\_prediction\_performance\_with\_bpe() (*in module dicee.evaluation*), 58  
 evaluate\_link\_prediction\_performance\_with\_bpe() (*in module dicee.evaluation.link\_prediction*), 49  
 evaluate\_link\_prediction\_performance\_with\_bpe\_reciprocals() (*in module dicee.eval\_static\_funcs*), 41  
 evaluate\_link\_prediction\_performance\_with\_bpe\_reciprocals() (*in module dicee.evaluation*), 58  
 evaluate\_link\_prediction\_performance\_with\_bpe\_reciprocals() (*in module dicee.evaluation.link\_prediction*), 49  
 evaluate\_link\_prediction\_performance\_with\_reciprocals() (*in module dicee.eval\_static\_funcs*), 40  
 evaluate\_link\_prediction\_performance\_with\_reciprocals() (*in module dicee.evaluation*), 57  
 evaluate\_link\_prediction\_performance\_with\_reciprocals() (*in module dicee.evaluation.link\_prediction*), 48  
 evaluate\_literal\_prediction() (*in module dicee.eval\_static\_funcs*), 41

`evaluate_literal_prediction()` (in module `dicee.evaluation`), 59  
`evaluate_literal_prediction()` (in module `dicee.evaluation.literal_prediction`), 51  
`evaluate_lp()` (`dicee.evaluation.Evaluator` method), 56  
`evaluate_lp()` (`dicee.evaluation.evaluator.Evaluator` method), 47  
`evaluate_lp()` (`dicee.Evaluator` method), 220  
`evaluate_lp()` (`dicee.evaluator.Evaluator` method), 65  
`evaluate_lp()` (in module `dicee.evaluation`), 58  
`evaluate_lp()` (in module `dicee.evaluation.link_prediction`), 49  
`evaluate_lp()` (in module `dicee.static_funcs_training`), 195  
`evaluate_lp_bpe_k_vs_all()` (`dicee.evaluation.Evaluator` method), 56  
`evaluate_lp_bpe_k_vs_all()` (`dicee.evaluation.evaluator.Evaluator` method), 47  
`evaluate_lp_bpe_k_vs_all()` (`dicee.Evaluator` method), 220  
`evaluate_lp_bpe_k_vs_all()` (`dicee.evaluator.Evaluator` method), 64  
`evaluate_lp_bpe_k_vs_all()` (in module `dicee.eval_static_funcs`), 41  
`evaluate_lp_bpe_k_vs_all()` (in module `dicee.evaluation`), 59  
`evaluate_lp_bpe_k_vs_all()` (in module `dicee.evaluation.link_prediction`), 50  
`evaluate_lp_k_vs_all()` (`dicee.evaluation.Evaluator` method), 56  
`evaluate_lp_k_vs_all()` (`dicee.evaluation.evaluator.Evaluator` method), 46  
`evaluate_lp_k_vs_all()` (`dicee.Evaluator` method), 219  
`evaluate_lp_k_vs_all()` (`dicee.evaluator.Evaluator` method), 64  
`evaluate_lp_with_byte()` (`dicee.evaluation.Evaluator` method), 56  
`evaluate_lp_with_byte()` (`dicee.evaluation.evaluator.Evaluator` method), 47  
`evaluate_lp_with_byte()` (`dicee.Evaluator` method), 220  
`evaluate_lp_with_byte()` (`dicee.evaluator.Evaluator` method), 64  
`Evaluator` (class in `dicee`), 218  
`Evaluator` (class in `dicee.evaluation`), 54  
`Evaluator` (class in `dicee.evaluation.evaluator`), 45  
`Evaluator` (class in `dicee.evaluator`), 62  
`evaluator` (`dicee.DICE_Trainer` attribute), 217  
`evaluator` (`dicee.Execute` attribute), 210, 211  
`evaluator` (`dicee.executer.Execute` attribute), 66  
`evaluator` (`dicee.trainer.DICE_Trainer` attribute), 203  
`evaluator` (`dicee.trainer.dice_trainer.DICE_Trainer` attribute), 198  
`every_x_epoch` (`dicee.callbacks.KGESaveCallback` attribute), 23  
`example_input_array` (`dicee.models.ensemble.EnsembleKGE` property), 105  
`Execute` (class in `dicee`), 210  
`Execute` (class in `dicee.executer`), 66  
`exists()` (`dicee.knowledge_graph.KG` method), 70  
`Experiment` (class in `dicee.analyse_experiments`), 19  
`experiment_dir` (`dicee.callbacks.LRScheduler` attribute), 27  
`experiment_dir` (`dicee.callbacks.PeriodicEvalCallback` attribute), 26  
`explicit` (`dicee.models.QMult` attribute), 155  
`explicit` (`dicee.models.quaternion.QMult` attribute), 116  
`exponential_function()` (in module `dicee.static_funcs`), 194  
`extract_input_outputs()` (`dicee.trainer.torch_trainer_ddp.NodeTrainer` method), 202  
`extract_input_outputs()` (in module `dicee.trainer.model_parallelism`), 200  
`extract_input_outputs_set_device()` (`dicee.trainer.torch_trainer.TorchTrainer` method), 201

## F

`f` (`dicee.callbacks.KronE` attribute), 25  
`fc` (`dicee.models.literal.LiteralEmbeddings` attribute), 110  
`fc1` (`dicee.models.AConEx` attribute), 150  
`fc1` (`dicee.models.AConvO` attribute), 163  
`fc1` (`dicee.models.AConvQ` attribute), 157  
`fc1` (`dicee.models.complex.AConEx` attribute), 102  
`fc1` (`dicee.models.complex.ConEx` attribute), 101  
`fc1` (`dicee.models.ConEx` attribute), 149  
`fc1` (`dicee.models.ConvO` attribute), 162  
`fc1` (`dicee.models.ConvQ` attribute), 156  
`fc1` (`dicee.models.octonion.AConvO` attribute), 113  
`fc1` (`dicee.models.octonion.ConvO` attribute), 112  
`fc1` (`dicee.models.quaternion.AConvQ` attribute), 117  
`fc1` (`dicee.models.quaternion.ConvQ` attribute), 117  
`fc_num_input` (`dicee.models.AConEx` attribute), 150  
`fc_num_input` (`dicee.models.AConvO` attribute), 163  
`fc_num_input` (`dicee.models.AConvQ` attribute), 157  
`fc_num_input` (`dicee.models.complex.AConEx` attribute), 102



`fc_num_input` (*dicее.models.complex.ConEx attribute*), 101  
`fc_num_input` (*dicее.models.ConEx attribute*), 149  
`fc_num_input` (*dicее.models.ConvO attribute*), 162  
`fc_num_input` (*dicее.models.ConvQ attribute*), 156  
`fc_num_input` (*dicее.models.octonion.AConvO attribute*), 113  
`fc_num_input` (*dicее.models.octonion.ConvO attribute*), 112  
`fc_num_input` (*dicее.models.quaternion.AConvQ attribute*), 117  
`fc_num_input` (*dicее.models.quaternion.ConvQ attribute*), 117  
`fc_out` (*dicее.models.literal.LiteralEmbeddings attribute*), 110  
`feature_map_dropout` (*dicее.models.AConEx attribute*), 150  
`feature_map_dropout` (*dicее.models.AConvO attribute*), 163  
`feature_map_dropout` (*dicее.models.AConvQ attribute*), 157  
`feature_map_dropout` (*dicее.models.complex.AConEx attribute*), 102  
`feature_map_dropout` (*dicее.models.complex.ConEx attribute*), 101  
`feature_map_dropout` (*dicее.models.ConEx attribute*), 150  
`feature_map_dropout` (*dicее.models.ConvO attribute*), 162  
`feature_map_dropout` (*dicее.models.ConvQ attribute*), 157  
`feature_map_dropout` (*dicее.models.octonion.AConvO attribute*), 113  
`feature_map_dropout` (*dicее.models.octonion.ConvO attribute*), 113  
`feature_map_dropout` (*dicее.models.quaternion.AConvQ attribute*), 117  
`feature_map_dropout` (*dicее.models.quaternion.ConvQ attribute*), 117  
`feature_map_dropout_rate` (*dicее.config.Namespace attribute*), 30  
`feature_map_dropout_rate` (*dicее.models.base\_model.BaseKGE attribute*), 90  
`feature_map_dropout_rate` (*dicее.models.BaseKGE attribute*), 139, 142, 148, 152, 158, 172, 175  
`fetch_worker()` (in module *dicее.read\_preprocess\_save\_load\_kg.util*), 186  
`fill_query()` (*dicее.query\_generator.QueryGenerator method*), 181  
`fill_query()` (*dicее.QueryGenerator method*), 216  
`find_good_batch_size()` (in module *dicее.trainer.model\_parallelism*), 200  
`find_missing_triples()` (*dicее.KGE method*), 214  
`find_missing_triples()` (*dicее.knowledge\_graph\_embeddings.KGE method*), 74  
`fit()` (*dicее.trainer.model\_parallelism.TensorParallel method*), 200  
`fit()` (*dicее.trainer.torch\_trainer.TorchDDPTrainer method*), 202  
`fit()` (*dicее.trainer.torch\_trainer.TorchTrainer method*), 201  
`FixedNegSampleDataset` (class in *dicее.dataset\_classes*), 36  
`flash` (*dicее.models.transformers.SelfAttention attribute*), 124  
`FMult` (class in *dicее.models*), 176  
`FMult` (class in *dicее.models.function\_space*), 106  
`FMult2` (class in *dicее.models*), 177  
`FMult2` (class in *dicее.models.function\_space*), 107  
`form_of_labelling` (*dicее.DICE\_Trainer attribute*), 217  
`form_of_labelling` (*dicее.trainer.DICE\_Trainer attribute*), 203  
`form_of_labelling` (*dicее.trainer.dice\_trainer.DICE\_Trainer attribute*), 198  
`forward()` (*dicее.models.base\_model.BaseKGE method*), 91  
`forward()` (*dicее.models.base\_model.IdentityClass static method*), 92  
`forward()` (*dicее.models.BaseKGE method*), 140, 143, 149, 153, 159, 172, 176  
`forward()` (*dicее.models.Block method*), 144  
`forward()` (*dicее.models.clifford.TransformerBlock method*), 96  
`forward()` (*dicее.models.clifford.TransformerMLP method*), 97  
`forward()` (*dicее.models.clifford.TransformerSelfAttention method*), 97  
`forward()` (*dicее.models.IdentityClass static method*), 141, 155, 161  
`forward()` (*dicее.models.literal.LiteralEmbeddings method*), 110  
`forward()` (*dicее.models.transformers.Block method*), 126  
`forward()` (*dicее.models.transformers.BytE method*), 122  
`forward()` (*dicее.models.transformers.GPT method*), 127  
`forward()` (*dicее.models.transformers.LayerNorm method*), 123  
`forward()` (*dicее.models.transformers.MLP method*), 125  
`forward()` (*dicее.models.transformers.SelfAttention method*), 124  
`forward_backward_update()` (*dicее.trainer.torch\_trainer.TorchTrainer method*), 201  
`forward_backward_update_loss()` (in module *dicее.trainer.model\_parallelism*), 200  
`forward_byte_pair_encoded_k_vs_all()` (*dicее.models.base\_model.BaseKGE method*), 91  
`forward_byte_pair_encoded_k_vs_all()` (*dicее.models.BaseKGE method*), 139, 143, 148, 153, 159, 172, 175  
`forward_byte_pair_encoded_triple()` (*dicее.models.base\_model.BaseKGE method*), 91  
`forward_byte_pair_encoded_triple()` (*dicее.models.BaseKGE method*), 139, 143, 148, 153, 159, 172, 175  
`forward_k_vs_all()` (*dicее.models.AConEx method*), 150  
`forward_k_vs_all()` (*dicее.models.AConvO method*), 163  
`forward_k_vs_all()` (*dicее.models.AConvQ method*), 157  
`forward_k_vs_all()` (*dicее.models.base\_model.BaseKGE method*), 91  
`forward_k_vs_all()` (*dicее.models.BaseKGE method*), 140, 143, 149, 153, 159, 173, 176

`forward_k_vs_all()` (*dicee.models.clifford.DeCaL method*), 99  
`forward_k_vs_all()` (*dicee.models.clifford.Keci method*), 95  
`forward_k_vs_all()` (*dicee.models.clifford.KeciTransformer method*), 96  
`forward_k_vs_all()` (*dicee.models.CoKE method*), 147  
`forward_k_vs_all()` (*dicee.models.ComplEx method*), 151  
`forward_k_vs_all()` (*dicee.models.complex.AConEx method*), 102  
`forward_k_vs_all()` (*dicee.models.complex.ComplEx method*), 103  
`forward_k_vs_all()` (*dicee.models.complex.ConEx method*), 102  
`forward_k_vs_all()` (*dicee.models.ConEx method*), 150  
`forward_k_vs_all()` (*dicee.models.ConvO method*), 163  
`forward_k_vs_all()` (*dicee.models.ConvQ method*), 157  
`forward_k_vs_all()` (*dicee.models.DeCaL method*), 168  
`forward_k_vs_all()` (*dicee.models.DistMult method*), 145  
`forward_k_vs_all()` (*dicee.models.DualE method*), 180  
`forward_k_vs_all()` (*dicee.models.dualE.DualE method*), 104  
`forward_k_vs_all()` (*dicee.models.Keci method*), 165  
`forward_k_vs_all()` (*dicee.models.KeciTransformer method*), 170  
`forward_k_vs_all()` (*dicee.models.octonion.AConvO method*), 113  
`forward_k_vs_all()` (*dicee.models.octonion.ConvO method*), 113  
`forward_k_vs_all()` (*dicee.models.octonion.OMult method*), 112  
`forward_k_vs_all()` (*dicee.models.OMult method*), 161  
`forward_k_vs_all()` (*dicee.models.pykeen\_models.PykeenKGE method*), 114  
`forward_k_vs_all()` (*dicee.models.PykeenKGE method*), 173  
`forward_k_vs_all()` (*dicee.models.QMult method*), 156  
`forward_k_vs_all()` (*dicee.models.quaternion.AConvQ method*), 117  
`forward_k_vs_all()` (*dicee.models.quaternion.ConvQ method*), 117  
`forward_k_vs_all()` (*dicee.models.quaternion.QMult method*), 116  
`forward_k_vs_all()` (*dicee.models.real.CoKE method*), 120  
`forward_k_vs_all()` (*dicee.models.real.DistMult method*), 118  
`forward_k_vs_all()` (*dicee.models.real.Shallom method*), 119  
`forward_k_vs_all()` (*dicee.models.real.TransE method*), 118  
`forward_k_vs_all()` (*dicee.models.Shallom method*), 145  
`forward_k_vs_all()` (*dicee.models.TransE method*), 145  
`forward_k_vs_sample()` (*dicee.models.AConEx method*), 150  
`forward_k_vs_sample()` (*dicee.models.base\_model.BaseKGE method*), 91  
`forward_k_vs_sample()` (*dicee.models.BaseKGE method*), 140, 143, 149, 153, 159, 173, 176  
`forward_k_vs_sample()` (*dicee.models.clifford.Keci method*), 95  
`forward_k_vs_sample()` (*dicee.models.CoKE method*), 147  
`forward_k_vs_sample()` (*dicee.models.ComplEx method*), 151  
`forward_k_vs_sample()` (*dicee.models.complex.AConEx method*), 102  
`forward_k_vs_sample()` (*dicee.models.complex.ComplEx method*), 103  
`forward_k_vs_sample()` (*dicee.models.complex.ConEx method*), 102  
`forward_k_vs_sample()` (*dicee.models.ConEx method*), 150  
`forward_k_vs_sample()` (*dicee.models.DistMult method*), 145  
`forward_k_vs_sample()` (*dicee.models.Keci method*), 166  
`forward_k_vs_sample()` (*dicee.models.pykeen\_models.PykeenKGE method*), 114  
`forward_k_vs_sample()` (*dicee.models.PykeenKGE method*), 174  
`forward_k_vs_sample()` (*dicee.models.QMult method*), 156  
`forward_k_vs_sample()` (*dicee.models.quaternion.QMult method*), 116  
`forward_k_vs_sample()` (*dicee.models.real.CoKE method*), 120  
`forward_k_vs_sample()` (*dicee.models.real.DistMult method*), 118  
`forward_k_vs_with_explicit()` (*dicee.models.clifford.Keci method*), 95  
`forward_k_vs_with_explicit()` (*dicee.models.Keci method*), 165  
`forward_triples()` (*dicee.models.AConEx method*), 150  
`forward_triples()` (*dicee.models.AConvO method*), 163  
`forward_triples()` (*dicee.models.AConvQ method*), 157  
`forward_triples()` (*dicee.models.base\_model.BaseKGE method*), 91  
`forward_triples()` (*dicee.models.BaseKGE method*), 140, 143, 149, 153, 159, 173, 176  
`forward_triples()` (*dicee.models.clifford.DeCaL method*), 98  
`forward_triples()` (*dicee.models.clifford.Keci method*), 96  
`forward_triples()` (*dicee.models.complex.AConEx method*), 102  
`forward_triples()` (*dicee.models.complex.ConEx method*), 102  
`forward_triples()` (*dicee.models.ConEx method*), 150  
`forward_triples()` (*dicee.models.ConvO method*), 162  
`forward_triples()` (*dicee.models.ConvQ method*), 157  
`forward_triples()` (*dicee.models.DeCaL method*), 167  
`forward_triples()` (*dicee.models.DualE method*), 180  
`forward_triples()` (*dicee.models.dualE.DualE method*), 104

`forward_triples()` (*dicee.models.FMult method*), 177  
`forward_triples()` (*dicee.models.FMult2 method*), 178  
`forward_triples()` (*dicee.models.function\_space.FMult method*), 106  
`forward_triples()` (*dicee.models.function\_space.FMult2 method*), 107  
`forward_triples()` (*dicee.models.function\_space.GFMult method*), 107  
`forward_triples()` (*dicee.models.function\_space.LFMult method*), 108  
`forward_triples()` (*dicee.models.function\_space.LFMult1 method*), 107  
`forward_triples()` (*dicee.models.GFMult method*), 177  
`forward_triples()` (*dicee.models.Keci method*), 166  
`forward_triples()` (*dicee.models.LFMult method*), 178  
`forward_triples()` (*dicee.models.LFMult1 method*), 178  
`forward_triples()` (*dicee.models.octonion.AConvO method*), 113  
`forward_triples()` (*dicee.models.octonion.ConvO method*), 113  
`forward_triples()` (*dicee.models.Pyke method*), 145  
`forward_triples()` (*dicee.models.pykeen\_models.PykeenKGE method*), 114  
`forward_triples()` (*dicee.models.PykeenKGE method*), 174  
`forward_triples()` (*dicee.models.quaternion.AConvQ method*), 117  
`forward_triples()` (*dicee.models.quaternion.ConvQ method*), 117  
`forward_triples()` (*dicee.models.real.Pyke method*), 119  
`forward_triples()` (*dicee.models.real.Shallom method*), 119  
`forward_triples()` (*dicee.models.Shallom method*), 145  
`freeze_entity_embeddings` (*dicee.models.literal.LiteralEmbeddings attribute*), 110  
`frequency` (*dicee.callbacks.Perturb attribute*), 26  
`from_pretrained()` (*dicee.models.transformers.GPT class method*), 127  
`from_pretrained_model_write_embeddings_into_csv()` (*in module dicee.static\_funcs*), 194  
`full_storage_path` (*dicee.analyse\_experiments.Experiment attribute*), 20  
`func_triple_to_bpe_representation` (*dicee.evaluation.Evaluator attribute*), 55  
`func_triple_to_bpe_representation` (*dicee.evaluation.evaluator.Evaluator attribute*), 45  
`func_triple_to_bpe_representation` (*dicee.Evaluator attribute*), 218  
`func_triple_to_bpe_representation` (*dicee.evaluator.Evaluator attribute*), 63  
`func_triple_to_bpe_representation()` (*dicee.knowledge\_graph.KG method*), 71  
`function()` (*dicee.models.FMult2 method*), 178  
`function()` (*dicee.models.function\_space.FMult2 method*), 107

## G

`gamma` (*dicee.models.FMult attribute*), 177  
`gamma` (*dicee.models.function\_space.FMult attribute*), 106  
`gate_residual` (*dicee.models.literal.LiteralEmbeddings attribute*), 109, 110  
`gated_residual_proj` (*dicee.models.literal.LiteralEmbeddings attribute*), 110  
`gelu` (*dicee.models.clifford.TransformerMLP attribute*), 97  
`gelu` (*dicee.models.transformers.MLP attribute*), 125  
`gen_test` (*dicee.query\_generator.QueryGenerator attribute*), 181  
`gen_test` (*dicee.QueryGenerator attribute*), 215  
`gen_valid` (*dicee.query\_generator.QueryGenerator attribute*), 181  
`gen_valid` (*dicee.QueryGenerator attribute*), 215  
`generate()` (*dicee.KGE method*), 212  
`generate()` (*dicee.knowledge\_graph\_embeddings.KGE method*), 71  
`generate()` (*dicee.models.transformers.BytE method*), 122  
`generate_queries()` (*dicee.query\_generator.QueryGenerator method*), 181  
`generate_queries()` (*dicee.QueryGenerator method*), 216  
`get_aswa_state_dict()` (*dicee.weight\_averaging.ASWA method*), 205  
`get_bpe_head_and_relation_representation()` (*dicee.models.base\_model.BaseKGE method*), 91  
`get_bpe_head_and_relation_representation()` (*dicee.models.BaseKGE method*), 140, 143, 149, 154, 160, 173, 176  
`get_bpe_token_representation()` (*dicee.abstracts.BaseInteractiveKGE method*), 15  
`get_callbacks()` (*in module dicee.trainer.dice\_trainer*), 198  
`get_default_arguments()` (*in module dicee.analyse\_experiments*), 19  
`get_default_arguments()` (*in module dicee.scripts.index\_serve*), 189  
`get_default_arguments()` (*in module dicee.scripts.run*), 190  
`get_ee_vocab()` (*in module dicee.read\_preprocess\_save\_load\_kg.util*), 186  
`get_ee_vocab()` (*in module dicee.static\_funcs*), 192  
`get_ee_vocab()` (*in module dicee.static\_preprocess\_funcs*), 197  
`get_embeddings()` (*dicee.models.base\_model.BaseKGE method*), 92  
`get_embeddings()` (*dicee.models.BaseKGE method*), 140, 143, 149, 154, 160, 173, 176  
`get_embeddings()` (*dicee.models.ensemble.EnsembleKGE method*), 105  
`get_embeddings()` (*dicee.models.real.Shallom method*), 119  
`get_embeddings()` (*dicee.models.Shallom method*), 145  
`get_entity_embeddings()` (*dicee.abstracts.BaseInteractiveKGE method*), 16

[get\\_entity\\_index\(\)](#) (*dicee.abstracts.BaseInteractiveKGE method*), 15  
[get\\_er\\_vocab\(\)](#) (in module *dicee.read\_preprocess\_save\_load\_kg.util*), 186  
[get\\_er\\_vocab\(\)](#) (in module *dicee.static\_funcs*), 191  
[get\\_er\\_vocab\(\)](#) (in module *dicee.static\_preprocess\_funcs*), 197  
[get\\_eval\\_report\(\)](#) (*dicee.abstracts.BaseInteractiveKGE method*), 15  
[get\\_head\\_relation\\_representation\(\)](#) (*dicee.models.base\_model.BaseKGE method*), 91  
[get\\_head\\_relation\\_representation\(\)](#) (*dicee.models.BaseKGE method*), 140, 143, 149, 154, 159, 173, 176  
[get\\_kronecker\\_triple\\_representation\(\)](#) (*dicee.callbacks.KronE method*), 25  
[get\\_mean\\_and\\_var\(\)](#) (*dicee.weight\_averaging.SWAG method*), 207  
[get\\_num\\_params\(\)](#) (*dicee.models.transformers.GPT method*), 127  
[get\\_padded\\_bpe\\_triple\\_representation\(\)](#) (*dicee.abstracts.BaseInteractiveKGE method*), 15  
[get\\_queries\(\)](#) (*dicee.query\_generator.QueryGenerator method*), 182  
[get\\_queries\(\)](#) (*dicee.QueryGenerator method*), 216  
[get\\_re\\_vocab\(\)](#) (in module *dicee.read\_preprocess\_save\_load\_kg.util*), 186  
[get\\_re\\_vocab\(\)](#) (in module *dicee.static\_funcs*), 192  
[get\\_re\\_vocab\(\)](#) (in module *dicee.static\_preprocess\_funcs*), 197  
[get\\_relation\\_embeddings\(\)](#) (*dicee.abstracts.BaseInteractiveKGE method*), 16  
[get\\_relation\\_index\(\)](#) (*dicee.abstracts.BaseInteractiveKGE method*), 15  
[get\\_sentence\\_representation\(\)](#) (*dicee.models.base\_model.BaseKGE method*), 91  
[get\\_sentence\\_representation\(\)](#) (*dicee.models.BaseKGE method*), 140, 143, 149, 154, 160, 173, 176  
[get\\_transductive\\_entity\\_embeddings\(\)](#) (*dicee.KGE method*), 212  
[get\\_transductive\\_entity\\_embeddings\(\)](#) (*dicee.knowledge\_graph\_embeddings.KGE method*), 71  
[get\\_triple\\_representation\(\)](#) (*dicee.models.base\_model.BaseKGE method*), 91  
[get\\_triple\\_representation\(\)](#) (*dicee.models.BaseKGE method*), 140, 143, 149, 153, 159, 173, 176  
[GFMult](#) (class in *dicee.models*), 177  
[GFMult](#) (class in *dicee.models.function\_space*), 106  
[global\\_rank](#) (*dicee.abstracts.AbstractTrainer attribute*), 13  
[global\\_rank](#) (*dicee.trainer.torch\_trainer\_ddp.NodeTrainer attribute*), 202  
[GPT](#) (class in *dicee.models.transformers*), 126  
[GPTConfig](#) (class in *dicee.models.transformers*), 126  
[gpus](#) (*dicee.config.Namespace attribute*), 28  
[gradient\\_accumulation\\_steps](#) (*dicee.config.Namespace attribute*), 29  
[ground\\_queries\(\)](#) (*dicee.query\_generator.QueryGenerator method*), 181  
[ground\\_queries\(\)](#) (*dicee.QueryGenerator method*), 216  
[gswa\\_n](#) (*dicee.weight\_averaging.SWAG attribute*), 207

## H

[hidden\\_dim](#) (*dicee.models.literal.LiteralEmbeddings attribute*), 110  
[hidden\\_dropout](#) (*dicee.models.base\_model.BaseKGE attribute*), 91  
[hidden\\_dropout](#) (*dicee.models.BaseKGE attribute*), 139, 142, 148, 153, 159, 172, 175  
[hidden\\_dropout\\_rate](#) (*dicee.config.Namespace attribute*), 30  
[hidden\\_dropout\\_rate](#) (*dicee.models.base\_model.BaseKGE attribute*), 90  
[hidden\\_dropout\\_rate](#) (*dicee.models.BaseKGE attribute*), 139, 142, 148, 152, 158, 171, 175  
[hidden\\_normalizer](#) (*dicee.models.base\_model.BaseKGE attribute*), 90  
[hidden\\_normalizer](#) (*dicee.models.BaseKGE attribute*), 139, 142, 148, 153, 159, 172, 175

## I

[IdentityClass](#) (class in *dicee.models*), 140, 154, 160  
[IdentityClass](#) (class in *dicee.models.base\_model*), 92  
[idx\\_entity\\_to\\_bpe\\_shaped](#) (*dicee.knowledge\_graph.KG attribute*), 70  
[index\(\)](#) (in module *dicee.scripts.index\_serve*), 189  
[index\\_triple\(\)](#) (*dicee.abstracts.BaseInteractiveKGE method*), 15  
[init\\_data\\_loader\(\)](#) (*dicee.DICE\_Trainer method*), 217  
[init\\_data\\_loader\(\)](#) (*dicee.trainer.DICE\_Trainer method*), 203  
[init\\_data\\_loader\(\)](#) (*dicee.trainer.dice\_trainer.DICE\_Trainer method*), 199  
[init\\_dataset\(\)](#) (*dicee.DICE\_Trainer method*), 217  
[init\\_dataset\(\)](#) (*dicee.trainer.DICE\_Trainer method*), 204  
[init\\_dataset\(\)](#) (*dicee.trainer.dice\_trainer.DICE\_Trainer method*), 199  
[init\\_param](#) (*dicee.config.Namespace attribute*), 29  
[init\\_params\\_with\\_sanity\\_checking\(\)](#) (*dicee.models.base\_model.BaseKGE method*), 91  
[init\\_params\\_with\\_sanity\\_checking\(\)](#) (*dicee.models.BaseKGE method*), 140, 143, 149, 153, 159, 172, 176  
[initial\\_eval\\_setting](#) (*dicee.weight\_averaging.ASWA attribute*), 204  
[initialize\\_or\\_load\\_model\(\)](#) (*dicee.DICE\_Trainer method*), 217  
[initialize\\_or\\_load\\_model\(\)](#) (*dicee.trainer.DICE\_Trainer method*), 203  
[initialize\\_or\\_load\\_model\(\)](#) (*dicee.trainer.dice\_trainer.DICE\_Trainer method*), 199  
[initialize\\_trainer\(\)](#) (*dicee.DICE\_Trainer method*), 217  
[initialize\\_trainer\(\)](#) (*dicee.trainer.DICE\_Trainer method*), 203

`initialize_trainer()` (*dicee.trainer.dice\_trainer.DICE\_Trainer* method), 199  
`initialize_trainer()` (in module *dicee.trainer.dice\_trainer*), 197  
`input_dp_ent_real` (*dicee.models.base\_model.BaseKGE* attribute), 91  
`input_dp_ent_real` (*dicee.models.BaseKGE* attribute), 139, 142, 148, 153, 159, 172, 175  
`input_dp_rel_real` (*dicee.models.base\_model.BaseKGE* attribute), 91  
`input_dp_rel_real` (*dicee.models.BaseKGE* attribute), 139, 142, 148, 153, 159, 172, 175  
`input_dropout_rate` (*dicee.config.Namespace* attribute), 30  
`input_dropout_rate` (*dicee.models.base\_model.BaseKGE* attribute), 90  
`input_dropout_rate` (*dicee.models.BaseKGE* attribute), 139, 142, 148, 152, 158, 171, 175  
`InteractiveQueryDecomposition` (class in *dicee.abstracts*), 16  
`intialize_model()` (in module *dicee.static\_funcs*), 193  
`is_continual_training` (*dicee.DICE\_Trainer* attribute), 217  
`is_continual_training` (*dicee.evaluation.Evaluator* attribute), 55  
`is_continual_training` (*dicee.evaluation.evaluator.Evaluator* attribute), 45  
`is_continual_training` (*dicee.Evaluator* attribute), 218  
`is_continual_training` (*dicee.evaluator.Evaluator* attribute), 63  
`is_continual_training` (*dicee.Execute* attribute), 210  
`is_continual_training` (*dicee.executer.Execute* attribute), 66  
`is_continual_training` (*dicee.trainer.DICE\_Trainer* attribute), 203  
`is_continual_training` (*dicee.trainer.dice\_trainer.DICE\_Trainer* attribute), 198  
`is_global_zero` (*dicee.abstracts.AbstractTrainer* attribute), 13  
`is_rank_zero()` (*dicee.Execute* method), 211  
`is_rank_zero()` (*dicee.executer.Execute* method), 67  
`is_seen()` (*dicee.abstracts.BaseInteractiveKGE* method), 15  
`is_sparql_endpoint_alive()` (in module *dicee.sanity\_checkers*), 188

## K

`k` (*dicee.models.FMult* attribute), 176  
`k` (*dicee.models.FMult2* attribute), 177  
`k` (*dicee.models.function\_space.FMult* attribute), 106  
`k` (*dicee.models.function\_space.FMult2* attribute), 107  
`k` (*dicee.models.function\_space.GFMult* attribute), 106  
`k` (*dicee.models.GFMult* attribute), 177  
`k_fold_cross_validation()` (*dicee.DICE\_Trainer* method), 217  
`k_fold_cross_validation()` (*dicee.trainer.DICE\_Trainer* method), 204  
`k_fold_cross_validation()` (*dicee.trainer.dice\_trainer.DICE\_Trainer* method), 199  
`k_vs_all_score()` (*dicee.models.clifford.Keci* method), 95  
`k_vs_all_score()` (*dicee.models.ComplEx* static method), 151  
`k_vs_all_score()` (*dicee.models.complex.ComplEx* static method), 103  
`k_vs_all_score()` (*dicee.models.DistMult* method), 144  
`k_vs_all_score()` (*dicee.models.Keci* method), 165  
`k_vs_all_score()` (*dicee.models.octonion.OMult* method), 112  
`k_vs_all_score()` (*dicee.models.OMult* method), 161  
`k_vs_all_score()` (*dicee.models.QMult* method), 156  
`k_vs_all_score()` (*dicee.models.quaternion.QMult* method), 116  
`k_vs_all_score()` (*dicee.models.real.DistMult* method), 118  
`Keci` (class in *dicee.models*), 163  
`Keci` (class in *dicee.models.clifford*), 93  
`KeciTransformer` (class in *dicee.models*), 170  
`KeciTransformer` (class in *dicee.models.clifford*), 96  
`kernel_size` (*dicee.config.Namespace* attribute), 30  
`kernel_size` (*dicee.models.base\_model.BaseKGE* attribute), 90  
`kernel_size` (*dicee.models.BaseKGE* attribute), 139, 142, 148, 152, 158, 172, 175  
`KGE` (class in *dicee.knowledge\_graph*), 68  
`kg` (*dicee.callbacks.PseudoLabellingCallback* attribute), 24  
`kg` (*dicee.read\_preprocess\_save\_load\_kg.LoadSaveToDisk* attribute), 188  
`kg` (*dicee.read\_preprocess\_save\_load\_kg.PreprocessKG* attribute), 187  
`kg` (*dicee.read\_preprocess\_save\_load\_kg.preprocess.PreprocessKG* attribute), 182  
`kg` (*dicee.read\_preprocess\_save\_load\_kg.read\_from\_disk.ReadFromDisk* attribute), 183  
`kg` (*dicee.read\_preprocess\_save\_load\_kg.ReadFromDisk* attribute), 188  
`kg` (*dicee.read\_preprocess\_save\_load\_kg.save\_load\_disk.LoadSaveToDisk* attribute), 183  
`KGE` (class in *dicee*), 212  
`KGE` (class in *dicee.knowledge\_graph\_embeddings*), 71  
`KGESaveCallback` (class in *dicee.callbacks*), 22  
`knowledge_graph` (*dicee.Execute* attribute), 210  
`knowledge_graph` (*dicee.executer.Execute* attribute), 66  
`KronE` (class in *dicee.callbacks*), 25



KvsAll (class in *dicee.dataset\_classes*), 34  
 kvsall\_score() (*dicee.models.DualE* method), 180  
 kvsall\_score() (*dicee.models.dualE.DualE* method), 104  
 KvsSampleDataset (class in *dicee.dataset\_classes*), 35

## L

label\_smoothing\_rate (*dicee.config.Namespace* attribute), 29  
 label\_smoothing\_rate (*dicee.dataset\_classes.AllvsAll* attribute), 34  
 label\_smoothing\_rate (*dicee.dataset\_classes.FixedNegSampleDataset* attribute), 36  
 label\_smoothing\_rate (*dicee.dataset\_classes.KvsAll* attribute), 34  
 label\_smoothing\_rate (*dicee.dataset\_classes.KvsSampleDataset* attribute), 35  
 label\_smoothing\_rate (*dicee.dataset\_classes.OnevsSample* attribute), 37  
 label\_smoothing\_rate (*dicee.dataset\_classes.TriplePredictionDataset* attribute), 37  
 layer\_norm (*dicee.models.literal.LiteralEmbeddings* attribute), 110  
 LayerNorm (class in *dicee.models.transformers*), 123  
 learning\_rate (*dicee.models.base\_model.BaseKGE* attribute), 90  
 learning\_rate (*dicee.models.BaseKGE* attribute), 139, 142, 148, 152, 158, 171, 175  
 length (*dicee.dataset\_classes.FixedNegSampleDataset* attribute), 36  
 length (*dicee.dataset\_classes.TriplePredictionDataset* attribute), 37  
 level (*dicee.callbacks.Perturb* attribute), 26  
 LFMult (class in *dicee.models*), 178  
 LFMult (class in *dicee.models.function\_space*), 108  
 LFMult1 (class in *dicee.models*), 178  
 LFMult1 (class in *dicee.models.function\_space*), 107  
 linear() (*dicee.models.function\_space.LFMult* method), 108  
 linear() (*dicee.models.LFMult* method), 179  
 list2tuple() (*dicee.query\_generator.QueryGenerator* method), 181  
 list2tuple() (*dicee.QueryGenerator* method), 216  
 LiteralDataset (class in *dicee.dataset\_classes*), 38  
 LiteralEmbeddings (class in *dicee.models.literal*), 109  
 lm\_head (*dicee.models.clifford.KeciTransformer* attribute), 96  
 lm\_head (*dicee.models.KeciTransformer* attribute), 170  
 lm\_head (*dicee.models.transformers.BytE* attribute), 122  
 lm\_head (*dicee.models.transformers.GPT* attribute), 127  
 ln\_1 (*dicee.models.Block* attribute), 144  
 ln\_1 (*dicee.models.clifford.TransformerBlock* attribute), 96  
 ln\_1 (*dicee.models.transformers.Block* attribute), 126  
 ln\_2 (*dicee.models.Block* attribute), 144  
 ln\_2 (*dicee.models.clifford.TransformerBlock* attribute), 96  
 ln\_2 (*dicee.models.transformers.Block* attribute), 126  
 ln\_f (*dicee.models.CoKE* attribute), 147  
 ln\_f (*dicee.models.real.CoKE* attribute), 120  
 load() (*dicee.read\_preprocess\_save\_load\_kg.LoadSaveToDisk* method), 188  
 load() (*dicee.read\_preprocess\_save\_load\_kg.save\_load\_disk.LoadSaveToDisk* method), 183  
 load\_and\_validate\_literal\_data() (*dicee.dataset\_classes.LiteralDataset* static method), 38  
 load\_from\_memmap() (*dicee.Execute* method), 211  
 load\_from\_memmap() (*dicee.executer.Execute* method), 67  
 load\_json() (in module *dicee.static\_funcs*), 194  
 load\_model() (in module *dicee.static\_funcs*), 193  
 load\_model\_ensemble() (in module *dicee.static\_funcs*), 193  
 load\_numpy() (in module *dicee.static\_funcs*), 194  
 load\_numpy\_ndarray() (in module *dicee.read\_preprocess\_save\_load\_kg.util*), 187  
 load\_pickle() (in module *dicee.read\_preprocess\_save\_load\_kg.util*), 187  
 load\_pickle() (in module *dicee.static\_funcs*), 192  
 load\_queries() (*dicee.query\_generator.QueryGenerator* method), 182  
 load\_queries() (*dicee.QueryGenerator* method), 216  
 load\_queries\_and\_answers() (*dicee.query\_generator.QueryGenerator* static method), 182  
 load\_queries\_and\_answers() (*dicee.QueryGenerator* static method), 216  
 load\_state\_dict() (*dicee.models.ensemble.EnsembleKGE* method), 105  
 load\_term\_mapping() (in module *dicee.static\_funcs*), 192  
 load\_term\_mapping() (in module *dicee.trainer.dice\_trainer*), 197  
 load\_with\_pandas() (in module *dicee.read\_preprocess\_save\_load\_kg.util*), 186  
 loader\_backend (*dicee.dataset\_classes.LiteralDataset* attribute), 38  
 LoadSaveToDisk (class in *dicee.read\_preprocess\_save\_load\_kg*), 188  
 LoadSaveToDisk (class in *dicee.read\_preprocess\_save\_load\_kg.save\_load\_disk*), 183  
 local\_rank (*dicee.abstracts.AbstractTrainer* attribute), 13  
 local\_rank (*dicee.Execute* attribute), 210

local\_rank (*dicee.executer.Execute* attribute), 66  
 local\_rank (*dicee.trainer.torch\_trainer\_ddp.NodeTrainer* attribute), 202  
 loss (*dicee.models.base\_model.BaseKGE* attribute), 90  
 loss (*dicee.models.BaseKGE* attribute), 139, 142, 148, 152, 158, 172, 175  
 loss\_func (*dicee.trainer.torch\_trainer\_ddp.NodeTrainer* attribute), 202  
 loss\_function (*dicee.trainer.torch\_trainer.TorchTrainer* attribute), 200  
 loss\_function() (*dicee.models.base\_model.BaseKGELightning* method), 85  
 loss\_function() (*dicee.models.BaseKGELightning* method), 134  
 loss\_function() (*dicee.models.transformers.BytE* method), 122  
 loss\_history (*dicee.models.base\_model.BaseKGE* attribute), 91  
 loss\_history (*dicee.models.BaseKGE* attribute), 139, 143, 148, 153, 159, 172, 175  
 loss\_history (*dicee.models.pykeen\_models.PykeenKGE* attribute), 114  
 loss\_history (*dicee.models.PykeenKGE* attribute), 173  
 loss\_history (*dicee.trainer.torch\_trainer\_ddp.NodeTrainer* attribute), 202  
 lr (*dicee.analyse\_experiments.Experiment* attribute), 20  
 lr (*dicee.config.Namespace* attribute), 28  
 lr\_init (*dicee.weight\_averaging.SWA* attribute), 206  
 lr\_init (*dicee.weight\_averaging.SWAG* attribute), 207  
 lr\_init (*dicee.weight\_averaging.TWA* attribute), 209  
 lr\_lambda (*dicee.callbacks.LRScheduler* attribute), 27  
 LRScheduler (class in *dicee.callbacks*), 26

## M

m (*dicee.models.function\_space.LFMult* attribute), 108  
 m (*dicee.models.LFMult* attribute), 178  
 main() (in module *dicee.scripts.index\_serve*), 190  
 main() (in module *dicee.scripts.run*), 190  
 make\_iterable\_verbose() (in module *dicee.evaluation*), 61  
 make\_iterable\_verbose() (in module *dicee.evaluation.utils*), 52  
 make\_iterable\_verbose() (in module *dicee.static\_funcs\_training*), 196  
 make\_iterable\_verbose() (in module *dicee.trainer.torch\_trainer\_ddp*), 201  
 mapping\_from\_first\_two\_cols\_to\_third() (in module *dicee.static\_preprocess\_funcs*), 197  
 margin (*dicee.models.Pyke* attribute), 145  
 margin (*dicee.models.real.Pyke* attribute), 119  
 margin (*dicee.models.real.TransE* attribute), 118  
 margin (*dicee.models.TransE* attribute), 145  
 mask\_emb (*dicee.models.CoKE* attribute), 147  
 mask\_emb (*dicee.models.real.CoKE* attribute), 120  
 max\_ans\_num (*dicee.query\_generator.QueryGenerator* attribute), 181  
 max\_ans\_num (*dicee.QueryGenerator* attribute), 215  
 max\_epochs (*dicee.callbacks.KGESaveCallback* attribute), 23  
 max\_epochs (*dicee.callbacks.PeriodicEvalCallback* attribute), 26  
 max\_epochs (*dicee.weight\_averaging.EMA* attribute), 208  
 max\_epochs (*dicee.weight\_averaging.SWA* attribute), 206  
 max\_epochs (*dicee.weight\_averaging.SWAG* attribute), 207  
 max\_epochs (*dicee.weight\_averaging.TWA* attribute), 209  
 max\_length\_subword\_tokens (*dicee.knowledge\_graph.KG* attribute), 70  
 max\_length\_subword\_tokens (*dicee.models.base\_model.BaseKGE* attribute), 91  
 max\_length\_subword\_tokens (*dicee.models.BaseKGE* attribute), 139, 143, 148, 153, 159, 172, 175  
 max\_num\_models (*dicee.weight\_averaging.SWAG* attribute), 207  
 max\_num\_of\_classes (*dicee.dataset\_classes.KvsSampleDataset* attribute), 35  
 mean (*dicee.weight\_averaging.SWAG* attribute), 207  
 mem\_of\_model() (*dicee.models.base\_model.BaseKGELightning* method), 84  
 mem\_of\_model() (*dicee.models.BaseKGELightning* method), 133  
 mem\_of\_model() (*dicee.models.ensemble.EnsembleKGE* method), 105  
 method (*dicee.callbacks.Perturb* attribute), 26  
 MLP (class in *dicee.models.transformers*), 124  
 mlp (*dicee.models.Block* attribute), 144  
 mlp (*dicee.models.clifford.TransformerBlock* attribute), 96  
 mlp (*dicee.models.transformers.Block* attribute), 126  
 mode (*dicee.query\_generator.QueryGenerator* attribute), 181  
 mode (*dicee.QueryGenerator* attribute), 215  
 model (*dicee.config.Namespace* attribute), 28  
 model (*dicee.models.pykeen\_models.PykeenKGE* attribute), 114  
 model (*dicee.models.PykeenKGE* attribute), 173  
 model (*dicee.trainer.torch\_trainer\_ddp.NodeTrainer* attribute), 202  
 model (*dicee.trainer.torch\_trainer.TorchTrainer* attribute), 200

- `model_kwargs` (*dicee.models.pykeen\_models.PykeenKGE attribute*), 114
- `model_kwargs` (*dicee.models.PykeenKGE attribute*), 173
- `model_name` (*dicee.analyse\_experiments.Experiment attribute*), 19
- `MODEL_REGISTRY` (*in module dicee.static\_funcs*), 191
- `module`
  - `dicee`, 12
  - `dicee.__main__`, 12
  - `dicee.abstracts`, 12
  - `dicee.analyse_experiments`, 19
  - `dicee.callbacks`, 21
  - `dicee.config`, 27
  - `dicee.dataset_classes`, 31
  - `dicee.eval_static_funcs`, 39
  - `dicee.evaluation`, 43
    - `dicee.evaluation.ensemble`, 43
    - `dicee.evaluation.evaluator`, 44
    - `dicee.evaluation.link_prediction`, 48
    - `dicee.evaluation.literal_prediction`, 50
    - `dicee.evaluation.utils`, 51
  - `dicee.evaluator`, 62
  - `dicee.executer`, 65
  - `dicee.knowledge_graph`, 68
  - `dicee.knowledge_graph_embeddings`, 71
  - `dicee.models`, 74
    - `dicee.models.adopt`, 74
    - `dicee.models.base_model`, 83
    - `dicee.models.clifford`, 92
    - `dicee.models.complex`, 101
    - `dicee.models.dualE`, 103
    - `dicee.models.ensemble`, 105
    - `dicee.models.function_space`, 105
    - `dicee.models.literal`, 109
    - `dicee.models.octonion`, 110
    - `dicee.models.pykeen_models`, 113
    - `dicee.models.quaternion`, 115
    - `dicee.models.real`, 118
    - `dicee.models.static_funcs`, 120
    - `dicee.models.transformers`, 121
  - `dicee.query_generator`, 180
  - `dicee.read_preprocess_save_load_kg`, 182
    - `dicee.read_preprocess_save_load_kg.preprocess`, 182
    - `dicee.read_preprocess_save_load_kg.read_from_disk`, 183
    - `dicee.read_preprocess_save_load_kg.save_load_disk`, 183
    - `dicee.read_preprocess_save_load_kg.util`, 184
  - `dicee.sanity_checkers`, 188
  - `dicee.scripts`, 189
    - `dicee.scripts.index_serve`, 189
    - `dicee.scripts.run`, 190
  - `dicee.static_funcs`, 190
    - `dicee.static_funcs_training`, 195
    - `dicee.static_preprocess_funcs`, 196
  - `dicee.trainer`, 197
    - `dicee.trainer.dice_trainer`, 197
    - `dicee.trainer.model_parallelism`, 199
    - `dicee.trainer.torch_trainer`, 200
    - `dicee.trainer.torch_trainer_ddp`, 201
  - `dicee.weight_averaging`, 204
- `modules()` (*dicee.models.ensemble.EnsembleKGE method*), 105
- `moving_average()` (*dicee.weight\_averaging.SWA static method*), 206
- `MultiClassClassificationDataset` (*class in dicee.dataset\_classes*), 32
- `MultiLabelDataset` (*class in dicee.dataset\_classes*), 33

## N

- `n` (*dicee.models.FMult2 attribute*), 177
- `n` (*dicee.models.function\_space.FMult2 attribute*), 107
- `n_embd` (*dicee.models.clifford.TransformerSelfAttention attribute*), 97
- `n_embd` (*dicee.models.CoKEConfig attribute*), 146



*n\_embd (dicee.models.real.CoKEConfig attribute), 119, 120*  
*n\_embd (dicee.models.transformers.GPTConfig attribute), 126*  
*n\_embd (dicee.models.transformers.SelfAttention attribute), 124*  
*n\_epochs\_eval\_model (dicee.callbacks.PeriodicEvalCallback attribute), 26*  
*n\_epochs\_eval\_model (dicee.config.Namespace attribute), 31*  
*n\_head (dicee.models.clifford.TransformerSelfAttention attribute), 97*  
*n\_head (dicee.models.CoKEConfig attribute), 146*  
*n\_head (dicee.models.real.CoKEConfig attribute), 119, 120*  
*n\_head (dicee.models.transformers.GPTConfig attribute), 126*  
*n\_head (dicee.models.transformers.SelfAttention attribute), 124*  
*n\_layer (dicee.models.CoKEConfig attribute), 146*  
*n\_layer (dicee.models.real.CoKEConfig attribute), 119, 120*  
*n\_layer (dicee.models.transformers.GPTConfig attribute), 126*  
*n\_layers (dicee.models.FMult2 attribute), 177*  
*n\_layers (dicee.models.function\_space.FMult2 attribute), 107*  
*name (dicee.abstracts.BaseInteractiveKGE property), 15*  
*name (dicee.models.AConEx attribute), 150*  
*name (dicee.models.AConvO attribute), 163*  
*name (dicee.models.AConvQ attribute), 157*  
*name (dicee.models.CKeci attribute), 166*  
*name (dicee.models.clifford.CKeci attribute), 97*  
*name (dicee.models.clifford.DeCaL attribute), 98*  
*name (dicee.models.clifford.Keci attribute), 93*  
*name (dicee.models.clifford.KeciTransformer attribute), 96*  
*name (dicee.models.CoKE attribute), 146*  
*name (dicee.models.ComplEx attribute), 151*  
*name (dicee.models.complex.AConEx attribute), 102*  
*name (dicee.models.complex.ComplEx attribute), 103*  
*name (dicee.models.complex.ConEx attribute), 101*  
*name (dicee.models.ConEx attribute), 149*  
*name (dicee.models.ConvO attribute), 162*  
*name (dicee.models.ConvQ attribute), 156*  
*name (dicee.models.DeCaL attribute), 167*  
*name (dicee.models.DistMult attribute), 144*  
*name (dicee.models.DualE attribute), 179*  
*name (dicee.models.dualE.DualE attribute), 104*  
*name (dicee.models.ensemble.EnsembleKGE attribute), 105*  
*name (dicee.models.FMult attribute), 176*  
*name (dicee.models.FMult2 attribute), 177*  
*name (dicee.models.function\_space.FMult attribute), 106*  
*name (dicee.models.function\_space.FMult2 attribute), 107*  
*name (dicee.models.function\_space.GFMult attribute), 106*  
*name (dicee.models.function\_space.LFMult attribute), 108*  
*name (dicee.models.function\_space.LFMult1 attribute), 107*  
*name (dicee.models.GFMult attribute), 177*  
*name (dicee.models.Keci attribute), 164*  
*name (dicee.models.KeciTransformer attribute), 170*  
*name (dicee.models.LFMult attribute), 178*  
*name (dicee.models.LFMult1 attribute), 178*  
*name (dicee.models.octonion.AConvO attribute), 113*  
*name (dicee.models.octonion.ConvO attribute), 112*  
*name (dicee.models.octonion.OMult attribute), 111*  
*name (dicee.models.OMult attribute), 161*  
*name (dicee.models.Pyke attribute), 145*  
*name (dicee.models.pykeen\_models.PykeenKGE attribute), 114*  
*name (dicee.models.PykeenKGE attribute), 173*  
*name (dicee.models.QMult attribute), 155*  
*name (dicee.models.quaternion.AConvQ attribute), 117*  
*name (dicee.models.quaternion.ConvQ attribute), 116*  
*name (dicee.models.quaternion.QMult attribute), 116*  
*name (dicee.models.real.CoKE attribute), 120*  
*name (dicee.models.real.DistMult attribute), 118*  
*name (dicee.models.real.Pyke attribute), 119*  
*name (dicee.models.real.Shallom attribute), 119*  
*name (dicee.models.real.TransE attribute), 118*  
*name (dicee.models.Shallom attribute), 145*  
*name (dicee.models.TransE attribute), 145*  
*name (dicee.models.transformers.Byte attribute), 122*

named\_children() (*dicee.models.ensemble.EnsembleKGE method*), 105  
 Namespace (*class in dicee.config*), 28  
 neg\_ratio (*dicee.config.Namespace attribute*), 29  
 neg\_ratio (*dicee.dataset\_classes.BPE\_NegativeSamplingDataset attribute*), 32  
 neg\_ratio (*dicee.dataset\_classes.KvsSampleDataset attribute*), 35  
 neg\_sample\_ratio (*dicee.dataset\_classes.FixedNegSampleDataset attribute*), 36  
 neg\_sample\_ratio (*dicee.dataset\_classes.OnevsSample attribute*), 37  
 neg\_sample\_ratio (*dicee.dataset\_classes.TriplePredictionDataset attribute*), 37  
 negnorm() (*dicee.abstracts.InteractiveQueryDecomposition method*), 16  
 neural\_searcher (*in module dicee.scripts.index\_serve*), 189  
 NeuralSearcher (*class in dicee.scripts.index\_serve*), 189  
 NodeTrainer (*class in dicee.trainer.torch\_trainer\_ddp*), 202  
 norm\_fc1 (*dicee.models.AConEx attribute*), 150  
 norm\_fc1 (*dicee.models.AConvO attribute*), 163  
 norm\_fc1 (*dicee.models.complex.AConEx attribute*), 102  
 norm\_fc1 (*dicee.models.complex.ConEx attribute*), 101  
 norm\_fc1 (*dicee.models.ConEx attribute*), 149  
 norm\_fc1 (*dicee.models.ConvO attribute*), 162  
 norm\_fc1 (*dicee.models.octonion.AConvO attribute*), 113  
 norm\_fc1 (*dicee.models.octonion.ConvO attribute*), 113  
 normalization (*dicee.analyse\_experiments.Experiment attribute*), 20  
 normalization (*dicee.config.Namespace attribute*), 29  
 normalization\_params (*dicee.dataset\_classes.LiteralDataset attribute*), 38  
 normalization\_type (*dicee.dataset\_classes.LiteralDataset attribute*), 38  
 normalize\_head\_entity\_embeddings (*dicee.models.base\_model.BaseKGE attribute*), 90  
 normalize\_head\_entity\_embeddings (*dicee.models.BaseKGE attribute*), 139, 142, 148, 153, 159, 172, 175  
 normalize\_relation\_embeddings (*dicee.models.base\_model.BaseKGE attribute*), 90  
 normalize\_relation\_embeddings (*dicee.models.BaseKGE attribute*), 139, 142, 148, 153, 159, 172, 175  
 normalize\_tail\_entity\_embeddings (*dicee.models.base\_model.BaseKGE attribute*), 90  
 normalize\_tail\_entity\_embeddings (*dicee.models.BaseKGE attribute*), 139, 142, 148, 153, 159, 172, 175  
 normalizer\_class (*dicee.models.base\_model.BaseKGE attribute*), 90  
 normalizer\_class (*dicee.models.BaseKGE attribute*), 139, 142, 148, 153, 159, 172, 175  
 num\_bpe\_entities (*dicee.dataset\_classes.BPE\_NegativeSamplingDataset attribute*), 32  
 num\_bpe\_entities (*dicee.knowledge\_graph.KG attribute*), 70  
 num\_core (*dicee.config.Namespace attribute*), 29  
 num\_datapoints (*dicee.dataset\_classes.BPE\_NegativeSamplingDataset attribute*), 32  
 num\_datapoints (*dicee.dataset\_classes.MultiLabelDataset attribute*), 33  
 num\_ent (*dicee.models.DualE attribute*), 180  
 num\_ent (*dicee.models.dualE.DualE attribute*), 104  
 num\_entities (*dicee.dataset\_classes.FixedNegSampleDataset attribute*), 36  
 num\_entities (*dicee.dataset\_classes.KvsSampleDataset attribute*), 35  
 num\_entities (*dicee.dataset\_classes.LiteralDataset attribute*), 38  
 num\_entities (*dicee.dataset\_classes.OnevsSample attribute*), 37  
 num\_entities (*dicee.dataset\_classes.TriplePredictionDataset attribute*), 37  
 num\_entities (*dicee.evaluation.Evaluator attribute*), 54, 55  
 num\_entities (*dicee.evaluation.evaluator.Evaluator attribute*), 45  
 num\_entities (*dicee.Evaluator attribute*), 218  
 num\_entities (*dicee.evaluator.Evaluator attribute*), 62, 63  
 num\_entities (*dicee.knowledge\_graph.KG attribute*), 69  
 num\_entities (*dicee.models.base\_model.BaseKGE attribute*), 90  
 num\_entities (*dicee.models.BaseKGE attribute*), 139, 142, 148, 152, 158, 171, 175  
 num\_epochs (*dicee.abstracts.AbstractPPECallback attribute*), 18  
 num\_epochs (*dicee.analyse\_experiments.Experiment attribute*), 20  
 num\_epochs (*dicee.config.Namespace attribute*), 28  
 num\_epochs (*dicee.trainer.torch\_trainer\_ddp.NodeTrainer attribute*), 202  
 num\_epochs (*dicee.weight\_averaging.ASWA attribute*), 204  
 num\_folds\_for\_cv (*dicee.config.Namespace attribute*), 29  
 num\_of\_data\_points (*dicee.dataset\_classes.MultiClassClassificationDataset attribute*), 33  
 num\_of\_data\_properties (*dicee.models.literal.LiteralEmbeddings attribute*), 109, 110  
 num\_of\_epochs (*dicee.callbacks.PseudoLabellingCallback attribute*), 24  
 num\_of\_output\_channels (*dicee.config.Namespace attribute*), 30  
 num\_of\_output\_channels (*dicee.models.base\_model.BaseKGE attribute*), 90  
 num\_of\_output\_channels (*dicee.models.BaseKGE attribute*), 139, 142, 148, 152, 158, 172, 175  
 num\_params (*dicee.analyse\_experiments.Experiment attribute*), 20  
 num\_relations (*dicee.dataset\_classes.FixedNegSampleDataset attribute*), 36  
 num\_relations (*dicee.dataset\_classes.OnevsSample attribute*), 37  
 num\_relations (*dicee.dataset\_classes.TriplePredictionDataset attribute*), 37  
 num\_relations (*dicee.evaluation.Evaluator attribute*), 54, 55

num\_relations (*dicee.evaluation.evaluator.Evaluator attribute*), 45  
 num\_relations (*dicee.Evaluator attribute*), 218, 219  
 num\_relations (*dicee.evaluator.Evaluator attribute*), 62, 63  
 num\_relations (*dicee.knowledge\_graph.KG attribute*), 69  
 num\_relations (*dicee.models.base\_model.BaseKGE attribute*), 90  
 num\_relations (*dicee.models.BaseKGE attribute*), 139, 142, 148, 152, 158, 171, 175  
 num\_sample (*dicee.models.FMult attribute*), 176  
 num\_sample (*dicee.models.function\_space.FMult attribute*), 106  
 num\_sample (*dicee.models.function\_space.GFMult attribute*), 106  
 num\_sample (*dicee.models.GFMult attribute*), 177  
 num\_samples (*dicee.weight\_averaging.TWA attribute*), 208  
 num\_tokens (*dicee.knowledge\_graph.KG attribute*), 70  
 num\_tokens (*dicee.models.base\_model.BaseKGE attribute*), 90  
 num\_tokens (*dicee.models.BaseKGE attribute*), 139, 142, 148, 152, 158, 171, 175  
 numpy\_data\_type\_changer () (*in module dicee.static\_funcs*), 193

## O

octonion\_mul () (*in module dicee.models*), 161  
 octonion\_mul () (*in module dicee.models.octonion*), 111  
 octonion\_mul\_norm () (*in module dicee.models*), 161  
 octonion\_mul\_norm () (*in module dicee.models.octonion*), 111  
 octonion\_normalizer () (*dicee.models.AConvO static method*), 163  
 octonion\_normalizer () (*dicee.models.ConvO static method*), 162  
 octonion\_normalizer () (*dicee.models.octonion.AConvO static method*), 113  
 octonion\_normalizer () (*dicee.models.octonion.ConvO static method*), 113  
 octonion\_normalizer () (*dicee.models.octonion.OMult static method*), 111  
 octonion\_normalizer () (*dicee.models.OMult static method*), 161  
 OMult (*class in dicee.models*), 161  
 OMult (*class in dicee.models.octonion*), 111  
 on\_epoch\_end () (*dicee.callbacks.KGESaveCallback method*), 23  
 on\_epoch\_end () (*dicee.callbacks.PseudoLabellingCallback method*), 24  
 on\_fit\_end () (*dicee.abstracts.AbstractCallback method*), 17  
 on\_fit\_end () (*dicee.abstracts.AbstractPPECallback method*), 18  
 on\_fit\_end () (*dicee.abstracts.AbstractTrainer method*), 13  
 on\_fit\_end () (*dicee.callbacks.AccumulateEpochLossCallback method*), 21  
 on\_fit\_end () (*dicee.callbacks.Eval method*), 24  
 on\_fit\_end () (*dicee.callbacks.KGESaveCallback method*), 23  
 on\_fit\_end () (*dicee.callbacks.LRScheduler method*), 27  
 on\_fit\_end () (*dicee.callbacks.PeriodicEvalCallback method*), 26  
 on\_fit\_end () (*dicee.callbacks.PrintCallback method*), 22  
 on\_fit\_end () (*dicee.weight\_averaging.ASWA method*), 205  
 on\_fit\_end () (*dicee.weight\_averaging.EMA method*), 208  
 on\_fit\_end () (*dicee.weight\_averaging.SWA method*), 206  
 on\_fit\_end () (*dicee.weight\_averaging.SWAG method*), 207  
 on\_fit\_end () (*dicee.weight\_averaging.TWA method*), 209  
 on\_fit\_start () (*dicee.abstracts.AbstractCallback method*), 17  
 on\_fit\_start () (*dicee.abstracts.AbstractPPECallback method*), 18  
 on\_fit\_start () (*dicee.abstracts.AbstractTrainer method*), 13  
 on\_fit\_start () (*dicee.callbacks.Eval method*), 24  
 on\_fit\_start () (*dicee.callbacks.KGESaveCallback method*), 23  
 on\_fit\_start () (*dicee.callbacks.KronE method*), 25  
 on\_fit\_start () (*dicee.callbacks.PrintCallback method*), 22  
 on\_init\_end () (*dicee.abstracts.AbstractCallback method*), 17  
 on\_init\_start () (*dicee.abstracts.AbstractCallback method*), 16  
 on\_train\_batch\_end () (*dicee.abstracts.AbstractCallback method*), 17  
 on\_train\_batch\_end () (*dicee.abstracts.AbstractTrainer method*), 14  
 on\_train\_batch\_end () (*dicee.callbacks.Eval method*), 25  
 on\_train\_batch\_end () (*dicee.callbacks.KGESaveCallback method*), 23  
 on\_train\_batch\_end () (*dicee.callbacks.LRScheduler method*), 27  
 on\_train\_batch\_end () (*dicee.callbacks.PrintCallback method*), 22  
 on\_train\_batch\_start () (*dicee.callbacks.Perturb method*), 26  
 on\_train\_epoch\_end () (*dicee.abstracts.AbstractCallback method*), 17  
 on\_train\_epoch\_end () (*dicee.abstracts.AbstractTrainer method*), 14  
 on\_train\_epoch\_end () (*dicee.callbacks.Eval method*), 25  
 on\_train\_epoch\_end () (*dicee.callbacks.KGESaveCallback method*), 23  
 on\_train\_epoch\_end () (*dicee.callbacks.PeriodicEvalCallback method*), 26  
 on\_train\_epoch\_end () (*dicee.callbacks.PrintCallback method*), 22

`on_train_epoch_end()` (*dicee.models.base\_model.BaseKGELightning method*), 85  
`on_train_epoch_end()` (*dicee.models.BaseKGELightning method*), 134  
`on_train_epoch_end()` (*dicee.weight\_averaging.ASWA method*), 205  
`on_train_epoch_end()` (*dicee.weight\_averaging.EMA method*), 208  
`on_train_epoch_end()` (*dicee.weight\_averaging.SWA method*), 206  
`on_train_epoch_end()` (*dicee.weight\_averaging.SWAG method*), 207  
`on_train_epoch_end()` (*dicee.weight\_averaging.TWA method*), 209  
`on_train_epoch_start()` (*dicee.abstracts.AbstractTrainer method*), 13  
`on_train_epoch_start()` (*dicee.weight\_averaging.EMA method*), 208  
`on_train_epoch_start()` (*dicee.weight\_averaging.SWA method*), 206  
`on_train_epoch_start()` (*dicee.weight\_averaging.SWAG method*), 207  
`on_train_epoch_start()` (*dicee.weight\_averaging.TWA method*), 209  
`on_train_start()` (*dicee.callbacks.LRScheduler method*), 27  
`OnevsAllDataset` (class in *dicee.dataset\_classes*), 35  
`OnevsSample` (class in *dicee.dataset\_classes*), 36  
`optim` (*dicee.config.Namespace attribute*), 28  
`optimizer` (*dicee.trainer.torch\_trainer\_ddp.NodeTrainer attribute*), 202  
`optimizer` (*dicee.trainer.torch\_trainer.TorchTrainer attribute*), 200  
`optimizer_name` (*dicee.models.base\_model.BaseKGE attribute*), 90  
`optimizer_name` (*dicee.models.BaseKGE attribute*), 139, 142, 148, 152, 158, 172, 175  
`ordered_bpe_entities` (*dicee.dataset\_classes.BPE\_NegativeSamplingDataset attribute*), 32  
`ordered_bpe_entities` (*dicee.knowledge\_graph.KG attribute*), 70  
`ordered_shaped_bpe_tokens` (*dicee.knowledge\_graph.KG attribute*), 69

## P

`p` (*dicee.config.Namespace attribute*), 30  
`p` (*dicee.models.clifford.DeCaL attribute*), 98  
`p` (*dicee.models.clifford.Keci attribute*), 93  
`p` (*dicee.models.DeCaL attribute*), 167  
`p` (*dicee.models.Keci attribute*), 164  
`P` (*dicee.weight\_averaging.TWA attribute*), 209  
`padding` (*dicee.knowledge\_graph.KG attribute*), 70  
`pandas_dataframe_indexer()` (in module *dicee.read\_preprocess\_save\_load\_kg.util*), 185  
`param_init` (*dicee.models.base\_model.BaseKGE attribute*), 90  
`param_init` (*dicee.models.BaseKGE attribute*), 139, 142, 148, 153, 159, 172, 175  
`parameters()` (*dicee.abstracts.BaseInteractiveKGE method*), 16  
`parameters()` (*dicee.models.ensemble.EnsembleKGE method*), 105  
`path` (*dicee.abstracts.AbstractPPECallback attribute*), 18  
`path` (*dicee.callbacks.AccumulateEpochLossCallback attribute*), 21  
`path` (*dicee.callbacks.Eval attribute*), 24  
`path` (*dicee.callbacks.KGESaveCallback attribute*), 23  
`path` (*dicee.weight\_averaging.ASWA attribute*), 204  
`path_dataset_folder` (*dicee.analyse\_experiments.Experiment attribute*), 20  
`path_for_deserialization` (*dicee.knowledge\_graph.KG attribute*), 69  
`path_for_serialization` (*dicee.knowledge\_graph.KG attribute*), 69  
`path_single_kg` (*dicee.config.Namespace attribute*), 28  
`path_single_kg` (*dicee.knowledge\_graph.KG attribute*), 69  
`path_to_store_single_run` (*dicee.config.Namespace attribute*), 28  
`PeriodicEvalCallback` (class in *dicee.callbacks*), 26  
`Perturb` (class in *dicee.callbacks*), 25  
`polars_dataframe_indexer()` (in module *dicee.read\_preprocess\_save\_load\_kg.util*), 184  
`poly_NN()` (*dicee.models.function\_space.LFMMult method*), 108  
`poly_NN()` (*dicee.models.LFMMult method*), 179  
`polynomial()` (*dicee.models.function\_space.LFMMult method*), 109  
`polynomial()` (*dicee.models.LFMMult method*), 179  
`pop()` (*dicee.models.function\_space.LFMMult method*), 109  
`pop()` (*dicee.models.LFMMult method*), 179  
`pos_emb` (*dicee.models.CoKE attribute*), 147  
`pos_emb` (*dicee.models.real.CoKE attribute*), 120  
`pq` (*dicee.analyse\_experiments.Experiment attribute*), 20  
`predict()` (*dicee.KGE method*), 213  
`predict()` (*dicee.knowledge\_graph\_embeddings.KGE method*), 72  
`predict_data_loader()` (*dicee.models.base\_model.BaseKGELightning method*), 87  
`predict_data_loader()` (*dicee.models.BaseKGELightning method*), 135  
`predict_literals()` (*dicee.KGE method*), 215  
`predict_literals()` (*dicee.knowledge\_graph\_embeddings.KGE method*), 74  
`predict_missing_head_entity()` (*dicee.KGE method*), 212

[predict\\_missing\\_head\\_entity\(\)](#) (*dicee.knowledge\_graph\_embeddings.KGE method*), 71  
[predict\\_missing\\_relations\(\)](#) (*dicee.KGE method*), 212  
[predict\\_missing\\_relations\(\)](#) (*dicee.knowledge\_graph\_embeddings.KGE method*), 72  
[predict\\_missing\\_tail\\_entity\(\)](#) (*dicee.KGE method*), 213  
[predict\\_missing\\_tail\\_entity\(\)](#) (*dicee.knowledge\_graph\_embeddings.KGE method*), 72  
[predict\\_topk\(\)](#) (*dicee.KGE method*), 213  
[predict\\_topk\(\)](#) (*dicee.knowledge\_graph\_embeddings.KGE method*), 72  
[preprocess\\_with\\_byte\\_pair\\_encoding\(\)](#) (*dicee.read\_preprocess\_save\_load\_kg.PreprocessKG method*), 187  
[preprocess\\_with\\_byte\\_pair\\_encoding\(\)](#) (*dicee.read\_preprocess\_save\_load\_kg.preprocess.PreprocessKG method*), 182  
[preprocess\\_with\\_byte\\_pair\\_encoding\\_with\\_padding\(\)](#) (*dicee.read\_preprocess\_save\_load\_kg.PreprocessKG method*), 187  
[preprocess\\_with\\_byte\\_pair\\_encoding\\_with\\_padding\(\)](#) (*dicee.read\_preprocess\_save\_load\_kg.preprocess.PreprocessKG method*), 182  
[preprocess\\_with\\_pandas\(\)](#) (*dicee.read\_preprocess\_save\_load\_kg.PreprocessKG method*), 187  
[preprocess\\_with\\_pandas\(\)](#) (*dicee.read\_preprocess\_save\_load\_kg.preprocess.PreprocessKG method*), 182  
[preprocess\\_with\\_polars\(\)](#) (*dicee.read\_preprocess\_save\_load\_kg.PreprocessKG method*), 187  
[preprocess\\_with\\_polars\(\)](#) (*dicee.read\_preprocess\_save\_load\_kg.preprocess.PreprocessKG method*), 182  
[preprocesses\\_input\\_args\(\)](#) (in module *dicee.static\_preprocess\_funcs*), 196  
[PreprocessKG](#) (class in *dicee.read\_preprocess\_save\_load\_kg*), 187  
[PreprocessKG](#) (class in *dicee.read\_preprocess\_save\_load\_kg.preprocess*), 182  
[PrintCallback](#) (class in *dicee.callbacks*), 21  
[process](#) (*dicee.trainer.torch\_trainer.TorchTrainer* attribute), 201  
[PseudoLabellingCallback](#) (class in *dicee.callbacks*), 23  
[Pyke](#) (class in *dicee.models*), 145  
[Pyke](#) (class in *dicee.models.real*), 119  
[pykeen\\_model\\_kwargs](#) (*dicee.config.Namespace* attribute), 30  
[PykeenKGE](#) (class in *dicee.models*), 173  
[PykeenKGE](#) (class in *dicee.models.pykeen\_models*), 114

## Q

[q](#) (*dicee.config.Namespace* attribute), 30  
[q](#) (*dicee.models.clifford.DeCaL* attribute), 98  
[q](#) (*dicee.models.clifford.Keci* attribute), 93  
[q](#) (*dicee.models.DeCaL* attribute), 167  
[q](#) (*dicee.models.Keci* attribute), 164  
[qdrant\\_client](#) (*dicee.scripts.index\_serve.NeuralSearcher* attribute), 189  
[QMult](#) (class in *dicee.models*), 155  
[QMult](#) (class in *dicee.models.quaternion*), 115  
[quaternion\\_mul\(\)](#) (in module *dicee.models*), 151  
[quaternion\\_mul\(\)](#) (in module *dicee.models.static\_funcs*), 121  
[quaternion\\_mul\\_with\\_unit\\_norm\(\)](#) (in module *dicee.models*), 155  
[quaternion\\_mul\\_with\\_unit\\_norm\(\)](#) (in module *dicee.models.quaternion*), 115  
[quaternion\\_multiplication\\_followed\\_by\\_inner\\_product\(\)](#) (*dicee.models.QMult* method), 155  
[quaternion\\_multiplication\\_followed\\_by\\_inner\\_product\(\)](#) (*dicee.models.quaternion.QMult* method), 116  
[quaternion\\_normalizer\(\)](#) (*dicee.models.QMult* static method), 156  
[quaternion\\_normalizer\(\)](#) (*dicee.models.quaternion.QMult* static method), 116  
[queries](#) (*dicee.scripts.index\_serve.StringListRequest* attribute), 190  
[query\\_name\\_to\\_struct](#) (*dicee.query\_generator.QueryGenerator* attribute), 181  
[query\\_name\\_to\\_struct](#) (*dicee.QueryGenerator* attribute), 216  
[QueryGenerator](#) (class in *dicee*), 215  
[QueryGenerator](#) (class in *dicee.query\_generator*), 181

## R

[r](#) (*dicee.models.clifford.DeCaL* attribute), 98  
[r](#) (*dicee.models.clifford.Keci* attribute), 93  
[r](#) (*dicee.models.DeCaL* attribute), 167  
[r](#) (*dicee.models.Keci* attribute), 164  
[random\\_seed](#) (*dicee.config.Namespace* attribute), 29  
[range\\_constraints\\_per\\_rel](#) (*dicee.evaluation.Evaluator* attribute), 55  
[range\\_constraints\\_per\\_rel](#) (*dicee.evaluation.evaluator.Evaluator* attribute), 45  
[range\\_constraints\\_per\\_rel](#) (*dicee.Evaluator* attribute), 219  
[range\\_constraints\\_per\\_rel](#) (*dicee.evaluator.Evaluator* attribute), 63  
[rank](#) (*dicee.Execute* attribute), 210  
[rank](#) (*dicee.executer.Execute* attribute), 66  
[ratio](#) (*dicee.callbacks.Perturb* attribute), 26  
[raw\\_test\\_set](#) (*dicee.knowledge\_graph.KG* attribute), 70  
[raw\\_train\\_set](#) (*dicee.knowledge\_graph.KG* attribute), 70  
[raw\\_valid\\_set](#) (*dicee.knowledge\_graph.KG* attribute), 70  
[re](#) (*dicee.models.clifford.DeCaL* attribute), 98



`re` (*dicee.models.DeCaL* attribute), 167  
`re_vocab` (*dicee.evaluation.Evaluator* attribute), 54, 55  
`re_vocab` (*dicee.evaluation.evaluator.Evaluator* attribute), 45  
`re_vocab` (*dicee.Evaluator* attribute), 218  
`re_vocab` (*dicee.evaluator.Evaluator* attribute), 62, 63  
`read_from_disk` () (in module *dicee.read\_preprocess\_save\_load\_kg.util*), 186  
`read_from_triple_store_with_pandas` () (in module *dicee.read\_preprocess\_save\_load\_kg.util*), 186  
`read_from_triple_store_with_polars` () (in module *dicee.read\_preprocess\_save\_load\_kg.util*), 186  
`read_only_few` (*dicee.config.Namespace* attribute), 29  
`read_only_few` (*dicee.knowledge\_graph.KG* attribute), 69  
`read_or_load_kg` () (in module *dicee.static\_funcs*), 193  
`read_with_pandas` () (in module *dicee.read\_preprocess\_save\_load\_kg.util*), 186  
`read_with_polars` () (in module *dicee.read\_preprocess\_save\_load\_kg.util*), 186  
`ReadFromDisk` (class in *dicee.read\_preprocess\_save\_load\_kg*), 188  
`ReadFromDisk` (class in *dicee.read\_preprocess\_save\_load\_kg.read\_from\_disk*), 183  
`reducer` (*dicee.scripts.index\_serve.StringListRequest* attribute), 190  
`reg_lambda` (*dicee.weight\_averaging.TWA* attribute), 208  
`rel2id` (*dicee.query\_generator.QueryGenerator* attribute), 181  
`rel2id` (*dicee.QueryGenerator* attribute), 215  
`relation_embeddings` (*dicee.models.AConvQ* attribute), 157  
`relation_embeddings` (*dicee.models.clifford.DeCaL* attribute), 98  
`relation_embeddings` (*dicee.models.ConvQ* attribute), 156  
`relation_embeddings` (*dicee.models.DeCaL* attribute), 167  
`relation_embeddings` (*dicee.models.DualE* attribute), 180  
`relation_embeddings` (*dicee.models.dualE.DualE* attribute), 104  
`relation_embeddings` (*dicee.models.FMult* attribute), 176  
`relation_embeddings` (*dicee.models.FMult2* attribute), 178  
`relation_embeddings` (*dicee.models.function\_space.FMult* attribute), 106  
`relation_embeddings` (*dicee.models.function\_space.FMult2* attribute), 107  
`relation_embeddings` (*dicee.models.function\_space.GFMult* attribute), 106  
`relation_embeddings` (*dicee.models.function\_space.LFMult* attribute), 108  
`relation_embeddings` (*dicee.models.function\_space.LFMult1* attribute), 107  
`relation_embeddings` (*dicee.models.GFMult* attribute), 177  
`relation_embeddings` (*dicee.models.LFMult* attribute), 178  
`relation_embeddings` (*dicee.models.LFMult1* attribute), 178  
`relation_embeddings` (*dicee.models.pykeen\_models.PykeenKGE* attribute), 114  
`relation_embeddings` (*dicee.models.PykeenKGE* attribute), 173  
`relation_embeddings` (*dicee.models.quaternion.AConvQ* attribute), 117  
`relation_embeddings` (*dicee.models.quaternion.ConvQ* attribute), 117  
`relation_to_idx` (*dicee.knowledge\_graph.KG* attribute), 69  
`relations_str` (*dicee.knowledge\_graph.KG* property), 70  
`reload_dataset` () (in module *dicee.dataset\_classes*), 39  
`report` (*dicee.DICE\_Trainer* attribute), 217  
`report` (*dicee.evaluation.Evaluator* attribute), 54, 55  
`report` (*dicee.evaluation.evaluator.Evaluator* attribute), 45, 46  
`report` (*dicee.Evaluator* attribute), 218, 219  
`report` (*dicee.evaluator.Evaluator* attribute), 63  
`report` (*dicee.Execute* attribute), 210  
`report` (*dicee.executer.Execute* attribute), 66  
`report` (*dicee.trainer.DICE\_Trainer* attribute), 203  
`report` (*dicee.trainer.dice\_trainer.DICE\_Trainer* attribute), 198  
`reports` (*dicee.callbacks.Eval* attribute), 24  
`reports` (*dicee.callbacks.PeriodicEvalCallback* attribute), 26  
`requires_grad_for_interactions` (*dicee.models.CKeci* attribute), 166  
`requires_grad_for_interactions` (*dicee.models.clifford.CKeci* attribute), 97  
`requires_grad_for_interactions` (*dicee.models.clifford.Keci* attribute), 93  
`requires_grad_for_interactions` (*dicee.models.Keci* attribute), 164  
`resid_dropout` (*dicee.models.clifford.TransformerSelfAttention* attribute), 97  
`resid_dropout` (*dicee.models.transformers.SelfAttention* attribute), 124  
`residual_convolution` () (*dicee.models.AConEx* method), 150  
`residual_convolution` () (*dicee.models.AConvO* method), 163  
`residual_convolution` () (*dicee.models.AConvQ* method), 157  
`residual_convolution` () (*dicee.models.complex.AConEx* method), 102  
`residual_convolution` () (*dicee.models.complex.ConEx* method), 101  
`residual_convolution` () (*dicee.models.ConEx* method), 150  
`residual_convolution` () (*dicee.models.ConvO* method), 162  
`residual_convolution` () (*dicee.models.ConvQ* method), 157  
`residual_convolution` () (*dicee.models.octonion.AConvO* method), 113

`residual_convolution()` (*dicее.models.octonion.ConvO method*), 113  
`residual_convolution()` (*dicее.models.quaternion.AConvQ method*), 117  
`residual_convolution()` (*dicее.models.quaternion.ConvQ method*), 117  
`retrieve_embedding()` (*dicее.scripts.index\_serve.NeuralSearcher method*), 189  
`retrieve_embeddings()` (*in module dicее.scripts.index\_serve*), 190  
`return_multi_hop_query_results()` (*dicее.KGE method*), 214  
`return_multi_hop_query_results()` (*dicее.knowledge\_graph\_embeddings.KGE method*), 73  
`root()` (*in module dicее.scripts.index\_serve*), 190  
`roots` (*dicее.models.FMult attribute*), 177  
`roots` (*dicее.models.function\_space.FMult attribute*), 106  
`roots` (*dicее.models.function\_space.GFMult attribute*), 106  
`roots` (*dicее.models.GFMult attribute*), 177  
`runtime` (*dicее.analyse\_experiments.Experiment attribute*), 20

## S

`sample()` (*dicее.weight\_averaging.SWAG method*), 207  
`sample_counter` (*dicее.abstracts.AbstractPPECallback attribute*), 18  
`sample_entity()` (*dicее.abstracts.BaseInteractiveKGE method*), 15  
`sample_relation()` (*dicее.abstracts.BaseInteractiveKGE method*), 15  
`sample_triples_ratio` (*dicее.config.Namespace attribute*), 29  
`sample_triples_ratio` (*dicее.knowledge\_graph.KG attribute*), 69  
`sample_weights()` (*dicее.weight\_averaging.TWA method*), 209  
`sampling_ratio` (*dicее.dataset\_classes.LiteralDataset attribute*), 38  
`sanity_check_callback_args()` (*in module dicее.sanity\_checkers*), 188  
`sanity_checking_with_arguments()` (*in module dicее.sanity\_checkers*), 188  
`save()` (*dicее.abstracts.BaseInteractiveKGE method*), 15  
`save()` (*dicее.read\_preprocess\_save\_load\_kg.LoadSaveToDisk method*), 188  
`save()` (*dicее.read\_preprocess\_save\_load\_kg.save\_load\_disk.LoadSaveToDisk method*), 183  
`save_checkpoint()` (*dicее.abstracts.AbstractTrainer static method*), 14  
`save_checkpoint_model()` (*in module dicее.static\_funcs*), 193  
`save_embeddings()` (*in module dicее.static\_funcs*), 194  
`save_embeddings_as_csv` (*dicее.config.Namespace attribute*), 28  
`save_every_n_epochs` (*dicее.config.Namespace attribute*), 30  
`save_experiment()` (*dicее.analyse\_experiments.Experiment method*), 20  
`save_model_at_every_epoch` (*dicее.config.Namespace attribute*), 29  
`save_model_every_n_epoch` (*dicее.callbacks.PeriodicEvalCallback attribute*), 26  
`save_numpy_ndarray()` (*in module dicее.read\_preprocess\_save\_load\_kg.util*), 186  
`save_numpy_ndarray()` (*in module dicее.static\_funcs*), 193  
`save_pickle()` (*in module dicее.read\_preprocess\_save\_load\_kg.util*), 187  
`save_pickle()` (*in module dicее.static\_funcs*), 192  
`save_queries()` (*dicее.query\_generator.QueryGenerator method*), 182  
`save_queries()` (*dicее.QueryGenerator method*), 216  
`save_queries_and_answers()` (*dicее.query\_generator.QueryGenerator static method*), 182  
`save_queries_and_answers()` (*dicее.QueryGenerator static method*), 216  
`save_trained_model()` (*dicее.Execute method*), 211  
`save_trained_model()` (*dicее.executer.Execute method*), 67  
`scalar_batch_NN()` (*dicее.models.function\_space.LFMult method*), 108  
`scalar_batch_NN()` (*dicее.models.LFMult method*), 179  
`scaler` (*dicее.callbacks.Perturb attribute*), 26  
`scaler` (*dicее.trainer.torch\_trainer\_ddp.NodeTrainer attribute*), 202  
`scheduler` (*dicее.callbacks.LRScheduler attribute*), 27  
`score()` (*dicее.models.clifford.Keci method*), 96  
`score()` (*dicее.models.CoKE method*), 147  
`score()` (*dicее.models.ComplEx static method*), 151  
`score()` (*dicее.models.complex.ComplEx static method*), 103  
`score()` (*dicее.models.DistMult method*), 145  
`score()` (*dicее.models.Keci method*), 166  
`score()` (*dicее.models.octonion.OMult method*), 112  
`score()` (*dicее.models.OMult method*), 161  
`score()` (*dicее.models.QMult method*), 156  
`score()` (*dicее.models.quaternion.QMult method*), 116  
`score()` (*dicее.models.real.CoKE method*), 120  
`score()` (*dicее.models.real.DistMult method*), 118  
`score()` (*dicее.models.real.TransE method*), 118  
`score()` (*dicее.models.TransE method*), 145  
`score_func` (*dicее.models.FMult2 attribute*), 177  
`score_func` (*dicее.models.function\_space.FMult2 attribute*), 107

scoring\_technique (dicee.analyse\_experiments.Experiment attribute), 20  
 scoring\_technique (dicee.config.Namespace attribute), 29  
 search() (dicee.scripts.index\_serve.NeuralSearcher method), 189  
 search\_embeddings() (in module dicee.scripts.index\_serve), 190  
 search\_embeddings\_batch() (in module dicee.scripts.index\_serve), 190  
 seed (dicee.dataset\_classes.FixedNegSampleDataset attribute), 36  
 seed (dicee.dataset\_classes.TriplePredictionDataset attribute), 37  
 seed (dicee.query\_generator.QueryGenerator attribute), 181  
 seed (dicee.QueryGenerator attribute), 215  
 select\_model() (in module dicee.static\_funcs), 193  
 selected\_optimizer (dicee.models.base\_model.BaseKGE attribute), 90  
 selected\_optimizer (dicee.models.BaseKGE attribute), 139, 142, 148, 153, 158, 172, 175  
 SelfAttention (class in dicee.models.transformers), 123  
 separator (dicee.config.Namespace attribute), 29  
 separator (dicee.knowledge\_graph.KG attribute), 69  
 seq\_len (dicee.models.clifford.KeciTransformer attribute), 96  
 seq\_len (dicee.models.KeciTransformer attribute), 170  
 sequential\_vocabulary\_construction() (dicee.read\_preprocess\_save\_load\_kg.PreprocessKG method), 187  
 sequential\_vocabulary\_construction() (dicee.read\_preprocess\_save\_load\_kg.preprocess.PreprocessKG method), 182  
 serve() (in module dicee.scripts.index\_serve), 190  
 set\_global\_seed() (dicee.query\_generator.QueryGenerator method), 181  
 set\_global\_seed() (dicee.QueryGenerator method), 216  
 set\_model\_eval\_mode() (dicee.abstracts.BaseInteractiveKGE method), 15  
 set\_model\_train\_mode() (dicee.abstracts.BaseInteractiveKGE method), 15  
 setup\_executor() (dicee.Execute method), 211  
 setup\_executor() (dicee.executor.Execute method), 67  
 Shallom (class in dicee.models), 145  
 Shallom (class in dicee.models.real), 119  
 shallom (dicee.models.real.Shallom attribute), 119  
 shallom (dicee.models.Shallom attribute), 145  
 single\_hop\_query\_answering() (dicee.KGE method), 214  
 single\_hop\_query\_answering() (dicee.knowledge\_graph\_embeddings.KGE method), 73  
 snapshot\_dir (dicee.callbacks.LRScheduler attribute), 27  
 snapshot\_loss (dicee.callbacks.LRScheduler attribute), 27  
 sparql\_endpoint (dicee.config.Namespace attribute), 28  
 sparql\_endpoint (dicee.knowledge\_graph.KG attribute), 69  
 sq\_mean (dicee.weight\_averaging.SWAG attribute), 207  
 start() (dicee.DICE\_Trainer method), 217  
 start() (dicee.Execute method), 211  
 start() (dicee.executor.Execute method), 67  
 start() (dicee.read\_preprocess\_save\_load\_kg.PreprocessKG method), 187  
 start() (dicee.read\_preprocess\_save\_load\_kg.preprocess.PreprocessKG method), 182  
 start() (dicee.read\_preprocess\_save\_load\_kg.read\_from\_disk.ReadFromDisk method), 183  
 start() (dicee.read\_preprocess\_save\_load\_kg.ReadFromDisk method), 188  
 start() (dicee.trainer.DICE\_Trainer method), 204  
 start() (dicee.trainer.dice\_trainer.DICE\_Trainer method), 199  
 start\_time (dicee.callbacks.PrintCallback attribute), 22  
 start\_time (dicee.Execute attribute), 211  
 start\_time (dicee.executor.Execute attribute), 66  
 state\_dict() (dicee.models.ensemble.EnsembleKGE method), 105  
 step() (dicee.models.ADOPT method), 131  
 step() (dicee.models.adopt.ADOPT method), 78  
 step() (dicee.models.ensemble.EnsembleKGE method), 105  
 step\_count (dicee.callbacks.LRScheduler attribute), 27  
 storage\_path (dicee.config.Namespace attribute), 28  
 storage\_path (dicee.DICE\_Trainer attribute), 217  
 storage\_path (dicee.trainer.DICE\_Trainer attribute), 203  
 storage\_path (dicee.trainer.dice\_trainer.DICE\_Trainer attribute), 198  
 store() (in module dicee.static\_funcs), 193  
 store\_ensemble() (dicee.abstracts.AbstractPPECallback method), 18  
 strategy (dicee.abstracts.AbstractTrainer attribute), 13  
 StringListRequest (class in dicee.scripts.index\_serve), 190  
 SWA (class in dicee.weight\_averaging), 205  
 swa (dicee.config.Namespace attribute), 30  
 swa\_c\_epochs (dicee.config.Namespace attribute), 31  
 swa\_c\_epochs (dicee.weight\_averaging.SWA attribute), 206  
 swa\_c\_epochs (dicee.weight\_averaging.SWAG attribute), 207  
 swa\_lr (dicee.weight\_averaging.SWA attribute), 206



swa\_lr (*dicee.weight\_averaging.SWAG attribute*), 207  
 swa\_model (*dicee.weight\_averaging.SWA attribute*), 206  
 swa\_n (*dicee.weight\_averaging.SWA attribute*), 206  
 swa\_start\_epoch (*dicee.config.Namespace attribute*), 31  
 swa\_start\_epoch (*dicee.weight\_averaging.SWA attribute*), 206  
 swa\_start\_epoch (*dicee.weight\_averaging.SWAG attribute*), 207  
 SWAG (*class in dicee.weight\_averaging*), 206  
 swag (*dicee.config.Namespace attribute*), 30

## T

T () (*dicee.models.DualE method*), 180  
 T () (*dicee.models.dualE.DualE method*), 104  
 t\_conorm () (*dicee.abstracts.InteractiveQueryDecomposition method*), 16  
 t\_norm () (*dicee.abstracts.InteractiveQueryDecomposition method*), 16  
 target\_dim (*dicee.dataset\_classes.AllvsAll attribute*), 34  
 target\_dim (*dicee.dataset\_classes.MultiLabelDataset attribute*), 33  
 target\_dim (*dicee.dataset\_classes.OnevsAllDataset attribute*), 36  
 target\_dim (*dicee.knowledge\_graph.KG attribute*), 70  
 temperature (*dicee.models.transformers.BytE attribute*), 122  
 tensor\_t\_norm () (*dicee.abstracts.InteractiveQueryDecomposition method*), 16  
 TensorParallel (*class in dicee.trainer.model\_parallelism*), 200  
 test\_data\_loader () (*dicee.models.base\_model.BaseKGELightning method*), 86  
 test\_data\_loader () (*dicee.models.BaseKGELightning method*), 134  
 test\_epoch\_end () (*dicee.models.base\_model.BaseKGELightning method*), 86  
 test\_epoch\_end () (*dicee.models.BaseKGELightning method*), 134  
 test\_h1 (*dicee.analyse\_experiments.Experiment attribute*), 20  
 test\_h3 (*dicee.analyse\_experiments.Experiment attribute*), 20  
 test\_h10 (*dicee.analyse\_experiments.Experiment attribute*), 20  
 test\_mrr (*dicee.analyse\_experiments.Experiment attribute*), 20  
 test\_path (*dicee.query\_generator.QueryGenerator attribute*), 181  
 test\_path (*dicee.QueryGenerator attribute*), 215  
 test\_set (*dicee.knowledge\_graph.KG attribute*), 69, 70  
 timeit () (*in module dicee.read\_preprocess\_save\_load\_kg.util*), 186  
 timeit () (*in module dicee.static\_funcs*), 192  
 timeit () (*in module dicee.static\_preprocess\_funcs*), 196  
 to () (*dicee.KGE method*), 212  
 to () (*dicee.knowledge\_graph\_embeddings.KGE method*), 71  
 to () (*dicee.models.ensemble.EnsembleKGE method*), 105  
 to\_df () (*dicee.analyse\_experiments.Experiment method*), 20  
 topk (*dicee.models.transformers.BytE attribute*), 122  
 topk (*dicee.scripts.index\_serve.NeuralSearcher attribute*), 189  
 torch\_ordered\_shaped\_bpe\_entities (*dicee.dataset\_classes.MultiLabelDataset attribute*), 33  
 TorchDDPTrainer (*class in dicee.trainer.torch\_trainer\_ddp*), 201  
 TorchTrainer (*class in dicee.trainer.torch\_trainer*), 200  
 total\_epochs (*dicee.callbacks.LRScheduler attribute*), 27  
 total\_steps (*dicee.callbacks.LRScheduler attribute*), 27  
 train () (*dicee.abstracts.BaseInteractiveTrainKGE method*), 19  
 train () (*dicee.trainer.torch\_trainer\_ddp.NodeTrainer method*), 202  
 train\_data (*dicee.dataset\_classes.AllvsAll attribute*), 34  
 train\_data (*dicee.dataset\_classes.KvsAll attribute*), 34  
 train\_data (*dicee.dataset\_classes.KvsSampleDataset attribute*), 35  
 train\_data (*dicee.dataset\_classes.MultiClassClassificationDataset attribute*), 33  
 train\_data (*dicee.dataset\_classes.OnevsAllDataset attribute*), 35  
 train\_data (*dicee.dataset\_classes.OnevsSample attribute*), 37  
 train\_data\_loader () (*dicee.models.base\_model.BaseKGELightning method*), 87  
 train\_data\_loader () (*dicee.models.BaseKGELighning method*), 136  
 train\_data\_loaders (*dicee.trainer.torch\_trainer.TorchTrainer attribute*), 201  
 train\_dataset\_loader (*dicee.trainer.torch\_trainer\_ddp.NodeTrainer attribute*), 202  
 train\_file\_path (*dicee.dataset\_classes.LiteralDataset attribute*), 38  
 train\_h1 (*dicee.analyse\_experiments.Experiment attribute*), 20  
 train\_h3 (*dicee.analyse\_experiments.Experiment attribute*), 20  
 train\_h10 (*dicee.analyse\_experiments.Experiment attribute*), 20  
 train\_indices\_target (*dicee.dataset\_classes.MultiLabelDataset attribute*), 33  
 train\_k\_vs\_all () (*dicee.abstracts.BaseInteractiveTrainKGE method*), 18  
 train\_literals () (*dicee.abstracts.BaseInteractiveTrainKGE method*), 19  
 train\_mode (*dicee.models.ensemble.EnsembleKGE attribute*), 105  
 train\_mrr (*dicee.analyse\_experiments.Experiment attribute*), 20

train\_path (*dicee.query\_generator.QueryGenerator* attribute), 181  
 train\_path (*dicee.QueryGenerator* attribute), 215  
 train\_set (*dicee.dataset\_classes.BPE\_NegativeSamplingDataset* attribute), 32  
 train\_set (*dicee.dataset\_classes.MultiLabelDataset* attribute), 33  
 train\_set (*dicee.dataset\_classes.TriplePredictionDataset* attribute), 37  
 train\_set (*dicee.knowledge\_graph.KG* attribute), 69, 70  
 train\_set\_target (*dicee.knowledge\_graph.KG* attribute), 70  
 train\_target (*dicee.dataset\_classes.AllvsAll* attribute), 34  
 train\_target (*dicee.dataset\_classes.KvsAll* attribute), 34  
 train\_target (*dicee.dataset\_classes.KvsSampleDataset* attribute), 35  
 train\_target\_indices (*dicee.knowledge\_graph.KG* attribute), 70  
 train\_triples (*dicee.dataset\_classes.FixedNegSampleDataset* attribute), 36  
 train\_triples() (*dicee.abstracts.BaseInteractiveTrainKGE* method), 18  
 trained\_model (*dicee.Execute* attribute), 210  
 trained\_model (*dicee.executer.Execute* attribute), 66  
 trainer (*dicee.config.Namespace* attribute), 29  
 trainer (*dicee.DICE\_Trainer* attribute), 217  
 trainer (*dicee.Execute* attribute), 210  
 trainer (*dicee.executer.Execute* attribute), 66  
 trainer (*dicee.trainer.DICE\_Trainer* attribute), 203  
 trainer (*dicee.trainer.dice\_trainer.DICE\_Trainer* attribute), 198  
 trainer (*dicee.trainer.torch\_trainer\_ddp.NodeTrainer* attribute), 202  
 training\_step (*dicee.trainer.torch\_trainer.TorchTrainer* attribute), 201  
 training\_step() (*dicee.models.base\_model.BaseKGELightning* method), 84  
 training\_step() (*dicee.models.BaseKGELightning* method), 133  
 training\_step() (*dicee.models.transformers.BytE* method), 122  
 training\_step\_outputs (*dicee.models.base\_model.BaseKGELightning* attribute), 84  
 training\_step\_outputs (*dicee.models.BaseKGELightning* attribute), 133  
 training\_technique (*dicee.knowledge\_graph.KG* attribute), 69  
 TransE (*class in dicee.models*), 145  
 TransE (*class in dicee.models.real*), 118  
 transformer (*dicee.models.clifford.KeciTransformer* attribute), 96  
 transformer (*dicee.models.KeciTransformer* attribute), 170  
 transformer (*dicee.models.transformers.BytE* attribute), 122  
 transformer (*dicee.models.transformers.GPT* attribute), 127  
 TransformerBlock (*class in dicee.models.clifford*), 96  
 TransformerMLP (*class in dicee.models.clifford*), 97  
 TransformerSelfAttention (*class in dicee.models.clifford*), 96  
 trapezoid() (*dicee.models.FMult2* method), 178  
 trapezoid() (*dicee.models.function\_space.FMult2* method), 107  
 tri\_score() (*dicee.models.function\_space.LFMult* method), 108  
 tri\_score() (*dicee.models.function\_space.LFMult1* method), 107  
 tri\_score() (*dicee.models.LFMult* method), 179  
 tri\_score() (*dicee.models.LFMult1* method), 178  
 triple\_score() (*dicee.KGE* method), 213  
 triple\_score() (*dicee.knowledge\_graph\_embeddings.KGE* method), 73  
 TriplePredictionDataset (*class in dicee.dataset\_classes*), 37  
 tuple2list() (*dicee.query\_generator.QueryGenerator* method), 181  
 tuple2list() (*dicee.QueryGenerator* method), 216  
 TWA (*class in dicee.weight\_averaging*), 208  
 twa (*dicee.config.Namespace* attribute), 30  
 twa\_c\_epochs (*dicee.weight\_averaging.TWA* attribute), 209  
 twa\_model (*dicee.weight\_averaging.TWA* attribute), 209  
 twa\_start\_epoch (*dicee.weight\_averaging.TWA* attribute), 208

## U

unlabelled\_size (*dicee.callbacks.PseudoLabellingCallback* attribute), 24  
 unmap() (*dicee.query\_generator.QueryGenerator* method), 181  
 unmap() (*dicee.QueryGenerator* method), 216  
 unmap\_query() (*dicee.query\_generator.QueryGenerator* method), 181  
 unmap\_query() (*dicee.QueryGenerator* method), 216  
 update\_hits() (*in module dicee.evaluation*), 61  
 update\_hits() (*in module dicee.evaluation.utils*), 53  
 use\_clifford\_mul (*dicee.models.clifford.KeciTransformer* attribute), 96  
 use\_clifford\_mul (*dicee.models.KeciTransformer* attribute), 170

## V

`val_aswa` (*dicee.weight\_averaging.ASWA attribute*), 205  
`val_data_loader()` (*dicee.models.base\_model.BaseKGELighting method*), 86  
`val_data_loader()` (*dicee.models.BaseKGELighting method*), 135  
`val_h1` (*dicee.analyse\_experiments.Experiment attribute*), 20  
`val_h3` (*dicee.analyse\_experiments.Experiment attribute*), 20  
`val_h10` (*dicee.analyse\_experiments.Experiment attribute*), 20  
`val_mrr` (*dicee.analyse\_experiments.Experiment attribute*), 20  
`val_path` (*dicee.query\_generator.QueryGenerator attribute*), 181  
`val_path` (*dicee.QueryGenerator attribute*), 215  
`VALID_SCORING_TECHNIQUES` (*in module dicee.evaluation.evaluator*), 45  
`valid_set` (*dicee.knowledge\_graph.KG attribute*), 69, 70  
`validate_knowledge_graph()` (*in module dicee.sanity\_checkers*), 188  
`var_clamp` (*dicee.weight\_averaging.SWAG attribute*), 207  
`vocab_preparation()` (*dicee.evaluation.Evaluator method*), 55  
`vocab_preparation()` (*dicee.evaluation.evaluator.Evaluator method*), 46  
`vocab_preparation()` (*dicee.Evaluator method*), 219  
`vocab_preparation()` (*dicee.evaluator.Evaluator method*), 63  
`vocab_size` (*dicee.models.CoKEConfig attribute*), 146  
`vocab_size` (*dicee.models.real.CoKEConfig attribute*), 119, 120  
`vocab_size` (*dicee.models.transformers.GPTConfig attribute*), 126  
`vocab_to_parquet()` (*in module dicee.static\_funcs*), 194  
`vtp_score()` (*dicee.models.function\_space.LFMult method*), 108  
`vtp_score()` (*dicee.models.function\_space.LFMultI method*), 108  
`vtp_score()` (*dicee.models.LFMult method*), 179  
`vtp_score()` (*dicee.models.LFMultI method*), 178

## W

`warmup_steps` (*dicee.callbacks.LRScheduler attribute*), 27  
`weight` (*dicee.models.transformers.LayerNorm attribute*), 123  
`weight_decay` (*dicee.config.Namespace attribute*), 29  
`weight_decay` (*dicee.models.base\_model.BaseKGE attribute*), 90  
`weight_decay` (*dicee.models.BaseKGE attribute*), 139, 142, 148, 152, 158, 172, 175  
`weight_samples` (*dicee.weight\_averaging.TWA attribute*), 209  
`weights` (*dicee.models.FMult attribute*), 177  
`weights` (*dicee.models.function\_space.FMult attribute*), 106  
`weights` (*dicee.models.function\_space.GFMult attribute*), 106  
`weights` (*dicee.models.GFMult attribute*), 177  
`world_size` (*dicee.Execute attribute*), 210  
`world_size` (*dicee.executer.Execute attribute*), 66  
`write_csv_from_model_parallel()` (*in module dicee.static\_funcs*), 194  
`write_links()` (*dicee.query\_generator.QueryGenerator method*), 181  
`write_links()` (*dicee.QueryGenerator method*), 216  
`write_report()` (*dicee.Execute method*), 211  
`write_report()` (*dicee.executer.Execute method*), 67

## X

`x_values` (*dicee.models.function\_space.LFMult attribute*), 108  
`x_values` (*dicee.models.LFMult attribute*), 178